

Constantin Gălăţan
Susana Gălăţan

Curs de



**Programare în Visual C# Studio 2008
Express Edition**



L&S SOFT

INFOBITS RO

Copyright 2021 © L&S SOFT

Toate drepturile asupra acestei lucrări aparțin editurii L&S SOFT.
Reproducerea integrală sau parțială a textului din această carte este posibilă doar cu acordul în scris al editurii L&S SOFT.

ISBN: 978-606-94898-4-0 (ebook)

ATENȚIE!

După confirmarea plății, fiecare carte poate fi descărcată de maximum 5 ori și este disponibilă 30 de zile.

Fiecare PDF este **securizat în 28 de zone cu watermark invizibil** (id comandă, e-mail, nume) pentru a nu putea fi distribuit pe alte căi virtuale.

Editura L&S SOFT

Adresa: Alea Aviatiei nr. 10, Voluntari, Ilfov;

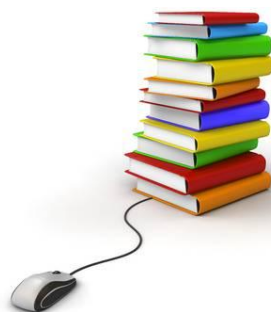
Mobil: 0727.731.947;

E-mail: hello@infobits.ro;

www.manuale-de-informatica.ro

infobits Academy

www.infobits.ro



CUPRINS

C# și platforma .NET	7
.NET Framework.....	8
Compilarea programelor pe platforma .NET	9
Instrumente de dezvoltare a aplicațiilor .NET	10
Principalele caracteristici ale arhitecturii .NET	10
Mediul Integrat Visual C# 2008 Express Edition	11
Instalare	11
Compilare în linie de comandă	12
Primul program C#	12
O scurtă analiză a programului salut.cs.....	14
Crearea unei aplicații de tip consolă	14
IntelliSense – o facilitate importantă a editorului de cod	16
Rezumatul capitolului.....	17
Întrebări și exerciții	17
Limbajul C# . Introducere	18
Structura unui program simplu C#	18
O aplicație de tip Consolă	18
Metoda Main - punctul de intrare în aplicație	19
Calificarea completă a numelor cu ajutorul operatorului	20
Parametri în linia de comandă	21
Variante ale metodei Main	23
Rezumatul capitolului.....	24
Întrebări și exerciții	24
Fundamentele limbajului C#.....	25
Tipuri.....	25
Tipuri predefinite	25
Tipuri definite de programator.....	27
Tipuri valoare și tipuri referință	29
Conversii între tipuri.....	31
Variabile și constante	32
Enumerări	33
Expresii	34
Tablouri	36
Tablouri unidimensionale	36
Tablouri multidimensionale	38
Tablouri neregulate	39
Instrucțiuni	42
Instrucțiunea de selecție if ... else	42
Instrucțiunea de selecție switch.....	43
Ciclul for	44
Ciclul foreach.....	44
Ciclul while.....	45
Ciclul do while	46
Instrucțiuni de salt necondiționat	46
Spații de nume	48
Directive de preprocesare	51
Directiva #define.....	52
Directiva #undefine	52

Rezumatul capitolului.....	53
Întrebări și exerciții	53
Programare Orientată pe Obiecte în C#.....	54
Obiecte și clase.....	54
Mecanismele fundamentale ale OOP.....	55
Clasele C#	56
Definirea claselor.....	57
Metode. Parametrii metodelor.....	59
Supraîncărcarea metodelor.....	63
Membrii statici ai unei clase	64
Constante	65
Constructorii	66
Cuvântul cheie <code>this</code>	69
Destructorul clasei	71
Proprietăți	71
Indexatori	76
Operatori de conversie.....	78
Clase interioare	80
Conținere	82
Clase parțiale	83
Clase sigilate.....	84
Supraîncărcarea operatorilor.....	84
Sintaxa	85
Supraîncărcarea operatorilor binari	85
Metoda <code>ToString()</code>	87
Supraîncărcarea operatorilor unari	87
Structuri	89
Interfețe	90
Moștenire.....	92
Specializare și generalizare.....	92
Implementarea moștenirii	93
Accesarea membrilor moșteniți	95
Constructorii claselor derivate	97
Membrii ascunși.....	99
Polimorfism	100
Conversia referințelor	100
Metode virtuale	101
Utilitatea polimorfismului.....	104
Rezumatul capitolului.....	105
Întrebări și exerciții	105
Trăsături esențiale ale limbajului C#	106
Delegări	106
Declarare.....	106
Crearea obiectelor delegare.....	106
Invocarea metodelor atașate unei delegări	107
Invocarea delegărilor cu tipuri de retur.....	108
Evenimente.....	109
Lucrul cu evenimente.....	109
Publicarea evenimentelor în mod specific .NET.....	112
Generice	114
Clase generice.....	115

Metode generice.....	117
Colecții.....	119
Clasa generică Stack<T>.....	119
Clasa generică List<T>.....	120
Clasa generică Dictionary<Tkey, Tvalue>.....	121
Tratarea excepțiilor.....	123
Manevrarea stringurilor.....	124
Operații și metode.....	125
Formatarea stringurilor.....	125
Transformarea stringurilor în valori numerice.....	126
Citirea unui șir de valori numerice.....	127
Citiri și afișări din fișiere de tip text.....	128
Rezumatul capitolului.....	130
Întrebări și exerciții.....	130

Aplicații de tip Windows Forms 131

Aplicații cu interfață grafică cu utilizatorul.....	131
Realizarea unei aplicații simple de tip Windows Forms.....	132
Controale, proprietăți și evenimente.....	134
Tratarea evenimentelor.....	135
Cum se crează handler-ele.....	135
O privire în spatele scenei.....	138
Declanșarea programatică a unui eveniment.....	142
Crearea programatică a unui control.....	143

Controalele Windows Forms..... 145

Controlul Button.....	145
Controalele Label și LinkLabel.....	148
Controalele RadioButton, CheckBox și GroupBox.....	151
Controlul TextBox.....	154
Controalele MenuStrip și ContextMenuStrip.....	161
Forme.....	165
Principalii membri ai clasei Form.....	165
Crearea formelor.....	166
Dialoguri modale și dialoguri nemodale.....	167
Butoane de validare a datelor.....	173
Dialoguri predefinite.....	177
Descrierea dialogurilor predefinite.....	177
Afișarea dialogurilor predefinite.....	178
MDI – Multiple Document Interface.....	184
Controlul RichTextBox.....	191
Controlul ToolTip.....	196
Controlul NotifyIcon.....	199
Fonturi.....	203
Stilurile Fonturilor.....	204
Fonturile instalate.....	206
Desenarea fonturilor.....	206
TabControl.....	208
Controalele ListBox, ComboBox și CheckedListBox.....	213
Controalele TrackBar, NumericUpDown și DomainUpDown.....	218
Controlul ProgressBar.....	223
Controlul Timer.....	224
Controalele PictureBox și ImageList.....	227

Controlul ListView	237
Controlul TreeView	245
Controalele Web Browser și StatusStrip	253
Integrarea WindowsMediaPlayer în aplicații	257
Desenare în .NET cu Visual C#.....	260
Clasa Graphics.....	260
Penițe pentru desenarea formelor	260
Pensule pentru umplerea formelor	264
Desenarea textului	266
XML cu C#	270
Sintaxa XML	270
Clase .NET pentru Xml	271
Citirea informațiilor dintr-un document XML	271
Descărcarea fișierelor XML de pe Internet.....	275
Citirea și analiza unui document XML cu XmlTextReader	276
Crearea conținutului XML cu XmlTextWriter	279
Baze de date și ADO.NET. Noțiuni introductive.....	282
Instrumente de lucru	282
Calitățile SGDBR-urilor	283
Furnizori de baze de date relaționale	284
Tehnologia ADO.NET – introducere	284
Caracteristicile tehnologiei ADO.NET	284
Arhitectura ADO.NET	285
Crearea unei baze de date în VCSE	288
Interogări cu Query Designer.....	292
Controlul DataGridView	299
Aplicații cu baze de date în modelul conectat.....	304
Utilizarea provider-ului pentru SQL Server 2005	304
Utilizarea provider-ului pentru OLE DB	309
Utilizarea provider-ului pentru ODBC	312
Aplicații cu baze de date în modelul deconectat.....	320
Construirea și utilizarea dataset-urilor	320
Accesarea tabelelor într-un dataset	321
Accesarea rândurilor și coloanelor într-o tabelă	321
Accesarea valorilor dintr-o tabelă a unui dataset	322
Propagarea schimbărilor din dataset spre baza de date	322
Dataset-urile și XML	326
Controalele și legarea datelor	327
Legarea simplă a datelor	327
Legarea complexă a datelor	328
Relații între tabele.....	335
Constrângerea Cheie Străină-Cheie Primară	335
Interogări. Proceduri stocate.....	338
Vederile unei baze de date (Views).....	346
Bibliografie.....	351

Partea I

Limbajul C#

Capitolul 1

C# și platforma .NET

Numele limbajului **C#** a fost inspirat din notația *#* (*diez*) din muzică, care indică faptul că nota muzicală urmată de *#* este mai înaltă cu un semiton. Este o similitudine cu numele limbajului C++, unde ++ reprezintă atât incrementarea unei variabile cu valoarea 1, dar și faptul că C++ este mai mult decât limbajul C.

Limbajul C# a fost dezvoltat în cadrul Microsoft. Principalii creatori ai limbajului sunt Anders Hejlsberg, Scott Wiltamuth și Peter Golde. Prima implementare C# larg distribuită a fost lansată de către Microsoft ca parte a inițiativei **.NET** în iulie 2000. Din acel moment, se poate vorbi despre o evoluție spectaculoasă. Mii de programatori de C, C++ și Java, au migrat cu ușurință spre C#, grație asemănării acestor limbaje, dar mai ales calităților noului limbaj. La acest moment, C# și-a câștigat și atrage în continuare numeroși adepți, devenind unul dintre cele mai utilizate limbaje din lume.

Creatorii C# au intenționat să înzestreze limbajul cu mai multe facilități. Succesul de care se bucură în prezent, confirmă calitățile sale:

- Este un limbaj de programare simplu, modern, de utilitate generală, cu productivitate mare în programare.
- Este un limbaj *orientat pe obiecte*.
- Permite dezvoltarea de aplicații industriale robuste, durabile.
- Oferă suport complet pentru dezvoltarea de componente software, foarte necesare de pildă în medii distribuite. De altfel, se poate caracteriza C# ca fiind nu numai *orientat obiect*, ci și *orientat spre componente*.

La aceste caracteristici generale se adaugă și alte trăsături, cum este de pildă suportul pentru internaționalizare, adică posibilitatea de a scrie aplicații care pot fi adaptate cu ușurință pentru a fi utilizate în diferite regiuni ale lumii unde se vorbesc limbi diferite, fără să fie necesare pentru aceasta schimbări în arhitectura software.

În strânsă legătură cu **Arhitectura .NET (.NET Framework)** pe care funcționează, C# gestionează în mod automat memoria utilizată. Eliberarea memoriei ocupate (*garbage collection*) de către obiectele care nu mai sunt necesare aplicației, este o facilitate importantă a limbajului. Programatorii nu mai trebuie să decidă singuri, așa cum o fac de pildă în C++, care este locul și momentul în care obiectele trebuie distruse.

În C# se scriu de asemenea aplicații pentru sisteme complexe care funcționează sub o mare varietate de sisteme de operare, cât și pentru sisteme

dedicate (*embedded systems*). Acestea din urmă se întind pe o arie largă, de la dispozitive portabile cum ar fi ceasuri digitale, telefoane mobile, MP3 playere, până la dispozitive staționare ca semafoare de trafic, sau controlere pentru automatizarea producției.

Din punct de vedere sintactic C# derivă din limbajul C++, dar include și influențe din alte limbaje, mai ales Java.

.NET Framework

Arhitectura .NET este o componentă software care oferă un mediu de programare și de execuție a aplicațiilor pentru sistemele de operare Microsoft. Este inclusă în sistemele de operare *Windows Server 2008* și *Windows Vista* și poate fi instalată pe *Windows XP* și *Windows Server 2003*.

.NET Framework este un mediu care permite dezvoltarea și rularea aplicațiilor și a serviciilor Web, independente de platformă.

Limbajul C# se află într-o strânsă legătură cu arhitectura .NET. Inițial, C# a fost dezvoltat de către Microsoft pentru crearea codului platformei .Net, la fel cum destinația inițială a limbajului C a fost aceea de a implementa sistemul de operare UNIX. .NET pune la dispoziție o colecție impresionantă de clase organizate în biblioteci, pe care C# le utilizează.

Este momentul să precizăm că C# funcționează având .NET ca infrastructură, dar .NET suportă și alte limbaje, cum este C++, Visual Basic sau Java. În oricare dintre aceste limbaje programați, aveți la dispoziție aceleași biblioteci de clase. .NET se realizează în acest fel **interoperabilitatea limbajelor**.

.NET este constituit din două entități importante:

- **Common Language Runtime (CLR)**

Acesta este **mediul de execuție** al programelor. Este modulul care se ocupă cu managementul și execuția codului scris în limbaje specifice **.NET**. **CLR** furnizează de asemenea servicii importante, cum sunt securitatea aplicațiilor, portabilitatea acestora, managementul memoriei și tratarea excepțiilor.

- **Base Class Library**

Este vorba despre *Biblioteca de Clase .NET*. Această bibliotecă acoperă o arie largă a necesităților de programare, incluzând interfața cu utilizatorul, conectarea cu bazele de date și accesarea datelor, dezvoltarea aplicațiilor web, comunicarea în rețele și altele. Codul bibliotecii este precompilat, fiind încapsulat de regulă în funcții, numite metode, pe care programatorul le poate apela din propriul program. La rândul lor, metodele aparțin claselor, iar clasele sunt organizate și separate între ele cu ajutorul spațiilor de nume (*namespaces*). Despre toate aceste noțiuni vom vorbi pe larg în capitolele următoare. Ceea ce trebuie reținut pentru moment, este că programatorii combină propriul cod cu codul Bibliotecii de Clase .NET pentru producerea de aplicații.

Compilarea programelor pe platforma .NET

Limbaje interpretate

Când programați într-un limbaj de programare interpretat, veți scrie codul și veți rula aplicația. Cu ajutorul unui interpretor specific limbajului, fiecare linie de cod este interpretată chiar în momentul rulării și este preschimbată imediat în cod mașină și executată. Partea bună este că puteți rula codul până la primul bug, puteți înlătura eroarea în cod iar apoi să continuați execuția codului. Limbajul nu necesită timp de compilare și de legare. Partea neplăcută este că limbajele interpretate sunt lente. Limbajul Basic a fost inițial un limbaj interpretat, dar mai apoi, începând cu anii 1980, au apărut și versiuni compilate. Marea majoritate a limbajelor de scriptare Web sunt de asemenea limbaje interpretate.

Limbaje compilate

Codul scris într-un asemenea limbaj, numit *cod sursă*, este translatat de către compilator într-un cod apropiat de nivelul mașinii, numit cod executabil (de exemplu codul conținut în fișierele (*.exe)). Dacă în timpul compilării apar erori, atunci este necesar să le înlăturați, după care veți compila din nou. Dacă aplicația trece acum de compilare fără erori de sintaxă, atunci se va produce codul executabil și veți putea să rulați aplicația. Limbajele C și C++ sunt exemple clasice de limbaje compilate.

Din această perspectivă C# este un limbaj compilat. Dar nu în sensul descris mai sus. Ca să înțelegeți, este necesar să știți că în urma compilării unui program C#, nu se crează un cod executabil. Se creează un fișier numit **assembly** care de regulă se identifică cu extensia **.exe** sau **.dll**. Un asemenea fișier nu poate fi executat pe un sistem pe care nu există infrastructura **.NET**. Fișierul conține un tip special de cod, numit **Limbaj Intermediar**, pe scurt **CIL** (*Common Intermediate Language*). Limbajul **CIL** definește un set de instrucțiuni portabile, independente de orice tip de procesor și platformă.

Figura ilustrează procesul de creare a codului executabil pe platforma .NET.

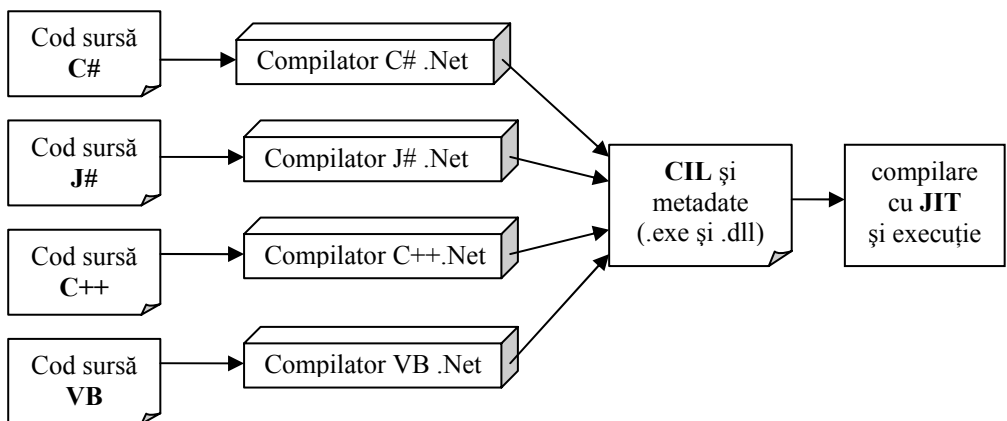


Fig. 1.1 Procesul de compilare pe platforma .NET

În momentul în care un program este executat, **CLR** activează un compilator special, numit **JIT** (*just in time*). Acesta preia codul **CIL** și îl transformă în cod executabil. Transformarea se face “la cerere”, în sensul că o secvență de cod se compilează doar în momentul în care este utilizată pentru prima oară. Un program compilat în format **CIL** poate rula pe orice sistem pe care s-a instalat **Common Language Runtime**. Aceasta asigură portabilitatea aplicațiilor **.NET**.

Fișierul .exe sau .dll produs la compilare conține pe lângă codul **CIL**, așa numitele **metadata**. Metadatele descriu datele utilizate de către aplicație.

Instrumente de dezvoltare a aplicațiilor .NET

Platforma **.NET** actuală a ajuns la versiunea **3.5**. Microsoft pune la dispoziția programatorilor două unelte pentru dezvoltarea aplicațiilor:

1. **Visual Studio .NET** și varianta *free* **Visual Studio Express 2008**.
2. **.NET Framework SDK**.

Pachetul de dezvoltare a aplicațiilor pentru **.NET 3.5** (*Microsoft .NET Framework 3.5 SDK*¹), include:

- **.NET Framework**
- **Compilatoare** în linie de comandă pentru limbajele de programare: **C#**, **C++**, **Visual Basic**, și **Jscript**.
- Instrumente pentru crearea, depanarea și configurarea aplicațiilor **.NET**.
- Exemple și documentație.

Trebuie să știți că există compilatoare pentru platforma **.NET** create de anumite firme sau organizații, pentru limbajele **Smalltalk**, **Perl**, **Cobol** sau **Pascal**, ca să enumerăm doar câteva disponibile pe piață.

Principalele caracteristici ale arhitecturii .NET

Independența de procesor și de platformă

Codul **CIL** este independent de sistemul de operare și de procesor.

De aceea, în scrierea aplicațiilor nu trebuie să fiți preocupați de caracteristicile hardware sau software ale sistemului. Spre surpriza multor programatori, aplicațiile **.NET** pot fi dezvoltate și executate pe sisteme de operare non-Microsoft, (Mac OS X, numeroase distribuții Linux și Solaris, ca să numim numai câteva).

Managementul automat al memoriei

Alocarea și eliberarea memoriei nu mai este o problemă care trebuie să-i preocupe pe programatori, datorită mecanismului automat de **garbage collection**.

Interoperabilitatea limbajelor

Este un fapt comun ca diversele componente ale unei aplicații să fie scrise în limbaje diferite, suportate de către platforma **.NET**.

¹ *Software Development Kit* – Kit de Dezvoltare a Aplicațiilor

Securitate

.NET furnizează un model comun de securitate, valabil pentru toate aplicațiile, care include un mecanism unificat de *tratare a excepțiilor*. O excepție este un eveniment neprevăzut, care întrerupe execuția unui program, atunci când de pildă, se execută o instrucțiune ilegală.

Portabilitate

Un program scris pentru platforma .NET poate rula fără nici o modificare pe oricare sistem pe care platforma este instalată.

Caracteristicilor de mai sus li se adaugă și altele, care ies însă din cadrul acestei lucrări.

Mediul Integrat Visual C# 2008 Express Edition

Visual C# 2008 Express Edition (pe scut: **VCSE**), este un mediu free de dezvoltare a aplicațiilor produs de Microsoft. Este un **IDE** (*integrated development environment*), care oferă un set de instrumente, între care un editor de cod pentru scrierea programelor C#, compilator, depanator, instrumente pentru *build automation* (automatizarea procesului de compilare) și altele. Kit-ul de instalare *C# Express*, include *Platforma .NET 3.5*, iar aceasta la rândul ei include între altele *Biblioteca de Clase*.

Cerințe de sistem

Sistemele de operare suportate sunt: *Windows Server 2003*, *Windows Vista*; *Windows XP*.

- Pentru Microsoft Windows XP, Service Pack 2
 - minim 192 MB de RAM (preferabil cel puțin 384 MB)
 - cel puțin un procesor de 1 GHz (preferabil > 1.6 GHz)
- Pentru Microsoft Windows Vista și Microsoft Windows Server 2003
 - minim 768 MB de RAM (preferabil cel puțin 1 GB)
 - cel puțin un procesor de 1,6 GHz (preferabil > 2.2 GHz)

Instalare

Visual C# 2008 Express Edition poate fi descărcat de pe site-ul Microsoft, la adresa <http://www.microsoft.com/express/download/>. Alternativ, în josul paginii aveți opțiunea de a descărca **Visual Studio Express Edition** pentru o instalare offline. Visual Studio conține mediile de programare **Visual C#**, **Visual Basic**, **Visual C++**, precum și serverul de baze de date **Microsoft SQL Server 2005**. Instalarea se face simplu, cu ajutorul unui wizard, însă pentru montarea imaginii DVD-ului (fișier cu extensia ISO) aveți nevoie de un utilitar cum ar fi de pildă *Daemon Tools*.

Ca observație, indiferent dacă veți instala *Visual Studio* sau doar *Visual C# 2008 Express*, veți opta întotdeauna pentru instalarea *.NET Framework*, *Visual C#*, *MS SQL Server* și **MSDN** (*Microsoft Developer Network*). MSDN conține o foarte bogată documentație de care nu vă puteți lipsi când dezvoltați aplicații.

Există și versiunile Visual C# 2008 și Visual Studio 2008 cu facilități suplimentare, dar care nu sunt gratuite.

Ajunși în acest punct, dorim să atragem atenția asupra faptului că o prejudecată răspândită privind dezvoltarea .NET este aceea că programatorii trebuie să instaleze Visual Studio ca să poată crea aplicații C#. Nu este adevărat. Puteți să compilați și să rulați orice tip de program .NET folosind kit-ul de dezvoltare a aplicațiilor **.NET Framework 3.5 Software Development Kit (SDK)**, care este downloadabil în mod gratuit. Acest SDK vă pune la dispoziție compilatoare, utilitare în linie de comandă, conține *Biblioteca de Clase .Net*, exemple de cod și o documentație completă.

Se pot crea aplicații C# în două moduri diferite:

1. Folosind *Notepad* sau oricare editor de text și apoi compilarea în linie de comandă.
2. Utilizând *Visual Studio Express Edition*. Aceasta este metoda preferabilă, datorită sprijinului considerabil pe care-l oferă mediul integrat în dezvoltarea aplicațiilor, mai ales a aceloră cu interfață cu utilizatorul.

Compilare în linie de comandă

Deși s-ar putea ca niciodată să nu vă decideți să dezvoltați aplicații mari folosind compilatorul C# în linie de comandă, este totuși important să înțelegeți cum se lucrează în linie de comandă fie doar și pentru următoarele motive:

- Nu aveți o copie a Visual C# 2008 Express, sau sistemul pe care-l nu satisface cerințele hardware sau software minimale;
- Doriți să proiectați un **build tool** automatizat, așa cum este *MSBuild* sau *Nant*, care necesită să cunoașteți opțiunile în linie de comandă ale utilităților.
- Doriți o înțelegere profundă a C#, doriți să vedeți ce se petrece în spatele scenei atunci când utilizați IDE-uri pentru dezvoltarea aplicațiilor.

Primul program C#

Este o tradiție păstrată în timp, ca o carte de programare să înceapă cu programul care afișează pe ecran "Salut lume !". Vom compila și executa acest program C# în linie de comandă, urmând pașii de mai jos:

1. Editarea codului sursă

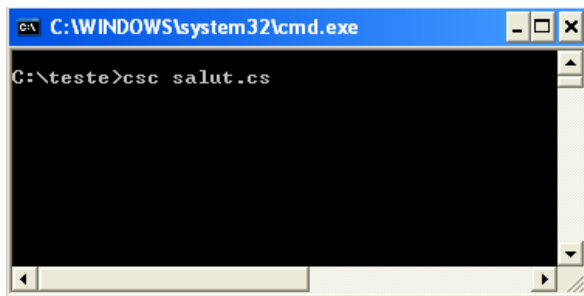
Deschideți *Notepad* și scrieți următorul cod:

```
using System;
class Salut
{
    static void Main()
    {
        Console.WriteLine("Salut lume!");
    }
}
```

În C# fișierele sursă au extensia **cs**. Salvați fișierul cu numele **salut.cs** într-un folder oarecare, de exemplu **C:\teste**.

2. Compilarea în linie de comandă

La promptul de comandă, compilați astfel: **csc salut.cs**



Așadar, compilatorul **csc.exe** (*C sharp compiler*) primește ca argument în linie de comandă numele fișierului sursă. Veți constata că în urma compilării, în folderul C:\teste s-a creat fișierul **salut.exe**. Acesta este un **assembly**, conținând codul intermediar **CIL**. Executarea lui este posibilă doar pe sistemele care au infrastructura .NET.

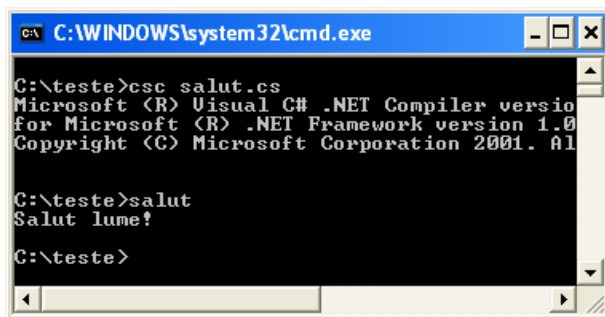
3. Executarea programului

Rularea programului se face simplu, prin tastarea numelui fișierului **.exe**:

```
c:\teste>salut
```

Programul afișează:

```
Salut lume!
```



Setarea variabilei de mediu PATH

Dacă platforma .NET a fost instalată odată cu *Visual Studio* sau **VCSE**, atunci la instalare puteți opta ca variabila de mediu **PATH** să rețină în mod implicit căile spre utilitarele în linie de comandă, inclusiv spre compilator, ceea ce vă scutește de efortul de a le introduce manual. Dacă **PATH** nu reține aceste căi, atunci nu veți putea lucra în linie de comandă, deoarece sistemul de operare nu va găsi aceste utilitare.

În cazul în care nu ați ales varianta setării PATH la instalare, o puteți face ulterior astfel:

- La promptul de comandă, mutați-vă în subdirectorul **Common7\Tools** al instalării.
- Rulați fișierul de comenzi **VSVARS32.bat** scriind: **VSVARS32**.

O scurtă analiză a programului salut.cs

Prima linie a programului, **using System**; spune implementării C# prin intermediul directivei **using**, că se vor utiliza clase aflate în spațiul de nume **System**. Așa cum veți vedea în capitolul următor, C# a preluat din C++ spațiile de nume (*namespaces*). Spațiile de nume funcționează ca niște containere, în care se definesc nume de clase pentru evitarea conflictelor de nume, dar și pentru separarea logică. Toate clasele din *Biblioteca de Clase .NET* se definesc în interiorul unor spații de nume. **System** este un spațiu de nume fundamental, care definește între altele, clasa **Console**. Aceasta din urmă, conține metode (funcțiile din C#) care scriu și citesc date de la consolă.

Observăm că la fel ca în C++, există metoda **Main()**, care se declară întotdeauna **static**.

IMPORTANT!

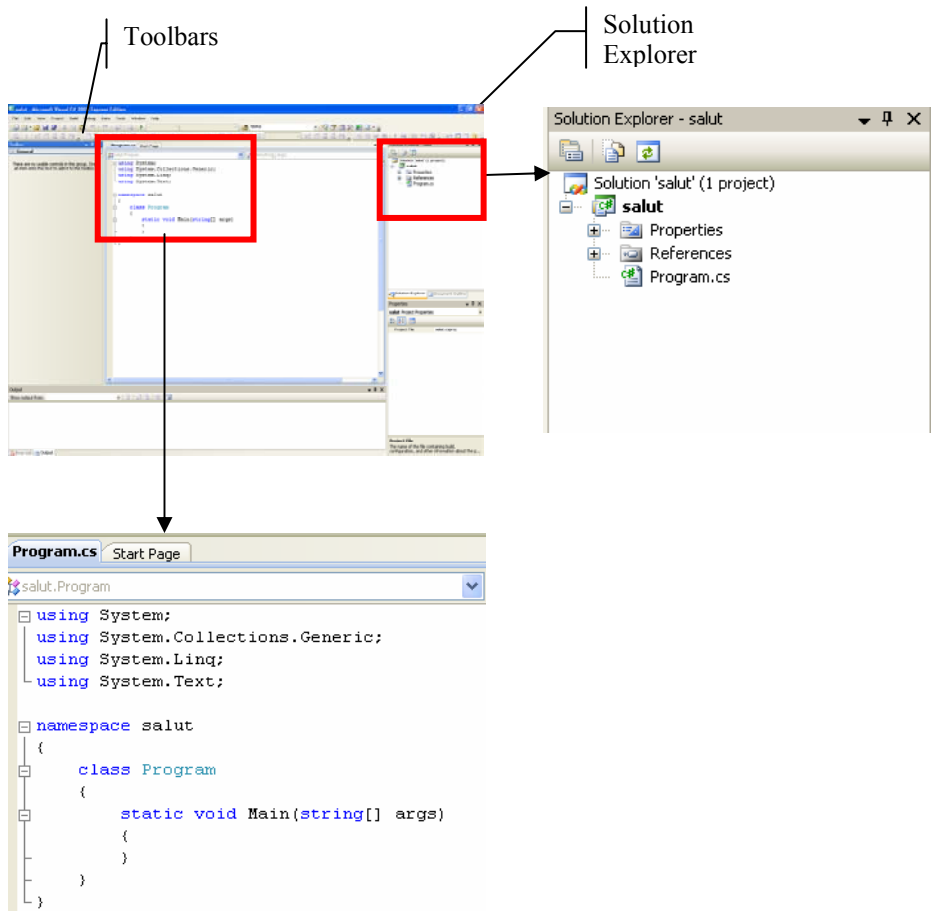
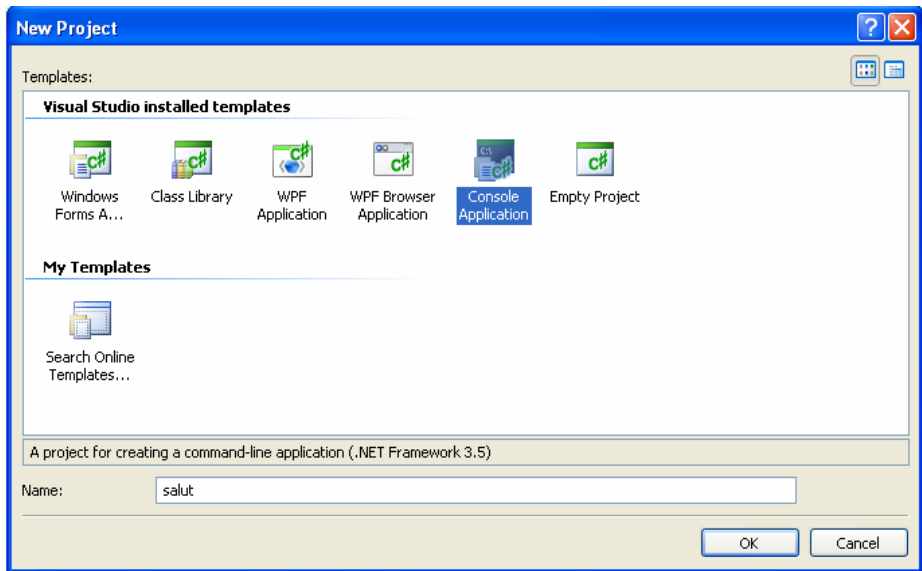
O deosebire față de C++ este faptul că trebuie să existe o clasă, în cazul de față clasa **Salut**, care găzduiește metoda **Main()**. Aceasta, deoarece C# este un **limbaj pur orientat obiect**, și nu pot exista definiții de funcții în afara claselor și nici variabile globale.

În acest sens, C# seamănă cu Java. Totuși, în Java numele clasei care conține metoda **main()** trebuie să coincidă în mod obligatoriu cu numele fișierului. Dacă veți schimba mai sus numele clasei, de exemplu **Hello**, o să constatați că nu veți avea nici o problemă la compilare.

Linia **Console.WriteLine("Salut lume!");** este apelul metodei statice **WriteLine** din clasa **Console**. O să revenim asupra metodelor statice în capitolele următoare. Pentru moment, rețineți că o metodă statică se apelează astfel: **nume_clasa.nume_metodă**.

Crearea unei aplicații de tip consolă

1. Deschideți mediul integrat de programare: **Start->All Programs-> Visual C# 2008 Express Edition**.
2. În meniul **File**, selectați **New Project**. Se deschide dialogul **New Project**. Acesta permite alegerea diferitor tipuri de aplicații. Selectați **Console Application** ca tip de proiect și schimbați numele aplicației în **salut**. Click **OK**. **Visual C# Express Edition 2008** crează un nou folder pentru proiect, cu același nume cu cel al proiectului. Deschide de asemenea fereastra principală și **Editorul de Cod** unde veți intra și veți modifica codul sursă C#.



Barele de instrumente (Toolbars) sunt în partea de sus a ferestrei principale. Acestea conțin icon-uri pentru crearea, încărcarea și salvarea proiectelor, editarea codului sursă, compilarea aplicației. Accesare: **View->Toolbars**. În partea a doua a lucrării, prin **Toolbar** vom identifica bara de instrumente care păstrează iconuri cu controalele *Windows Forms*.

Solution Explorer este un panou foarte util care afișează fișierele care constituie proiectul. Cel mai important fișier este **Program.cs**, care conține codul sursă al aplicației.

3. Este permis să schimbați numele clasei în **Editorul de Cod**. De exemplu schimbați **Program** cu **Salut**.
4. Scrieți codul de mai jos :

```
using System;

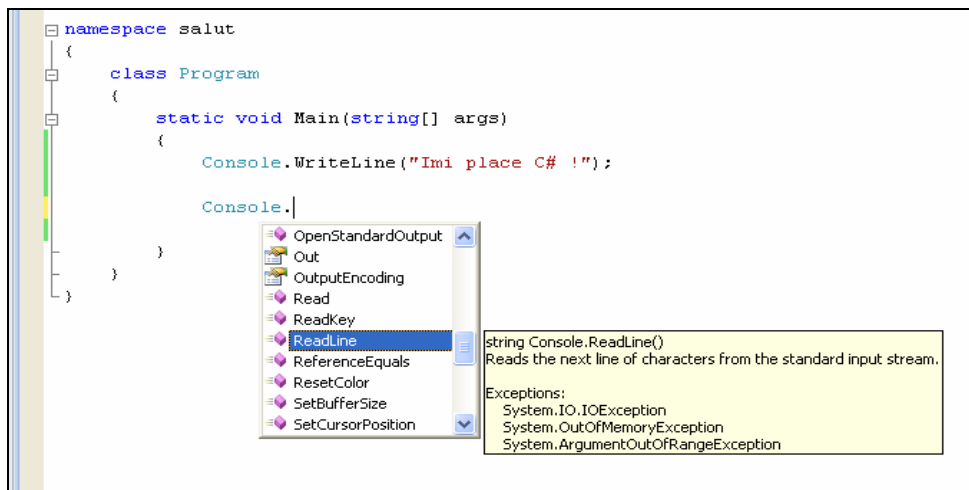
static void Main(string[] args)
{
    Console.WriteLine("Imi place C# !");
    Console.ReadLine();
}
```

5. Rularea programului.

Programul este gata de compilare și rulare. Se apasă **F5** ori click pe iconul ►

IntelliSense – o facilitate importantă a editorului de cod

Când se scrie un nume sau un cuvânt cheie în editor, se poate utiliza instrumentul numit **IntelliSense** care este parte a **panoului de cod**. De exemplu, când metoda **ReadLine** apare evidențiată în listă, Apăsați Enter sau Tab sau faceți dublu-click și **ReadLine** va fi adăugată în cod:



Avantajul utilizării **IntelliSense** este pe de o parte faptul că programatorul nu trebuie să memoreze toate tipurile și metodele vizibile într-un anumit context deoarece acestea apar în mod automat în listă. Pe de altă parte, poate fi sigur că ceea ce scrie este corect.

Rezumatul capitolului

- C# este un limbaj din familia C++, *orientat pe obiecte*, cu mare productivitate în programare.
- Infrastructura pe care se programează în C# este **.NET Framework**.
- **.NET Framework** este un mediu care permite dezvoltarea și rularea aplicațiilor și a serviciilor Web, independente de platformă.
- Una dintre componentele de bază a *.NET Framework* este **Biblioteca de Clase .NET**.
- Mediul *Visual C# Express Edition 2008* este un instrument de programare *free*, cu care puteți dezvolta aplicații pentru sistemele de operare *Windows*.

Întrebări și exerciții

1. Precizați câteva dintre calitățile limbajului C#.
2. Ce reprezintă *Common Language Runtime* ?
3. Descrieți modul în care se produce compilarea unui program C#. Care este rolul compilatorului **JIT** ?
4. Ce este un **assembly** ?
5. Descrieți câteva dintre caracteristicile Platformei .NET.

Capitolul 2

Limbajul C# . Introducere

Limbajul C# este deosebit de simplu, cu numai 80 de cuvinte cheie și 12 tipuri încorporate. Dar C# este în același timp un limbaj foarte productiv din punct de vedere al programării aplicațiilor și pe deplin adaptat conceptelor moderne de programare.

Fiind un limbaj orientat pe obiecte, o caracteristică esențială a sa este suportul pentru definirea și lucrul cu clasele. Clasele definesc tipuri noi de date. Variabilele de tip clasă se numesc obiecte. Uneori un obiect este abstract, cum ar fi o tabelă de date sau un *thread* (program sau proces lansat în execuție). Alteori obiectele sunt mai tangibile, cum sunt un buton sau o fereastră a aplicației. Obiectele sunt un element fundamental în programare datorită rolului lor în modelarea problemelor din practică care trebuie rezolvate. Programarea orientată pe obiecte se sprijină pe trei piloni fundamentali: **încapsularea datelor**, **moștenire** și **polimorfism**.

Clasele C# conțin **câmpuri** și **proprietăți** care rețin informații despre obiecte și **metode** (funcții aparținând clasei). Metodele încapsulează cod care descrie ceea ce poate face obiectul, acțiunile și capabilitățile sale. Obiectele la rândul lor, interacționează, comunică între ele.

C# este un limbaj care oferă suport explicit pentru tratarea **evenimentelor**. Vom reveni pe larg asupra tuturor acestor concepte în capitolele următoare.

C# admite **programarea generică**. Programarea generică este un stil de programare diferit de programarea orientată pe obiecte. Scopul programării generice este scrierea de cod care să fie independent de tipul datelor. De exemplu, imaginați-vă o funcție care primește ca parametru un șir numeric pe care îl sortează. Algoritmul de sortare nu depinde de tipul elementelor șirului. Acestea pot fi de pildă întregi sau reale. În această situație în C# puteți defini o funcție generică, care va folosi după caz, șiruri întregi sau reale.

C#, așa cum s-a văzut în capitolul precedent utilizează un **garbage collector** (colector de deșeuri) pentru eliberarea memoriei ocupate de obiectele de care aplicația nu mai are nevoie. Obiectele C# se alocă întotdeauna dinamic, adică în timpul execuției programului și se distrug în mod automat.

Structura unui program simplu C#

O aplicație de tip Consolă

Creați în VCSE o aplicație de tip consolă și rulați programul următor:

```
/*  
    Programul declarară variabile, initializează  
    variabile și face atribuiri  
*/
```

```
namespace SpatiulMeu
{
    class Test
    {
        static void Main(string[] args)
        {
            int a;    // Declararea unei variabile locale
            a = 20;    // Atribuire
            double b = 10;    // Declarare și inițializare
            string s = "obiect de tip string";

            System.Console.WriteLine("a = " + a);
            System.Console.WriteLine("b = " + b);
            System.Console.WriteLine("s este " + s);
        }
    }
}
```

Cu **Ctrl + F5** compilați și lansați programul, care va afișa pe ecran:

```
a = 20
b = 10
s este obiect de tip string
```

Vom analiza în continuare structura acestui program.

Metoda **Main** - punctul de intrare în aplicație

Fiecare program necesită un punct de intrare (*entry point*) și în cazul C#, aceasta este metoda statică **Main**. La lansarea în execuție a programului, mediul de execuție .NET (CLR) caută și apelează metoda **Main**. Un program poate să conțină mai multe clase, dar numai una dintre acestea definește metoda **Main**.

Tipuri abstracte

Programul declară un tip (clasa cu numele **Test**) și un membru al clasei (metoda **Main**). Este o diferență față de C++, unde de regulă tipurile abstracte se declară în fișiere *header*, iar definițiile acestora în fișiere *.cpp* separate.

Variabile locale

Metodele unei clase pot să declare variabile locale, așa cum este variabila întregă **a**, variabila reală **b** și variabila de tip **string**, s. Tipul **string** este un tip definit în *Biblioteca de Clase .NET*, fiind destinat lucrului cu șiruri de caractere. Variabilele locale pot fi inițializate cu valori la momentul declarării.

Comentarii

C# admite comentarii multilinie (`/* ... */`) și comentarii de o singură linie (`//`), întocmai ca C++.

Clase

C# lucrează cu clase. Fiind un limbaj pur orientat obiect, nu puteți scrie un program C# fără clase. Totul se petrece în interiorul claselor. Tot ceea ce aparține unei clase, va fi definit în interiorul ei. Nu există variabile globale și nici funcții globale. Un program trebuie să aibă cel puțin o clasă, aceea care conține metoda **Main**. O clasă se definește cu ajutorul cuvântului cheie **class**.

Exemplu:

```
class Persoana
{
    // membrii clasei
}
```

Observație:

Programatorii C++ trebuie să fie avizați de faptul că în C# după ultima paranteză închisă a clasei, nu se pune ; .

Spații de nume definite de programator

Există spații de nume predefinite. Ne referim la acelea care fac parte din *Biblioteca de Clase .NET*. În același timp C# permite programatorilor să-și creeze propriile spații de nume pentru organizarea logică a aplicațiilor mari. Ați remarcat desigur **namespace SpatiulMeu** definit în primul program. Este bine de reținut că prezența lui este opțională și că atât clasa care conține metoda **Main**, cât și celelalte clase ale programului pot să fie incluse sau nu în interiorul unui *namespace*.

Calificarea completă a numelor cu ajutorul operatorului .

Afișările pe ecran s-au făcut prin apelul metodei statice **WriteLine** din clasa **Console**. Clasa **Console** este definită în spațiul de nume **System**. Pentru utilizarea claselor C# din *Biblioteca de Clase .NET*, dar și a metodelor statice ale acestor clase, există alternativele:

1. Utilizarea directivei **using**;

Directiva **using** face vizibile în program toate numele de clase dintr-un spațiu de nume. Reamintim că spațiile de nume sunt containere logice care servesc la separarea numelor de tipuri într-o bibliotecă de clase sau într-un program.

Exemple:

- **using System;** - toate numele definite în spațiul de nume **System** devin accesibile.
- **using System.Windows.Forms** - clasele din spațiul de nume **Forms** devin vizibile în program. Observați că spațiile de nume se pot include unele în altele. De pildă **System** conține mai multe clase, dar și spații de nume printre care spațiul de nume **Windows**. Spațiul de nume **Windows** la rândului lui include mai multe spații de nume, între care și **Forms**.

2. Calificarea completă a numelui

Calificarea completă a numelui (*fully qualified name*) presupune precizarea căii complete până la numele metodei. În programul anterior, apelul `System.Console.WriteLine("a = " + a)`; ilustrează modul corect de apelare. Operatorul `.` servește la separarea spațiilor de nume. Se pornește de la spațiul de nume exterior (`System`) și se continuă cu specificarea tuturor spațiilor de nume până se ajunge la metoda dorită. Să scriem două programe simple:

```
// Primul program folosește directiva using
using System;

class Test
{
    static void Main()
    {
        Console.WriteLine("C#");
    }
}

// Al doilea program precizează calea completă
// a numelui metodei
class Test
{
    static void Main()
    {
        System.Console.WriteLine("C#");
    }
}
```

Cele două programe compilează și rulează identic.

Parametri în linia de comandă

Metoda `Main` poate avea un parametru. Acesta este un șir de stringuri, care reține argumentele în linia de comandă care se transmit programului. Acestea sunt informații care pot fi pasate programului de către utilizator. Creați un proiect de tip consolă cu numele `arg`. În editorul de cod introduceți programul:

```
using System;
class Program
{
    static void Main(string[] args)
    {
        for (int i = 0; i < args.Length; i++)
            Console.Write(args[i] + " ");
    }
}
```

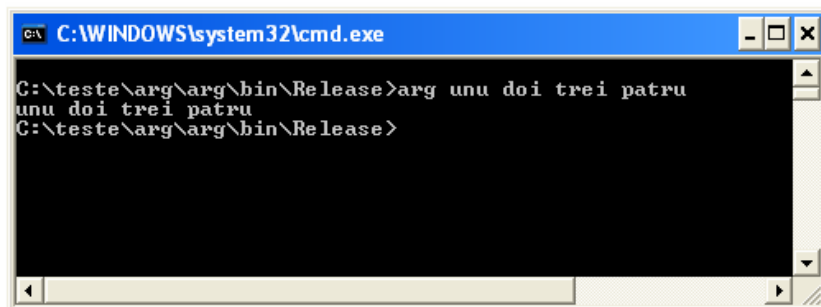
Compilați proiectul (faceți **Build**) cu **F6**. În folderul `\bin\Release` al aplicației se va produce fișierul **assembly** cu numele **arg.exe**.

OBSERVAȚII

- Vom reveni în capitolul următor asupra buclei `for`. În acest moment rețineți sintaxa preluată din limbajul C.
- **Length** este o proprietate publică a clasei abstracte **Array**, care returnează numărul de elemente ale șirului. Toate tablourile unidimensionale din C# moștenesc clasa **Array**, deci pot utiliza proprietățile acesteia.

Transmiterea argumentelor de la promptul sistemului de operare

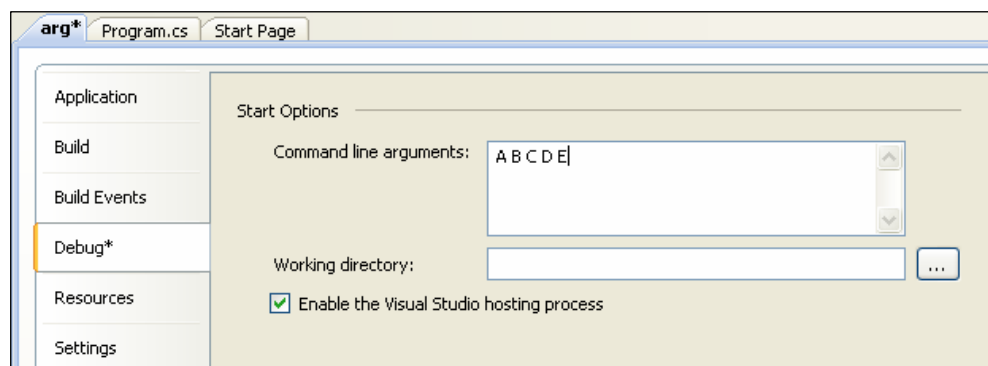
Interpretorul în linie de comandă al sistemului de operare este **cmd.exe**. Acționați **Start->Run** și introduceți comanda **cmd**. La promptul sistemului schimbați calea în folderul `\bin\Release` al proiectului. Tastați numele programului, urmat de câteva argumente separate prin spații: **arg unu doi trei patru**. Programul afișează pe ecran: **unu doi trei patru**



Argumentele sunt de regulă informații de care programul are nevoie, de exemplu numele fișierelor de intrare și de ieșire.

Transmiterea argumentelor în linie de comandă din Visual C# Express

În fereastra *Solution Explorer*, faceți dublu click pe iconul *Properties*. În fereastra din dreapta, click pe tab-ul *Debug*. Introduceți argumentele în linie de comandă în *TextBox*-ul cu eticheta *Command Line Arguments*:



Faceți **Build** și rulați acționând **Ctrl + F5**. Programul afișează: **A B C D E**

Variante ale metodei Main

La crearea unei aplicații de tip consolă, **VCSE** generează următoarea definiție pentru metoda **Main**:

```
static void Main(string[] args) { /*...*/ }
```

Este declarată **static**, are tipul de retur **void** și un șir de stringuri, ca și parametru de intrare. C# admite și alte *signaturi* ale funcției **Main**, ca mai jos:

```
static int Main(string[] args) { /*...*/ } // Tip de retur int,
                                           // un șir de stringuri ca argument
static void Main() { /*...*/ }           // Fără tip de retur, fără argumente

static int Main() { /*...*/ }           // Tip de retur int, fără argumente
```

Veți alege un prototip sau altul, în funcție de necesități. Dacă doriți ca aplicația să î ntoarcă sistemului de operare o valoare întregă, de pildă un cod de eroare când programul se termină, atunci veți prefera o semnătură care returnează **int**. Dacă aplicația folosește argumente transmise de către utilizator din linie de comandă, atunci veți opta pentru o versiune cu un parametru de tip șir de stringuri.

IMPORTANT

- C# este case-sensitive. Aceasta înseamnă că **Main** este corect, iar **main** greșit, că **WriteLine** nu este tot una cu **writeline**.
- Cuvintele cheie din C#, (**return**, **int**, **void**, **if**, etc) se scriu cu litere mici.
- Pentru tipurile definite de programator, pentru spațiile de nume și pentru metode, există convenția pe care este indicat să o respectați și anume: prima literă este mare și fiecare cuvânt conținut începe tot cu literă mare.

Exemple: **Console.ReadLine()** – metodă
System.TimeZone – spațiu de nume
System.Data.SqlClient – clasă

Rolul cuvântului cheie **static** în prototipul metodei **Main**

Main este membră a unei clase, de pildă clasa **TestAfisare**. Dar o clasă nu este altceva decât o specificație a unui tip de date. În lipsa unui obiect de tip **TestAfisare**, nu există nici un membru al clasei. Totuși, **Main** trebuie să fie apelată pentru că este punct de intrare în program. Pentru ruperea cercului vicios, se definește **Main** ca metodă statică. Metodele statice au tocmai această proprietate interesantă, că ele există și pot fi apelate în lipsa unui obiect de tipul clasei.

Accesul public sau privat la metoda Main

La fel ca în cazul altor limbaje orientate obiect, în C# se specifică modul de acces la membrii clasei cu ajutorul *modificatorilor de acces* **public** sau **private**. Vom reveni în capitolul următor asupra acestor cuvinte cheie. Pe scurt, cuvântul **private** aplicat unui membru de clasă face ca acel membru să nu poată fi accesat din exteriorul clasei. *Visual C# 2008 Express* definește mod implicit metoda **Main** ca fiind **private**, pentru ca alte aplicații să nu o poată apela. Aveți libertatea să specificați modificatorul public sau private. Construcțiile următoare sunt legale:

```
public static void Main()  
{  
}
```

```
private static void Main()  
{  
}
```

Rezumatul capitolului

- Metoda **Main** este punctul de intrare într-un program C#.
- Un program trebuie să aibă cel puțin o clasă, aceea care conține metoda **Main**.
- **Main** are câteva versiuni. Prin tabloul de stringuri pe care **Main** îl are ca parametru, se pot pasa programului argumente în linia de comandă.
- C# nu admite variabile globale. Toate variabilele declarate sunt locale.
- Toate clasele .NET sunt definite în interiorul unor spații de nume.

Întrebări și exerciții

1. Ce se înțelege prin *garbage collection* ?
2. Care este rolul cuvântului cheie static în *signatura* metodei **Main** ?
3. Ce se înțelege prin *fully qualified name* ?
4. Scrieți un program C# care se rulează într-o aplicație de tip consolă. Programul afișează toți parametrii transmiși în linie de comandă. Nu se vor utiliza directive de compilare.
5. Ce rol are operatorul `.` într-un program C# ?

Capitolul 3

Fundamentele limbajului C#

Acest capitol descrie elemente de bază ale limbajului C#: tipurile de date, sintaxa, expresiile, operatorii, directivele de procesare.

Toate programele din acest capitol se pot testa cu ajutorul unei aplicații de tip consolă în *Visual C# 2008 Express Edition*.

Tipuri

C# este un limbaj **puternic tipizat** (*strongly-typed*). Un asemenea limbaj impune programatorului să declare un tip pentru fiecare obiect creat, iar compilatorul verifică compatibilitatea dintre tipurile obiectelor și valorile care le sunt atribuite.

Tipurile sistem din C#, se împart în două categorii:

- **Tipuri predefinite** (*built-in*), cum sunt `int`, `char`, etc.
- **Tipuri definite de programator** (*user-defined*). Acestea se definesc cu ajutorul claselor, structurilor sau interfețelor.

Un alt sistem de clasificare a tipurilor în C# împarte tipurile astfel:

- **Tipuri referință**
- **Tipuri valoare**

Tipurile valoare sunt variabile locale care se memorează în segmentul de stivă al programului, iar tipurile referință se alocă dinamic în **Heap**.

C# admite și **tipul pointer**, însă pointerii sunt folosiți foarte rar și doar în situațiile când se lucrează cu *unmanaged code*, adică cod nespecific platformei .Net.

Tipuri predefinite

C# oferă toate tipurile de bază necesare unui limbaj de programare modern. În C# toate tipurile sunt obiecte, chiar și cele mai elementare tipuri. Cuvintele cheie pentru tipurile predefinite (`int`, `char`, `double`, etc.) sunt mapate direct peste tipuri **struct** din spațiul de nume **System**. De exemplu, cuvântului cheie `int` îi corespunde clasa **System.Int32** din *Biblioteca de Clase .NET*. Aveți libertatea să declarați o variabilă întreagă astfel:

```
System.Int32 x; , în loc de int x;
```

Maparea tipurilor primitive peste tipuri din .NET asigură faptul că tipurile create în C# pot interacționa cu tipuri create în alte limbaje care lucrează pe platforma .Net. Tipurile referință predefinite sunt **object** și **string**.

Tipul `object`

Tipul `object` este rădăcina întregii ierarhii de tipuri și clasa de bază a tuturor claselor din **.NET Framework**. Aceasta înseamnă pe de o parte că toate clasele din **.NET** moștenesc clasa `object` și pe de altă parte că oricare clasă definită de programator derivă în mod implicit din `object`.

Tipul `string`

Tipul `string` reține un șir de caractere din setul **Unicode**. **Unicode** este un standard care permite reprezentarea tuturor caracterelor scrise în orice limbă scrisă, nu doar în limbile de circulație internațională (în jur de 100.000 caractere).

Tabelul 3.1 *Tipuri C# predefinite*

Tip	Dimensiune (bytes)	Tip .Net asociat	Descriere
<code>object</code>	variabilă	<code>Object</code>	Tipul de bază al tuturor tipurilor
<code>string</code>	variabilă	<code>String</code>	Tip șir de caractere.
<code>byte</code>	1	<code>Byte</code>	Tip întreg fără semn. [0, 255]
<code>char</code>	2	<code>Char</code>	Tip caracter. Reține caractere Unicode
<code>bool</code>	1	<code>Boolean</code>	Valori true sau false
<code>sbyte</code>	1	<code>SByte</code>	Tip întreg cu semn. [-128, 127]
<code>short</code>	2	<code>Int16</code>	Tip întreg cu semn. [-32768, 32767]
<code>ushort</code>	2	<code>UInt16</code>	Tip întreg fără semn. [0, 65535]
<code>int</code>	4	<code>Int32</code>	Tip întreg cu semn. [-2.147.483.648, 2.147.483.647]
<code>uint</code>	4	<code>UInt32</code>	Tip întreg fără semn. [0, 4.294.967.295]
<code>float</code>	4	<code>Single</code>	Tip real cu virgulă mobilă, simplă precizie. Valori cuprinse între $\pm 1,5 \times 10^{-45}$ și $\pm 3,4 \times 10^{38}$ cu 7 cifre semnificative
<code>double</code>	8	<code>Double</code>	Tip real cu virgulă mobilă, dublă precizie. Valori cuprinse între $\pm 5,0 \times 10^{-324}$ și $\pm 1,8 \times 10^{308}$ cu 15 cifre semnificative
<code>decimal</code>	12	<code>Decimal</code>	Tip zecimal cu virgulă fixă. Precizie de cel puțin 28 cifre semnificative.
<code>long</code>	8	<code>Int64</code>	Tip întreg cu semn. $[-2^{63}, 2^{63} - 1]$
<code>ulong</code>	8	<code>UInt64</code>	Tip întreg fără semn. $[0, 2^{64}]$

Tipul `decimal` se folosește în calcule financiare, atunci când erorile de rotunjire datorate virgulei mobile sunt inacceptabile.

Declarări de variabile

Toate declarările de mai jos sunt legale:

```
object o1, o2 = null;
string s = "Salut";
sbyte b = 6;
short h = 13;
```

```
ushort u = 7;
int a = 10;
long p = 3, q = 8L;
ulong x = 2UL, y = 5L, z = 10U, w = 8;
float f1 = -4.5, f2 = 3.8F;
double d1 = 1.3, d2 = 0.2D;
char c = 'T';
Decimal d = 12.4M;    // M desemnează o constantă decimal
```

Tipuri definite de programator

În C# programatorii pot defini propriile tipuri cu ajutorul cuvintelor cheie : **class**, **struct**, **interface**, **delegate** și **enum**. Tipul tablou este de asemenea un tip *user-defined*.

Tipul class

În C# clasele se creează cu ajutorul cuvântului cheie **class**:

```
class nume_clasa
{
    modificador_acces Tip nume_membru;
    // alți membri
}
```

Exemplu:

```
class Intreg
{
    public int x;    // x - câmp (membru) al clasei
}
```

De reținut :

- Variabilele de tip clasă se numesc **obiecte** sau **instanțe** ale clasei.
- Crearea unui obiect de tip clasă, se numește **instanțiere**.

Instanțierea se face cu ajutorul operatorului **new**, urmat de apelul constructorului clasei. Vom reveni asupra acestor noțiuni în capitolul următor. Vom crea două obiecte de tip **Intreg**:

```
Intreg r1, r2;    // Declară două referințe
r1 = new Intreg();    // Creează un obiect referit de r1
r2 = new Intreg();    // Creează un obiect referit de r2
```

Mai sus, **r1** și **r2** se numesc **referințe**.

IMPORTANT

Accesarea membrilor clasei se face cu sintaxa: `referința.membru`

Modificatorul `public` care prefixează declararea câmpului `x`, indică faptul că `x` poate fi accesat din afara clasei sale astfel: `r.x`.

Ilustrăm aceste informații în programul `intreg.cs`:

```
using System;

class Intreg
{
    public int x;
}

class TestIntreg
{
    static void Main()
    {
        Intreg r = new Intreg();
        r.x = 10;    // Atribuire valoarea 10 câmpului x
        Console.Write(r.x);    // Afisează 10
    }
}
```

Observați că un program C# poate să conțină una sau mai multe clase, dintre care una singură conține metoda `Main`.

Structuri

Structurile se construiesc prin utilizarea cuvântului cheie `struct`. Au fost introduse în C# pentru a permite programatorului să definească tipuri valoare. Sunt similare claselor, dar le lipsesc unele caracteristici, cum ar fi moștenirea. Obiectele de tip `struct` se depozitează în stivă. Se creează și se accesează mai rapid decât cele alocate dinamic. Sunt de preferat atunci când aveți nevoie de obiecte mici ca dimensiuni, sau care se construiesc într-un ciclu.

Sintaxa este similară cu cea a claselor:

Exemplu

```
using System;

struct Persoana
{
    public string nume;
    public int varsta;
}
```

```
class TestStruct
{
    static void Main()
    {
        // Datele obiectului p se memorează în stivă
        Persoana p = new Persoana();
        p.nume = "Iulia";
        p.varsta = 8;
        Console.Write("Numele {0}\nVarsta {1} ani",
                      p.nume, p.varsta);
    }
}
```

La rulare, programul afișează:

```
Numele Iulia
Varsta 8 ani
```

Notă

- Metodele `Write` și `WriteLine` din clasa `Console`, admit formatarea afișării în maniera funcției `printf()` din `C`. Modificatorii `{0}` și `{1}` specifică locul și ordinea în care se vor afișa valorile argumentelor `p.nume`, respectiv `p.varsta`.
- `\n` este o secvență escape, reprezentând caracterul (*newline*).

Tipuri valoare și tipuri referință

În C# tipurile se clasifică în **tipuri valoare** și **tipuri referință**.

Tipurile valoare sunt toate tipurile predefinite (`char`, `int`, `float`, etc.), cu excepția tipurilor `string` și `object`. C# permite în plus definirea de tipuri valoare cu ajutorul cuvântului cheie `struct`. Tipurile valoare se caracterizează prin faptul că se memorează în segmentul de stivă al programului.

Tipurile referință, se definesc de către programator. În această categorie se includ tipurile *clasă*, tipurile *interfață*, tipurile *delegat* și tipurile *array* (tablourile). Se numesc tipuri referință, deoarece variabilele folosite pentru manipularea lor sunt referințe la obiecte alocate dinamic în **Heap**. O variabilă de tip referință, păstrează de fapt adresa obiectului din **Heap** pe care îl referă, astfel că există o asemănare între referințe și pointerii C++.

Ca să putem lămuri pe deplin diferențele între tipurile referință și tipurile valoare, vom vedea cum se crează obiectele.

IMPORTANT

Tipurile valoare diferă de **tipurile referință** prin faptul că variabilele de tip valoare conțin datele obiectului, în timp ce variabilele de tip referință conțin referințe la obiecte.

Pentru a lămuri aceasta, vom analiza programul:

```

class C      // Tip referință
{
}

struct S     // Tip valoare
{
}

class StackHeap
{
    static void Main(string[] args)
    {
        int x = 7;      // Variabilă locală.
        S s = new S(); // s - variabilă locală

        C r;             // Referința r se memorează pe Stivă.
        r = new C();     // Alocă în Heap un obiect referit de r
    }
}

```

Unde se vor memora variabilele **x**, **s**, **r** și obiectele create cu **new** ? Priviți figura:

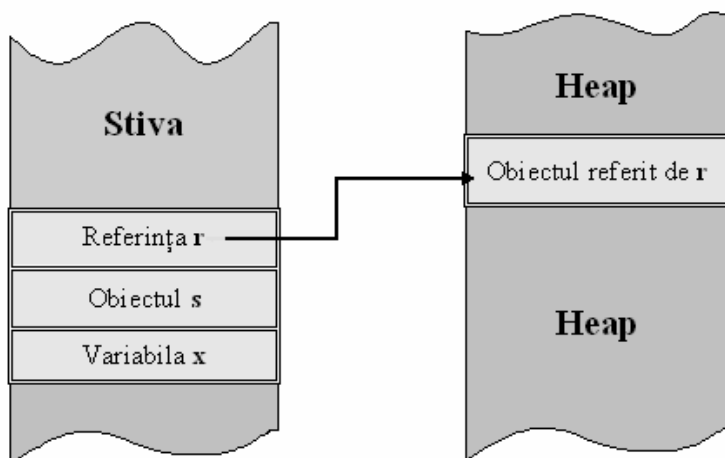


Figura 3.2. Memorarea datelor tipurilor *valoare* și *referință*

Observăm câteva aspecte importante:

- Obiectul cu numele **s** de tip **struct**, creat cu **new** se memorează în stivă.
- Referința **r** la obiectul de tip **class** se memorează în stivă, iar obiectul propriu-zis creat cu **new** se alocă în **Heap**.
- Variabila **x** se memorează în stivă.

DE REȚINUT:

- Datele variabilelor de **tip valoare**, se memorează pe stivă (Stack). O excepție de la această regulă o constituie situația în care tipuri valoare sunt încorporate unui tip referință (de pildă, sunt date membre ale unei clase). În acest caz datele se memorează în *Heap*.
- În cazul **tipurilor referință**, referințele sunt în stivă, iar datele se alocă dinamic în *Heap*.

Conversii între tipuri

Obiectele de un anumit tip se pot converti în obiecte de un alt tip în mod implicit sau explicit. Tipurile predefinite suportă de asemenea conversii.

Conversiile implicite

Acest tip de conversie se produce în mod automat. O face compilatorul și aveți garanția că se produce fără pierdere de informație.

Exemplu:

```
short a = 4;
int n = a;    // Conversie implicită spre tipul int.
              // Nu se pierde informație
```

Conversiile explicite

Dacă încercați să atribuiți unei variabile de tip **short** valoarea unei variabile de tip **int**, compilatorul nu va efectua conversia implicită, deoarece aceasta poate cauza pierdere de informație. Aveți totuși posibilitatea de a efectua această conversie cu ajutorul operatorului de conversie explicită (*cast operator*) :

```
int n = 10000;
short s1 = n;           // Incorect. Nu va compila
short s2 = (short)n;    // Corect. Conversie explicită
                       // cu pierdere de informație
```

Secvența următoare ilustrează modul în care conversiile au loc în C#:

```
// Declarații
object o = null;
int a = 2000;
short b = 3;
char c = 'T';
long d = 10;
double f = 1.2;
// Testarea convertibilitatii
a = d;           // Eroare.
a = (int)d;      // Corect. Conversie explicită
o = b;           // Corect. Toate tipurile se convertesc
                // în mod implicit la tipul object
b = o;           // Eroare.
```

```
f = b;           // Corect. Conversie implicita
b = f;           // Eroare.
ulong e = -3;    // Eroare.
a = c;           // Corect. Conversie implicita
c = a;           // Eroare.
c = (char)a;     // Corect. Conversie explicită
```

Variabile și constante

Variabile

O variabilă este o locație de memorie care are un tip asociat. În programul de mai jos, **a** și **b** sunt variabile:

```
using System;

static void Main(string[] args)
{
    int a = 2;
    int b;
    b = 3;
    System.Console.WriteLine(a + " " + b);
}
```

Programul afișează: 2 3

Comentați instrucțiunea **b = 3;** Veți constata faptul că programul are o eroare. Compilatorul anunță că ați încercat să utilizați o variabilă căreia nu i-a fost atribuită o valoare.

IMPORTANT

C# nu permite utilizarea variabilelor cărora programul nu le-a asociat anterior o valoare.

Constante

O constantă este o variabilă inițializată, a cărei valoare nu mai poate fi modificată în program. Constantele se introduc cu ajutorul calificativului **const**:

```
using System;
static void Main(string[] args)
{
    const long x = 100; // Corect
    const string s;     // Eroare. Constanta neinitializata
    s = "Salut";
    System.Console.WriteLine(x + " " + s);
}
```

Programul nu va compila, deoarece stringul **s** a fost declarat constant dar nu i s-a atribuit o valoare inițială.

Enumerări

Enumerările sunt tipuri de date de **tip valoare**, create de programator în scopul de a grupa sub un nume un set de constante simbolice.

Exemplul 1:

```
using System;

enum Saptamana
{
    Luni, Marti, Miercuri, Joi,
    Vineri, Sambata, Duminica
}

class TestEnum
{
    static void Main()
    {
        Saptamana ziua = Saptamana.Luni;
        if ( ziua == Saptamana.Luni)
            Console.WriteLine("Luni e {0} Marti e {1}",
                               (int)Saptamana.Luni, (int)Saptamana.Marti);
    }
}
```

Programul afișează: Luni e 0 Marti e 1

De reținut:

- Membrii unei enumerări se numesc enumeratori. Lista membrilor enumerării, se separă prin virgulă.
- Variabila **ziua** de tip **enum**, reține doar unul dintre membrii enumerării (**Luni**).
- Valorile membrilor enumerării se obțin prin conversii explicite la **int** și sunt egale cu 0, 1, 2, ... , dacă nu se specifică alte valori pentru enumeratori.

Exemplul 2

Membrilor unei enumerări li se pot asocia valori de tip întreg:

```
using System;

enum Note
{
    Mate = 10,
    Info = 9,
    Romana = 8
}
```

```
class TestEnum
{
    static void Main()
    {
        Console.WriteLine("mate: {0}\ninfo: {1}\nromana: {2}",
            (int)Note.Mate, (int)Note.Info, (int)Note.Romana);
    }
}
```

Programul afișează:

```
mate: 10
info: 9
romana: 8
```

Enumerările sunt o alternativă puternică în raport cu constantele. Utilizarea enumerărilor face codul mai clar, mai bine documentat.

Expresii

Expresiile în C# sunt similare cu expresiile din C++ și Java. Expresiile se construiesc folosind operanzi și operatori și în general întorc o valoare în urma evaluării lor. Operanzii sunt variabile de memorie, nume de metode, de tablouri, de obiecte, etc.

Tabelul de mai jos prezintă operatorii C# în ordinea descreșterii **precedenței** (priorității) lor.

Tabelul 3.3. Operatorii C#

Grupul de operatori	Operator	Expresii	Descriere
Primari	.	x.m	Acces la membrii clasei
	()	f (x)	Apel de metodă
	[]	a [x]	Acces la elem. tablourilor
	++	x++	Postincrementare
	--	x--	Postdecrementare
	new	new T (x) new T [x]	Creare de obiecte Creare de tablouri
	typeof	typeof (T)	Informații despre tipul T
	checked unchecked	checked (x) unchecked (x)	Evalează expresia x în medii <i>checked</i> și <i>unchecked</i>
Unari	+ -	+x -x	Identitate și negație
	!	!x	Negație logică
	~	~x	Negație pe biți
	++ --	++x --x	Preincrementare Predecrementare
	()	(T) x	Conversie explicită
Multiplicativi	* / %	x*y x/y x%y	Înmulțire, împărțire, modulo

Aditivi	+ -	x+y x-y	Adunare, scădere
Shiftare	<<	x << y	Deplasare pe biți la stânga și la dreapta
	>>	x >> y	
Relaționali și de testare a tipului	< > <= >=	x<y x>y x<=y x>=y	Operatori aritmetici
	is	x is T	true , dacă x este convertibil la T . Altfel, false
	as	x as T	Returnează x convertit la T sau null dacă conversia e imposibilă
Egalitate	== !=	x == y x!=y	Egal și diferit
AND logic	&	x & y	AND logic pe biți
XOR logic	^	x ^ y	XOR logic pe biți
OR logic	 	x y	OR logic pe biți
AND condițional	&&	x && y	Evaluează y numai dacă x este true
OR condițional	 	x y	Evaluează y numai dacă x este false
Condițional	?:	x ? y : z	Evaluează y dacă x este true . Altfel, evaluează z
Atribuire	= *= /=	x = y x *= y	Atribuire simplă și atribuirii compuse
	%= += -=	etc.	
	<<= >>= &= ^=		
	 =		

Observație:

Din tabel se constată că apelurile metodelor sunt expresii, accesarea unui element într-un tablou este o expresie, crearea de obiecte sau de tablouri sunt de asemenea expresii.

Când o expresie conține mai mulți operatori, precedența operatorilor determină ordinea de evaluare a operațiilor.

De exemplu, $x - y / z$ se evaluează ca $x - (y / z)$ deoarece precedența operatorului $/$ este mai mare decât cea a operatorului $-$.

Când un operand se găsește între doi operatori cu aceeași precedență, atunci *asociativitatea operatorilor* controlează ordinea operațiilor.

În afara operatorilor de atribuire, toți operatorii binari se asociază de la stânga spre dreapta. De exemplu: $a + b + c$ se asociază $(a + b) + c$.

Operatorii de atribuire și operatorul condițional ($?:$) se asociază de la dreapta spre stânga. De exemplu: $a = b = c$ se evaluează în ordinea $a = (b = c)$.

Precedența și asociativitatea se pot controla cu ajutorul parantezelor. De exemplu: $a+b/c$ împarte b la c și apoi adună a la rezultat, dar $(a+b)/c$ adună a cu b și împarte rezultatul la c .

[11]

Tablouri

Un tablou este o structură de date conținând variabile de același tip. În C# tablourile sunt **tipuri referință**. Tablourile pot fi **unidimensionale** sau **multidimensionale**. Cele multidimensionale, la rândul lor pot fi dreptunghiulare sau **neregulate** (*jagged*).

Tablouri unidimensionale

Declararea tablourilor unidimensionale se face precizând un tip de dată urmat de operatorul de indexare și numele tabloului. Crearea tabloului se face cu **new** urmat de tip și dimensiunea tabloului cuprinsă între paranteze pătrate.

```
Declarare: Tip[] a;  
Create:    a = new Tip[n];
```

Tablourile se indexează de la 0. Un tablou cu **n** elemente este indexat de la 0 la **n-1**.

Exemplul 1:

Programul de mai jos definește un tablou cu **5** valori de tip **int**.

```
using System;  
  
class TabUnidim1  
{  
    static void Main()  
    {  
        // Tablou cu 5 elemente de tip int  
        int[] a;           // Declarare  
        a = new int[5];    // Create  
        for (int i = 0; i < a.Length; i++)  
            a[i] = i + 1;  
  
        for (int i = 0; i < a.Length; i++)  
            Console.WriteLine("a[{0}] = {1}\n", i, a[i]);  
    }  
}
```

Programul afișează:

```
a[0] = 1  
a[1] = 2  
a[2] = 3  
a[3] = 4  
a[4] = 5
```

IMPORTANT !

*Tablourile sunt în realitate **obiecte** care moștenesc clasa **Array**. De aceea, metodele și proprietățile acestei clase pot fi folosite pentru tablouri. Proprietatea **Length** a clasei **Array** returnează numărul de elemente ale tabloului.*

Exemplul 2:

Mai jos, **a**, **b**, **c**, sunt tablouri de **double**, **char**, **string**, respectiv obiecte de tip **A**.

```
class TabUnidim2
{
    class A
    {
    }

    static void Main()
    {
        double[] a = new double[100];
        char[] b = new char[50];
        string[] s = new string[1000];

        // Tablou de 7 obiecte de tip A
        A[] ob = new A[7];
        for (int i = 0; i < ob.Length; i++)
            if ( ob[i] == null ) //
                Console.Write("null ");

        Decimal[] d = new Decimal[10];
        for (int i = 0; i < d.Length; i++)
            Console.Write(d[i] + " ");
    }
}
```

Programul afișează:

```
null null null null null null null 0 0 0 0 0 0 0 0 0 0
```

Observații:

1. C# vă permite să definiți clase imbricate (*nested*), așa cum este cazul clasei **A**, definită în interiorul clasei **TabUndim2**.
2. Valorile implicite ale tablourilor numerice sunt setate la 0, iar elementele referință sunt setate la **null**.

Inițializare

Tablourile unidimensionale se pot inițializa cu valori de același tip cu tipul tabloului:

Exemple:

```
// Tablou cu 4 elemente.  
int[] a = new int[] { 7, 4, 3, 0 };  
  
// Sintaxă alternativă. Tablou cu 3 elemente  
int[] b = {10, 2, 17};  
  
float[] c = { 1.2F, 7.9F, -8F };  
  
string[] s1 = new string[] { "UNU", "DOI", "TREI" };  
  
// Sintaxă alternativă. Tablou cu 3 elemente  
string[] s2 = { "UNU", "DOI", "TREI" };
```

Tablouri multidimensionale

Un tablou poate avea una, două sau mai multe dimensiuni.

Exemple:

```
// Tablou bidimensional cu 3 linii și 5 coloane  
double[,] a = new double[3, 5];  
  
// Tablou tridimensional cu dimensiunile 3, 7 și 5  
double[, ,] a = new double[3, 7, 5];
```

Accesarea elementelor

Accesarea elementelor unui tablou multidimensional se face cu sintaxa cunoscută din limbajul Pascal. De exemplu, elementul aflat pe linia 2 și coloana 0 într-o matrice se accesează astfel: `a[2, 0]`.

Inițializare

Tablourile multidimensionale pot fi inițializate ca în exemplele următoare:

```
// Tablou cu 2 linii și 3 coloane  
int[,] x = new int[, ]{ { 1, 2, 3 }, { 2, 3, 0 } };  
  
// Identic, dar cu sintaxa alternativa  
int[,] y = { { 1, 2, 3 }, { 2, 3, 0 } };
```

DE REȚINUT

Dacă ați creat mai întâi referința, iar tabloul va fi creat ulterior, atunci sintaxa alternativă de inițializare nu se mai poate folosi. În acest caz este obligatorie utilizarea operatorului `new`.

Exemplu:

```
using System;

class TabBidim
{
    static void Main()
    {
        int[,] y;
        // y = { { 1, 2, 3 }, { 2, 3, 0 } }; // Illegal

        int[,] x;
        x = new int[,] { { 1, 2, 3 }, { 2, 3, 0 } }; // OK

        for (int i = 0; i < 2; i++)
        {
            for (int j = 0; j < 3; j++)
                Console.Write(x[i, j] + " ");
            Console.WriteLine();
        }
    }
}
```

leșire:

```
1 2 3
2 3 0
```

Tablouri neregulate

Un tablou neregulat (**jagged array**) este un tablou ale cărui elemente sunt tablouri. Elementele pot fi tablouri unidimensionale sau chiar multidimensionale.

Exemplu:

Un *jagged array* cu două elemente, fiecare dintre acestea fiind un șir de `int`, se declară astfel:

```
int[][] a = new int[2][];
```

Tabloul nu este încă complet definit. Este nevoie de inițializarea fiecărui element:

```
a[0] = new int[3]; // tablou unidimensional cu 3 elemente
a[1] = new int[5]; // tablou unidimensional cu 5 elemente
```

Inițializare

C# permite câteva forme de inițializare a tablourilor neregulate:

1. Fie tabloul neregulat:

```
int[][] a = new int[3][];
```

Elementele tabloului se pot inițializa cu sintaxa:

```
a[0] = new int[] { 2, 7, 3, 8 };  
a[1] = new int[] { 0, -1 };  
a[2] = new int[] { 1, 9, 4 };
```

Tabloul are întradevăr, formă neregulată.

2	7	3	8
0	-1		
1	9	4	

2. A doua variantă de inițializare a unui tablou neregulat este următoarea:

```
int[][] a = new int[][]  
{  
    new int[] { 1, 4 },  
    new int[] { 5, 9, 0, 7, 2 },  
    new int[] { 6, 8, 3 }  
};
```

3. A treia variantă de inițializare este:

```
int[][] a =  
{  
    new int[] { 1, 4 },  
    new int[] { 5, 9, 0, 7, 2 },  
    new int[] { 6, 8, 3 }  
};
```

4. Cu ajutorul cuvântului cheie **var**, se pot inițializa atât variabile simple cât și tablouri cu **tipuri implicite**:

```
var d = 3.5;          // var înlocuiește double  
var s = "Salut";      // var înlocuiește string  
  
// Tablou unidimensional. var înlocuiește string[]  
var p = new[] { "C#", null, "C++", "Java" };  
  
// Tablou neregulat. var înlocuiește int[][]  
var q = new[]  
{  
    new[] { 7, 4 },  
    new[] { 2, 0, -4, 8 }  
};
```

În fiecare caz compilatorul deduce tipul de dată din tipul constantelor de inițializare.

Accesarea elementelor

Pentru un tablou neregulat ale cărui elemente sunt tablouri unidimensionale, accesarea se face la fel ca la tablourile din C++: `a[i][j]`.

Exemplu:

```
using System;

class JaggedArray
{
    static void Main()
    {
        int[][] a =
        {
            new int[] { 1, 4 },
            new int[] { 5, 9, 0, 7, 2 },
            new int[] { 6, 8, 3 }
        };

        for (int i = 0; i < a.Length; i++)
        {
            for (int j = 0; j < a[i].Length; j++)
                Console.Write(a[i][j] + " ");
            Console.WriteLine();
        }
    }
}
```

Ieșire:

```
1 4
5 9 0 7 2
6 8 3
```

Tablouri neregulate cu elemente de tip tablou multidimensional

Elementele unui *jagged array* pot fi tablouri multidimensionale.

Exemplu:

Elementele tabloului neregulat `a` sunt tablouri dreptunghiulare de valori `int`:

```
int[,] a = new int[3][,]
{
    new int[,] { {2, 9}, {4, 7}, {5, 0} },
    new int[,] { {1, 5}, {3, 12} },
    new int[,] { {8, 6}, {2, 9}, {0, 6}, {3, -1} }
};
```

Accesarea elementelor tabloului se face astfel: `a[i][j, k]`, unde `i` este al `i`-lea tablou din `a`, iar `j` și `k` este poziția valorii căutate în matricea `i`. De exemplu, valoarea `-1` din `a`, se obține: `a[2][3, 1]`.

Instrucțiuni

În C# o declarație (*statement*) poate fi: o expresie, o instrucțiune simplă, o instrucțiune compusă, o instrucțiune de control al fluxului de execuție (`if`, `for`, `while`, etc.), un apel de funcție, un bloc de cod, dar și o declarație de variabilă locală.

Un program C# este o secvență de asemenea *statements* care se evaluează în ordine. În continuare, vom folosi cuvântul *instrucțiune* pentru *statement*. Fiecare instrucțiune se termină cu `;`. Instrucțiunile C# sunt preluate cu unele mici modificări din C și C++. În continuare vom prezenta instrucțiunile care controlează fluxul de execuție al programului.

Instrucțiunea de selecție `if ... else`

Instrucțiunea `if...else` este împrumutată din limbajul C. Controlează fluxul programului prin evaluarea unei expresii booleane. În funcție de valoarea de adevăr, selectează pentru executare o secvență de instrucțiuni sau o alta.

```
if (expresie_booleană)
    instrucțiunea1;
else
    instrucțiunea2;
```

sau

```
if (expresie_booleană)
{
    bloc_instrucțiunii1;
}
else
{
    bloc_instrucțiunii2;
}
```

Exemplu:

```
string s = Console.ReadLine();

int x = int.Parse(s);
if ( x >= 0 )
    Console.Write("pozitiv");
else
    Console.Write("negativ");
```

În cazul instrucțiunilor **if** imbricate, trebuie să ții seama de regula conform căreia un **else** se asociază cu cel mai apropiat **if** care nu are **else**:

```
if (expresie1)
    if (expresie2)
        instrucțiune1;
    else
        instrucțiune2; // se asociază cu if (expresie2)
```

De reținut:

1. Metoda **ReadLine()** din clasa **System.Console** citește linia următoare din **stream-ul standard** de intrare (tastatură).
2. Metoda **Parse(s)**, membră a structurii **System.Int32** convertește stringul **s** într-un număr într-un întreg pe 32 de biți. Amintiți-vă că tipul **int** este un *alias* pentru structura **Int32**.

Din păcate biblioteca **.NET** nu furnizează operatorul **>>**, așa cum o face C++, capabil să extragă *bytes* din *stream-uri* și să-i convertească direct spre tipul dorit (Exemplu C++: **int x; cin >> x**). Metodele **Read()** sau **ReadLine()** care se definesc ca membre ale diferitor clase din **.NET**, citesc întotdeauna din *stream-urile* de intrare un caracter, un bloc de caractere sau o linie și returnează caracterul citit sau un string. Drept urmare, de câte ori aveți nevoie să citiți valori numerice, veți obține un string, pe care îl veți converti cu **Parse()**. Toate tipurile de date întregi **Int16**, **Int32**, **SByte**, etc, definesc această metodă.

Instrucțiunea de selecție **switch**

Instrucțiunile **if** imbricate sunt greu de scris corect, greu de citit și greu de depanat. **switch** este o instrucțiune de control mai potrivită atunci când aveți un set complex de alegeri de făcut. Sintaxa este identică cu cea din C++.

Sintaxa:

```
switch ( n )
{
    case val1: bloc_instrucțiunii;
                break;
    case val2: bloc_instrucțiunii;
                break;
    // alte cazuri
    default: bloc_instrucțiunii;           // opțional
}
```

n este o variabilă de tip întreg, de tip **char** sau de tip **string**. **val1**, **val2**, ... etc, sunt valorile pe care le poate lua **n**. Controlul este transferat etichetei **case** care potrivește valorii lui **n**.

Diferența față de **switch** din C++ este aceea că C# nu suportă căderea implicită la secțiunea următoare (*fall through*). Exemplul următor nu va compila:

```
int k = 0;
switch ( k )
{
    case 0:
        Console.WriteLine( "cazul 0" );
        // goto case 1;
    case 1:
    case 2:
        Console.WriteLine( "cazul 1 si 2" );
        break;
}
```

Căderea prin ramuri se poate face numai pentru cazuri vide, cum e **case 1**. Dacă doriți un salt direct la o anume etichetă, folosiți instrucțiunea de salt necondiționat **goto**. Dacă scoateți de sub comentariu **goto case 1**; în secvența de program precedentă, atunci pe ecran se va afișa:

```
cazul 0
cazul 1 si 2
```

Ciclul for

Este o instrucțiune identică cu cea din C++.

Sintaxa:

```
for (expresie1; expresie2; expresie3)
{
    bloc_instrucțiuni;
}
```

Dacă **bloc_instrucțiuni** e format din o singură instrucțiune, atunci acoladele sunt opționale. Se evaluează mai întâi **expresie1**. Apoi se evaluează **expresie2**. Aceasta controlează bucla. Dacă **expresie2** este evaluată **true**, se execută instrucțiunile cuprinse între acolade, apoi **expresie3**. Se reia evaluarea **expresie2** și ciclul continuă până când **expresie2** este evaluată **false**. Apoi controlul este cedat instrucțiunii următoare ciclului **for**.

Toate expresiile sunt opționale. Dacă **expresie2** lipsește, condiția de test este evaluată **true**:

```
for( ;; )
    Console.Write("Ciclu infinit!");
```

Ciclul foreach

foreach este nou în familia de limbaje **C**. Este folosit pentru iterarea printre elementele unui tablou sau ale unei colecții. Colecțiile sunt containere de tip generic, care pot reține secvențe omogene de obiecte, așa cum vom vedea mai târziu.

Sintaxa:

```
foreach (Tip identificator in expresie)  
    bloc_instrucțiuni;
```

Exemplu:

```
using System;  
  
class TestForeach  
{  
    static void Main()  
    {  
        int[] a = {10, 20, 30};  
        foreach (int x in a)  
            Console.Write(x + " ");  
        Console.WriteLine();  
  
        string[] st = { "UNU", "DOI", "TREI" };  
        foreach (string s in st)  
            Console.Write(s + " ");  
    }  
}
```

leșire:

```
10 20 30  
UNU DOI TREI
```

Ciclul while

Este împrumutat fără modificări din C++.

Sintaxa:

```
while (expresie)  
{  
    bloc_instrucțiuni;  
}
```

Dacă **bloc_instrucțiuni** e format dintr-o o singură instrucțiune, atunci acoladele sunt opționale. Se evaluează **expresie**. Dacă este evaluată **true**, atunci se execută **bloc_instrucțiuni**. Se reia evaluarea expresiei. Ciclul continuă cât timp este evaluată **true**.

Exemplu:

```
int x = 5;
while ( x > 0 )
    x--;
```

Ciclul `do while`

Este preluat fără modificări din C++. Este un ciclu cu test final, deci `bloc_instrucțiuni` se evaluează cel puțin o dată.

Sintaxă:

```
do
{
    bloc_instrucțiuni;
} while (expresie);
```

Dacă `bloc_instrucțiuni` e format dintr-o o singură instrucțiune, atunci acoladele sunt opționale. Se evaluează `bloc_instrucțiuni`, după care `expresie`. Cât timp aceasta se evaluează `true`, ciclul continuă. În momentul în care `expresie` se evaluează `false`, se transferă controlul instrucțiunii imediat următoare ciclului.

Exemplu:

```
int x = 5;
do
{
    x--;
} while ( x > 0 );
```

Observație

Rezultatul evaluării expresiilor care controlează buclele C# trebuie să fie o valoare *booleană*, adică `true` sau `false`. Secvența următoare nu va compila:

```
int x = 5;
do
{
    x--;
} while ( x ); // Ilegal! x este evaluat la o valoare int
```

Instrucțiuni de salt necondiționat

C# admite instrucțiunile de salt `goto`, `break` și `continue`.

Instrucțiunea `goto`

Utilizarea `goto` este nerecomandabilă, deoarece produce cod greu de înțeles (*cod spaghetti*). În cazul în care doriți totuși să o folosiți urmați pașii:

1. Creați o etichetă (identificador urmat de :)
2. `goto` etichetă

Exemplu:

```
using System;

public class GoTo
{
    static void Main()
    {
        int x = 1;
        repeta:           // Eticheta
        Console.Write( x + " ");
        x++;
        if (x <= 5)
            goto repeta; // Salt la eticheta
    }
}
```

Ieșire:

1 2 3 4 5

Instrucțiunile break și continue

Dacă în blocul de instrucțiuni al unei bucle se execută **continue**, atunci are loc un salt la începutul buclei. Instrucțiunea **break** duce la ieșirea forțată din ciclu.

Exemplu:

```
using System;

public class TestBreakContinue
{
    static void Main()
    {
        for (int i = 1; i <= 100; i++)
        {
            if (i % 2 == 1)    // Pentru valori impare
                continue;    // salt la expresia i < 10
            Console.Write(i + " ");
            if (i == 10) break;    // "Rupe" bucla
        }
    }
}
```

Ieșire:

0 2 4 6 8 10

Sugestie

Nu este indicat să utilizați intens aceste instrucțiuni, deoarece creează puncte multiple de ieșire din cicluri, ceea ce duce la cod dificil de înțeles și de întreținut (cod spaghetti).

Spații de nume

În capitolele **2** și **3** s-au discutat primele noțiuni referitoare la **spații de nume** (*namespaces*). Am arătat că sunt containere logice care găzduiesc definiții ale tipurilor de date. Au fost introduse în C# pentru evitarea conflictelor de nume în utilizarea bibliotecilor de clase, dar și pentru organizarea și sistematizarea tipurilor de date. De exemplu, dacă într-o aplicație utilizați două biblioteci neprotejate prin spații de nume, care provin de la furnizori diferiți, este posibil ca un același nume de clasă, să-i spunem *Button*, să fie definit în ambele biblioteci, ceea ce ar crea o coliziune în program, inacceptabilă de către compilator.

Întreaga **Biblioteca de Clase .NET** se găsește în interiorul acestor *namespaces*. În afara spațiilor de nume ale bibliotecii *.NET Framework* sau ale bibliotecilor altor furnizori, C# vă permite să vă creați propriile spații de nume. Se definesc foarte simplu cu ajutorul cuvântului cheie **namespace**.

Exemplu:

```
namespace MySpace
{
    static void Main()
    {
    }
}
```

Spații de nume imbricate (*nested*)

Într-un spațiu de nume puteți defini nu numai tipuri de date ci și alte spații de nume.

Exemplu:

```
namespace X
{
    namespace Y
    {
    }
    namespace Z
    {
        namespace W
        {
        }
    }
    static void Main()
    {
    }
}
```

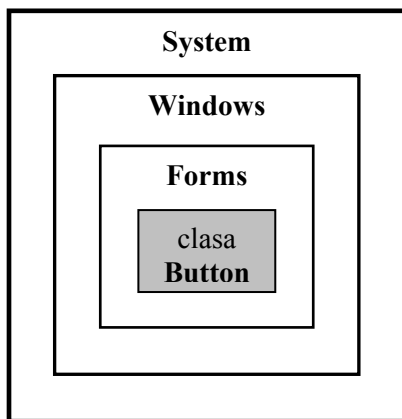

În interiorul lui **x** s-au definit alte două spații de nume **y** și **z** și metoda **Main()**. În **z** s-a definit **w**.

Accesarea numelor de tipuri

Accesarea numelor definite într-un *namespace* se face precizând toate spațiile de nume până la numele clasei dorite, separate între ele cu operatorul punct (**.**).

Exemplu:

Cum vom accesa numele clasei **Button** ?



În figură s-a ilustrat faptul că spațiul de nume **System** include spațiul de nume **Windows**, care la rândul lui include **Forms**. Clasa **Button**, ca de altfel toate clasele de tip fereastră se definesc în **Forms**.

Cum accesăm numele **Button** ? Vom crea un obiect de tip **Button** astfel:

```
System.Windows.Forms.Button b =
new System.Windows.Forms.Button();
```

Cam complicat, nu-i așa ? Din fericire, există directiva **using**.

Figura 3.1 Accesarea numelor

Programele C# încep de regulă cu o secțiune de directive **using**. ([4])
using precizează spațiile de nume cel mai utilizate în program. În felul acesta programatorul nu mai este nevoit să scrie întreaga cale (*fully qualified name*) până la clasa căutată.

a) Accesarea unui nume de clasă folosind calificarea completă a numelui acesteia:

```
public class NestedNamespaces
{
    static void Main()
    {
        System.Windows.Forms.Button b =
            new System.Windows.Forms.Button();
    }
}
```

b) Accesarea unui nume de clasă cu utilizarea directivei `using`:

```
using System.Windows.Forms;

public class NestedNamespaces
{
    static void Main()
    {
        Button b = new Button();
    }
}
```

Notă:

Pentru ca programul de la punctul **b)** să compileze, trebuie să aduceți proiectului de tip consolă o *referință*, la spațiul de nume `System.Windows.Forms` astfel:

- În **Solution Explorer**, click dreapta pe numele proiectului.
- Alegeți *Add reference*.
- În *tab-sheet* –ul *.Net* alegeți `System.Windows.Forms` și apăsați OK.

ATENȚIE !

O directivă `using` nu vă permite accesul la nici un spațiu de nume definit în spațiul de nume pe care îl specificați.

Exemplu:

```
using System;

public class WrongAcces
{
    static void Main()
    {
        Windows.Forms.Button b = new Windows.Forms.Button();
    }
}
```

Programul de mai sus nu compilează, deoarece `Windows` este un spațiu de nume imbricat în `System` și nu puteți avea acces la acesta, ci doar la numele de tipuri (clase sau structuri) din `System`. Cu directiva de mai sus, este corect să scrieți astfel:

```
System.Windows.Forms.Button b =
    new System.Windows.Forms.Button();
```

Alias-uri

Pentru simplificarea scrierii puteți declara nume alternative pentru spațiile de nume, astfel:

a) Introducerea unui **alias** pentru spații imbricate, definite de programator:

```
using N = N1.N2.N3;    // Creează un alias N

namespace N1
{
    namespace N2
    {
        namespace N3
        {
            public class C { }
        }
    }
}

public class TestAlias
{
    static void Main()
    {
        N.C ob = new N.C();
    }
}
```

b). Introducerea unui **alias** pentru spații de nume predefinite:

```
using F = System.Windows.Forms;

public class TestAlias
{
    static void Main()
    {
        F.ComboBox cb = new F.ComboBox();
    }
}
```

Directive de preprocesare

Înainte ca programul să fie compilat, un alt program numit *preprocesor*, rulează și pregătește programul pentru compilare. Preprocesorul detectează directivele de preprocesare și stabilește niște reguli de compilare în raport cu acestea. Directivele încep cu caracterul **#**. Le veți introduce în situații în care doriți ca unele porțiuni de cod să nu fie compilate în anumite condiții. De exemplu, în etapa de **build final** a unei aplicații, nu doriți să mai compilați porțiuni de cod pe care l-ați folosit pentru depanare.

Directiva #define

Numele simbolice se definesc cu directiva **#define**, înaintea oricărei secvențe de cod din program. Alte directive pot fi definite și în alte zone ale programului.

Exemplu:

```
#define DEPANARE
// Cod neafectat de directiva #define
#if DEPANARE
// Cod care se compilează dacă este definit numele DEPANARE
#else
// Cod care se compileaza daca nu este definit DEPANARE
#endif
// Cod neafectat de preprocesor
```

Când preprocesorul rulează, el caută directiva **#define**. Dacă aceasta există, ia notă de identificatorul **DEPANARE**. Directiva **#if** testează existența identificatorului. În funcție de aceasta, setează pentru compilare o porțiune de cod sau alta.

Orice cod care nu este cuprins între **#if** și **#endif** este compilat de către compilator.

Directiva #undef

Dacă într-un punct în program doriți ca un anume identificator definit cu **#define** să nu mai fie definit, deci să nu mai fie cunoscut de către preprocesor, atunci folosiți directiva **#undef**:

```
#define DEPANARE
#define MAXIM

// Cod neafectat de directiva #define
#if DEPANARE
// Cod care se compilează
#endif

#undef DEPANARE

#if DEPANARE
// Cod care NU se compilează
#elif MAXIM
// Cod care compilează pentru că nu e definit DEPANARE,
// dar este definit MAXIM
#endif
```

Să observăm că **#elif** testează existența unui nume simbolic doar în cazul în care testul **#if** eșuează.

Rezumatul capitolului

- Tipurile predefinite din C# (`int`, `char`, ..., `string`), sunt mapate direct peste tipuri `struct` din spațiul de nume `System`. De exemplu, tipului `int` îi corespunde clasa `System.Int32` din *Biblioteca de Clase .NET*.
- Tipurile definite cu ajutorul cuvintelor cheie `class` sau `struct` se numesc tipuri definite de programator.
- Variabilele de tip clasă se numesc **obiecte** sau **instanțe** ale clasei.
- **Tipurile valoare** sunt toate tipurile predefinite (`char`, `int`, `float`, etc.), cu excepția tipurilor `string` și `object`.
- **Tipurile referință**, se definesc de către programator. În această categorie se includ tipurile *clasă*, tipurile *interfață*, tipurile *delegat* și tipurile *array* (tablourile).
- Datele variabilelor de **tip valoare**, se memorează pe stivă (*Stack*).
- În cazul **tipurilor referință**, referințele sunt în stivă, iar datele se alocă dinamic în *Heap*.
- În C# tablourile sunt **tipuri referință**. Tablourile pot fi **unidimensionale** sau **multidimensionale**. Cele multidimensionale, la rândul lor pot fi dreptunghiulare sau **neregulate** (*jagged*).
- Instrucțiunile de control al fluxului de execuție a programului (`if`, `for`, `while`, etc) sunt similare sintactic cu cele din limbajul C++.
- Întreaga **Biblioteca de Clase .NET** se găsește în interiorul spațiilor de nume. În afara spațiilor denume predefinite, programatorii pot să definească propriile spații de nume cu ajutorul cuvântului cheie `namespace`.
- Directivele de preprocesare se introduc în situațiile în care se dorește ca unele porțiuni de cod să nu fie compilate în anumite condiții.

Întrebări și exerciții

1. Care este diferența dintre tipurile **valoare** și tipurile **referință** ?
2. În ce condiții se produc erori la compilare atunci când declarați o variabilă? Dar o constantă ?
3. În care zone ale memoriei RAM se memorează membrii unei clase ?
4. Declarați și inițializați un tablou bidimensional dreptunghiular cu **4** linii și **3** coloane.
5. Declarați și inițializați un tablou bidimensional neregulat cu **3** linii.
6. Definiți două spații de nume: **x** și **y**, unde **y** este conținut în **x**. În **y** definiți clasa **A**. Scrieți două programe C# în care se instanțiază clasa **A**, utilizând moduri diferite de accesare a numelui clasei.

Capitolul 4

Programare Orientată pe Obiecte în C#

Programarea Orientată pe Obiecte (OOP) este cea mai influentă paradigmă de programare a timpurilor noastre. Prin “paradigmă” se înțelege stil de programare. În raport cu stilul de programare procedural, OOP reprezintă o cu totul altă abordare a programării.

În limbajele C sau Pascal ca și în toate celelalte limbaje procedurale, se pot defini tipuri de date și în mod separat funcții care acționează asupra acestora. Separarea tipurilor de date de funcțiile corespunzătoare, impune programatorului eforturi considerabile pentru a menține logica grupării acestora, mai ales când aplicația are mii de linii de cod.

În OOP funcțiile sunt grupate în același obiect, împreună cu datele asupra cărora operează. Datele sunt mai complexe în OOP și mai bine organizate. Organizarea superioară se dovedește foarte utilă atunci când se scriu aplicații foarte mari, când se lucrează în echipă și când se scrie cod pentru biblioteci care vor fi folosite de clienți.

Obiecte și clase

Limbajul C# este un limbaj *orientat-obiect*. Limbajele orientate pe obiecte au calitatea că permit programatorului să-și creeze noi tipuri complexe de date. Tipurile se obțin prin definirea de clase. Clasele sunt baza programării orientată pe obiecte. O clasă reunește cod și date care descriu o anumită categorie de obiecte. Clasa este apoi folosită pentru construirea de obiecte. Crearea unui obiect se numește **instanțiere**. Obiectul creat se numește **instanță** a clasei.

Diferența dintre un obiect și o clasă este aceeași ca diferența între o pisică și descrierea unei pisici. Descrierea este clasa, iar pisica este obiectul. De altfel, clasele și obiectele au apărut în programare tocmai din necesitatea de a modela obiectele din realitate, dar și obiecte abstracte.

O clasă **Pisica**, specifică trăsăturile care ne interesează la o pisică: nume, rasă, greutate, culoarea blănii, etc. De asemenea, clasa descrie acțiunile pe care o pisică le poate face: să sară, să zgârie, să toarcă, etc. Pisica *Kitty*, rasa *Birmaneză*, greutatea - 4 Kg, culoarea blănii – *gri*, este o instanță a clasei, deci un obiect de tip **Pisica**.

Marele avantaj al claselor este că încapsulează caracteristicile și capabilitățile obiectului într-o singură unitate de cod. O clasă descrie complet obiectele de tipul său, nemaifiind nevoie în acest scop de cod exterior clasei.

Mai trebuie să rețineți următoarele: programarea orientată pe obiecte nu este doar programare cu obiecte. Între obiecte pot exista relații, obiectele pot comunica între ele, obiectele pot folosi alte obiecte.

Mecanismele fundamentale ale OOP

Toate limbajele orientate pe obiecte au câteva caracteristici comune:

1. Abstractizarea

Abstractizarea este simplificarea realității prin modelarea unor clase care păstrează doar trăsăturile obiectului real care sunt esențiale și suficiente pentru atingerea scopului propus. Membrii clasei (câmpuri, metode, etc.) definesc aceste trăsături.

2. Încapsularea

Un obiect încapsulează date și cod care acționează asupra acestor date. Clasa oferă protecție membrilor săi. Nu toți membrii clasei pot fi accesați din afara ei. O regulă importantă în OOP este aceea că datele membre trebuie să rămână private, să fie inaccesibile din lumea exterioară clasei, pentru a preîntâmpina modificările accidentale. Partea accesibilă a clasei o constituie membrii publici, de regulă metodele. Aceștia pot fi utilizați de către porțiuni din program care lucrează cu obiectele clasei. Membrii publici ai unei clase formează **interfața** clasei.

3. Moștenirea

Moștenirea este mecanismul prin care un limbaj orientat-obiect vă permite să definiți noi clase care încorporează și extind o clasă deja definită. Noile clase se numesc **clase derivate**. Ele moștenesc atributele și comportamentul clasei pe care o extind, fiind versiuni specializate ale clasei de bază. Clasa pe care o extindeți se numește clasă de bază. Ca exemplu, să presupunem că doriți pentru aplicația dumneavoastră un buton care să aibă funcționalitatea unui buton de apăsare, dar să aibă în plus anumite proprietăți: să fie rotund, să aibă un mic punct luminos central care clipește când este apăsă, etc. Aveți două opțiuni: să scrieți o clasă care să îndeplinească toate cerințele sau să scrieți o clasă **MyButton**, care să moștenească de pildă clasa **System.Windows.Forms.Button**. A doua variantă este avantajoasă, deoarece clasa **MyButton** moștenește întregul comportament și caracteristicile unui buton obișnuit și veți avea nevoie să adăugați doar codul cerut de cerințele suplimentare. **Reutilizarea codului** este unul dintre beneficiile importante pe care le aduce moștenirea.

4. Polimorfismul

În programarea orientată pe obiecte, *polimorfismul* (cuvânt care vine din greacă și care înseamnă “a avea mai multe forme”) este caracteristica unei interfețe de a fi folosită cu acțiuni multiple. O funcție sau un obiect au un comportament polimorfic, dacă acestea au mai mult de o singură formă.

Polimorfismul, caracterizat de expresia “*interfață comună, acțiuni multiple*”, este deci capacitatea de a folosi în funcție de context, mai multe forme ale unui tip sau metodă, fără să ne intereseze detaliile de implementare.

Există câteva tipuri de polimorfism. Unul dintre acestea se obține în OOP cu ajutorul moștenirii. Să luăm un exemplu din viața reală: Un dresor de circ are trei animale : un șarpe, un porumbel, și un tigru. Când vrea să cheme unul dintre ele,

le trimite un unic mesaj : “*la masa !*”. Ca răspuns la acest mesaj, șarpele se *tărăște*, porumbelul *zboară*, iar tigrul *aleargă*. Să ne imaginăm clasele **Sarpe**, **Porumbel** și **Tigrul**. Toate clasele moștenesc clasa **Animal**. Fiecare clasă și-a definit aceeași interfață, metoda “*LaMasa*”, dar cu acțiuni diferite. Obiecte diferite (animalele) răspund în mod diferit aceluiași mesaj (metoda *LaMasa()*). Pentru dresor nu este important cum procedează fiecare animal pentru a veni (detaliile de implementare ale metodei *LaMasa()*), ci contează doar faptul că animalul vine la masă.

Clasele C#

În afara celor cincisprezece tipuri *predefinite* în C#, puteți crea propriile tipuri de date (*user-defined types*). Există șase categorii de tipuri pe care le puteți crea:

1. Tipuri **class**
2. Tipuri **struct**
3. Tipuri **tablou**
4. Tipuri **enum**
5. Tipuri **delegate**
6. Tipuri **interface**

Odată ce ați definit un tip de dată îl puteți utiliza ca și când ar fi un tip predefinit.

Dintre tipurile enumerate mai sus, **class** este cel mai important. O clasă este o structură de date care poate depozita **date** și executa **cod**. Primele noțiuni despre clasele C# au fost discutate în capitolul 3.

Definiție

*Clasele sunt entități logice care modelează obiecte din lumea reală sau obiecte abstracte. Clasele încapsulează **date** și **funcții** care operează cu aceste date.*

Caracteristicile obiectului se memorează în **câmpurile** clasei.

Capabilitățile, acțiunile obiectului, se codifică în **metodele** clasei.

- **Câmpurile** (acestea sunt **date membre**) sunt **variabile** care rețin informații despre obiect.
- **Metodele** sunt **funcții membre** ale clasei.

Câmpurile și metodele unei clase se numesc **membrii clasei**. O clasă poate avea un număr oricât mare de date membre și de funcții membre. Membrii pot fi oricare combinație a următoarelor tipuri de membri:

Date membre	Funcții membre
▪ Câmpuri ▪ Constante	▪ Metode ▪ Proprietăți ▪ Constructori

	▪ Destructori ▪ Operatori ▪ Indexatori ▪ Evenimente
--	--

Definirea claselor

O clasă se declară cu ajutorul cuvântului cheie `class`.

Sintaxa minimală:

```
class nume_clasă
{
    // Câmpuri
    [modificator de acces] Tip nume;

    // Metode
    [modificator de acces] TipRetur NumeMetodă(Parametri);
}
```

Modificatorii de acces controlează regiunile programului care pot accesa membrii clasei. Tabelul descrie cele șase tipuri de modificatori:

Atributul	Descriere
public	Acces nelimitat
private	Acces limitat la propria clasă
protected	Acces limitat la propria clasă și la clasele derivate
internal	Acces limitat la programul care conține clasa
protected internal	Acces limitat la program și la clasele derivate

Dacă modificatorul de acces lipsește, atunci membrul respectiv este în mod implicit **private**. Parantezele pătrate indică opționalitatea.

Cum se lucrează cu obiecte ?

1. Mai întâi se definește clasa. **Exemplu:**

```
class X
{
    // membrii clasei
}
```

2. Acum puteți utiliza tipul **x** pentru a crea obiecte:

```
X r = new X();
```

`r` este referință la obiectul de tip **x** care se alocă dinamic cu `new X()` în (Heap-ul sistemului). `x()` este constructorul clasei **x**.

Despre constructori vom discuta într-unul dintre paragrafele următoare.

3. Folosiți referința pentru a accesa membrii clasei:

```
r.membru; // Cu operatorul . se accesează membrii clasei
```

Exemplul 1:

```
class Program
{
    static void Main()
    {
    }
}
```

Clasa **Program** are ca membru doar metoda statică **Main()**. Această metodă este punctul de intrare în program (*entry point*). Membrii statici ai unei clase există independent de obiectele clasei. Din acest motiv, **Main()** poate fi apelată de către platforma **.NET**, fără să fie nevoie de un obiect de tip **Program**.

Exemplul 2:

```
class Test
{
    int x = 100;           // Câmp privat

    static void Main()
    {
//      x = 10;           // Gresit! x trebuie să aparțină
                        //      unui obiect
        Test r;           // Declară o referință
        r = new Test();    // Crează un obiect
        r.x = 20;          // Corect. x este privat, dar
    }                      // Main() e membră a clasei
}
}
```

Exemplul 3:

```
using System;

class Test
{
    private int x;         // Câmp privat

    // Metode publice
    public int GetX()
    {
        return x;
    }
}
```

```

public void SetX(int n)
{
    x = n;           // Metodele clasei au acces
}                  // la câmpurile private
}

class Program
{
    static void Main()
    {
        Test r;      // Declară o referință
        r = new Test(); // Creează un obiect
        // r.x = 20;   // Greșit. x este private
        r.SetX(10);    // Corect. SetX() este public
        Console.WriteLine(r.GetX());
    }
}

```

leșire: 10

Programul de mai sus are două clase. În clasa **Program**, metoda **Main()**, crează un obiect de tip **Test**, care mai apoi este utilizat pentru accesarea membrilor săi publici.

Metode. Parametrii metodelor.

Metoda este un bloc de cod care poate fi apelat din diferite părți ale programului și chiar din alte programe. Când o metodă este apelată, își execută codul, apoi transferă controlul codului care a apelat-o. Metodele pot fi implementate ca membri ai claselor sau ai structurilor sau ca prototipuri în interfețe. Metodele pot să aibă sau să nu aibă parametri și pot să returneze sau nu valori. Dacă nu returnează valori, returnează **void**. De asemenea, metodele pot fi **statice** sau **ne-statice**.

ATENȚIE !

Spre deosebire de C sau C++, în C# nu există metode globale. Metodele nu se pot declara în afara unui tip de date.

Sintaxa minimală a unei metode este:

```

[ModificatorAcces] TipRetur NumeMetodă (ListaParametri)
{
    // codul metodei
}

```

Apelul unei metode:

NumeMetodă (*ListaArgumente*) ;

Argumentele (sau *parametrii actuali*) sunt valorile sau obiectele cu care se apelează metodele.

Parametrii unei metode pot fi precedați în C# de câteva cuvinte cheie, numite *modificatori*. Modificatorii sunt: **ref**, **out**, **params**. Aceștia controlează modul în care argumentele sunt transmise unei metode.

Parametri valoare

În mod implicit, *tipurile valoare* sunt pasate metodelor **prin valoare**. Aceasta înseamnă că atunci când un obiect de tip valoare este transmis ca argument unei metode, o copie temporară a obiectului este creată de către metodă pe stivă. Când metoda își termină execuția, copiiile temporare sunt distruse. În general aveți nevoie doar de valoarea argumentelor, însă sunt situații când doriți să modificați valoarea argumentelor. În acest caz, mecanismul transmiterii prin valoare nu vă ajută.

Exemplu:

```
using System;

class S
{
    public void Schimba(int x, int y) // x, y - parametri
    {
        //          formali
        int aux;          // x, y, aux - variabile locale
        aux = x; x = y; y = aux;
    }
}

class Test
{
    static void Main()
    {
        S s = new S();
        int a = 10, b = 20; // a, b - variabile locale

        // a, b - argumente pasate prin valoare
        s.Schimba(a, b);
        Console.WriteLine("a = {0}, b = {1}", a, b);
    }
}
```

Programul afișează: **a = 10, b = 20**

Observăm că **a** și **b** au rămas nemodificați în urma apelului, deoarece la transmiterea prin valoare **x** și **y** sunt copii locale ale argumentelor **a** și **b**. Argumentele nu trebuie să fie în mod obligatoriu variabile. Poate fi argument orice expresie care se evaluează la tipul parametrului formal.

Parametri referință

Dorim să modificăm programul anterior, astfel încât valorile **a** și **b** să se modifice după apel. Se aplică modificatorul **ref** asupra parametrilor formali și asupra parametrilor actuali astfel:

```
public void Schimba(ref int x, ref int y)
{
    int aux;
    aux = x; x = y; y = aux;
}

int a = 10, b = 20;           // Inițializare obligatorie
s.Schimba(ref a, ref b);     // Apel
//s.Schimba(2*a, b - 1);    // Greșit! Argumentele nu
                             // sunt variabile de memorie
```

Rulați programul. Va afișa: **a = 20, b = 10**

Când dorim să pasăm argumente prin referință, atunci:

- Trebuie utilizat modificatorul **ref** atât în definiție, cât și în apelul metodei.
- Este obligatoriu ca argumentul transmis funcției să fie o variabilă, iar acestei variabile trebuie să-i fie atribuită o valoare înainte de a fi utilizată ca parametru actual.

Parametrii referință au următoarele **caracteristici**:

- Nu alocă memorie pe stiva programului pentru parametrii formali.
- Parametrul formal este de fapt un alt nume (un *alias*) pentru variabila de apel, referind aceeași zonă de memorie. Din această cauză, orice modificare asupra valorii parametrului formal în timpul execuției metodei, se exercită direct asupra parametrului actual și persistă după apelul metodei.

Parametri de ieșire

Parametri de ieșire sunt utilizați pentru a transmite date din interiorul metodei spre codul care a apelat-o. Nu contează valorile inițiale ale argumentelor, ci doar cele finale.

Cerințe:

- Trebuie utilizat modificatorul **out** atât în definiție, cât și în apelul metodei.
- Este obligatoriu ca argumentul transmis funcției trebuie să fie o variabilă și nu o expresie. Acestei variabile nu este nevoie să-i fie atribuită o valoare înainte de a fi utilizată ca parametru actual.

Exemplu:

```
class OutParam
{
    public void Metoda(out int x)
    {
        // int y = x; // Eroare!
        x = 100;      // Inițializare înainte de
        x++;          // a se citi din variabila x
    }
}
```

```
class Program
{
    static void Main(string[] args)
    {
        OutParam op = new OutParam();
        int z;    // Nu e nevoie de initializare
        op.Metoda(out z);
        System.Console.WriteLine(z);
    }
}
```

La fel ca în cazul parametrilor referință, parametrii de ieșire (*output parameters*) sunt *alias*-uri pentru parametri actuali. Orice schimbare asupra unui parametru de ieșire, se produce de fapt asupra argumentului funcției.

Parametrii de ieșire au următoarele **caracteristici**:

- În interiorul metodei, parametrilor de ieșire trebuie să li se atribuie valori înainte de a se utiliza valoarea acestora. Din acest motiv, valorile inițiale ale parametrilor actuali (argumente) sunt neneesare și nici nu este nevoie să inițializați argumentele înainte de apelul metodei.
- Tuturor parametrilor de ieșire trebuie să le fie atribuie valori înainte de revenirea din apel.

Parametri tablou

Parametrii tablou (*parameter array*) admit zero sau mai mulți parametri actuali pentru un același parametru formal.

Restricții:

- Poate exista doar un singur *parametru tablou* în lista de parametri formali.
- Dacă există un *parametru tablou*, atunci acesta este ultimul în lista de parametri formali.
- Pretinde modificatorul **params** în declarație, dar nu este permis în apelul metodei.

Parametrii tablou se declară încadrând tipul parametrului la stânga cu modificatorul **params**, iar la dreapta cu operatorul de indexare: `[]`.

Exemplu:

```
class Params
{
    public void Print(params int[] x)
    {
        foreach (int i in x)
            System.Console.WriteLine(i + " ");
    }
}
```


Membrii statici ai unei clase

În mod implicit, metodele și câmpurile clasei sunt ne-stactice. Pentru a putea invoca o metodă ne-statică, aveți nevoie de un obiect de tipul clasei. Metodele statice se pot apela în lipsa obiectelor. Ele au comportamentul unor funcții globale, care însă nu sunt vizibile decât în cadrul clasei. De asemenea, câmpurile declarate **static**, se alocă în memorie și pot fi folosite, chiar dacă nu ați construit nici un obiect.

Accesarea membrilor statici.

Fie clasa:

```
class C
{
    public static int x;
    public static void F() {}
}
```

Accesarea acestor membri se face prin numele clasei, astfel: **C.x** și **C.F()** ;

Exemplul 1:

```
class Program
{
    static int x = 10;
    static void F()
    {
    }
    int y = 20;           // Membri ne-statici
    void G()
    {
    }
    static void Main(string[] args)
    {
        // Membrii statici aparțin clasei
        x = 100;           // Corect
        F();               // Corect

        Program.x = 100;   // Corect! Accesare prin
        Program.F();       // numele clasei.

        // Membrii ne-statici aparțin obiectelor
        // y = 200;         // Greșit
        // G();             // Greșit
        Program p = new Program();
        p.y = 100;         // Corect
        p.G();             // Corect
    }
}
```


Observație:

C# admite inițializarea datelor membre, la momentul declarării lor, așa ca mai sus: `int y = 20;` sau `int x = 10;`

Exemplul 2:

Când accesați membrii statici din afara clasei trebuie să folosiți sintaxa: `NumeClasă.MembruStatic;`

```
class ClasaMea
{
    public static string s = "Ionel";
    public static void F() { }
}

class Test
{
    static void Main(string[] args)
    {
        // s = "Marcel";           // Greșit
        ClasaMea.s = "Marcel";    // Corect
        // F();                    // Greșit
        ClasaMea.F();             // Corect
    }
}
```

Constante

Dacă doriți să păstrați valori care nu se vor schimba pe parcursul execuției programului și în acest scop vreți să le protejați la scriere, veți declara constante. În C#, constantele apar în două forme:

1. **Constante locale.**
2. **Constante membre** ale clasei.

Atât constantele membre, cât și cele locale, se introduc cu cuvântul cheie **const**. **Constantele locale** se declară în corpul metodelor și sunt vizibile doar în corpul acestora:

```
void Metoda(int a)
{
    const int x = 100;        // Inițializare obligatorie
    // x++                    // Eroare!
    a = x;                    // Corect
}
```

Constantele membre se comportă la fel ca valorile statice. Sunt vizibile în fiecare instanță (obiect) al clasei și sunt disponibile chiar în lipsa obiectelor clasei.

Exemplu:

```
using System;

class C
{
    public const int H = 24;
}

class TestConst
{
    static void Main(string[] args)
    {
        Console.Write(C.H);
    }
}
```

De reținut:

- Constantele trebuie inițializate la declarare.
- Constantele membre se accesează la fel ca membrii statici :
NumeClasă.NumeConstantă.

Constructorii

Constructorul de instanță este o **metodă specială** a clasei, care are rolul de a crea, a construi obiecte. O clasă are cel puțin un constructor. De fiecare dată când instanțiați un obiect de tip **struct** sau **class**, se apelează constructorul clasei.

Exemplu:

```
class Masina
{
    public string marca;
    public int pret;
}

class Test
{
    static void Main(string[] args)
    {
        Masina m = new Masina(); // Apelul constructorului
    }
}
```

Observați că instanțierea obiectului presupune de fapt apelul constructorului clasei: **Masina()**. Toate clasele au un constructor. Puteți defini proprii constructori. În cazul în care nu o faceți, compilatorul va genera un constructor în mod implicit. Clasa **Masina** nu definește un constructor. În această situație, compilatorul generează un **constructor implicit**, fără parametri, **Masina()**, care construiește obiectul, dar nu întreprinde altă acțiune.

Constructorul implicit inițializează câmpurile clasei cu următoarele valori implicite:

- Tipurile numerice (`int`, `short`, etc.) și tipul `enum` cu valoarea 0.
- Tipul `bool` cu valoarea `false`.
- Tipul `char` cu `'\0'`.
- Tipurile referință (inclusiv `string` sau tablouri) cu `null`.

În mod frecvent veți dori să stabiliți starea inițială a obiectului prin setarea unor valori inițiale ale datelor membre. Pentru aceasta, veți defini unul sau mai mulți constructori. De exemplu, pentru clasa `Masina`, puteți scrie:

```
using System;
```

```
class Masina
{
    public string marca;
    public int pret;
    public Masina(string m, int p) // Constructor
    {
        marca = m;
        pret = p;
    }
}

class Test
{
    static void Main(string[] args)
    {
        // Masina m = new Masina(); // Gresit! Nu mai există
        // constructorul implicit!
        // Se invocă constructorul Masina(string, int)
        Masina m = new Masina("Audi", 50000);
        Console.Write(m.marca + " " + m.pret);
    }
}
```

Ieșire: Audi 50000

ATENȚIE !

În momentul în care ați definit cel puțin un constructor, compilatorul nu mai sintetizează constructorul implicit. De aceea, dacă doriți și unul fără parametri, trebuie să-l definiți.

Exemplu:

```
class Masina
{
    public string marca;
    public int pret;
```

```
public Masina()                // 1
{
    marca = "Fiat";
    pret = 1000;
}
public Masina(string m)        // 2
{
    marca = m;
    pret = 20000;
}
public Masina(int p)           // 3
{
    marca = "Ford";
    pret = p;
}
public Masina(string m, int p) // 4
{
    marca = m;
    pret = p;
}
}
```

Acum puteți crea obiecte de tip **Mașina** în mai multe moduri:

```
Masina m1 = new Masina();           // constructor 1
Masina m2 = new Masina("Opel");     // constructor 2
Masina m3 = new Masina(2000);       // constructor 3
Masina m4 = new Masina("Renault", 15000); // constructor 4
```

Caracteristicile metodelor constructor

- Sunt metode care se apelează ori de câte ori, un obiect este creat.
- Au același nume ca cel al clasei.
- Nu au tip de retur, deci nu returnează nimic, nici măcar **void**.
- Pot fi supraîncărcați, la fel ca oricare metodă a clasei.
- Pot să fie declarați **static**, pentru inițializarea membrilor statici ai clasei.
- De regulă se declară **public**, însă dacă doriți ca un anumit constructor să nu poată fi invocat din afara clasei, îl declarați **private**. Procedați în acest fel, mai ales când o clasă conține numai membri statici. În această situație este inutil să permiteți crearea de instanțe ale clasei (obiecte).

Constructori de copiere

Dacă doriți ca un obiect să fie creat ca o copie fidelă a unui alt obiect, atunci este convenabil să definiți un constructor de copiere (*copy constructor*). C# nu furnizează un constructor de copiere implicit, așa cum oferă un constructor de instanță implicit. De aceea va trebui să-l definiți dumneavoastră.

Exemplu:

```
using System;

class C
{
    private int x; // Câmp privat

    public C(int a) // Constructor de instanță
    {
        x = a;
    }
    public C(C ob) // Constructor de copiere
    {
        x = ob.x;
    }
    public int GetX() { return x; } // Metodă accesor
}

class TestCopyCtor
{
    static void Main(string[] args)
    {
        C c1 = new C(10); // Apeleaza C(int)
        C c2 = new C(c1); // Apeleaza C(C)
        Console.WriteLine(c2.GetX()); // Afișează: 10
    }
}
```

Din exemplu vedem că parametrul constructorului de copiere are tipul clasei, iar argumentul de apel este un obiect (**c1**) al clasei.

Cuvântul cheie **this**

this este o referință la obiectul curent. Toate metodele nestatice ale clasei posedă această referință și au acces la ea.

Exemplificăm utilizarea referinței **this** în următoarele circumstanțe:

1. Pentru rezolvarea ambiguităților care apar când numele parametrilor formali ai unei metode coincid cu numele altor membri.
2. Returnarea de către o metodă a unei referințe la obiectul curent.

```
using System;

class Copil
{
    private string nume;
    private int varsta;
```

```
// Parametrii formali au aceleași nume cu ale câmpurilor
public Copil(string nume, int varsta)
{
    this.nume = nume;        // cazul 1
    this.varsta = varsta;    // cazul 1
}
public Copil ModificVarsta(int varsta)
{
    this.varsta = varsta;    // cazul 1
    return this;            // cazul 2
}
public void Afiseaza()
{
    Console.WriteLine(nume + " " + varsta);
}
}

class Test_this
{
    static void Main(string[] args)
    {
        Copil c = new Copil("Alin", 17);
        c.Afiseaza();        // Ieșire: Alin 17
        c.ModificVarsta(18);
        c.Afiseaza();        // Ieșire: Alin 18
    }
}
```

Ce s-ar fi întâmplat dacă constructorul clasei s-ar fi definit astfel :

```
public Copil(string nume, int varsta)
{
    nume = nume;
    varsta = varsta;
}
```

Nu este eroare la compilare. Dar parametrul formal **nume** “ascunde” câmpul clasei, astfel încât metoda nu accesează câmpul privat al clasei, ci realizează o inutilă autoatribuire a parametrului formal. Oricare membru al clasei poate fi accesat în metodele clasei astfel: **this.membru**. În acest fel, **this.nume = nume**; realizează o inițializare corectă a câmpului **nume**.

Mai observăm că metoda **ModificVarsta()** returnează o referință la obiectul curent. Cine este de fapt obiectul curent ? Asta nu vom ști, până în momentul în care metoda este apelată: **c.ModificVarsta(18)**; Prin urmare, **c** referă obiectul curent și **this** referă evident același obiect. De altfel, în momentul apelului, **this** referă deja obiectul alocat cu **new**, pentru că **c** transferă în **this** referința către obiect.

Există și alte situații în care aveți nevoie de **this**, așa cum vom vedea mai departe.

Destructorul clasei

Destructorul clasei este o metodă specială care distruge instanțele claselor. Dacă o clasă poate avea mai mulți constructori, în schimb nu poate avea decât un singur destructor. Un destructor se declară astfel:

```
class Avion
{
    // Membrii clasei
    ~Avion() // Destructor
    {
        // Eliberarea resurselor gestionate de obiect
    }
}
```

În majoritatea covârșitoare a cazurilor, nu veți avea de implementat un destructor, deoarece C# are un **garbage collector** care distruge obiectele nefolositoare în mod automat. Veți implementa un destructor doar în situații în care obiectele gestionează resurse care nu sunt controlate de *.NET Framework*.

Nu vom intra în alte detalii care ies din cadrul propus în această lucrare. Este bine să rețineți totuși câteva informații utile în legătură cu destructorii:

IMPORTANT

- Destructorii au același nume cu cel al clasei, prefixat cu caracterul ~.
- O clasă poate avea un singur destructor.
- Destructorii nu se apelează niciodată. Ei se invocă în mod automat.
- Constructorii nu au parametri și nu acceptă modificatori.
- În structuri nu se pot defini destructori.
- Destructorii nu pot fi supraîncărcați.

Proprietăți

Proprietățile reprezintă o facilitate importantă a limbajului C#. Sunt membri ai clasei care permit accesul direct la starea obiectului. Starea unui obiect este dată de valorile datelor membre. Cu ajutorul proprietăților veți accesa câmpurile private, ca și când ar fi fost declarate public, fără ca prin aceasta să se încalce principiul protecției datelor, care cere ca datele membre să fie private. În realitate, proprietățile implementează niște metode speciale, numite **accesori**.

Exemplu:

```
using System;

class Copil
{
    private string nume;           // Câmpuri private
    private int varsta;
```

```
public Copil(string n, int v)           // Constructor
{
    nume = n;
    varsta = v;
}

public string Nume                     // Proprietatea Nume
{
    get
    {
        return nume;
    }
    set
    {
        nume = value;
    }
}

public int Varsta                     // Proprietatea Varsta
{
    get
    {
        return varsta;
    }
    set
    {
        varsta = value;
    }
}
}

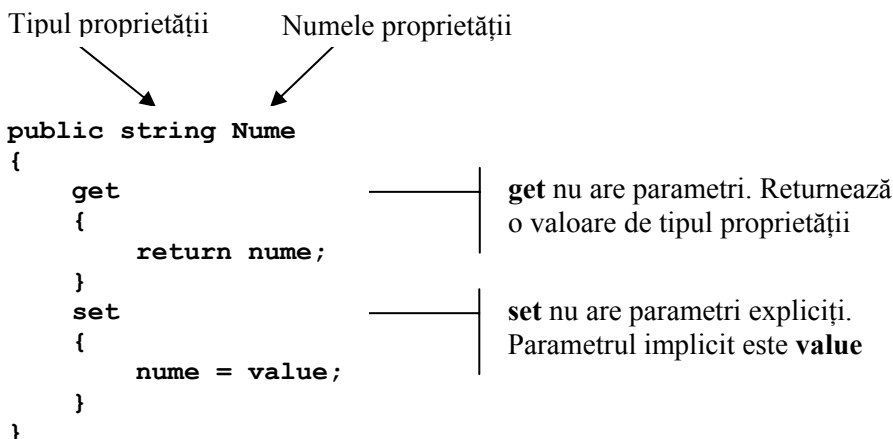
class TestProprietati
{
    static void Main(string[] args)
    {
        Copil c = new Copil("Valentin", 18);
        Console.WriteLine(c.Nume + " " + c.Varsta); // get
        c.Nume = "Andrei";           // set
        c.Varsta = 20;                // set
        Console.WriteLine(c.Nume + " " + c.Varsta); // get
    }
}
```

Fiecare proprietate returnează și eventual setează și valoarea unui câmp privat. S-au implementat două proprietăți: **Nume** și **Varsta**. Toate proprietățile au tip.

Prototipul proprietății este format din tipul returnat de către proprietate (aici **string**, respectiv **int**) și numele proprietății (**Nume** și **Varsta**). Tipul proprietății, este tipul câmpului pe care îl gestionează proprietatea.

Corpul proprietății conține doi accesorii: **get** și **set**. Unul din ei poate să lipsească.

- **get** are următoarele caracteristici:
 - ✓ Nu are parametri
 - ✓ Are tip de retur de același tip cu al proprietății.
- **set** are următoarele caracteristici:
 - ✓ Are un singur parametru implicit, numit **value**, de același tip cu proprietatea. **value** reține valoarea câmpului gestionat de proprietate.
 - ✓ Are tipul de retur **void**.



Accesorii pot fi declarați în orice ordine. Nici o altă metodă în afară de **get** și **set** nu poate exista în corpul proprietății. Accesorii se apelează în mod implicit.

Citirea și scrierea într-o proprietate se face la fel ca pentru un câmp public:

- Expresia **c.Nume = "Andrei"**; atribuie câmpului privat **nume** valoarea "Andrei" în felul următor: **set** se apelează în mod implicit, iar parametrul său implicit **value**, primește valoarea "Andrei". În corpul metodei, prin **nume = value**; are loc atribuirea necesară.
- Expresia **c.Nume** (scrisă în contextul unei afișări) returnează valoarea proprietății, deci a câmpului **nume**, astfel: **get** se apelează în mod implicit. Metoda **get** returnază valoarea proprietății, deci a câmpului privat **nume**.

Proprietățile nu încalcă principiul protecției datelor

Aparent, un câmp declarat **public** (o practică descurajată în OOP) se accesează în mod identic cu o proprietate asociată unui câmp privat, iar proprietatea pare că nu protejează câmpul de acțiuni nedorite. Nu este de loc așa.

În primul rând aveți libertatea să creați proprietăți:

- **Read-Only**, adică proprietăți care nu implementează metoda **get**.
- **Write-Only**, adică proprietăți care nu implementează metoda **set**.
- Nu puteți defini o proprietate din care să lipsească atât **get** cât și **set**.

În al doilea rând, metoda `set` se poate implementa de asemenea manieră, încât câmpul asociat proprietății (*backing field*) să fie complet protejat de acțiuni nedorite.

Exemplu:

În `set` puteți plasa orice cod defensiv de protecție a câmpului asociat proprietății:

```
using System;

class Intreg
{
    private int n;           // Câmp privat
    public int N
    {
        get { return n; }

        set
        { // Cod care protejează n de acțiuni nedorite
            if (value > 1000) n = 1000;
            else
                if (value < 0) n = 0;
                else n = value;
        }
    }
}

class Test
{
    static void Main(string[] args)
    {
        Intreg i = new Intreg();
        i.N = 1200;           // set
        Console.Write(i.N + " "); // get
        i.N = -10;           // set
        Console.Write(i.N);   // get
    }
}
```

Înșire: 1000 0

Proprietăți statice

Proprietățile se pot declara `static`. Procedați astfel când doriți să le asociați unor câmpuri statice. Proprietățile statice nu pot accesa alți membrii ai clasei, decât pe cei statici și există independent de existența instanțelor clasei. Se accesează prin numele clasei, la fel ca alți membri statici.

Exemplu:

```
class Numar
{
    private static int nr;           // Câmp static
    public static int Nr             // Proprietate statică
    {
        get { return nr; }
        set { nr = value; }
    }
}

class TestConst
{
    static void Main(string[] args)
    {
        Numar.Nr = 100;    // set
        System.Console.Write(Numar.Nr); // get
    }
}
```

Convenție de notare

Pentru claritate, se obișnuiește ca numele proprietății să fie același cu cel al câmpului privat asociat. Diferența este că în timp ce câmpul începe cu literă mică, proprietatea începe cu literă mare. Sfatul nostru este să urmați această convenție.

Proprietăți implementate în mod automat

Aceste tipuri de proprietăți, au următoarele caracteristici:

- Nu se asociază nici unui câmp al clasei.
- Compilatorul alocă memorie pentru valorile furnizate, în funcție de tipul proprietății.
- Accesorii **get** și **set** nu au corp.
- Nu pot fi *read-only* (nu poate lipsi **get**) și nici *write-only* (nu poate lipsi **set**).

Exemplu:

```
using System;

class X
{
    public int Valoare    // Proprietate
    {
        // implementată
        get; set;        // în mod automat
    }
}
```

```
class TestAutoPropertie
{
    static void Main(string[] args)
    {
        X x = new X();
        x.Valoare = 10;           // set
        Console.Write(x.Valoare); // get
    }
}
```

Veți utiliza o proprietate automată atunci când nu aveți nevoie să impuneți restricții speciale valorilor sale.

Restricțiile proprietăților

- Proprietățile nu definesc o locație de memorie. Din acest motiv, nu pot fi transmise cu modificatorii **out** sau **ref**.
- Nu pot fi supraîncărcate.

Indexatori

Indexatorii permit instanțelor unei clase sau ale unei structuri să fie indexate, la fel ca elementele unui tablou. Indexatorii și proprietățile sunt similari din mai multe puncte de vedere:

- Sintaxă asemănătoare.
- Pot avea unul sau doi accesorii.
- Nu definesc o locație de memorie.
- Codul accesoriilor poate să fie asociat sau să nu fie cu un câmp al clasei.

Există și **diferențe**:

- O proprietate accesează de regulă un singur câmp privat, în timp ce un indexator poate accesa mai mulți membrii ai clasei.
- Numele unui indexator este întotdeauna **this**.
- Un indexator este întotdeauna un membru de instanță. Din această cauză nu poate fi declarat **static**.

Sintaxă:

```
TipReturnat this[int index]
{
    get
    {
        // Se returnează valoarea precizată prin index
    }
    set
    {
        // Se modifică valoarea precizată prin index
    }
}
```

TipReturnat este tipul de bază al indexării. Parametrul *index* primește valoarea indexului elementului care va fi accesat.

Exemplu:

```
using System;

class Tablou
{
    private int[] a;          // Tablou conținut - Câmp privat
    private int n;           // Dimensiunea tabloului
    private const int MAX = 1000;

    public Tablou(int dim)    // Constructorul clasei
    {
        n = dim;
        a = new int[n];
    }

    public int this[int index] // Indexator
    {
        get    // Returnează a[index] doar dacă index este
        {      // în intervalul [0, MAX]
            if (index < 0)
                return a[0];
            else
                if (index > MAX)
                    return a[MAX];
                else
                    return a[index];
        }

        set    // Setează a[index] doar dacă index este
        {      // în intervalul [0, MAX]
            if (index < 0)
                a[0] = value;
            else
                if (index > MAX)
                    a[MAX] = value;
                else
                    a[index] = value;
        }
    }
}

class TestIndexer
{
    static void Main(string[] args)
    {
        Tablou t = new Tablou(100);
        for (int i = 0; i < 10; i++)
            t[i] = i;          // set
        t[-1] = 10;           // set
    }
}
```

```

        for (int i = 0; i < 10; i++)
            Console.Write(t[i] + " "); // get
    }
}

```

leșire:

10 1 2 3 4 5 6 7 8 9

În exemplul de mai sus se aplică operatorul `[]` referinței `t`, accesându-se în felul acesta elementele tabloului `a`, conținut în clasă.

Dacă o clasă conține ca și câmp privat o colecție, iar acea colecție suportă operația de indexare, atunci puteți defini un indexator care vă va permite să accesați elementele colecției cu ajutorul operatorului `[]` aplicat referinței la obiectul de tipul clasei (mai sus, `t[i]`).

Operatori de conversie

În capitolul 3, paragraful *Conversii între tipuri*, am văzut că între tipurile predefinite se pot face conversii *implicite* sau *explicite*. Conversiile explicite se realizează cu ajutorul operatorului `( )`.

Exemplu:

```

double x = 2.3;
int y = 6;
x = y;           // Conversie implicită
y = (double)x;   // Conversie explicită

```

C# vă permite să definiți un tip special de operatori în clase și structuri, care realizează conversii implicite sau explicite între tipul clasei sau a structurii și alte tipuri predefinite sau tipuri `class` sau `struct`.

Operatorii de conversie sunt metode statice, introduse cu cuvântul cheie `operator`.

Sintaxă:

```

public static explicit operator Tip1 (Tip2 t)
{
    // Cod care convertește în mod explicit t spre TipulClasei
}

```

sau

```

public static implicit operator Tip1(Tip2 t)
{
    // Cod care convertește în mod implicit t spre TipulClasei
}

```

Tip1 este tipul spre care se face conversia. *Tip2* este tipul care se convertește spre *Tip1*. Unul dintre cele două tipuri trebuie să fie de tipul clasei sau structurii care conține operatorul de conversie.

Exemplul 1:

Definim doi operatori care realizează conversii explicite de la tipul **Bancnota** spre **int** și de la **int** spre tipul **Bancnota**:

```
// conversii_explicite.cs
using System;

class Bancnota
{
    private int valoare;

    public Bancnota(int v) // Constructor
    {
        valoare = v;
    }

    // Operator de conversie explicită de la int la Bancnota
    public static explicit operator Bancnota(int v)
    {
        return new Bancnota(v);
    }

    // Operator de conversie explicită de la Bancnota la int
    public static explicit operator int(Bancnota b)
    {
        return b.valoare;
    }
}

class TestConversieExplicita
{
    static void Main()
    {
        int val = 100;
        Bancnota b = (Bancnota)val; // Conversie explicită
        Console.WriteLine((int)b);  // Conversie explicită
    }
}
```

Exemplul 2:

Modificăm programul anterior, astfel încât să obținem operatori de conversii implicite:

```
// conversii_implicite.cs
using System;

class Bancnota
{
    private int valoare;

    public Bancnota(int v) // Constructor
    {
        valoare = v;
    }

    // Operator de conversie implicită de la int la Bancnota
    public static implicit operator Bancnota(int v)
    {
        return new Bancnota(v);
    }

    // Operator de conversie implicită de la Bancnota la int
    public static implicit operator int(Bancnota b)
    {
        return b.valoare;
    }
}

class TestConversieImplicita
{
    static void Main()
    {
        int val = 100;
        Bancnota b = val; // Conversie implicită
        Console.WriteLine(b); // Conversie implicită
    }
}
```

Observați că nu a mai fost necesară utilizarea operatorului () - *type cast operator*.

Clase interioare

O clasă interioară (*inner class* sau *nested class*) este o clasă a cărei definiție se găsește în interiorul definiției altei clase.

1. Pentru a instanția un obiect de tipul clasei interioare, nu este nevoie de un obiect de tipul clasei exterioare:

Exemplu:

```
using System;

class A // Clasa container
{
    public class B // Clasa interioară
    {
        public void F() { Console.Write("F()"); }
    }
}

class TestInnerClass
{
    static void Main()
    {
        A.B obj = new A.B(); // Creează un obiect de tip B
        obj.F(); // Afișează: F();
    }
}
```

2. Metodele clasei interioare pot accesa toți membrii clasei care o conține, prin intermediul operatorului `\.\'`:

Exemplu:

```
using System;

public class Exterior
{
    private int x;
    public Exterior(int y) // Constructor
    {
        x = y;
    }

    public class Interior
    {
        public void Scrie(Exterior o)
        {
            // Acces permis la membrii clasei Exterior
            System.Console.WriteLine(o.x);
        }
    }
}

public class TestInnerClass
{
    static void Main()
```

```

{
    Exterior o1 = new Exterior(100);

    // Instanțierea unui obiect de tip Interior
    Exterior.Interior o2 = new Exterior.Interior();
    o2.Scrie(o1);    // Afișează: 100
}
}

```

Conținere

Când o clasă are ca membri unul sau mai multe obiecte de tip clasă, vorbim despre **conținere** (*containment*) sau **compoziție**.

În figură, ilustrăm clasa **Calculator**. Un calculator are mai multe componente. Acestea sunt instanțe ale altor clase: **Monitor**, **Tastatura**, **Mouse** și **UnitateCentrala**.

IMPORTANT

Tipul de relație care se modelează în cazul conținerii este: **"HAS A"** (**ARE UN**, **ARE O**).

Exemplu:

Programul de mai jos definește trei clase: **Motor**, **Caroserie**, **Masina**. Clasa **Masina** are ca membri privați câte o referință spre obiecte de tip **Motor** și **Caroserie**.

```

class Motor { /* ... */ }

class Caroserie { /* ... */ }

class Masina
{
    private string marca;
    // Mașina ARE O caroserie c și ARE UN motor m
    private Caroserie c;
    private Motor m;
    // Constructorul clasei
    public Masina(Caroserie c, Motor m, string marca)
    {
        this.c = c;
        this.m = m;
        this.marca = marca;
    }
}

```

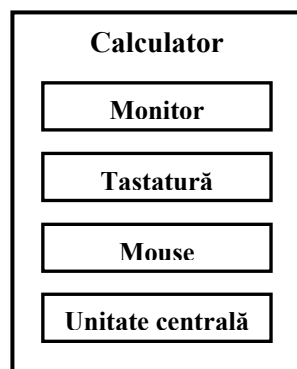


Figura 4.1 Conținere

```
class TestContinere
{
    public static void Main()
    {
        Caroserie C = new Caroserie();
        Motor M = new Motor();
        Masina mea = new Masina(C, M, "Volvo");
    }
}
```

Clase parțiale

Definițiile claselor, structurilor și interfețelor se pot împărți în mai multe fișiere sursă. Ca alternativă, puteți fragmenta o clasă în interiorul aceluiași fișier. Pentru a separa în blocuri o definiție de clasă, utilizați cuvântul cheie **parțial**.

Exemplu:

```
using System;

partial class A
{
    public void F()
    {
        System.Console.Write ("F() ");
    }
}

partial class A
{
    public void G()
    {
        System.Console.WriteLine("G()");
    }
}

public class TestPartialClass
{
    static void Main()
    {
        A a = new A();
        a.F();
        a.G();
    }
}
```

Înșirarea programului : F() G()

Practic, este vorba despre o singură clasă cu numele **A**, a cărei definiție a fost fragmentată. Compilatorul unifică definițiile parțiale și instanțiază în mod corect obiectul **a** de tip **A**.

Mediul *Visual C# 2008* generează în mod curent asemenea definiții parțiale. Dacă creai un proiect de tip *Windows Forms*, vei constata că definiția de clasă a formei principale, să-i spunem **Form1**, este fragmentată în două fișiere diferite: *Form1.cs* și *Form1.Designer.cs*.

Clase sigilate

sealed este un cuvânt cheie. Când modificatorul **sealed** este aplicat unei clase, are efectul de a împiedica alte clase să o moștenească.

Exemplu:

```
sealed class A { /*...*/}  
  
class B : A { /*...*/ }    // Eroare
```

Supraîncărcarea operatorilor

Toți operatorii unari și binari au implementări predefinite care sunt disponibile în orice expresie în care sunt implicate tipuri predefinite. C# vă permite să redefinești semnificația unui operator standard, astfel încât operatorul respectiv să poată fi aplicat instanțelor clasei voastre. În acest fel sporii funcționalitatea claselor.

Ce înțelegem prin supraîncărcarea operatorilor (*operator overloading*)? Este un mecanism prin care instanțele clasei pot fi integrate în expresii aritmetice sau logice în calitate de operanzi, cu utilizarea operatorilor specifici tipurilor predefinite: +, -, *, /, <, >, etc. Când definești o metodă de tip operator, de exemplu pentru operația de adunare, atunci spunem că “am supraîncărcat operatorul +”.

Să presupunem că avem o clasă care descrie o fracție și dorim să efectuăm operații cu instanțe de tip **Fractie**, în felul acesta:

```
Fractie f1 = new Fractie();  
Fractie f2 = new Fractie();  
Fractie f = f1 + f2;
```

În C# operatorii supraîncărcați sunt metode statice ai cărui parametri sunt operanzii, iar valoarea returnată este rezultatul operației.

Sintaxa de declarare a unui operator pretinde folosirea cuvântului cheie **operator**, urmat de simbolul operatorului care se redefineste.

Sintaxa

În cazul operatorilor binari:

```
public static Tip operator op(Tip1 operand1, Tip2 operand2)
{
    // operații
}
```

În cazul operatorilor unari:

```
public static Tip operator op(Tip operand)
{
    // operații
}
```

op este operatorul care se supraîncarcă: +, -, /, %, etc.

Pentru operatorii unari, operandul trebuie să fie de același tip cu clasa pentru care supraîncărcăți operatorul. Pentru operatorii binari, cel puțin unul dintre cei doi operanzi trebuie să fie de tipul clasei. *Tip* este tipul de retur al operației și de regulă este un obiect de tipul clasei.

Supraîncărcarea operatorilor binari

În exemplul de mai jos, clasa **Fractie**, supraîncarcă operatorii +, ==, !=:

```
// fractie.cs
using System;

public class Fractie
{
    private int numarator;
    private int numitor;
    private static int Cmmdc(int a, int b)
    {
        if (b == 0) return a;
        return Cmmdc(b, a % b);
    }
    public Fractie(int numarator, int numitor) // Constructor
    {
        int div = Cmmdc(numarator, numitor);
        numarator /= div; numitor /= div;
        this.numarator = numarator;
        this.numitor = numitor;
    }

    public static Fractie operator +(Fractie f1, Fractie f2)
    {
        int A = f1.numarator * f2.numitor +
            f2.numarator * f1.numitor;
```

```

        int B = f1.numitor * f2.numitor;
        int cmmdc = Cmmdc(A, B);
        A /= cmmdc;
        B /= cmmdc;
        return new Fractie(A, B);
    }

    public static bool operator ==(Fractie f1, Fractie f2)
    {
        if (f1.numitor == f2.numitor &&
            f1.numarator == f2.numarator)
            return true;
        return false;
    }

    public static bool operator !=(Fractie f1, Fractie f2)
    {
        return !(f1 == f2);
    }

    // Suprascrie Object.ToString()
    public override string ToString()
    {
        String s = numarator.ToString() + "/" +
            numitor.ToString();
        return s;
    }
}

public class TestFractie
{
    static void Main()
    {
        Fractie a = new Fractie(3, 4);
        Console.WriteLine("a = {0}", a.ToString());
        Fractie b = new Fractie(2, 4);
        Console.WriteLine("b = {0}", b.ToString());

        Fractie c = a + b; // operator+
        Console.WriteLine("c = a + b = {0}",
            c.ToString());
        Fractie d = new Fractie(2, 4);

        if (d == b) // operator==
            Console.WriteLine("d = b = {0}", d);

        if (a != b) // operator!=
            Console.WriteLine(a + " != " + b);

        a += b; // operator +=
    }
}

```

```
        Console.WriteLine("a = " + a);  
    }  
}
```

leșire:

```
a = 3/4  
b = 1/2  
c = a + b = 5/4  
d = b = 1/2  
3/4 != 1/2  
a = 5/4
```

De reținut:

- Operatorul de atribuire = nu se supraîncarcă. El este furnizat în mod implicit de către compilator (`c = a + b`).
- Operatorii unari au un parametru, iar cei binari doi parametri: în ambele cazuri, cel puțin unul dintre operanzi trebuie să fie de tipul clasei sau al structurii.
- Operatorii unari au un singur parametru.
- C# pretinde ca în situația în care supraîncărcați `==` atunci trebuie să supraîncărcați și `!=`. Similar, operatorii `<` și `>` trebuie supraîncărcați în perechi, la fel ca și `<=` cu `>=`.
- Dacă supraîncărcați operatorul `+`, atunci C# sintetizează automat `+=`. Aceeași regulă este valabilă și pentru `+` cu `+=`, `*` cu `*=`, etc.

Metoda ToString()

În clasa **Fractie**, am definit metoda **ToString()**. De fapt, am redefinit pentru **Fractie**, această funcție. **ToString()** este o metodă a clasei **object**, clasa de bază a tuturor claselor. Rolul ei este de a returna o reprezentare a obiectului clasei într-un obiect de tip **string**. De câte ori doriți o conversie particulară spre **string** a obiectelor clasei, redefiniți această metodă moștenită. Pentru aceasta folosiți cuvântul cheie **override**. Observați că metoda **ToString()** se apelează în mod implicit atunci când transferați obiectul într-un context în care se cere un string: apelul `Console.WriteLine("a = " + a);` pune `a` într-un context în care se așteaptă un string, deci `a` se convertește la **string** prin apelul implicit al metodei **ToString()**.

Supraîncărcarea operatorilor unari

Pentru exemplificarea supraîncărcării operatorilor unari definim o clasă simplă:

```
using System;  
  
public class Intreg  
{
```

```
private int n;
public Intreg(int i)    // Constructor
{
    n = i;
}
// Operatorul de incrementare
public static Intreg operator ++(Intreg x)
{
    return new Intreg(++x.n);
}
// Operatorul unar -
public static Intreg operator -(Intreg x)
{
    return new Intreg(-x.n);
}
public override string ToString()
{
    String s = string.Format("{0}", n);
    return s;
}
}

public class TestOpUnari
{
    static void Main()
    {
        Intreg x = new Intreg(10);
        x++;    // Incrementare
        Console.WriteLine(x + " " + -x); // Scrie: 11 -11
    }
}
```

NOTĂ:

Nu este nevoie să creați două versiuni diferite ale operatorului ++ ca să suporte incrementare prefixată și sufixată. O singură versiune este suficientă, iar compilatorul are grijă să implementeze diferențierea între prefixare și sufixare.

IMPORTANT

Este recomandabil să nu modificați operanzii pasați metodelor operator. În loc de aceasta, creați noi instanțe de tipul valorii de retur și returnați aceste instanțe. Urmând această practică, veți evita probleme la depanare.

Operatorii care pot fi supraîncărcați:

Operatori unari: +, -, !, ~, ++, --, true, false

Operatori binari: +, -, *, /, %, &, |, ^, >>, <<

Operatori relaționali: == cu !=, < cu >, <= cu >= (aceștia trebuie supraîncărcați în perechi).

Acțiuni nepermise la supraîncărcarea operatorilor:

- Crearea unui nou operator (se pot supraîncărca doar operatori predefiniți).
- Schimbarea sintaxei unui operator.
- Redefinirea modului de lucru a unui operator cu tipuri predefinite.
- Schimbarea precedenței sau asociativității unui operator.

Structuri

Structurile sunt tipuri de date definite de programator, asemănătoare claselor. Se definesc cu ajutorul cuvântului cheie **struct**.

Structurile sunt asemănătoare claselor prin faptul că pot să conțină câmpuri, metode, constructori, proprietăți, operatori, tipuri imbricate, indexatori.

Diferă de clase în următoarele privințe:

- Structurile sunt *tipuri valoare*, iar clasele sunt *tipuri referință*.
- Nu suportă moștenirea.
- Nu suportă constructori fără parametri.
- Nu au destructori.

Sintaxa:

```
[ModificatorAcces] struct NumeStructură
{
    // membrii structurii
}
```

Exemplu:

```
using System;

struct Punct
{
    private double x;
    private double y;

    public Punct(double _x, double _y) // Constructor
    {
        x = _x;
        y = _y;
    }

    public double X // Proprietatea X
    {
        get { return x; }
    }
}
```

```

        set { x = value; }
    }
    public double Y                                // Proprietatea Y
    {
        get { return y; }

        set { y = value; }
    }
}

class TestStruct
{
    static void Main(string[] args)
    {
        Punct p1 = new Punct(2.3, 3.5);
        Console.WriteLine("x = {0}, y = {1}", p1.X, p1.Y);
    }
}

```

Structurile sunt recomandabile pentru obiecte mici, așa cum este un obiect de tip **Punct**, care trebuie instanțiate în număr mare, eventual într-o buclă. Pentru că sunt tipuri valoare, se construiesc pe stiva programului, care se accesează mai rapid decât memoria *Heap*.

Interfețe

O interfață este un tip referință care descrie un set de metode, dar nu le implementează. Clasele și structurile pot implementa interfețele. Când o clasă implementează o interfață, trebuie să implementeze toate metodele acelei interfețe. În felul acesta, clasa “semnează un contract”, pe care se obligă să-l respecte.

Sintaxa minimală:

```

[modificatori] interface NumeInterfață
{
    // Corpul interfeței
}

```

Modificatorii pot fi **public**, **private**, **protected**, **internal**, **protected internal**. Cuvântul cheie **interface** precedă numele interfeței. Este o practică comună ca numele interfeței să înceapă cu litera **I**. Exemple: **IComparable**, **ICloneable**, **IControl**, etc.

Implementarea interfețelor de către clase

O clasă sau o structură poate implementa una sau mai multe interfețe, ca mai jos:

```
interface I1 { /*...*/ }
interface I2 { /*...*/ }

class C : I1, I2
{ /*...*/ }
```

Exemplu:

```
using System;

interface IPrintable
{
    void Print(string s);
}

public class Mail : IPrintable
{
    private string s;
    public Mail(string s)      // Constructor
    {
        this.s = s;
    }
    // Implementarea metodei interfeței (obligatoriu!)
    public void Print(string a)
    {
        Console.WriteLine(s + a);
    }
}

public class TestOpUnari
{
    static void Main()
    {
        Mail m = new Mail("Prietenilor mei ");
        m.Print("Salut!");
    }
}
```

leșire:

Prietenilor mei Salut!

Ceea ce trebuie să rețineți despre interfețe este:

- O interfață nu se poate instanția.
- Interfețele nu conțin câmpuri, ci doar metode, proprietăți, evenimente.
- Interfețele nu conțin implementări ale metodelor.
- Clasele și structurile pot moșteni (impelmenta) una sau mai multe interfețe.
- O interfață poate la rândul ei să moștenească o altă interfață.

Moștenire

Moștenirea este unul dintre cele mai importante și mai puternice concepte în *Programarea Orientată pe Obiecte*. Moștenirea vă permite să definiți o nouă clasă care încorporează și extinde o clasă existentă.

Diagrama de mai jos prezintă o ierarhie de clase bazată pe moștenire. Un animal **este o** ființă, un om **este o** ființă, un câine **este un** animal, un bărbat **este** om, ș.a.m.d. Vom generaliza cu afirmația următoare:

Moștenirea modelează relația **IS A (ESTE UN, ESTE O)**.

Spunem că **Om** și **Animal** moștenesc clasa **Ființă**. Ele sunt **clase derivate** din clasa **Ființă**, iar aceasta din urmă este **clasă de bază** pentru **Om** și **Animal**. Un obiect de tipul clasei derivate, conține ca **subobiect**, un obiect de tipul

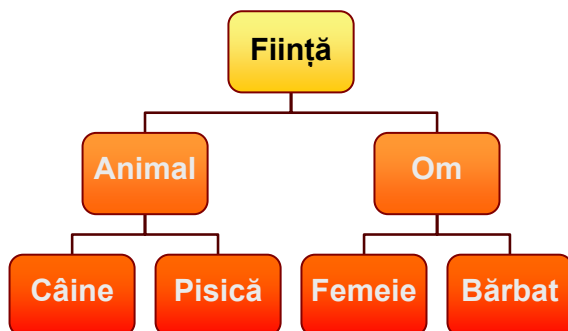


Figura 4.2 Ierarhie de clase bazată pe moștenire

clasei de bază. Așadar, un obiect de tip **Om** conține ca subobiect un obiect de tip **Ființă**, iar un obiect de tip **Femeie**, conține un subobiect de tip **Om**, care la rândul său include un subobiect de tip **Ființă**.

Specializare și generalizare

Una dintre cele mai importante relații între obiecte în lumea reală este **specializarea**, care poate fi descrisă ca o relație **IS A**. Când spunem că un câine **este un** animal, ne gândim de fapt că un câine este un tip specializat de animal. Câinele **este un** animal pentru că are toate caracteristicile unui animal, dar specializează aceste caracteristici conform speciei sale. O pisică **este un** animal, deci are în comun cu câinele caracteristicile animalelor (ochi, gură, etc) dar diferă față de un câine prin caracteristici specifice pisicilor.

Pe de altă parte, tipul **Animal**, **generalizează** tipurile **Pisică** și **Câine**. Aceste relații sunt ierarhice. Ele formează un arbore. Urcând în arbore, generalizăm; coborând, specializăm.

Moștenirea presupune atât **specializare** cât și **generalizare**. În C#, dubla relație de specializare și de generalizare este implementată folosind principiul moștenirii.

Implementarea moștenirii

Fie clasa **A** care moștenește clasa **B**. Operatorul `:` exprimă relația de moștenire:

```
class B          // B - Clasa de bază
{
    // Membrii clasei B
}

class A : B      // A - Clasa derivată
{
    // Membrii clasei A
}
```

Sintaxa **A : B** se numește *specificație de clasă de bază (base class specification)*.

Membrii clasei derivate sunt:

- Membrii definiți în propria clasă.
- Membrii clasei de bază.

Se spune despre o clasă că **extinde** clasa sa de bază, pentru că include membrii clasei de bază, plus orice caracteristică și funcționalitate suplimentară furnizată de propria declarație.

Clasa **B** din figura 4.2 are un câmp și o metodă. Clasa **A**, în dreapta, își definește proprii membri: un câmp și o metodă și în plus moștenește clasa **B**, deci are un câmp și o metodă suplimentare pentru că un obiect de tip **A** include un subobiect de tip **B**.

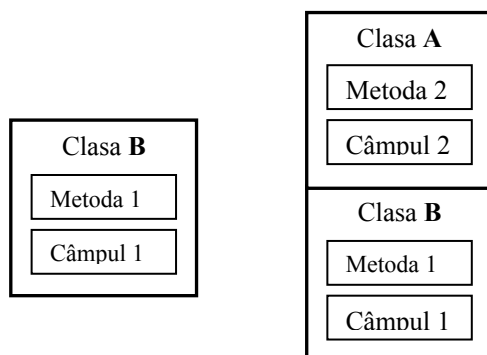


Figura 4.3. Clasa A moștenește clasa B

IMPORTANT

În C# toate clasele sunt clase derivate. Oricare clasă derivă în mod direct sau indirect din clasa `object`. Dacă o clasă nu specifică în mod explicit derivarea dintr-o altă clasă, atunci ea moștenește direct clasa `object`. În felul acesta, `object` este baza tuturor ierarhiilor de clase.

Exemplu:

Cele două declarații ale clasei `Avion` specifică același lucru: faptul că `Avion` derivă din `object`:

```
// Derivare implicită           // Derivare explicită
// din object                  // din object
class Avion                    class Avion : object
{
    // membrii clasei          // membrii clasei
}
```

Alte câteva aspecte trebuie reținute în legătură cu moștenirea în C#:

➤ **O clasă poate moșteni o singură altă clasă și oricâte interfețe**

În C# o clasă poate moșteni o singură clasă de bază. Acest tip de moștenire se numește **moștenire singulară** (*single inheritance*).

O clasă C# poate moșteni în schimb, oricâte interfețe.

Fie clasele `A` și `B` și interfețele `I1`, `I2`, `I3`. Dacă `A` moștenește `B` și interfețele `I1`, `I2`, `I3`, atunci în lista care specifică relația de moștenire, clasa `B` trebuie să fie precizată înaintea interfețelor:

```
// Corect                      // Incorect
class A : B, I1, I2, I3        class A : I1, B, I2, I3
{
}                               {
}
```

➤ **C# nu admite moștenirea multiplă**

Moștenirea multiplă (*multiple inheritance*) are loc atunci când o clasă are mai multe clase de bază. Dintre limbajele importante, doar C++ suportă acest tip de moștenire.

```
// Incorect în C#
class A : B, C, D
{
}
```

➤ **Înălțimea arborelui relațiilor de moștenire poate fi oricât de mare**

Mai jos, `A`, `B`, și `C` sunt clase. Rădăcina ierarhiei este `object`. Din `object` derivă `D`, din `D` derivă `C`, din `C` derivă `B`, iar din `B` derivă `A`:

```
// Corect
class C : D { }
class B : C { }
class A : B { }
```

Accesarea membrilor moșteniți

O instanță a unei clase moștenește toți membrii clasei de bază, cu excepția constructorilor. Totuși accesul la membrii clasei de bază poate fi restricționat, cu ajutorul *modificatorilor de acces*. Aceștia sunt: **public**, **private**, **protected**, **internal** și **protected internal**. Programul următor, testează modul în care modificatorii de acces controlează modul de acces la membrii clasei de bază.

```
using System;
```

```
class Animal      // Clasa de bază
{
    private string hrana;           // Câmpuri
    protected int  speranta_viata;
    public string  Hrana            // Proprietate
    {
        set { hrana = value; }
        get { return hrana; }
    }
    protected void SeHraneste()    // Metodă
    {
        Console.WriteLine("Animalul se hraneste");
    }
}

class Pisica : Animal              // Pisica - clasă derivată
{
    private string rasa;
    public string  Rasa
    {
        set { rasa = value; }
        get { return rasa; }
    }
    public void Toarce()
    {
        /* Membrii protejați și cei publici ai clasei de bază
           pot fi accesați din metodele clasei derivate */

        // hrana = "lapte";          // Eroare! -câmp privat
        speranta_viata = 12;         // Corect! -câmp protejat
        Hrana = "lapte";             // Corect! -proprietate publică
        SeHraneste();                // Corect! -metodă protejată
        Console.WriteLine("Pisica toarce");
    }
}
```

```

public class TestAccesMembri
{
    static void Main()
    {
        Animal a = new Animal();
        // a.hrana = "carne";           // Eroare! -câmp privat
        // a.speranta_viata = 10;       // Eroare! -câmp protejat
        a.Hrana = "carne";           // Corect! -proprietate publică
        // a.SeHraneste();             // Eroare! -metodă protejată

        /* Toți membrii clasei Animal sunt membri ai clasei
           Pisica. Membrii private și protected nu pot fi
           accesați din exteriorul clasei Pisica */

        Pisica p = new Pisica();
        // p.hrana = "lapte";           // Eroare! -câmp privat
        // p.speranta_viata = 12;       // Eroare! -câmp protejat
        p.Hrana = "lapte";           // Corect! -proprietate publică
        // p.SeHraneste();             // Eroare! -metodă protejată
        // p.rasa = "siameza";         // Eroare! -câmp privat
        p.Rasa = "Siameza";         // Corect! -proprietate publică
        p.Toarce();                 // Corect! -metodă publică
    }
}

```

leșire:

Animalul se hraneste
Pisica toarce

Se desprind următoarele reguli:

- Metodele unei clase au acces la toți membrii declarați în acea clasă, indiferent de nivelul de protecție a membrilor.
- Metodele clasei derivate pot accesa membrii publici și pe cei protejați ai clasei de bază.
- Metodele clasei derivate nu pot accesa membrii privați ai clasei de bază.
- Din exteriorul clasei derivate și a celei de bază, se pot accesa numai membrii publici ai clasei de bază. Membrii protejați se comportă în acest caz la fel ca cei privați.

Membrii unei clase marcați cu modificatorii **internal**, sunt vizibili (accesibili) pentru toate clasele din același fișier sau din același *assembly*.

Membrii unei clase marcați **protected internal** sunt vizibili tuturor care o moștenesc și în plus, tuturor claselor din același *assembly*.

Constructorii claselor derivate

Constructorii clasei de bază nu se moștesc. Constructorul clasei derivate apelează în schimb constructorul bazei, pentru a construi porțiunea din obiect specifică bazei.

Exemplu:

```
using System;

class Baza
{
    public Baza()
    {
        Console.WriteLine("Constructor Baza");
    }
}

class Derivat : Baza
{
    public Derivat()
    {
        Console.WriteLine("Constructor Derivat");
    }
}

public class TestConstructor
{
    static void Main()
    {
        Derivat d;
        d = new Derivat(); // Apelul constructorului clasei
                           // derivate
    }
}
```

leșire:

```
Constructor Baza
Constructor Derivat
```

Exemplul de mai sus pune în evidență faptul că la construirea unei instanțe a clasei, constructorul `Derivat()` execută constructorul clasei `Baza` înaintea executării propriului cod. Are loc un *apel implicit* al constructorului fără parametri `Baza()`.

Limbajul C# permite și **apelul explicit** al constructorului bazei. Sintaxa corespunzătoare este:

```
public Derivat() : base()
{
}
```

Dacă constructorul bazei are parametri, atunci *apelul implicit* nu poate avea loc. Constructorul derivat trebuie să invoce în mod explicit constructorul bazei, furnizându-i și argumentele corespunzătoare:

Exemplu:

```
using System;

class Baza
{
    private int x;
    public Baza(int y)
    {
        x = y;
    }
}

class Derivat : Baza
{
    private char c;
    public Derivat(char a, int b) : base(b)
    {
        c = a;
    }
}

public class TestConstructor
{
    static void Main()
    {
        Derivat d = new Derivat('T', 10);
    }
}
```

Constructorul clasei derivate trebuie să aibă suficienți parametri pentru a inițializa și câmpurile bazei.

Cum se construiește o instanță a clasei derivate ?

Când se invocă constructorul clasei derivate, ordinea construcției instanței este:

1. Inițializarea membrilor de tip instanță a clasei derivate.
2. Apelul constructorului clasei de bază.
3. Executarea corpului constructorului clasei derivate.

În cazul în care lanțul ierarhic al moștenirii conține mai multe clase, atunci fiecare constructor își execută mai întâi constructorul bazei sale înaintea propriului corp.

Membrii ascunși

Dacă în clasa derivată aveți un câmp cu același nume cu al unuia din clasa de bază, sau aveți o metodă cu aceeași *signatură* (același nume și aceeași listă de parametri formali) cu a uneia din clasa de bază, atunci numele membrilor din clasa de bază sunt ascunse metodelor clasei derivate. Pentru a accesa membrii ascunși ai bazei, se întrebuițează **new** în fața membrilor bazei și cuvântul cheie **base**, pentru accesarea membrilor ascunși.

Exemplu:

```
using System;
```

```
class Baza
```

```
{
    public int camp = 10;
    public void Metoda()
    {
        Console.WriteLine("Baza.Metoda()");
    }
}
```

```
class Derivat : Baza
```

```
{
    new public int camp = 20;
    new public void Metoda()
    {
        base.Metoda();
        Console.WriteLine("Derivat.Metoda() " + base.camp);
    }
}
```

```
public class TestMembriAscunsi
```

```
{
    static void Main()
    {
        Derivat d = new Derivat();
        d.Metoda();
    }
}
```

Ieșire:

```
Baza.Metoda()
Derivat.Metoda() 10
Baza.Metoda()
```

Polimorfism

Polimorfismul este unul dintre conceptele fundamentale ale programării orientate pe obiecte. Reprezintă caracteristica unei entități de a se comporta în moduri diferite, în funcție de context. În particular, este caracteristica unei variabile referință de a putea referi obiecte de tipuri diferite. C# admite polimorfismul bazat pe moștenire. Acest tip de polimorfism vă permite să invocați *runtime* metode ale claselor derivate cu ajutorul unei referințe la clasa de bază.

Conversia referințelor

Dacă aveți o referință la un obiect al clasei derivate, puteți obține o referință la partea de bază a obiectului, folosind operatorul de conversie () ca mai jos:

```
class Baza
{
    public void Metoda()
    {
        System.Console.WriteLine("Baza.Metoda()");
    }
}

class Derivat : Baza
{
    public void Metoda() // sau new public void Metoda()
    {
        System.Console.WriteLine("Derivat.Metoda()");
    }
}

public class TestConversieRef
{
    static void Main()
    {
        Derivat d = new Derivat();
        d.Metoda();

        // Upcast
        Baza b = (Baza)d;
        b.Metoda();
    }
}

leșire:
Derivat.Metoda()
Baza.Metoda()
```

Se constată că referința la partea de bază a obiectului nu “poate vedea” restul obiectului clasei derivate, deoarece “privește” printr-o referință **b** la clasa de bază. Prin **b** veți putea să invocați numai **Metoda()** clasei de bază.

Conversia unei referințe a unui obiect derivat spre o referință la clasa de bază se numește **upcast**.

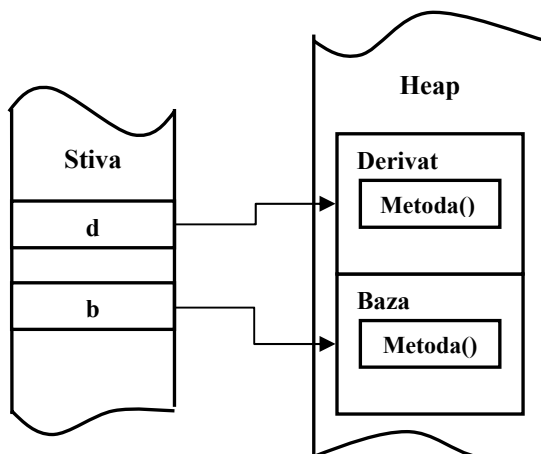


Figura 4.4. Conversia referințelor

Metode virtuale

În paragraful anterior am arătat că într-o clasă derivată puteți defini o metodă cu același prototip cu al unei metode din clasa de bază, dar că o referință la un obiect de tipul clasei de bază nu poate invoca noua metodă definită cu **new**.

Pentru ca o referință la un obiect al bazei să poată accesa membri ai obiectelor derivate, altfel spus, pentru a obține polimorfism bazat pe moștenire, este nevoie de următoarele:

- Clasa de bază declară acel membru **virtual**.
- Clasa derivată redefinește acel membru, cu exact aceeași semnătură și tip de retur, adăugând cuvântul cheie **override**, înaintea tipului de retur.

Exemplu:

```
using System;
namespace Polimorfism
{
    class Baza
    {
        virtual public void Metoda()
        {
            Console.WriteLine("Metoda() din Baza");
        }
    }
    class Derivat : Baza
    {
        override public void Metoda()
        {
            Console.WriteLine("Metoda() din Derivat");
        }
    }
    class TestPolimorfism
    {
        static void Main(string[] args)
```

```

    {
        Derivat d = new Derivat();
        d.Metoda(); // Ieșire: "Metoda() din Derivat"
        // Polimorfism
        Baza b = new Baza();
        b.Metoda(); // Ieșire: "Metoda() din Baza"
        b = new Derivat();
        b.Metoda(); // Ieșire: "Metoda() din Derivat"
    }
}

```

Primul apel `b.Metoda()`; invocă metoda din clasa de bază, deoarece tipul referinței este `Baza` și tipul obiectului referit este `Baza`. Al doilea apel invocă metoda redefinită în clasa derivată. Referința `b` este în continuare de tip `Baza`, dar tipul obiectul referit este de tipul clasei derivate. Acesta este manifestarea polimorfismului bazat pe moștenire.

În cazul unui lanț de derivare care pornește de la o clasă de bază cu metode virtuale, în clasele derivate aveți opțiunea de a redefini sau nu acele metode.

Exemplu:

```

using System;
class A
{
    virtual public void F()
    { Console.WriteLine("F() din A"); }
}

class B : A { } // B nu redefinește F()
class C : B
{
    override public void F()
    { Console.WriteLine("F() din C"); }
}

class D : C { } // D nu redefinește F()

class Test
{
    static void Main(string[] args)
    {
        A a = new B(); // a referă un obiect de tip B
        a.F();         // Ieșire: "F() din A"
        a = new C();   // a referă un obiect de tip C
        a.F();         // Ieșire: "F() din C"
        a = new D();   // a referă un obiect de tip D
        a.F();         // Ieșire: "F() din C"
    }
}

```

Dacă o referință de tipul clasei de bază referă un obiect de tip derivat și prin aceea referință invocați o metodă virtuală, atunci se apelează cea mai apropiată metodă **override** definită pe lanțul ierarhic. Dacă nu există metode redefinite, atunci se invocă metoda virtuală din clasa de bază.

Restricții

Reținem următoarele aspecte:

- Se pot declara **virtual** următorii membri: **metode**, **proprietăți**, **evenimente** și **indexatori**.
- Câmpurile nu pot fi declarate **virtual**.
- Metodele redefinite cu **override** trebuie să aibă același nivel de accesibilitate cu metodele virtuale corespunzătoare. De exemplu nu este corect ca o metodă virtuală declarată **public** să fie redefinită cu o metodă declarată **private**.
- Nu se pot redefini metodele *non-virtuale*.
- Nu pot fi redefini metodele statice.

Modificatorul **sealed**

Aplicat unei clase, modificatorul **sealed** împiedică alte clase să o moștenească. **sealed** poate fi aplicat și unei metode sau unei proprietăți care redefinește o metodă sau o proprietate a clasei de bază. În această situație, se permite altor clase să moștenească clasa care are un membru **sealed**, dar metoda sau proprietatea **sealed** nu mai poate fi redefinită în clasele derivate.

Exemplu:

```
class A
{
    protected virtual void F() { /*...*/ }
    protected virtual void G() { /*...*/ }
}

class B : A
{
    sealed protected override void F() { /*...*/ }
    protected override void G() { /*...*/ }
}

class D : B    // Corect. B poate fi moștenită
{
    protected override void F() { /*...*/ }    // Eroare !
    protected override void G() { /*...*/ }    // Corect
}
```

Când se aplică unei metode sau unei proprietăți, modificatorul **sealed** trebuie să fie folosit întotdeauna cu **override**.

Utilitatea polimorfismului

În practică se operează deseori cu colecții de obiecte. Mai precis, cu colecții de referințe la obiecte. Dacă obiectele sunt legate printr-o relație de moștenire având o clasă de bază comună, atunci nu trebuie ca ele să fie de același tip. Dacă toate obiectele redefinesc o metodă virtuală a clasei de bază, atunci puteți invoca această metodă pentru fiecare obiect.

```
using System;
class LimbaVorbitor // Clasa de bază
{
    virtual public void SeVorbeste() { } // nu va fi invocată
}
class Engleza : LimbaVorbitor
{
    override public void SeVorbeste()
    { Console.WriteLine("Engleza"); }
}
class Franceza : LimbaVorbitor
{
    override public void SeVorbeste()
    { Console.WriteLine("Franceza"); }
}
class Spaniola : LimbaVorbitor
{
    override public void SeVorbeste()
    { Console.WriteLine("Spaniola"); }
}
class Catalana : Spaniola
{
    override public void SeVorbeste()
    { Console.WriteLine("Catalana"); }
}
}
class Test
{
    static void Main(string[] args)
    {
        LimbaVorbitor[] L = new LimbaVorbitor[4];
        L[0] = new Engleza(); L[1] = new Franceza();
        L[2] = new Spaniola(); L[3] = new Catalana();

        foreach (LimbaVorbitor lv in L)
            lv.SeVorbeste();
    }
}
leșire:
Engleza
Franceza
Spaniola
Catalana
```


Tabloul **LimbaVorbita** reține referințe ale unor obiecte de tipuri diferite, dar care sunt legate prin relație de moștenire, având ca bază a ierarhiei clasa **LimbaVorbita**. Dacă relația de moștenire nu ar exista, atunci referințele ar fi trebuit să fie de același tip. Programul implementează polimorfismul. Interfața de apelare unică (**lv.SeVorbeste()**) care se utilizează pentru fiecare obiect, duce la acțiuni specifice, în funcție de obiectul referit.

Rezumatul capitolului

- Clasele sunt entități logice care modelează obiecte din lumea reală sau obiecte abstracte. Clasele încapsulează **date** și **metode** care operează cu aceste date.
- **Constructorii** unei clase sunt metode speciale care au rolul de a crea, a construi obiecte.
- **Destructorul** clasei este o metodă specială care distruge instanțele claselor.
- **Proprietățile** sunt membri ai clasei cu ajutorul cărora se accesează câmpurile private ca și când ar fi fost declarate public, fără ca prin aceasta să se încalce principiul protecției datelor.
- **Supraîncărcarea operatorilor** este mecanismul prin care instanțele clasei pot fi integrate în expresii aritmetice sau logice în calitate de operanzi, cu utilizarea operatorilor specifici tipurilor predefinite: +, -, *, /, <, >, etc.
- **Moștenirea** este mecanismul care permite să se definească o nouă clasă care încorporează și extinde o clasă existentă.
- **Polimorfismul** este caracteristica unei variabile referință de a putea referi obiecte de tipuri diferite. C# admite polimorfism bazat pe relația de moștenire.

Întrebări și exerciții

1. Care este diferența dintre un obiect și o clasă ? Dar dintre o referință și un obiect ?
2. Indicați principalele deosebiri care există între membrii statici și cei nestatici ai unei clase.
3. Ce relație se modelează la **moștenire** ? Dar la **conținere** ?
4. Implementați o clasă cu numele **Persoana** și o a doua clasă cu numele **Elev**. Clasele trebuie să aibă constructori, câmpuri private și proprietăți.
5. Scrieți o clasă cu numele **BigNumber** care implementează operații cu numere mari. Supraîncărcați cel puțin operatorii de adunare, scădere și înmulțire.

Capitolul 5

Trăsături esențiale ale limbajului C#

Acest capitol prezintă delegările, evenimentele, genericele, colecțiile, mecanismul de tratare a excepțiilor în C#, un subcapitol dedicat manevrării stringurilor și un paragraf care se ocupă de operațiunile de intrare și ieșire cu fișiere text.

Delegări

O delegare este un tip referință, utilizat să încapsuleze o listă ordonată de metode cu aceeași semnătură și același tip de retur. Lista de metode se numește *lista de invocare*. Când un delegat este invocat, el apelează toate metodele din lista de invocare. O delegare cu o singură metodă în lista sa este similară cu un pointer la funcții în C++, însă o delegare este un tip referință și oferă siguranța tipurilor (*type-safe*). Capacitatea unei delegări de a invoca mai multe metode se numește **multicasting**.

Declarare

O delegare este un tip, așa cum și clasele sunt tipuri. Un *tip delegare* trebuie declarat înainte de crearea obiectelor de tipul său. Declararea este creată cu cuvântul cheie **delegate**, urmat de tipul de retur și de *signatura* metodelor pe care delegarea le acceptă.

Exemplu:

```
delegate void DelegareaMea(int x);
```

Expresia declară tipul delegat **DelegareaMea**. Obiectele de tipul acesta vor accepta numai metode cu un singur parametru de tip **int** și cu tipul de retur **void**. Tipurile delegat nu au corp.

Crearea obiectelor delegare

O delegare este un tip referință. Un asemenea tip, conține referința spre obiect, și obiectul propriu-zis.

Referințele (aici, referința se numește **d**) se declară simplu:

```
DelegareaMea d;
```

Obiectele de tip delegare se creează în două moduri :

1. Cu sintaxa specifică instanțierii obiectelor:

```
d = new DelegareaMea(ReferințăObiect.Metodă);
sau
d = new DelegareaMea(NumeClasă.MetodăStatică);
```

Este important de reținut că delegările pot încapsula atât metode de instanță cât și metode statice.

2. Cu sintaxa simplificată, care necesită doar precizarea metodelor atașate delegării:

```
d = ReferințăObiect.Metodă;
sau
d = NumeClasă.MetodăStatică;
```

Invocarea metodelor atașate unei delegări

Prezentăm un exemplu care declară un tip delegat și o referință la acest tip. Apoi atașează delegării câteva metode și în cel din urmă le invocă.

```
using System;

class Simplu
{
    // Metodă de instanță care potrivește delegării
    public void F(string s)
    {
        Console.WriteLine(s + "F() ");
    }
    // Metodă statică care potrivește delegării
    public static void G(string s)
    {
        Console.WriteLine(s + "G() ");
    }
}

// Declară tipul delegat Del
delegate void Del(string s);

class Program
{
    static void Main()
    {
        Del d;    // Declară o referință de tip Del
        Simplu s = new Simplu();

        // Acum atașăm metode delegării d
        d = s.F;    // Lista de invocare: F()
        d += Simplu.G;    // Lista de invocare: F(), G()
        d += Simplu.G;    // Lista de invocare: F(), G(), G()
```

```
// Delegarea d invocă acum toate metodele din listă
d("Prima invocare: ");

d -= s.F;          // Lista de invocare: G()
d("A doua invocare: ");
}
}
```

leșire:

```
Prima invocare: F()
Prima invocare: G()
Prima invocare: G()
A doua invocare: G()
A doua invocare: G()
```

Exemplul anterior descrie modul în care puteți atribui o metodă listei de invocare a delegării, cu operatorul de atribuire =, apoi puteți adăuga sau elimina alte metode cu același prototip cu operatorii += sau -=.

Invocarea delegărilor cu tipuri de retur

Dacă o delegare are o valoare de retur și mai mult de o metodă în lista de invocare, atunci valoarea care se returnează în urma invocării delegării este valoarea returnată de ultima metodă din lista de invocare. Valorile de retur ale tuturor celorlalte metode din listă sunt ignorate.

Exemplu:

```
using System;

// Declară tipul delegat DelTest
delegate int DelTest();

class Test
{
    private int x = 0;
    // Metode cu același prototip cu al delegării
    public int F()
    {
        x += 2;
        return x;
    }
    public int G()
    {
        x += 6;
        return x;
    }
}

class Program
{
```

```
static void Main()
{
    Test s = new Test();
    DelTest d;    // Referință la tipul delegat

    // Atașăm metode delegării d
    d = s.F;      // Lista de invocare: F()
    d += s.G;     // Lista de invocare: F() , G()
    d += s.F;     // Lista de invocare: F() , G() , F()
    Console.WriteLine(d()); // Afișează: 10
}
}
```

Evenimente

Aplicațiile cu interfață grafică cu utilizatorul sunt sensibile la evenimente ca: click cu mouse-ul pe suprafața unei ferestre, apăsarea unei taste, deplasarea mouse-ului deasupra unui control, etc. Sistemul de operare înștiințează fereastra activă despre apariția unei acțiuni, iar programatorul poate decide dacă va trata acest eveniment. Evenimentele pot avea o mare diversitate și nu sunt întotdeauna cauzate de o acțiune directă a utilizatorului aplicației; de exemplu curgerea unui interval de timp, terminarea copierii unor fișiere, primirea unui mail.

Evenimentele C# permit unei clase sau un obiect să notifice, să înștiințeze alte clase sau obiecte că ceva s-a întâmplat.

În terminologia specifică, clasa care semnalează evenimentul se numește *publisher*, iar clasele care sunt informate despre faptul că a avut loc evenimentul se numesc *subscribers*. Clasele care subscriu evenimentului (clasele *subscriber*) definesc metode pentru tratarea acestui eveniment (*event handler-e*).

Există o mare asemănare între delegări și evenimente.

Un eveniment este un membru public al clasei care publică evenimentul. Atunci când are loc o acțiune, acest membru al clasei *publisher* se activează, invocând toate metodele care au subscris evenimentului. Activarea se numește *declanșare (firing the event)*.

Lucrul cu evenimente

Pentru utilizarea evenimentelor, programatorul trebuie să scrie cod după cum urmează:

1. **Declară un tip delegat.** Această declarație poate avea loc în clasa *publisher* sau în afara oricărei clase, pentru că este o declarație de tip. Evenimentul și *handler-ele* de evenimente trebuie să aibă o semnătură și un tip de retur identice.

2. **Declară evenimentul.** Evenimentul se declară ca membru public în clasa *publisher*, cu ajutorul cuvântului cheie **event**,. El depozitează și invocă lista de *handler*-e.
3. **Scrie cod care declanșează evenimentul.** Codul se scrie în clasa *publisher*. Apelarea evenimentului duce la invocarea tuturor metodelor *handler* înregistrate cu acest eveniment.
4. **Declară *event handler*-ele.** Se declară în clasele *subscriber* și sunt metode (*event handler*-e) care se execută când evenimentul se declanșează.
5. **Înregistrează *handler*-ele.** Aceasta presupune conectarea evenimentului la metodele *handler*. Codul se scrie în clasele *subscriber* sau în alte clase, dar nu în *publisher*.

Este important de reținut că prin mecanismul de semnalare și de tratare a evenimentelor, se realizează o decuplare a obiectelor de tip *publisher* de obiectele de tip *subscriber*. Clasa *publisher* „nu știe” nimic despre obiectele sau clasele *subscriber* care vor fi notificate.

Programul care urmează, declară tipul delegat **Timp**. Metodele *handler* trebuie să potrivească acestui tip, adică să nu aibă parametri formali și tipul de retur să fie **void**. Clasa **Publisher** (evident, poate fi oricare alt nume), declară evenimentul ca și câmp public cu numele **eveniment**, de tipul **Timp**. Metoda **Declanseaza()** declanșează evenimentul prin apelul **eveniment()**. De fapt, evenimentul ca atare este trecerea a 3 secunde, iar **eveniment()** declanșează apelul metodelor *handler* din lista de invocare a evenimentului. Programul declară două clase *subscriber*, **A** și **B**, care definesc câte o metodă de tratare a evenimentului care se semnalează în clasa **Subscriber**. Înregistrarea *handler*-elor are loc în **Main()**, dar s-ar fi putut face și în metode ale claselor *subscriber*.

```
// event.cs
using System;

public delegate void Timp(); // Declararea tipului delegat

// Publisher nu stie nimic despre obiectele pe care le va
// notifica sau despre metodele inregistrate evenimentului
public class Publisher
{
    public event Timp eveniment; // Declararea evenimentului

    public void Declanseaza()
    {
        while (true)
        {
            // Execuția programului se întrerupe 3 secunde
            System.Threading.Thread.Sleep(3000);

            // Ne asigurăm că există metode înregistrate
            if (eveniment != null)
                eveniment(); // Declanșează evenimentul
        }
    }
}
```

```
        } // o dată la trei secunde
    }
}

// Clase Subscriber
class A
{
    public void HandlerA()
    {
        Console.WriteLine("Obiect A, notificat la {0}",
                           DateTime.Now);
    }
}

class B
{
    public void HandlerB()
    {
        Console.WriteLine("Obiect B, notificat la {0}",
                           DateTime.Now);
    }
}

class Test
{
    static void Main()
    {
        Publisher p = new Publisher();
        // Obiectele a și b vor fi notificate la declanșarea
        // unui eveniment
        A a = new A();
        B b = new B();

        // Clasele A și B subscriu acestui eveniment
        p.eventiment += a.HandlerA; // înregistrarea metodelor
        p.eventiment += b.HandlerB; // handler

        // Apelează metoda care declanșează evenimentul
        p.Declanseaza();
    }
}
```

Programul afișează la fiecare trei secunde, câte două linii:

```
Obiect A, notificat la 12.06.2008 10:13:22
Obiect B, notificat la 12.06.2008 10:13:22
Obiect A, notificat la 12.06.2008 10:13:25
Obiect B, notificat la 12.06.2008 10:13:25
...
```

Pentru determinarea datei și a orei curente, se utilizează proprietatea `Now`, a clasei `System.DateTime`.

IMPORTANT

Înregistrarea metodelor atașate unui eveniment se face cu operatorul `+=`, iar îndepărtarea lor din lista de invocare, se face cu `-=`.

Publicarea evenimentelor în mod specific .NET

Tratarea evenimentelor presupune existența unui tip delegat. Acesta poate fi creat de dumneavoastră, însă o mai bună alternativă este folosirea tipului delegat predefinit al platformei .NET.

Când evenimentul nu transmite date handler-elor

```
public
delegate void EventHandler(object sender, EventArgs e);
```

`sender` este o referință la obiectul (de tip `publisher`) care declanșează evenimentul, iar `e` este o referință la un obiect de tip `EventArgs`. Declararea de mai sus nu trebuie făcută în program, pentru că o face platforma .NET. Ceea ce aveți de făcut în programul `events.cs`, este să ștergeți linia care declară tipul delegat `Timp` și să vă declarați un eveniment în clasa `Publisher`, astfel:

```
public event EventHandler eveniment;
```

Handler-ele trebuie să aibă desigur, aceeași semnătură și tip de retur, cu a tipului `EventHandler`:

```
public void HandlerA(object sender, EventArgs e)
și
public void HandlerB(object sender, EventArgs e)
```

O ultimă modificare pe care o faceți, este înlocuirea apelului `eveniment()` ; cu apelul `eveniment(this, new EventArgs())`;

Când evenimentul transmite date handler-elor

Obiectul de tip `EventArgs` nu conține în realitate date utile și este folosit atunci când un eveniment nu transmite date de stare unui *event handler*. Dacă doriți să transmiteți *handler*-elor informații suplimentare despre eveniment, puteți crea o clasă, de exemplu `MyEventArgs`, care menține aceste informații. Această clasă trebuie să fie derivată (să moștenească) clasa `EventHandler`, așa cum se vede în programul care urmează. În această situație, veți crea un tip delegat cu o semnătură compatibilă .NET. Vom rescrie programul anterior. Tipul delegat se va declara ca mai jos:


```
public delegate void Timp(object sender, MyEventArgs ev);
```

sender este o referință la *object*, deci poate referi obiecte de orice tip, inclusiv **Publisher**. În apelul `eveniment(this, ev);` argumentele sunt: referința `this` la obiectul de tip **Publisher** și referința `ev` la un obiect de tip **MyEventArgs**.

```
using System;
```

```
public class MyEventArgs : EventArgs
```

```
{
    private DateTime momentul;    // Câmp
    public DateTime Momentul      // Proprietate
    {
        set
        {
            momentul = value;
        }
        get
        {
            return this.momentul;
        }
    }
}
```

```
// Declară tipul delegat cu prototipul cerut de .Net
```

```
public delegate void Timp(object sender, MyEventArgs ev);
```

```
public class Publisher
```

```
{
    public event Timp eveniment;    // Declar evenimentul

    public void Declanseaza()
    {
        while (true)
        {
            // Execuția programului se întrerupe 3 secunde
            System.Threading.Thread.Sleep(3000);
            if (eveniment != null) // Ne asigurăm ca există
            {                      // metode înregistrate
                MyEventArgs ev = new MyEventArgs();
                ev.Momentul = DateTime.Now;
                // Declanșează invocarea handler-elor
                eveniment(this, ev);
            }
        }
    }
}
```

```
// Clase Subscriber
```

```
class A
{
    public void HandlerA(object sender, EventArgs e)
    {
        Console.WriteLine("Obiect A, notificat la {0}",
                           e.Momentul);
    }
}

class B
{
    public void HandlerB(object sender, EventArgs e)
    {
        Console.WriteLine("Obiect B, notificat la {0}",
                           e.Momentul);
    }
}

class Test
{
    static void Main()
    {
        Publisher m = new Publisher();
        A a = new A(); // Obiectele a si b vor fi notificate
        B b = new B(); // la aparitia evenimentului

        // Înregistrarea metodelor handler
        m.event += a.HandlerA;
        m.event += b.HandlerB;

        m.Declanseaza();
    }
}
```

Înșirarea programului este identică cu cea a programului anterior.

De reținut:

Dacă vrei să scrieți cod eficient, veți trata evenimentele folosind clasele de bază **EventHandler** și **EventArgs**, iar delegările vor avea prototipul compatibil **.NET**, chiar dacă C# admite orice model delegat.

Generice

Programarea Generică sau programarea cu șabloane este un stil de programare diferit de *Programarea Orientată pe Obiecte*. Au în comun abstractizarea datelor și reutilizarea codului, dar abordările sunt diferite. În timp ce

OOP încapsulează în același obiect date care reflectă starea obiectului, împreună cu metode care descriu capacitățile lui, scopul programării generice este scrierea de cod care să fie independent de tipurile de date.

Clase generice

Să presupunem că ai definit o clasă **Stiva** ca mai jos:

```
public class Stiva
{
    private int[] a;
    private int n;

    public Stiva(int max)
    { a = new int[max]; }

    public void Push(int val)
    { a[n++] = val; }

    public int Pop()
    { return a[--n]; }
}
```

Acesta stivă memorează în câmpul **a** de tip tablou de **int**, un număr de valori întregi. Dacă dorești să depozitați valori de tip **double**, **string** sau oricare alt tip, sunteți nevoiți să rescrieți codul, înlocuind tipul **int** cu noul tip. C# vă oferă posibilitatea să scrieți doar o singură definiție de clasă, care să lucreze cu toate tipurile dorite. Veți scrie o clasă generică.

Clasele generice sunt **tipuri parametrizate**. Clasa generică se va numi **Stiva<T>** și se va rescrie astfel:

```
// stiva_generic.cs
using System;

public class Stiva<T>
{
    private T[] a;
    private int n;
    public Stiva(int max)    // Constructor
    { a = new T[max]; }
    public void Push(T val)
    { a[n++] = val; }
    public T Pop()
    { return a[--n]; }
}
```

```

public class TestGeneric
{
    static void Main()
    {
        // Se creează o stivă de int (T = int)
        Stiva<int> s1 = new Stiva<int>(100);
        s1.Push(2);
        s1.Push(4);
        s1.Push(6);
        Console.WriteLine(s1.Pop() + " " + s1.Pop() + " " +
                           s1.Pop());
        // Se creează o stivă de string (T = string)
        Stiva<string> s2 = new Stiva<string>(50);
        s2.Push("Marcel");
        s2.Push("Ionel");
        s2.Push("Alin");
        Console.WriteLine(s2.Pop() + " " + s2.Pop() + " " +
                           s2.Pop());
    }
}

```

leșire:

6 4 2

Alin Ionel Marcel

Numele clasei generice este `Stiva<T>`. De fapt, nu este o clasă reală, ci este un șablon de clasă. Pe baza șablonului de clasă se vor genera clase reale, pentru valori particulare ale parametrului `T`.

Parametrul șablonului este `T`. Se numește *parametru tip* sau *parametru generic*.

Expresia `Stiva<int> s1 = new Stiva<int>(100);` creează pe baza șablonului `Stiva<T>` o clasă cu numele `Stiva<int>`, apoi instanțiază un obiect de acest tip, deci o stivă de întregi.

Argumentele cu care se înlocuiesc *parametrii tip* se numesc *argumente tip*. Mai sus, `T` este *parametru tip*, iar `int` este *argumentul tip*.

O clasă poate avea mai mulți parametri generici. Programul următor declară o clasă generică (nu uitați, o clasă generică este un șablon de clasă), cu doi *parametri tip*, `T1` și `T2`:

```

using System;

public class A<T1, T2>
{
    private T1 a;
    private T2 b;
    public A(T1 a, T2 b) // Constructor
    {
        this.a = a;
        this.b = b;
    }
}

```

```
public void Print()
{
    Console.WriteLine(a + " " + b);
}

public class TestGeneric
{
    static void Main()
    {
        A<string, int> m1 = new A<string, int>("Alin", 18);
        m1.Print();
        A<string, string> m2 = new A<string, string>("UNU",
                                                    "DOI");
        m2.Print();
        A<double, char> m3 = new A<double, char>(2.3, 'F');
        m3.Print();
    }
}
```

Ieșire:

Alin 18

1 2

2.3 F

Expresia `A<string, int> m1 = new A<string, int>("Alin", 18);` generează pe baza șablonului `A<T1, T2>` o clasă cu numele `A<string, int>`, apoi instanțiază un obiect `m1` de acest tip. Similar se generează și celelalte clase, respectiv obiecte. În felul acesta, metoda `Print()` a primit o funcționalitate extinsă.

Metode generice

Toate metodele unei clase generice sunt la rândul lor generice, deoarece pot utiliza parametri generici ai clasei. În afara genericității implicite, puteți defini metode cu proprii parametri generici, care nu depind de cei ai clasei.

Exemplu:

```
using System;

public class C<U>
{
    private U u;

    public C(U u) // Constructor generic
    {
```

```

        this.u = u;
    }
    // Metoda generică - are proprii parametri generici
    public void F<V1, V2>(V1 a, V2 b)
    {
        Console.WriteLine(a.ToString() + " " + u.ToString() +
                           " " + b.ToString());
    }
}

public class TestMetodaGenerica
{
    static void Main()
    {
        C<int> c1 = new C<int>(104);    // U = int
        c1.F<string, string>("Sa traiti", "ani!");

        C<char> c2 = new C<char>('F');  // U = char
        c2.F<string, int>("Iulia", 8);
    }
}

```

leșire:

Să traiti 104 ani!
Iulia F 8

Expresia `C<int> c1 = new C<int>(104);` generează clasa `C<int>` pe baza șablonului de clasă `C<T>`.

Expresia `c1.F<string, string>("Sa traiti", "ani!");` generează metoda `F<string, string>()` pe baza șablonului de metodă `F<V1, V2>(V1 a, V2 b)`, apoi apelează metoda cu două argumente de tip `string`. Metodele cu proprii parametri generici își sporesc funcționalitatea în raport cu celelalte metode ale unei clase șablon.

Avantajele programării generice

Acest model de programare permite implementarea algoritmilor generici. Pentru asemenea algoritmi, datele se manipulează în același fel ca în cazul algoritmilor non-generici, în timp ce tipurile de date utilizate pot să fie schimbate de către programator după necesități. Genericele se remarcă prin calitatea și eleganța codului, sintaxa ușor de înțeles. Codul se poate reduce semnificativ ca volum atunci când aveți sarcini de programare cu cod repetabil.

Folosirea tipurilor generice este recomandată de asemenea pentru siguranța tipurilor care se creează (corectitudinea tipurilor se verifică în timpul compilării), pentru performanța în timpul rulării și nu în ultimul rând pentru creșterea productivității în programare.

Colecții

Platforma **.Net** conține clase specializate pentru depozitarea datelor. Aceste clase implementează stive, cozi, liste, tabele de dispersie (*hash-tables*). Colecțiile non-generice sunt definite în spațiul de nume **System.Collections**. Colecțiile generice se definesc în **System.Collections.Generic**.

Cele două tipuri de colecții implementează aproximativ aceleași tipuri de structuri de date, însă cele generice sunt mai performante și furnizează o mai mare siguranță a tipurilor. Prezintă modul de utilizare a câtorva containere generice.

Clasa generică **Stack<T>**

Stack<T> este o colecție de instanțe de același tip **T**, care implementează operații specifice unei stive (*LIFO*). Managementul memoriei se face automat.

Exemplu:

```
using System;
using System.Collections.Generic;

class StivaGenerica
{
    public static void Main()
    {
        // Declară o stivă vidă cu elemente de tip string
        Stack<string> st = new Stack<string>();

        // Adaugă câteva elemente în stivă
        st.Push("UNU"); st.Push("DOI");
        st.Push("TREI"); st.Push("PATRU");
        Console.WriteLine("Nr. de elemente: {0}", st.Count);

        foreach (string s in st) // Se parcurge stiva
            Console.WriteLine(s);

        // Pop() scoate elementul din vârful stivei
        Console.WriteLine("Scoate '{0}'", st.Pop());

        // Peek() returnează elementul din vârful stivei
        // fără să-l scoată din stivă
        Console.WriteLine("Varful stivei: {0}", st.Peek());
        st.Clear(); // Șterge toate elementele
    }
}
```

Ieșire:

```
Nr. de elemente: 4
PATRU
TREI
```

```
DOI
UNU
Scoate 'PATRU'
Varful stivei: 3
```

Clasa generică `List<T>`

Clasa reprezintă o listă de obiecte care poate fi accesată prin index. Implementează metode pentru căutarea, sortarea și manipularea obiectelor. Este echivalentul clasei non-generice `ArrayList`, dar mai performantă decât aceasta.

Exemplu:

```
using System;
using System.Collections.Generic;

public class ListaGenerica
{
    public static void Main()
    {
        // Creează lista copii cu elemente de tip string
        List<string> copii = new List<string>();

        // Adaugă elemente în listă
        copii.Add("Ionel");   copii.Add("Radu");
        copii.Add("Viorel");  copii.Add("Adisor");
        copii.Add("Nelutu");

        // Parcurge colecția
        foreach (string c in copii)
            Console.Write(c + " ");
        Console.WriteLine("\nNr. copii: {0}", copii.Count);

        // Contains() returnează true dacă un element există
        Console.WriteLine("Viorel exista ? {0}\n",
                          copii.Contains("Viorel"));

        Console.WriteLine("Insereaza \"Alin\" " +
                          "pe pozitia 2:");
        copii.Insert(2, "Alin");

        foreach (string c in copii)
            Console.Write(c + " ");
        Console.WriteLine("\ncopii[3] = {0}", copii[3]);

        Console.WriteLine("\nSorteaza alfabetic: ");
        copii.Sort();
        for (int i = 0; i < copii.Count; i++)
            Console.Write(copii[i] + " ");
    }
}
```



```
Console.WriteLine("\n\nSterge \"Nelutu\":");
copii.Remove("Nelutu");

foreach (string c in copii)
    Console.Write(c + " ");

Console.WriteLine("\n\nSterge lista de copii:");
copii.Clear();
Console.WriteLine("Nr. copii: {0}", copii.Count);
}
}
```

Ieșire:

Ionel Radu Viorel Adisor Nelutu

Nr. copii: 5

Viorel exista ? true

Insereaza "Alin" pe pozitia 2:

Ionel Radul Alin Viorel Adisor Nelutu

copii[3] = Viorel

Sorteaza alfabetic:

Adisor Alin Ionel Nelutu Radu Viorel

Sterge "Nelutu":

Adisor Alin Ionel Radu Viorel

Sterge lista de copii:

Nr. de copi: 0

Clasa generică Dictionary<Tkey, Tvalue>

Clasa este echivalentul clasei `map` din **STL, C++**. Realizează o mapare, o corespondență biunivocă între o mulțime de chei și o mulțime de valori. Cheile sunt unice, în sensul că nu pot exista mai multe chei identice în dicționar. Fiecărei chei îi corespunde o singură valoare asociată.

Clasa oferă metode care implementează operații rapide de inserare a perechilor cheie-valoare, de ștergere și de căutare a valorii după cheia asociată. În dicționar, perechile cheie-valoare sunt încapsulate în obiecte de tip `KeyValuePair`.

Exemplu:

```
using System;
using System.Collections.Generic;
```

```

public class AgendaTelefonica
{
    public static void Main()
    {
        // Un dicționar cu cheia string și valoarea int
        Dictionary<string, int> T =
            new Dictionary<string, int>();
        // Adaugă elemente în dicționar.
        T.Add("Ionescu", 209791); // Inserează cu Add()
        T.Add("Pop", 232145);
        T["Vlad"] = 213048;       // Inserează cu operatorul
        T["Cazacu"] = 219465;     // de indexare

        Console.WriteLine("Cheia Vlad are valoarea: {0}",
            T["Vlad"]);

        // Cu operatorul de indexare poate schimba valoarea
        // asociată unei chei existente
        T["Vlad"] = 215773;
        Console.WriteLine("Cheia Vlad are valoarea: {0}\n",
            T["Vlad"]);

        // Dacă cheia nu există în dicționar, se adaugă o
        // nouă pereche cheie-valoare
        T["Dragnea"] = 279950;

        // ContainsKey() se folosește pentru a testa
        // existența unei chei înainte de inserare
        // if (!T.ContainsKey("Simion"))
            T.Add("Simion", 200371);

        // În dicționar elementele se memorează ca perechi
        // cheie-valoare în obiecte de tip KeyValuePair
        foreach (KeyValuePair<string, int> p in T)
            Console.WriteLine("{0, -7} {1, 10}",
                p.Key, p.Value);

        // Șterge o pereche cheie-valoare
        T.Remove("Ionescu");
        if ( !T.ContainsKey("Ionescu") )
            Console.WriteLine("\nCheia Ionescu nu exista");
    }
}

```

leșire:

Cheia Vlad are valoarea: 213048

Cheia Vlad are valoarea: 215773

Ionescu	209791
Pop	232145
Vlad	215773
Cazacu	219465
Dragnea	279950
Simion	200371

Cheia Ionescu nu exista

Tratarea excepțiilor

În timpul execuției unui program pot apărea situații excepționale, cum ar fi operații ilegale executate de propriul cod, care pot duce la întreruperea execuției programului sau la un comportament neașteptat. Aceste situații se numesc excepții. C# oferă un mecanism de tratare a excepțiilor, bazat pe cuvintele cheie **try**, **catch** și **finally**.

Erorile din timpul rulării programului se propagă în program cu ajutorul acestui mecanism de tratare a excepțiilor. Veți include codul care este probabil să arunce excepții, într-un bloc **try**. Când excepția se produce, fluxul de execuție al programului este dirijat direct în blocul de cod numit **catch**, care “prinde” și tratează excepția. Excepțiile “neprinse”, sunt captate de către un *handler* furnizat de către sistem, care afișează un mesaj de eroare.

Tipurile excepțiilor care pot să apară sunt reprezentate de către clase specializate ale platformei .NET, clase care derivă din clasa **Exception**.

Pentru tratarea excepțiilor, veți proceda astfel:

```
try
{
    // Cod care poate arunca excepții
}
catch(System.Exception e)
{
    // Cod care tratează excepția
}
finally
{
    // Cod care se execută după try-catch indiferent
    // dacă se aruncă sau nu excepții
}
```

Exemplu:

```
using System;

class TestExcepții
```

```
{
    public static void Main()
    {
        int[] a = { 2, 4, 6, 8 };
        try
        {
            Console.WriteLine(a[4]);
        }
        catch (Exception e)
        {
            Console.WriteLine("Exceptie!\n" + e.ToString());
        }
        finally
        {
            Console.WriteLine(a[3]);
        }
    }
}
```

leșire:**Exceptie!**

System.IndexOutOfRangeException: Index was outside the bounds of the array at TestExceptii.Main() in c:\teste\Program.cs: line 7

8

Blocul **catch** declară o variabilă de tip excepție (e) care poate fi utilizată pentru a obține informații suplimentare. Codul din blocul **finally** se execută indiferent dacă se aruncă sau nu o excepție în blocul **try**, permițând programului să elibereze resursele (fișiere deschise, memorie alocată, etc.). Dacă excepția se produce, atunci blocul **finally** se execută după **catch**.

Blocul **finally** poate să lipsească. Pentru același bloc **try**, se pot declara mai multe blocuri **catch**, fiecare dintre ele, precizând o posibilă excepție care se poate lansa din **try**.

Manevrarea stringurilor

Tipul **string** în C# este un *alias* pentru clasa **System.String** din *.Net Framework*. Este un tip referință. Obiectele de tip **string** încapsulează un șir de caractere în format *Unicode*. Stringurile sunt "imutabile". Odată creat, un obiect de tip **string** nu mai poate fi schimbat; toate operațiile care modifică un string returnează un alt string modificat.

Operații și metode

string definește operatorii relaționali **=**, **!=** și operatorii de concatenare **+**, **+=**. Definește de asemenea mai multe metode utile. Vom exemplifica utilizarea câtorva dintre ele.

```
using System;

class TestString
{
    public static void Main()
    {
        string s1 = "Salut", s2 = "salut";
        if (s1 != s2) // Compară stringurile, nu obiectele
            Console.WriteLine("s1 != s2");
        if ( s1.ToUpper() == s2.ToUpper() )
            Console.WriteLine(s1.ToUpper()); //Afișează SALUT

        s1 += s2;    // s1 este acum "Salutsalut"
        // Inserează șirul " " începând cu poziția 5
        string s = s1.Insert(5, " "); // s1 nu se modifică!
        Console.WriteLine(s); // Afișează: "Salut salut"

        // Extrage din s, începând cu poziția 6,
        // un substring format din 3 caractere
        s = s.Substring(6, 3);
        Console.WriteLine(s); // Afișează: "Salut salut"
    }
}
```

Formatarea stringurilor

Pentru formatarea stringurilor există în clasa **String** metodele statice **Format()**, supraîncărcate.

Exemplu:

```
using System;
class TestString
{
    public static void Main()
    {
        int x = 123; double y = 23.4589;
        string s = String.Format("x = {0}, y = {1}", x, y);
        Console.WriteLine(s); // Afișează: 123 23.4589
    }
}
```

Mai sus, **{0}** se referă la primul obiect (**x**) din lista de parametri, iar **{1}** identifică cel de-al doilea obiect (**y**).

Opțiunile de formatare sunt diverse. Puteți stabili aliniere *justify* la stânga sau la dreapta și formatul de afișare a numerelor:

Exemplu:

```
float x = 123.3456F; double y = 23.4589; string s;
s = String.Format("x = {0,12:E3}\ny = {1,12:F2}", x, y);
Console.WriteLine(s);
```

Afișează:

```
x =      1.233E+002
y =      23.46
```

Specificatorul {0,12:E3} se interpretează astfel: 0 – identifică primul obiect (x), 12 este lățimea câmpului de afișare, E cere afișare în format științific, iar 3 este numărul de zecimale care se afișează.

Specificatorul {1,12:F2} se interpretează astfel: 1 – identifică al doilea obiect (y), 12 este lățimea câmpului de afișare, F impune afișare în virgulă fixă, iar 2 este numărul de zecimale care se afișează. Dacă doriți aliniere la stânga, se pune semnul – după virgulă: {1,-12:F2}.

Stringurile se pot formata pentru afișare după aceleași reguli și cu ajutorul metodelor **Console.Write()** și **Console.WriteLine()**:

Exemplu:

```
double x = 23.4589;
Console.WriteLine("|x = {0,10:F2}|", x);
Console.WriteLine("|x = {0,-10:F2}|", x);
```

Afișează:

```
|x =      23.46|
|x = 23.46   |
```

Transformarea stringurilor în valori numerice

În C# nu există metode sau operatori care să citească date numerice din *stream*-uri și să le formateze direct în valori numerice, așa cum sunt funcțiile **scanf()** din limbajul C, sau operatorul de extracție >> din C++. De regulă, veți citi datele ca stringuri. Din aceste stringuri, veți extrage valorile numerice.

Tipurile predefinite (**int**, **double**, **float**, etc.) definesc metoda **Parse()**. Aceasta preia stringul citit și îl transformă în valoarea numerică corespunzătoare.

Exemplu :

```
int n = int.Parse("652"); // n = 652
double d = double.Parse(" -20.235"); // d = -20.235
```

```
// Citim un număr real de la tastatură:  
string s = Console.ReadLine(); // s = "91.045"  
double f = double.Parse(s); // f = 91.045
```

Citirea unui șir de valori numerice

Dacă este nevoie să citiți un șir de valori numerice dintr-un *stream*, de exemplu de la tastatură, atunci trebuie să precizați caracterul sau caracterele separatoare între numere.

Un caz simplu este acela în care toate numerele sunt despărțite printr-un singur spațiu și se găsesc pe o singură linie. Se citește linia într-un string, apoi se desparte stringul în substringuri reprezentând numerele. Pentru aceasta, utilizați metoda `Split()`. Aceasta returnează un tablou de stringuri reprezentând numerele citite.

Exemplu:

```
string linie = Console.ReadLine();  
string[] s = linie.Split(' '); // Separatorul este  
int x; // caracterul spațiu  
foreach (string nr in s) // Parcurge tabloul de stringuri  
{  
    x = int.Parse(nr); // Le transformă în valori int  
    Console.Write(x + " ");  
}
```

Metoda `Split()` este supraîncărcată. Una dintre versiuni are ca parametru un șir de caractere `char[]`. În acest șir veți introduce toate caracterele separatoare pe care le considerați necesare. Al doilea parametru al metodei, vă permite să înlăturați toate substringurile vide din tabloul de stringuri rezultat în urma splitării. Se obțin stringuri vide în situația când în stringul inițial apar doi sau mai mulți separatori consecutivi.

Exemplu:

```
using System;  
public class TestSplit  
{  
    static void Main()  
    {  
        char[] sep = { '\n', ' ', '.', '!' };  
        string s = "12 27\n... 496!";  
        string[] w = null;  
        // Tabloul w preia substringurile fără separatori din s  
        w = s.Split(sep, StringSplitOptions.RemoveEmptyEntries);  
  
        for (int i = 0; i < w.Length; i++)  
        {  
            int x = int.Parse(w[i]);  
        }  
    }  
}
```

```

        Console.Write(w[i] + "_");
    }
}
}

```

Ieșire:

12_27_496_

Cunoscătorii limbajului C au remarcat desigur similitudinea între metoda `Split()` și funcția `strtok()`.

Citiri și afișări din fișiere de tip text

Toate intrările și ieșirile în C#, la fel ca în C++ sau Java, presupun folosirea *stream*-urilor. Un *stream* este o reprezentare abstractă a unui dispozitiv fizic (memorie, fișier, rețea, etc.), în care informația se poate accesa doar câte un *byte* odată. Într-un *stream* informația circulă într-un singur sens. De exemplu, un program nu va scrie datele direct într-un fișier, ci le va scrie într-un *stream* de ieșire care reprezintă fișierul.

Există *stream*-uri de intrare și *stream*-uri de ieșire. Din *stream*-urile de intrare programul citește date, iar în cele de ieșire scrie date. Păstrând dispozitivul în formă abstractă, destinația sau sursa *stream*-ului poate fi ascunsă. Aceasta permite reutilizarea codului, iar codul este similar atunci când aplicațiile scriu sau citesc din fișiere sau din rețea, de pe disc, din memorie, sau din oricare alt dispozitiv.

.NET Framework definește în spațiul de nume **System.IO** acele clase care reprezintă *stream*-uri cu fișiere. Sunt mai multe astfel de clase. În acest paragraf ne referim la doar un singur aspect: citirea și scrierea datelor numerice din fișiere text. Iată un program pentru exemplificare:

```

using System;
using System.IO;

class FileReadWrite
{
    static void Main()
    {
        string fin = @"C:\teste\numere.in";
        string fout = @"C:\teste\numere.out";

        // Deschide fișierele de intrare și de ieșire
        StreamReader sr = new StreamReader(fin);
        StreamWriter sw = new StreamWriter(fout);

        string[] s = null;
        string linie = null;
    }
}

```



```

int x = 0;

// Citește câte o linie până la întâlnirea
// sfârșitului de fișier
while ( (linie = sr.ReadLine()) != null )
{
    s = linie.Split(' '); // Desparte linia în stringuri
    foreach (string nr in s)
    {
        x = int.Parse(nr); // Obține valoarea numerică
        sw.Write("{0, -5}", x); // O scrie în fișier
    }
    sw.WriteLine();
}
sr.Close(); // Închide fișierele
sw.Close();
}
}

```

Exemplu : Dacă `numere.in` conține valorile:

```

12 335 561
0 25 5 76
99 1 773 67 3

```

Atunci `numere.out` va avea:

```

12    335    561
0     25     5     76
99    1     773   67    3

```

Observații:

- Clasele `StreamReader` și `StreamWriter` citesc, respectiv scriu caractere din *stream*-uri. Conțin metodele `ReadLine()`, respectiv `WriteLine()`. Acestea citesc sau scriu din *stream* până la caracterul *newline*.
- O constantă șir de caractere se numește șir **verbatim**, dacă se prefațează cu caracterul `@`. Exemplu: `@ "C:\teste\numere.in"`. Efectul este că secvențele escape din interior nu se mai evaluează. Deci șirul verbatim din exemplu este echivalent cu `"C:\\teste\\numere.in"`.
- În expresia `sw.Write("{0, -5}", x);`, `0` semnifică primul obiect de afișat, adică `x`, iar `-5` cere alinierea la stânga a rezultatului pe un câmp de lățime 5.
- Dacă fișierul de intrare poate avea ca separatori între două numere pe aceeași linie mai mult decât un singur spațiu, atunci veți folosi versiunea `Split()` cu separatori, așa cum am arătat în paragraful "Transformarea stringurilor în valori numerice".

Rezumatul capitolului

- O delegare este un tip referință, utilizat să încapsuleze o listă ordonată de metode cu aceeași semnătură și același tip de retur. Lista de metode se numește *lista de invocare*. Când un delegat este invocat, el apelează toate metodele din lista de invocare.
- Evenimentele C# permit unei clase sau un obiect să notifice, să înștiințeze alte clase sau obiecte că ceva s-a întâmplat.
- Un eveniment este un membru public al clasei care publică evenimentul. Atunci când are loc o acțiune, acest membru al clasei *publisher* se activează, invocând toate metodele care au subscris evenimentului.
- Scopul programării generice este scrierea de cod care să fie independent de tipurile de date.
- Există clase generice și metode generice. Clasele generice sunt *tipuri parametrizate*. Parametrii lor se numesc **parametri tip** sau **parametri generici**.
- Colecțiile sunt clase specializate pentru depozitarea datelor. Biblioteca .NET definește colecții generice și colecții non-generice.
- Colecțiile implementează stive, cozi, liste, tabele de dispersie (*hash-tables*).
- Tipurile predefinite (**int**, **double**, **float**, etc.) definesc metoda **Parse()**. Aceasta preia stringul citit și îl transformă în valoarea numerică corespunzătoare.
- Clasele **StreamReader** și **StreamWriter** citesc, respectiv scriu caractere din *stream*-uri. Conțin metodele **ReadLine()**, respectiv **WriteLine()**. Acestea citesc sau scriu din *stream* până la caracterul *newline*.

Întrebări și exerciții

1. Ce rol îndeplinește un tip delegat într-un program ?
2. Cum puteți elimina o metodă din lista de invocare a unei delegări ?
3. De ce se spune că aplicațiile Windows sunt *conduse de evenimente* ?
4. Evidențiați asemănările și deosebirile dintre tipul tablou și tipul **List<T>**.
5. Să se citească din fișierul text *matrice.in* elementele unui tablou bidimensional de valori întregi. Să se afișeze în fișierul *matrice.out* pătratul valorii fiecărui element al tabloului.

Partea a II - a

Programare Windows cu Visual C# 2008 Express Edition

Capitolul 6

Aplicații de tip Windows Forms

Visual C# 2008 Express Edition (VCSE) oferă suport pentru dezvoltarea următoarelor tipuri de aplicații:

- Aplicații Windows de tip *Windows Forms*.
- Aplicații Windows de tip **WPF** (*Windows Presentation Foundation*).
- Aplicații de tip consolă.
- Aplicații de tip bibliotecă dinamică.

Aplicațiile de tip *Windows Forms* și cele *WPF* facilitează prin *designer*-ele integrate dezvoltarea interfețelor grafice cu utilizatorul (*user interface*).

Aplicații cu interfață grafică cu utilizatorul

Biblioteca .NET conține un număr mare de clase definite în spațiul de nume **System.Windows.Forms**, cum sunt: **Button**, **TextBox**, **ComboBox**, **Label**, etc. Instanțele acestor clase identifică controalele *Windows*. Controalele sunt elemente de interfață grafică ale unei aplicații *Windows*.

Un programator hardcore poate să codeze cu ajutorul acestor clase o aplicație cu interfață grafică oricât de complicată fără a utiliza mediul integrat, însă munca este imensă. Este de preferat să lăsați *Visual C#* să genereze pentru voi acel cod necesar interfeței grafice, ca să vă puteți concentra pe funcționalitatea aplicației. Veți utiliza pentru aceasta **Windows Form Designer** și **Toolbox**. Cu mouse-ul, prin *drag and drop* veți alege controalele necesare din **Toolbox** și le veți aranja pe suprafața formelor în **Windows Form Designer**. Veți seta proprietățile controalelor. Pe măsură ce faceți toate acestea, designerul generează codul C# aferent, pe care îl scrie în fișierul `<nume>.designer.cs`, unde `<nume>` este numele formei respective.

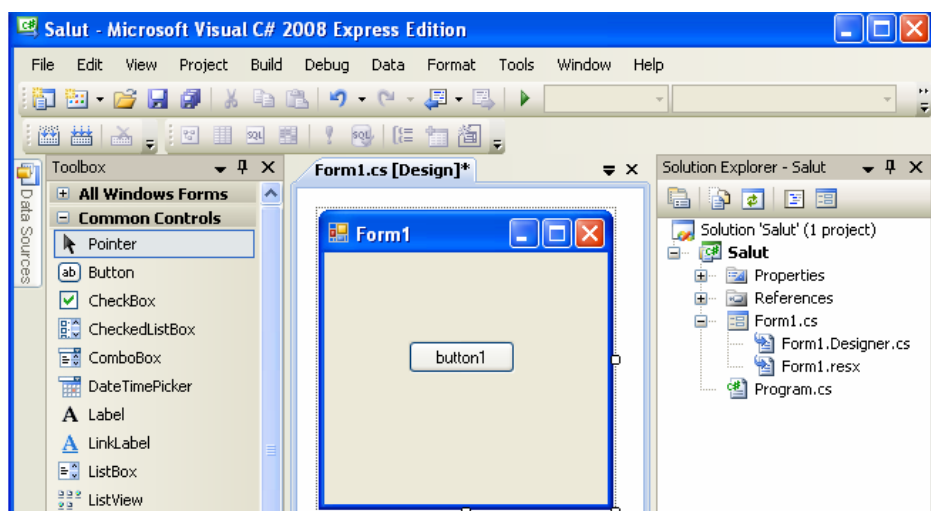
Așadar, sunt trei etape importante în crearea unei aplicații cu interfață grafică:

1. **Adăugarea controalelor** pe suprafața formelor.
2. **Setarea proprietăților inițiale ale controalelor** din fereastra **Properties**.
3. **Scrierea handlerelor** pentru evenimente.

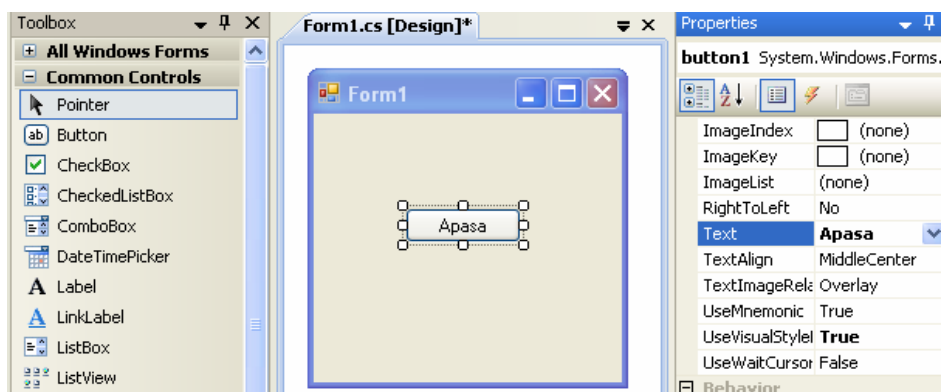
Realizarea unei aplicații simple de tip Windows Forms

Vom crea o aplicație numită **Salut**, care are un singur buton plasat pe o formă. La apăsarea lui, apare o fereastră sistem de tip **MessageBox**, care afișează un mesaj de salut. Urmăți pașii:

1. În meniul **File**, click **New Project**.
2. În fereastra **New Project**, în panoul **Templates**, alegeți **Windows Forms Application**.
3. În câmpul **Name**, scrieți **Salut**, apoi click **OK**.
În acest fel ați creat un nou proiect **Windows Forms**. Acum urmează:
4. Din **Toolbox**, trageți un buton pe suprafața formei.



5. Click dreapta pe butonul cu eticheta `button1`. Alegeți *Properties* din meniul contextual.
6. În fereastra **Properties**, schimbați valoarea proprietății **Text** în *Apasa*.



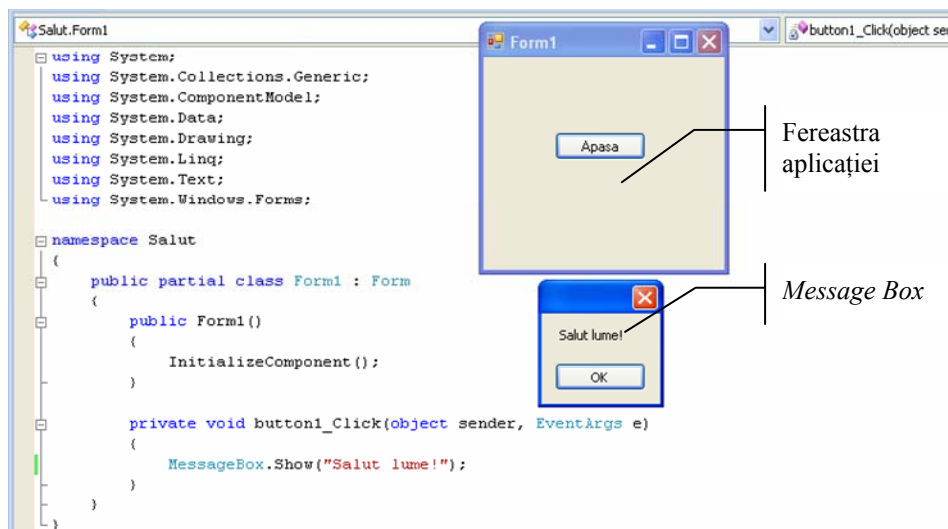
7. Urmează tratarea evenimentului **Click**, pentru ca aplicația să răspundă acestui eveniment. Aveți două variante:
 - a. Dublu click pe suprafața butonului.
 - b. În fereastra **Properties**, click pe iconul "fulger", numit *Events*. Din lista de evenimente la care poate răspunde butonul, selectați evenimentul **Click** și apăsați **Enter**.

În ambele situații se deschide **Editorul de Cod**. Fișierul deschis este **Form1.cs**, iar metoda nou creată este **button1_Click**. Acesta este *handler*-ul de eveniment. Va fi invocată în mod automat la click pe buton.

8. În corpul metodei, scrieți codul:

```
private void button1_Click(object sender, EventArgs e)
{
    MessageBox.Show("Salut lume!");
}
```

9. Compilați și rulați aplicația cu F5.



Observație:

Pentru afișarea mesajelor de informare se utilizează metoda statică `Show()` a clasei `System.Windows.Forms.MessageBox`.

Controale, proprietăți și evenimente**Controale**

Controalele sunt instanțe ale unor clase .NET definite în spațiul de nume **System.Windows.Forms**. Marea majoritate a acestor clase derivă din clasa `Control`, care definește funcționalitatea de bază a oricărui control. Așa se explică faptul că unele proprietăți și evenimente sunt comune tuturor controalelor. Clasele de tip control sunt organizate ierarhic, pe baza rețelei de moștenire, așa cum se vede din diagrama parțială de mai jos:

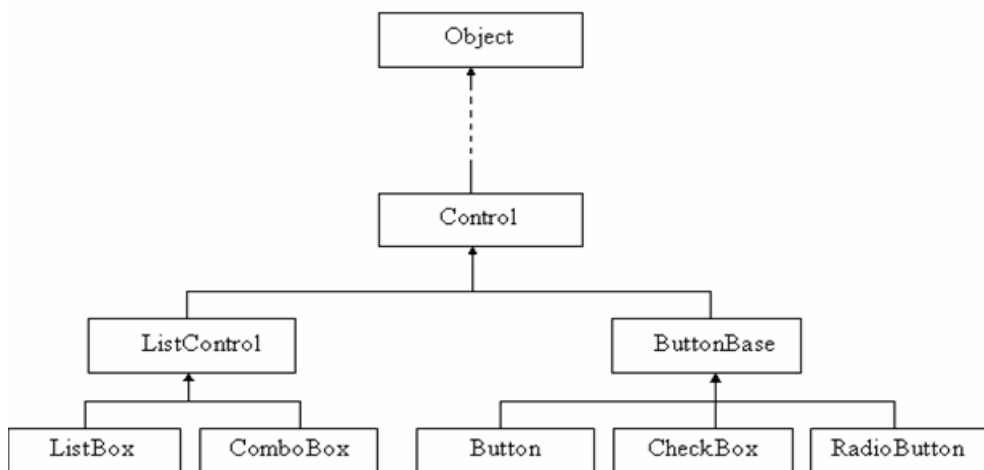


Fig. 6.1 Structura ierarhică a controalelor .NET

În mod evident, diagrama de mai sus este incompletă. Numărul de controale este mult mai mare.

Proprietăți

Proprietățile sunt membri ai claselor din care fac parte. Ele definesc caracteristicile controalelor, de pildă culoarea, poziția, dimensiunile acestora. Controalele moștenesc proprietățile claselor părinte. În unele cazuri le suprascriu (*override*), pentru a obține un comportament particular și definesc de asemenea altele noi.

Fereastra **Properties** a mediului integrat este un instrument important. Selectând un control, aveți acces vizual la proprietățile și evenimentele pe care le suportă controlul. Controalele se creează cu adevărat *run-time*, însă proprietățile lor inițiale pot fi stabilite *design-time* în această fereastră. Desigur că aceste proprietăți se pot modifica programatic în timpul execuției programului.

Evenimente

Începând cu această parte a lucrării, ne vom preocupa doar de evenimentele pe care le generează controalele Windows.

IMPORTANT

*Programele cu interfață grafică cu utilizatorul sunt **conduse de evenimente** (event-driven)*

În momentul în care utilizatorul acționează asupra unui control, cum ar fi click pe un buton, sistemul de operare “simte” și transmite controlului un mesaj. Controlul generează atunci un eveniment specific acelei acțiuni, ca un semn că ceva s-a întâmplat. Programatorul poate să trateze sau nu acel eveniment. Dacă alege să o facă, atunci el trebuie să scrie o metodă *handler*, așa cum este `button1_Click()` în paragraful anterior. Această metodă se apelează în momentul în care evenimentul are loc, iar codul ei asigură funcționalitatea dorită a controlului.

Toate controalele au evenimente pe care le pot genera. Amintiți-vă că evenimentele sunt membrii ai claselor de tip control definite de .NET.

Tratarea evenimentelor

În partea teoretică a lucrării am discutat despre mecanismul tratării evenimentelor în C#. Un obiect publică un eveniment, iar alte obiecte subscriu acestui eveniment. Când evenimentul se declanșează, toate obiectele care au scris sunt informate, în sensul că *handlerele* acestora se vor invoca.

Să presupunem că avem o formă, pe care ați așezat un buton. Vrem să tratăm evenimentul click pe buton. Cine este *publisher* ? Desigur, butonul. Și cine este *subscriber* ? Este fereastra părinte, adică forma. Deci în clasa atașată formei vom defini o metodă handler. Cum are loc subscrierea ? Vom vedea cu exemplul practic care urmează. Din fericire, codul necesar subscrierii și definiția metodei de tratare a evenimentului se generează în mod automat, atunci când utilizați panoul **Properties**.

Cum se crează handler-ele

Pentru același control puteți trata mai multe evenimente, definind desigur câte un *handler* specific. Mai mult decât atât, un același *handler* poate fi utilizat pentru mai multe controale, așa cum vom vedea în continuare.

Vom realiza un proiect cu trei controale de tipuri diferite. Pentru fiecare control tratăm evenimentul **Click**. În mod evident, putem scrie câte un handler care să răspundă fiecărui control. Vom proceda însă altfel. Vom crea un singur handler pentru **Click** pe oricare control. Acest lucru este posibil, datorită semnăturii speciale a handlerelor .NET:

```
private void NumeHandler(object sender, EventArgs e)
{
    // Cod care tratează evenimentul
}
```

sender este o referință la obiectul (controlul) care a generat evenimentul. Astfel, în corpul handlerului putem identifica acel control și putem trata în mod diferențiat.

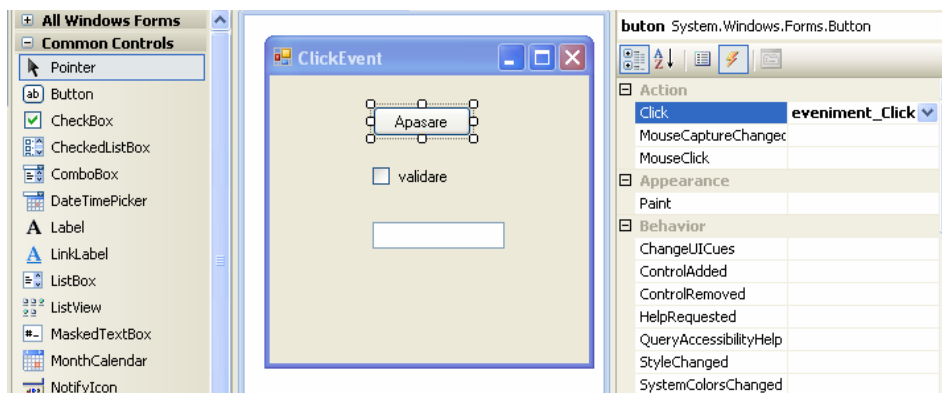
Aplicația *ClickEvent*

Pentru realizarea proiectului, urmați pașii de mai jos:

1. Creați un proiect de tip *Windows Forms*, cu numele **ClickEvent**.
2. Faceți click drept pe suprafața formei și alegeți **Properties**.
3. Schimbați (opțional) titlul formei (implicit este *Form1*) în *ClickEvent*, modificând valoarea proprietății **Text**. Modificați proprietatea **Name** la valoarea *FormaMea*. Este noul nume al clasei formei. Numele implicit era *Form1*.
4. (Opțional). În **Solution Explorer** faceți click drept pe fișierul *Form1.cs* și redenumiți-l *FormaMea.cs*. Dacă fereastra **Solution Explorer** nu e vizibilă, atunci din meniul **View**, alegeți **Solution Explorer**.
5. Trageți cu mouse-ul pe suprafața formei din **Toolbox** trei controale diferite, de exemplu un **Button**, un **CheckBox** și un **TextBox**.
6. Selectați butonul și setați proprietatea **Text** la valoarea *Apasare*, iar proprietatea **Name** la valoarea *buton* (implicit aceasta era *button1*).
7. Selectați check box-ul și setați proprietatea **Text** la valoarea *Validare*, iar proprietatea **Name** la valoarea *verif* (implicit aceasta era *checkBox1*).
8. Selectați căsuța de text și atribuiți proprietății **Name** valoarea *edit* (implicit aceasta era *textBox1*).
9. Selectați butonul și din **Properties** apăsați butonul *Events* (fulgerul). Veți crea un *handler* pentru tratarea evenimentului **Click** generat de buton. Aveți varianta simplă de a face dublu click pe eticheta *Click* din **Properties** sau un simplu dublu click pe suprafața butonului. În această variantă se generează metoda cu prototipul:

```
private void buton_Click(object sender, EventArgs e)
```

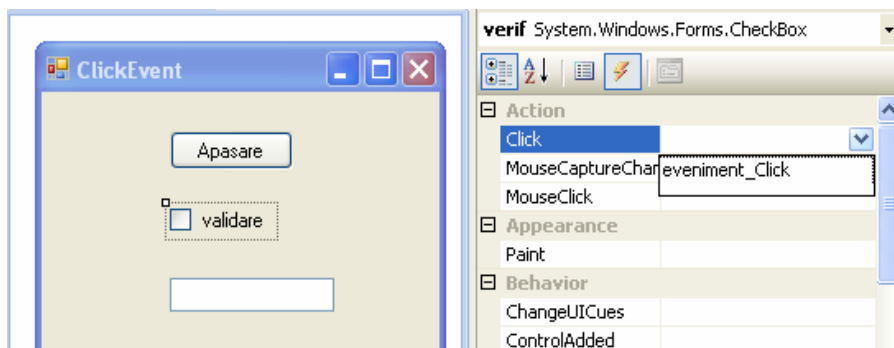
A doua variantă este să dați alt nume metodei, din fereastra **Properties**, editând câmpul din dreapta etichetei *Click*. Puteți pune de pildă numele **eveniment_Click**, apoi apăsați **Enter**.



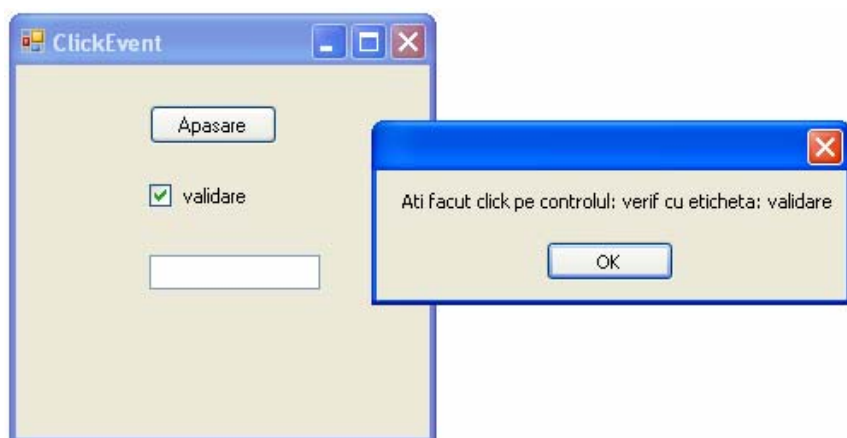
10. În **Editorul de Cod**, completați corpul metodei generate, ca mai jos:

```
private void eveniment_Click(object sender, EventArgs e)
{
    Control c = (Control)sender; // Conversie explicită
                                // (downcast)
    MessageBox.Show("Ati facut click pe controlul: " +
                    c.Name + " cu eticheta: " + c.Text);
}
```

11. Acum subscriem cu același *handler*, pentru evenimentele generate de **CheckBox** și **TextBox**. Selectați pe rând câte unul dintre controale, și din fereastra **Properties**, apăsați butonul **Events**, apoi alegeți din lista din dreapta etichetei **Click** același *handler*: **eveniment_Click**.



12. Compilați și rulați cu **F5** (sau click pe butonul cu iconul  din bara de instrumente Standard).



Observații:

- În corpul handlerului, `c.Name` și `c.Text` sunt proprietăți ale controalelor, reprezentând numele obiectului care reprezintă controlul, respectiv eticheta afișată pe control. `TextBox` nu are proprietatea `Text`.
- Expresia `Control c = (Control)sender;` necesită o conversie explicită (*downcast*), deoarece `sender` e de tip `object`, `Control` e subclasă pentru `Object` (sau `object`), iar conversiile implicite au loc numai de la subclase la superclase.

O privire în spatele scenei

Ne referim la proiectul realizat în paragraful anterior. Am utilizat **Windows Forms Designer** care include între altele **Editorul de Cod** și fereastra **Properties**, pentru scrierea codului și tratarea evenimentul **Click**. Pe măsură ce operam în mod vizual, mediul integrat lucra pentru noi, translatând acțiunile în cod C#. Să vedem:

A). După primul pas de creare a proiectului, aveam o formă goală, cu eticheta *Form1*. S-au generat fișierele de bază ale proiectului: **Form1.cs**, **Form1.Designer.cs** și **Program.cs**.

B). În urma plasării controalelor pe formă și a setării proprietăților **Name** și **Text** pentru formă, buton, butonul de validare și controlul de tip **TextBox**, dar și a redenumirii fișierelor proiectului, în **Solution Explorer** puteți identifica: **FormaMea.cs**, **FormaMea.Designer.cs**, **Program.cs**.

Fișierul **Program.cs**

Fișierul conține metoda **Main()**. Este punctul de intrare în aplicație. În **Solution Explorer**, faceți click drept pe **Program.cs** și alegeți **View Code**:

```
// fragment din codul Program.cs
static void Main()
{
    Application.EnableVisualStyles();
    Application.SetCompatibleTextRenderingDefault(false);
    Application.Run(new FormaMea());
}
```

Partea importantă este faptul că metoda **Main()** din clasa **Program** apelează metoda **Run()**. **Run()** e metodă statică a clasei **Application**. **Run()** lansează în execuție aplicația. O instanță a clasei **FormaMea** este creată și este făcută vizibilă. În acest fișier ar trebui să nu modificați nimic.

Fișierul **FormaMea.cs**

Fișierul implementează în general constructorii clasei, și toate metodele definite de programator.

În **Solution Explorer**, faceți click drept pe fișierul **FormaMea.cs** și alegeți **View Code**:

```
// fragment din codul FormaMea.cs
namespace ClickEvent
{
    public partial class FormaMea : Form
    {
        public FormaMea()
        {
            InitializeComponent();
        }

        private void eveniment_Click(object sender, EventArgs e)
        {
            Control c = (Control)sender; // Conversie explicită
                                         // (downcast)
            MessageBox.Show("Ati facut click pe controlul: " +
                           c.Name + " cu eticheta: " + c.Text);
        }
    }
}
```

Observații:

- Fișierul definește clasa **FormaMea**. Cuvântul cheie **partial** spune faptul că definiția clasei se va completa în alt fișier (**FormaMea.Designer.cs**).
- **FormaMea** moștenește clasa **Form** din **.NET**, pentru ca să aibă toate caracteristicile unei forme Windows.
- Constructorul clasei, care se invocă la crearea formei, apelează metoda **InitializeComponent()**. Această metodă execută tot codul necesar la

inițializarea aplicației: crearea controalelor, setarea proprietăților inițiale, etc. Cu alte cuvinte, tot ce ați setat *design-time*.

- Practic, dumneavoastră veți coda mai mult în acest fișier. Este locul unde definiți de regulă metode, proprietăți și evenimente noi ale clasei.
- Oricând doriți ca o metodă să fie apelată odată cu crearea formei, puteți alegeți constructorul clasei în acest scop:

```
public FormaMea()    // Constructor
{
    // Aici puteți apela metode.
    // Controalele nu există încă
    InitializeComponent();
    // Si aici puteți apela metode.
    // Forma și controalele sunt deja create
}
```

- Când doriți ca un cod să fie executat ca răspuns la o acțiune a user-ului, veți identifica evenimentul care se declanșează la aceea acțiune, și-l veți trata. Veți scrie codul dorit în *handler*-ul de eveniment.

Fișierul *FormaMea.Designer.cs*

Fișierul reține tot ce s-a stabilit în etapa de *design*, relativ la formă și controale. Codul fișierului se generează în mod automat.

În **Solution Explorer**, faceți click drept pe fișierul **FormaMea.Designer.cs** și alegeți **View Code**:

```
// fragment din codul FormaMea.Designer.cs
namespace ClickEvent
{
    partial class FormaMea
    {
        // ...
        private void InitializeComponent()
        {
            // ...
        }
        private System.Windows.Forms.Button buton;
        private System.Windows.Forms.CheckBox verific;
        private System.Windows.Forms.TextBox edit;
    }
}
```

- **IMPORTANT!** Fișierul completează definiția clasei **FormaMea** reținând toate setările facute *design-time*.
- **IMPORTANT!** Remarcăm trei câmpuri private: referințele **buton**, **verif** și **edit**. Câmpurile au apărut sub numele: **button1**, **checkBox1** și

`textBox1`, în momentul în care ați plasat controalele pe formă. Numele actuale s-au generat automat, atunci când din fereastra **Properties** ați modificat proprietatea **Name**.

- **IMPORTANT!** Cele trei referințe sunt **conținute** în clasa `FormaMea`. Amintiți-vă relația care se modelează la **conținere**: **HAS A**. Întradevăr, forma **are un** sau **conține un** buton, **are un** check box și **are un** text box.

Metoda `InitializeComponent()`

Redăm fragmentele semnificative de cod din această metodă:

```
private void InitializeComponent()
{
    // Instanțierea (crearea) celor trei controale
    // prin apelul constructorilor
    this.buton = new System.Windows.Forms.Button();
    this.verif = new System.Windows.Forms.CheckBox();
    this.edit = new System.Windows.Forms.TextBox();

    // Setarea proprietăților inițiale ale controlului
    // buton ca urmare a acțiunilor de design în fereastra
    // Properties și Form Designer.
    this.buton.Location = new System.Drawing.Point(78, 23);
    this.buton.Name = "buton";
    this.buton.Size = new System.Drawing.Size(75, 23);
    this.buton.Text = "Apasare";

    // Subscrierea la evenimentul Click
    this.buton.Click
        += new System.EventHandler(this.eveniment_Click);

    // Setări similare pentru celelalte două controale
    // ...

    // Adăugarea controalelor pe formă
    this.Controls.Add(this.edit);
    this.Controls.Add(this.verif);
    this.Controls.Add(this.buton);

    // Setarea proprietăților formei
    this.Name = "FormaMea";
    this.Text = "ClickEvent";

    // Afișarea controalelor pe formă
    this.ResumeLayout(false);
    this.PerformLayout();
}
```

Se impun câteva precizări cu privire la metoda `InitializeComponent()`:

1. Codul metodei se generează în mod automat. **Nu este indicat să modificați manual acest cod.** Operați cu **Form Designer** și fereastra **Properties** și modificările se produc de la sine.
2. Metoda construiește controalele prin apelul constructorilor acestora.
3. Metoda stabilește proprietățile inițiale ale controalelor și ale formei.
4. Metoda stabilește subscrierea formei la evenimentele generate de către controale. Subscrierea se face astfel:

```
this.buton.Click += new  
    System.EventHandler(this.eveniment_Click);  
this.edit.Click += new  
    System.EventHandler(this.eveniment_Click);  
this.verif.Click += new  
    System.EventHandler(this.eveniment_Click);
```

`this` este referința la obiectul de tip `FormaMea`. O subscriere ca aceasta este la fel de corectă:

```
buton.Click += eveniment_Click;  
edit.Click += eveniment_Click;  
verif.Click += eveniment_Click;
```

Altfel spus, obiectul referit de `this` (forma) subscrie la evenimentele **Click** generate de către fiecare control, setând pentru fiecare caz, același *handler*: `eveniment_Click`.

5. Puteți, cu siguranță, să tratați un eveniment oarecare, scriind manual codul necesar subscrierii și definind un handler adecvat, fără utilizarea panoului **Properties**. Totuși, e mult mai comod să utilizați facilitățile mediului integrat.
6. Ca idee generală, numele fișierului `<nume_formă>.Designer.cs`, ne spune că este generat în întregime de către compilator. În general, nu se fac modificări manuale în acest fișier.

Declanșarea programatică a unui eveniment

Puteți constrânge programatic un control să genereze evenimentul **Click**, chiar dacă userul nu a acționat click pe acel control. Se utilizează metoda `PerformClick()`. Vom face un mic proiect:

Aplicația *RaiseEvent*

1. În meniul **File**, alegeți **NewProject**.
2. În panoul **Templates**, selectați **Windows Forms Application**.
3. În câmpul **Name**, scrieți de exemplu *RaiseEvent* ca nume de proiect.

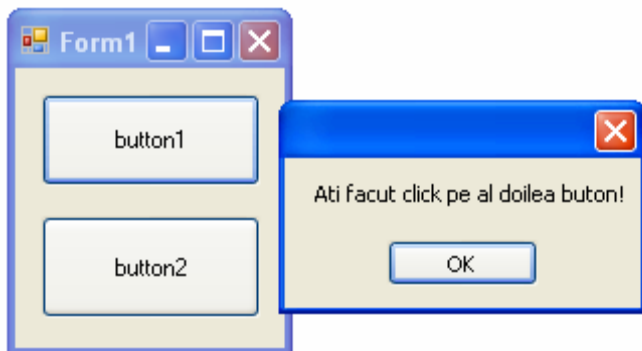
4. Din **Toolbox**, aduceți pe formă două butoane.
5. Faceți dublu click pe primul buton pentru a trata evenimentul **Click**. În **Editorul de Cod**, scrieți în corpul *handler*-ului:

```
private void button1_Click(object sender, EventArgs e)
{
    // Declanșează evenimentul Click pentru al doilea
    // buton
    button2.PerformClick();
}
```

6. Faceți dublu click pe al doilea buton pentru a trata evenimentul **Click**. În **Editorul de Cod**, scrieți în corpul *handler*-ului:

```
private void button2_Click(object sender, EventArgs e)
{
    MessageBox.Show("Ati facut click butonul doi !");
}
```

7. Apăsăți **F5** pentru compilare și rulare.



Mesajul se afișează indiferent de butonul pe care faceți click.

Crearea programatică a unui control

Până în prezent, controalele prezente pe formă le-au fost setate proprietățile *design-time*. Crearea lor are loc totuși *run-time*, odată cu crearea formei. Puteți utiliza oricare *handler* de eveniment pentru a crea și a seta proprietăți *run-time* pentru oricare control Windows.

Aplicația *DtpRuntime*

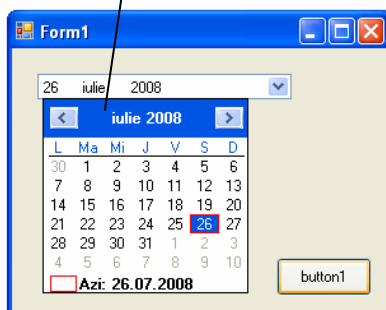
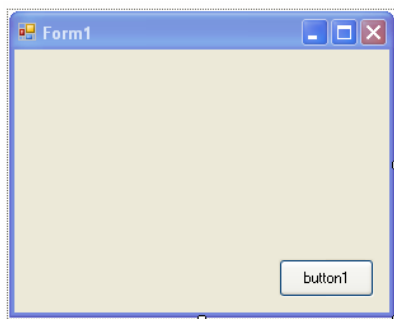
Realizăm o aplicație care în timpul execuției, prin apăsarea unui buton, crează un nou control pe formă, de exemplu un `DateTimePicker`. Un control `DateTimePicker` afișează data curentă.

1. În meniul **File**, alegeți **NewProject**.
2. În panoul **Templates**, selectați **Windows Forms Application**.
3. În câmpul **Name**, scrieți de exemplu *DtpRuntime* ca nume de proiect.
4. Din **Toolbox**, trageți pe formă un buton.
5. Faceți dublu click pe suprafața butonului pentru a trata evenimentul **Click**. În **Editorul de Cod**, scrieți în corpul *handler*-ului:

```
private void button1_Click(object sender, EventArgs e)
{
    // Crează runtime un obiect de tip D.T.P.
    DateTimePicker dtp = new DateTimePicker();

    // Location e poziția colțului stânga sus al D.T.P.
    dtp.Location = new Point(20, 20);

    // Adaugă controlul pe formă
    this.Controls.Add(dtp);
}
```



Capitolul 7

Controalele Windows Forms

Există o mare diversitate de controale predefinite .NET. Într-o clasificare relativă după funcționalitate, distingem:

- Controale pentru declanșarea evenimentelor, cum este **Button**.
- Controale pentru editare de text, cum sunt **TextBox** sau **RichTextBox**.
- Controale pentru afișare de informații pentru utilizator, cum sunt **Label** și **LinkLabel**.
- Controale pentru afișare de liste, cum sunt **ListView** sau **ListBox**.
- Controale pentru afișarea informațiilor din baze de date, cum este **DataGridView**.
- Controale de tip container, cum sunt **GroupBox** sau **Panel**.
- Controale de tip meniu sau bară de instrumente, cum sunt **MenuStrip** sau **ToolStrip**.

La cele de mai sus se mai adaugă și alte tipuri de controale, care vor fi descrise în cele ce urmează, însoțite de exemple practice.

Controlul Button

În spațiul de nume **System.Windows.Forms** se definesc trei controale care derivă din clasa **ButtonBase**: **Button**, **CheckBox** și **RadioButton**.

Button este de departe cel mai comun control. Pentru a inspecta proprietățile și evenimentele unui buton, este suficient să creai rapid o aplicație de tip *Windows Forms*, să aduceți pe formă un buton și să parcurgeți fereastra **Properties**. De multe ori însă nu este de ajuns pentru a vă lămuri. De aceea, vă sfătuim ca să aveți mereu deschis *Helpul* mediului integrat și să vă obișnuiți să căutați rapid acolo oricare clasă, proprietate sau cod exemplificator. Din meniul **Help**, alegeți **Contents**. În panoul din stânga, expandați nodul **.Net Framework SDK**. Apoi expandați **.Net Framework Class Library**. Expandând mai departe nodul **System.Windows.Forms Namespace**, întreaga galerie de clase de tip control vă este accesibilă cu toate informațiile necesare.

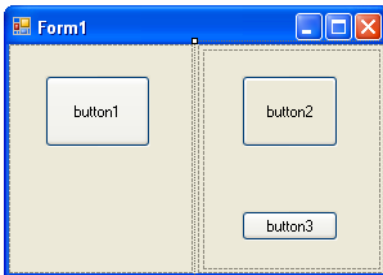
Revenind la **Button**, acesta poate genera mai multe tipuri de evenimente, însă în proporție covârșitoare, evenimentul **Click** este cel mai folosit.

Aplicația ButtonExample

Vom face o mică aplicație care tratează alte două tipuri de evenimente și pune în evidență câteva dintre proprietățile butoanelor.

1. În meniul **File**, alegeți **NewProject**.

2. În panoul **Templates**, selectați *Windows Forms Application*.
3. În câmpul **Name**, scrieți de exemplu **ButtonExample** ca nume al proiectului.
4. Din **Toolbox**, trageți cu mouse-ul pe suprafața formei un control de tip **SplitContainer**. Acesta are aici rolul de a izola partea din stânga de cea din dreapta a formei.
5. În panoul din stânga aduceți un buton, iar în cel din dreapta, plasați două butoane.



6. Acționați dublu click pe *button1* pentru a trata evenimentul **Click**. În corpul handlerului introduceți codul:

```
private void button1_Click(object sender, EventArgs e)
{
    // Dacă butonul este andocat (umple panoul)
    if (button1.Dock == DockStyle.Fill)
    {
        // Îl readucem la forma și eticheta inițiale
        button1.Dock = DockStyle.None;
        button1.Text = "button1";
    }
    else
    {
        // Îl andocăm și schimbăm eticheta
        button1.Dock = DockStyle.Fill;
        button1.Text = "Fill !!";
    }
}
```

7. Selectați *button2* și din fereastra **Properties** setați proprietatea **BackColor** la valoarea **Red**, și proprietatea **Text** la valoarea **Rosu**.
8. Selectați *button2*, iar din fereastra **Properties**, apăsați butonul *Events* (fulgerul). Faceți dublu click pe evenimentul **MouseEnter**. Acesta se declanșează atunci când mouse-ul intră pe suprafața controlului. În corpul *handler*-ului scrieți codul:

```
private void button2_MouseEnter(object sender,
                                EventArgs e)
{
    // Setăm fundalul la culoarea galbenă
    button2.BackColor = Color.Yellow;
    button2.Text = "Galben";
}
```

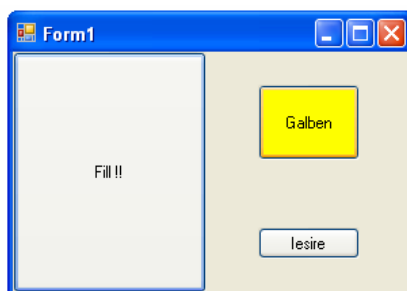
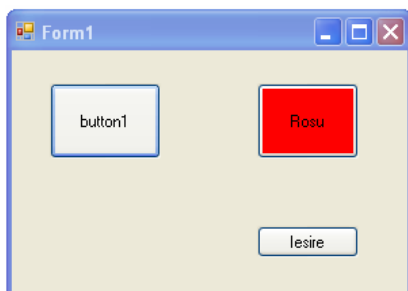
9. Selectați *button2* și faceți dublu click pe evenimentul **MouseLeave**. Acesta se declanșează atunci când mouse-ul iese de pe suprafața controlului. În corpul *handler*-ului scrieți codul:

```
private void button2_MouseLeave(object sender,
                                EventArgs e)
{
    // Resetăm culoarea fundalului și textul
    button2.BackColor = Color.Red;
    button2.Text = "Rosu";
}
```

10. Selectați *button3* și din fereastra **Properties** setați proprietatea **Text** la valoarea **Iesire**.
11. Faceți dublu click pe butonul cu eticheta *Iesire*, pentru a trata evenimentul **Click**. Introduceți în corpul handlerului de eveniment, codul:

```
private void button3_Click(object sender, EventArgs e)
{
    Application.Exit(); // Iesire din aplicație
}
```

12. Apăsați pentru rulare **F5**, sau iconul .



La click pe *button1*, acesta se andochează, apoi la un nou click, revine la starea inițială. La intrarea cu mouse-ul deasupra butonului cu eticheta *Roșu*, acesta își schimbă culoarea și eticheta, iar la ieșirea mouse-ului revine la starea inițială. Utilizați metoda **Application.Exit()** de câte ori doriți să încheiați aplicația.

Controalele *Label* și *LinkLabel*

Controalele **Label** au menirea de a plasa un text pe o formă, cu informații pentru utilizator. Așa cum puteți vedea din fereastra **Properties**, controalele **Label** suportă evenimente, însă de regulă nu le veți trata.

.NET Framework definește și clasa **LinkLabel**, derivată din **Label**. Aceasta afișează o porțiune din text ca un *hyperlink*. La un click pe *hyperlink*, puteți deschide pagini WEB, lansa în execuție diverse aplicații și în general puteți utiliza *handler*-ul de tratare a evenimentului **Click**, pentru orice acțiune.

Aplicația *LabelExample*

Vom realiza un proiect care exemplifică cel mai simplu mod de utilizare a etichetelor.

1. În meniul **File**, alegeți **NewProject**.
2. În panoul **Templates**, selectați **Windows Forms Application**.
3. În câmpul **Name**, scrieți de exemplu **LabelExample** pentru numele proiectului.
4. Din **Toolbox**, trageți cu mouse-ul pe suprafața formei un control de tip **Label**, și două controale de tip **LinkLabel**.
5. Selectați controlul de tip **Label** și în fereastra **Properties** setați valoarea proprietății **Text** la valoarea "Un control de tip **Label**".
6. Selectați primul control **LinkLabel** și în fereastra **Properties** setați proprietatea **Text** la valoarea "Vizitați www.yahoo.com", apoi expandați nodul **LinkArea** și setați proprietatea **Start** la valoarea **9**. Aceasta înseamnă că link-ul începe după al 9-lea caracter al valorii proprietății **Text**.

AutoEllipsis	False
ContextMenuStrip	(none)
Enabled	True
☒ LinkArea	9; 24
Start	9
Length	24
LinkBehavior	SystemDefault

7. Selectați primul control de tip **LinkLabel**, iar în fereastra **Properties**, apăsați butonul **Events**. Faceți dublu click pe evenimentul **LinkClicked**. Scrieți în corpul *handler*-ului codul:

```
private void linkLabel1_LinkClicked(object sender,
    LinkLabelLinkClickedEventArgs e)
```

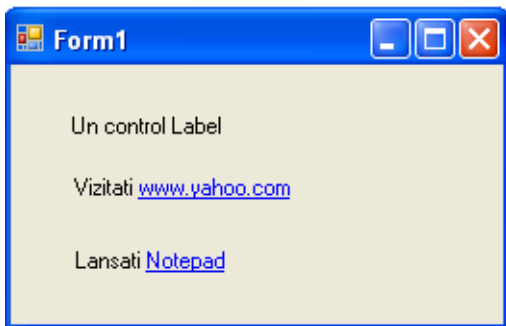
```

{
    // Schimbă culoarea link-ului după click
    linkLabel1.LinkVisited = true;

    // Schimbă textul în controlul Label
    label1.Text = "Se viziteaza www.yahoo.com";

    // Foloseste metoda Process.Start() pentru a
    // deschide un URL cu browserul implicit
    System.Diagnostics.Process.Start(
        "http://www.yahoo.com");
}
    
```

8. Selectați al doilea control **LinkLabel**. În fereastra **Properties** setați proprietatea **Text** la valoarea "*Lansati Notepad*", apoi expandați nodul **LinkArea** și setați proprietatea **Start** la valoarea **8**. Aceasta înseamnă că *link*-ul începe după al 8-lea caracter al valorii proprietății **Text**:



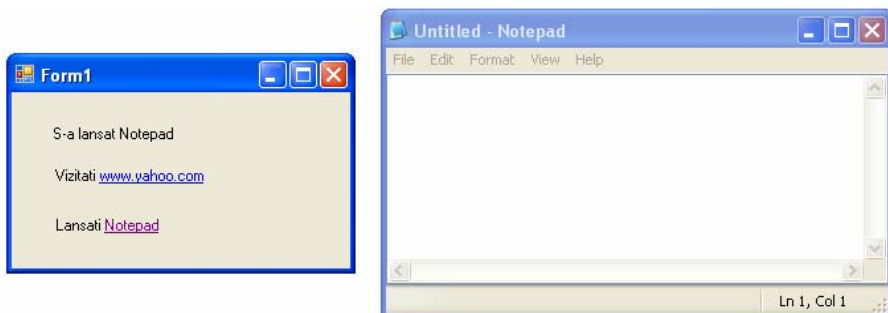
9. Selectați controlul **LinkLabel** cu textul "*Lansati Notepad*", iar în fereastra **Properties** apăsați butonul **Events**. Faceți dublu click pe evenimentul **LinkClicked**. Scrieți în corpul *handler*-ului codul:

```

private void linkLabel2_LinkClicked(object sender,
    LinkLabelLinkClickedEventArgs e)
{
    linkLabel2.LinkVisited = true;
    label1.Text = "S-a lansat Notepad";

    // Folosește metoda Start() pentru a lansa Notepad
    System.Diagnostics.Process.Start("notepad");
}
    
```

10. Lansați în execuție cu **F5**.



Observații:

- Proprietatea **LinkArea** a unui control **LinkLabel**, este o structură care reține poziția primului caracter al link-ului și numărul de caractere din link. Iată cum puteți crea programatic un **LinkLabel** care va fi adăugat pe o formă:

```
void AddLinkLabel()
{
    // Crează o etichetă
    LinkLabel lnk = new LinkLabel();

    lnk.Text = "Vizitati www.yahoo.com";

    // 9 -poziția primului caracter al link-ului
    lnk.LinkArea = new LinkArea(9, 24);

    // Poziția de amplasare pe formă
    lnk.Location = new Point(20, 20);

    // Adaugă controlul pe forma curentă
    this.Controls.Add(lnk);
}
```

Această metodă se poate apela în constructorul clasei formei, sau într-un *handler* de eveniment.

- Metoda **Process.Start()** din spațiul de nume **System.Diagnostics** este supraîncărcată și este utilă oricând doriți să lansați în execuție o altă aplicație din propria aplicație. De exemplu, apelul următor deschide fișierul *Salut.doc* cu *WordPad*:

```
System.Diagnostics.Process.Start("wordpad.exe",
                                @"c:\Salut.doc");
```

Controalele **RadioButton**, **CheckBox** și **GroupBox**

Așa cum am mai arătat, controalele **Button**, **RadioButton** și **CheckBox** derivă direct din clasa **ButtonBase**.

Un buton radio poate fi apăsat sau nu. Se folosește atunci când utilizatorul trebuie să facă o singură alegere între mai multe opțiuni, iar acele opțiuni se exclud reciproc. De exemplu, trebuie să marcheze dacă e bărbat sau femeie, dacă e căsătorit sau nu, etc.

Butoanele radio nu se pun niciodată direct pe formă. Ele se grupează într-un control container, de regulă un **GroupBox**. În interiorul unui control container, radio butoanele devin coerente din punct de vedere logic, în sensul că un singur buton poate fi selectat la un moment dat.

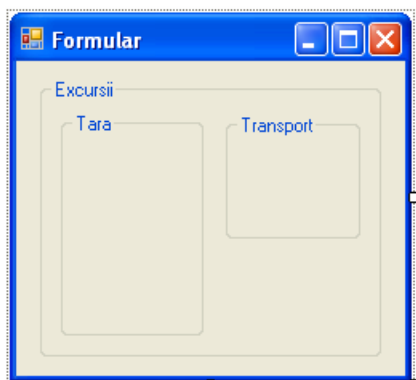
Butoanele de validare (*checkbox*-urile) permit utilizatorului să aleagă una sau mai multe opțiuni. De exemplu, utilizatorul poate să bifeze mai multe sporturi preferate. Pot să fie sau nu incluse în controale container.

Controalele **RadioButton** și **CheckBox** au proprietatea numită **Checked**, care indică dacă controlul este selectat sau nu.

Aplicația **GroupRadioCheck**

Proiectul ilustrează un mod simplu de utilizare a controalelor **RadioButton**, **CheckBox** și **GroupBox**.

1. Din meniul **File**, alegeți **NewProject**.
2. În panoul **Templates**, selectați **Windows Forms Application**.
3. În câmpul **Name**, scrieți de exemplu **GroupRadioCheck** pentru numele proiectului.
4. Selectați forma. În fereastra **Properties** setați proprietatea **Text** la valoarea *Formular*.
5. Din **Toolbox**, trageți cu mouse-ul pe suprafața formei un control de tip **GroupBox**. În fereastra **Properties**, etichetați controlul, setând proprietatea **Text** la valoarea *Excursii*.
6. Din **Toolbox**, trageți cu mouse-ul pe suprafața primului **GroupBox**, încă două controale de tip **GroupBox**. În fereastra **Properties**, selectați-le pe rând și etichetați-le astfel: *Tara*, respectiv *Transport*.



7. Pe suprafața groupbox-ului cu numele *Tara*, aduceți din **Toolbox** cinci radio butoane. Selectați-le pe rând și din fereastra **Properties** setați-le proprietățile **Text** (etichetele) la valorile: *Austria*, *Grecia*, *Franta*, *Italia*, *Germania*.
8. Pe suprafața groupbox-ului cu numele *Transport*, aduceți din **Toolbox** două butoane de validare. Selectați-le pe rând și din fereastra **Properties** setați-le proprietățile **Text** (etichetele) la valorile: *Avionul*, respectiv *Balonul*.
9. Pe suprafața grup box-ului cu eticheta *Excursii*, aduceți două butoane. Selectați primul buton și setați proprietatea **Text** la valoarea *&Verifica*. Ampersandul are ca efect sublinierea caracterului care îi urmează, informând userul că poate să acceseze butonul de la tastatură cu combinația: *Alt + litera subliniată*. Selectați cel de-al doilea buton și setați-i proprietatea **Text** la valoarea *&Iesire*.



10. Faceți dublu click pe butonul *Verifica*, pentru a trata evenimentul **Click**. În *handler*-ul de eveniment, scrieți codul evidențiat cu **bold**:

```
private void button1_Click(object sender,  
                           EventArgs e)
```



```

{
    string msg = "Vom pleca in ";
    // Verifică care dintre butoanele radio e apăsat
    if (radioButton1.Checked)
        msg += radioButton1.Text;
    if (radioButton2.Checked)
        msg += radioButton2.Text;
    if (radioButton3.Checked)
        msg += radioButton3.Text;
    if (radioButton4.Checked)
        msg += radioButton4.Text;
    if (radioButton5.Checked)
        msg += radioButton5.Text;

    // Verifică starea butoanelor de validare
    bool firstDest = false;
    if (checkBox1.Checked)
    {
        msg += " cu " + checkBox1.Text;
        firstDest = true;
    }
    if (checkBox2.Checked)
        if (firstDest)
            msg += " si cu " + checkBox2.Text;
        else
            msg += " cu " + checkBox2.Text;

    // Afișează mesajul
    MessageBox.Show(msg + " !");

    // Restabilește starea inițială
    checkBox1.Checked = false;
    checkBox2.Checked = false;
    radioButton1.Checked = true;
}

```

11. Faceți dublu click pe butonul *Iesire*, pentru tratarea evenimentului **Click**. În *Editorul de Cod* veți scrie:

```

private void button2_Click(object sender, EventArgs e)
{
    Application.Exit(); // Iesire din aplicație
}

```

12. Rulați aplicația cu **F5** sau acționați butonul  din *Standard Toolbar*.

La rulare, obțineți:



Controlul TextBox

Platforma .NET oferă două controale pentru editare de text: **TextBox** și **RichTextBox**. Ambele clase derivă din clasa **TextBoxBase**. **TextBox** este un control cu mai puține facilități decât **RichTextBox**, însă este foarte util pentru introducerea de date mici ca dimensiuni de la tastatură.

Principalii membri ai clasei TextBox

De la clasa părinte **TextBoxBase**, moștenește proprietăți și metode pentru manipularea textului, cum este selectarea textului, copiere, tăiere și lipire din *clipboard*, ca și un mare număr de evenimente. Descriem o parte dintre membrii clasei.

Proprietăți :

Multiline	- Returnează sau setează o valoare booleană care indică dacă <i>text box</i> -ul poate avea mai multe linii
AcceptsReturn	- Returnează sau setează o valoare care indică dacă la apăsarea Enter într-un TextBox multilinie se va trece la linie nouă sau se activează butonul implicit al formei
PasswordChar	- Returnează sau setează un caracter folosit să mascheze parola
Text	- Returnează sau modifică textul în control

Metode:

Copy ()	- Copiază textul selectat în <i>Clipboard</i>
Paste ()	- Lipește în control conținutul <i>Clipboard</i> -ului

Cut()	- Mută textul selectat în <i>Clipboard</i>
Clear()	- Șterge textul din control
Select()	- Selectează un interval de text în control
SetBounds()	- Setează limitele controlului, la locația și la dimensiunile specificate
Undo()	- Anulează ultima operație efectuată în <i>text box</i>

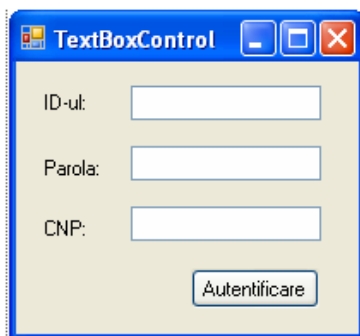
Evenimente:

Click	- Se declanșează la click în control
GotFocus	- Se declanșează când controlul primește focusul
Leave	- Se declanșează când focusul părăsește controlul
TextChanged	- Se declanșează când proprietatea Text se schimbă.

Aplicația TextBoxExample

Proiectul următor relevă câteva dintre caracteristicile controlului **TextBox**.

1. Creați un proiect de tip **Windows Forms Application**, cu numele *TextBoxExample*.
2. Din **Toolbox**, plasați cu mouse-ul pe suprafața formei trei controale de tip **Label**, trei controale de tip **TextBox** și un buton, apoi aranjați-le așa ca în figură:



3. Selectați primul control de tip **TextBox**. Din fereastra **Properties** setați proprietatea **Name** la valoarea **idTextBox**. Setati în același mod pentru al doilea control proprietatea **Name** la valoarea **parolaTextBox** și **cnpTextBox** pentru al treilea control de tip **TextBox**. Pentru eticheta **CNP**, setați proprietatea **Name** la valoarea **cnpLabel**, iar pentru buton, **autentifButton**.

4. Inițial, numai primul câmp text este activ, celelalte două fiind dezactivate. În acest scop se utilizează proprietatea **Enabled**. Dorim ca încă de la apariția formei controalele **parolaTextBox** și **cnpTextBox** să fie dezactivate. Pentru aceasta, vom scrie codul necesar în constructorul formei. Selectați forma, apoi în **Properties** setați câmpul **Name** la valoarea **TextBoxEx**. În felul acesta, ați modificat numele clasei formei, care inițial era **Form1**. Faceți click drept în **SolutionExplorer** pe numele formei și alegeți **View Code**. În constructorul clasei introduceți următoarele:

```
public TextBoxExample()    // Constructorul formei
{
    InitializeComponent();
    // Caracterul '*' maschează parola
    parolaTextBox.PasswordChar = '*';

    // Controlul de tip TextBox pentru parolă
    // este inițial vizibil, dar dezactivat
    parolaTextBox.Enabled = false;

    // Eticheta și TextBox-ul pentru CNP, ca și
    // butonul Autentificare sunt inițial invizibile
    cnpLabel.Visible = false;
    cnpTextBox.Visible = false;
    autentifButton.Visible = false;
}
```

5. Pentru testarea validității ID-ului introdus, tratați evenimentul **PreviewKeyDown**:

```
private void idTextBox_PreviewKeyDown(object sender,
                                       PreviewKeyDownEventArgs e)
{
    // Dacă tasta apăsată este Tab
    if (e.KeyCode == Keys.Tab)
        if (idTextBox.Text == "ionel")
        {
            // ID corect, activăm câmpul pentru parolă
            parolaTextBox.Enabled = true;
        }
        else
            MessageBox.Show("Utilizatorul nu exista!");
}
```

6. Tratați evenimentul **PreviewKeyDown** pentru verificarea parolei:

```
private void parolaTextBox_PreviewKeyDown
(object sender, PreviewKeyDownEventArgs e)
{
    // Dacă tasta apăsată este Enter
    if (e.KeyCode == Keys.Enter)
```

```

{
    // Dacă câmpul Text are valoarea "parola"
    if (parolaTextBox.Text == "parola")
    {
        // Eticheta și controlul de tip text
        // pentru CNP devin vizibile
        cnpLabel.Visible = true;
        cnpTextBox.Visible = true;

        // TextBox-ul pentru CNP primește focusul
        cnpTextBox.Focus();

        // Butonul devine vizibil pe formă
        autentifButton.Visible = true;
    }
    else
    {
        MessageBox.Show("Parola incorecta!");
        // Șterge textul din control
        parolaTextBox.Clear();
    }
}
}

```

7. Pentru CNP, dorim să nu permitem introducerea altor caractere decât cifre. Pentru aceasta, tratați evenimentul **KeyPress** și pentru toate caracterele cu codul ASCII mai mic decât 48 sau mai mare decât 57, setați proprietatea **Handled** la valoarea **true**. În felul acesta, evenimentul nu mai este transmis controlului, iar caracterul apăsător nu va mai fi afișat.

```

private void cnpTextBox_KeyPress(object sender,
                                   KeyPressEventArgs e)
{
    // Dacă nu e cifră, nu se afișează
    if (e.KeyChar < 48 || e.KeyChar > 57)
        e.Handled = true;
}

```

8. Tratați evenimentul **Click** pentru butonul *Autentificare*. Dublu click pe buton în *Form Designer*. Veți cere ca numărul total de cifre să fie 13:

```

private void autentifButton_Click(object sender,
                                   EventArgs e)
{
    // Dacă lungimea CNP este incorectă
    if (cnpTextBox.Text.Length != 13)
    {
        MessageBox.Show("CNP incorect!");
        // Se șterge textul din control
        cnpTextBox.Clear();
    }
}

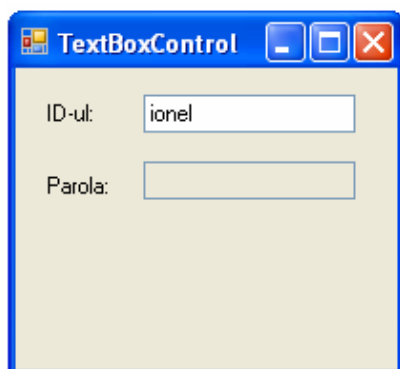
```

```
else  
    MessageBox.Show("Autentificare cu succes!");  
}
```

9. Compilați și rulați cu **F5**.

Utilizatorul va completa primul câmp de text cu ID-ul "*ionel*", după care va apăsa **Tab**, al doilea câmp cu parola "*parola*", după care apasă **Enter**, iar în al treilea va trece un număr format din **13** cifre, apoi va face click pe buton. Aplicația verifică corectitudinea ID-ului, a parolei și a CNP-ului, într-un mod, desigur rudimentar, dar suficient pentru ceea ce ne-am propus.

La rulare, înainte de introducerea parolei, avem:



După introducerea parolei "*parola*" și a unui ID format din 13 cifre:



Veți constată că acele caractere tastate care nu sunt cifre nu apar în *TextBox*-ul pentru *CNP*.

De reținut:

- Proprietatea **PasswordChar** stabilește caracterul care maschează textul introdus de utilizator.

- Proprietățile **Enabled** și **Visible** au valorile **true** sau **false**. Implicit este **true**. Valoarea **false** pentru **Enabled**, aduce controlul în stare inactivă, în timp ce **false** pentru **Visible** face controlul invizibil pe formă. Sunt caracteristici ale tuturor controalelor, întrucât se moștenesc de la clasa de bază **Control**.
- Evenimentele **PreviewKeyDown** sau **KeyPress** au fost alese și în funcție de abilitățile parametrilor *handler*-elor lor. De exemplu, obiectul **KeyPressEventArgs**, parametru al metodei **cnpTextBox_KeyPress** conține proprietatea **KeyChar**, necesară identificării codului ASCII al caracterului introdus.
- Metoda **Focus()**, moștenită de la clasa **Control**, setează focusul pe controlul curent.

Elemente de Stil

Este recomandabil ca în aplicațiile în care folosiți mai multe controale, să introduceți propriile nume pentru referințe și tipuri. Numele trebuie să fie sugestive, adică să indice menirea și tipul controlului. Exemplu: **idTextBox**, **parolaTextBox**, etc. Numele claselor trebuie să înceapă cu literă mare, de exemplu **TextBoxExample**. În felul acesta veți distinge ușor referințele și tipurile.

Focusul Controalelor

Controlul care primește intrările de la tastatură este cel care “are **focusul**”. În oricare moment, un singur control al aplicației poate avea focusul. Schimbarea focusului se face acționând tastele săgeți, tasta **Tab**, sau mouse-ul.

Ordinea în care controalele primesc focusul se stabilește programatic cu ajutorul proprietății **TabIndex**. Controlul cu valoarea **0** pentru **TabIndex** primește primul focusul. Visual C# 2008 permite stabilirea *design time* a focusului controlilor. În meniul **View**, alegeți **Tab Order**, apoi faceți click pe etichete pentru modificări.

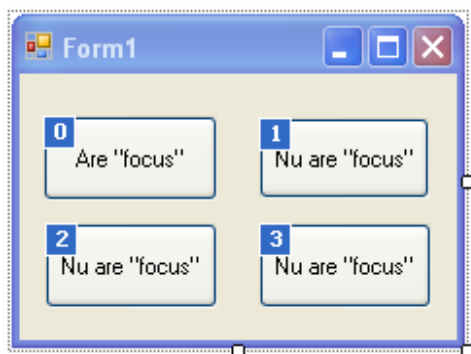


Figura 7.1 Focus stabilit design time

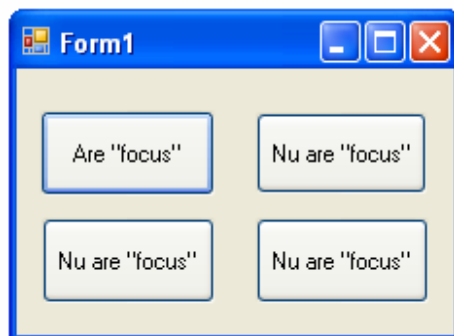


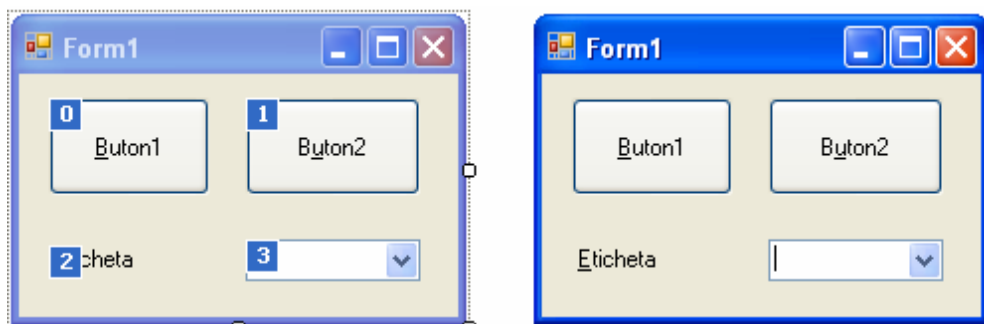
Figura 7.2 Focusul la rulare

Chei de accesare a controalelor

În timpul rulării, un control poate primi în mod direct focusul de la tastatură, sărind peste ordinea stabilită prin **TabIndex**, cu ajutorul unei combinații de taste: **Alt + Caracter**. Este nevoie pentru aceasta ca valoarea proprietății **Text** să conțină caracterul ampersand. Pentru exemplul de mai jos, proprietatea **Text** are valorile: **&Buton1**, **B&uton2**, **&Eticheta**.

Label-urile au **TabIndex**, chiar dacă ele nu pot primi focusul. Dacă le accesați cu o combinație de chei, controlul care urmează etichetei în ordinea de indexare, va primi focusul.

După combinația **Alt+E**, **ComboBox**-ul va primi focusul:



Controalele invizibile sau cele dezactivate nu pot fi accesate cu tasta **Tab**, și nu pot primi focusul, chiar dacă li s-a asociat un index de tabulare *design time* sau *run time*.

Probleme propuse

1. Realizați o aplicație care folosește un text box ca editor de text. Explorați proprietățile clasei **TextBox** și testați-le pentru acest control (Exemplu: culoarea fontului, dimensiune, culoare *background*, etc).
2. Realizați o aplicație care utilizează un **TextBox** pentru introducerea textului. La fiecare caracter introdus, culoarea fundalului și a textului trebuie să se schimbe (alegeți câte o listă de culori complementare pentru text și fundal, care se vor repeta ciclic).
3. Realizați un mic proiect a cărui fereastră reprezintă un formular care trebuie completat de către un elev, atunci când se înscrie la facultate. Pentru preluarea datelor, se vor utiliza controale de tip **TextBox**, **CheckBox** și **RadioButton**. La apăsarea unui buton, toate datele introduse se vor lista într-un control de tip **TextBox**, într-o formatare aleasă de dumneavoastră.
4. Creați o aplicație care preia adresa și numărul de telefon ale unei persoane (introduse în controale de tip text diferite) și afișează într-un **MessageBox** toate informațiile culese.
5. Implementați o aplicație care preia o comandă de produse efectuată de către un client și afișează toate elemente sale

Controalele **MenuStrip** și **ContextMenuStrip**

Meniurile sunt controale familiare în peisajul aplicațiilor de zi cu zi. De exemplu, utilitarul clasic **Notepad** are un meniu din care utilizatorul poate seta diverse acțiuni de formatare a textului sau de editare. Cu ajutorul controlului de tip container **MenuStrip** se pot crea deosebit de ușor asemenea meniuri.

Meniurile de context definite cu **ContextMenuStrip** se accesează cu click drept și sunt folosite deseori în editare de text sau oriunde doriți să aveți acces rapid la anumite opțiuni, cum ar fi *Cut*, *Copy*, *Paste*. Un meniu contextual se asignează întotdeauna unui control, aceluși control pe care faceți click drept.

Principalii membri ai clasei **MenuStrip**

Clasa este bogată în proprietăți, metode și evenimente. Din fericire, nu aveți nevoie decât extrem de rar să modificați valorile implicite ale proprietăților. De asemenea, din mulțimea de evenimente, tratați de regulă doar evenimentul Click.

Urmează o prezentare selectivă a membrilor:

Proprietăți :

Anchor	- Determină modul în care se redimensionează controlul în raport cu containerul său.
BackgroundImage	- Returnează sau setează imaginea afișată în control
ClientSize	- Returnează sau setează înălțimea și lățimea zonei client a controlului.
ContextMenu	- Setează meniul de context asociat
Text	- Reprezintă textul asociat controlului

Metode:

FindForm()	- Returnează o referință la forma pe care se găsește controlul
Hide()	- Ascunde controlul
Show()	- Afișează controlul

Evenimente:

Click	- Se declanșează la click în control
MenuActivate	- Se declanșează când utilizatorul accesează meniul cu tastatura sau mouse-ul
Leave	- Se declanșează când focusul părăsește controlul
TextChanged	- Se declanșează când proprietatea Text se schimbă.

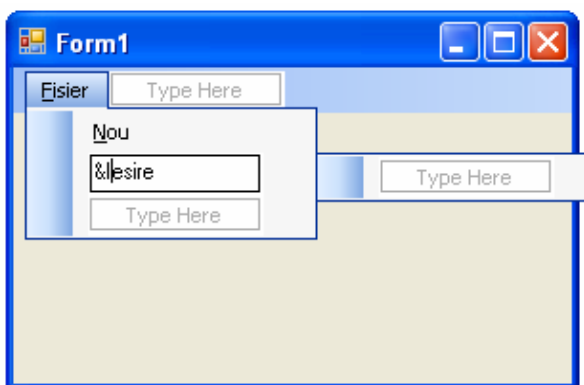
Aplicația *MenuExample*

Proiectul pe care îl vom realiza pentru acomodarea cu cele două controale implementează:

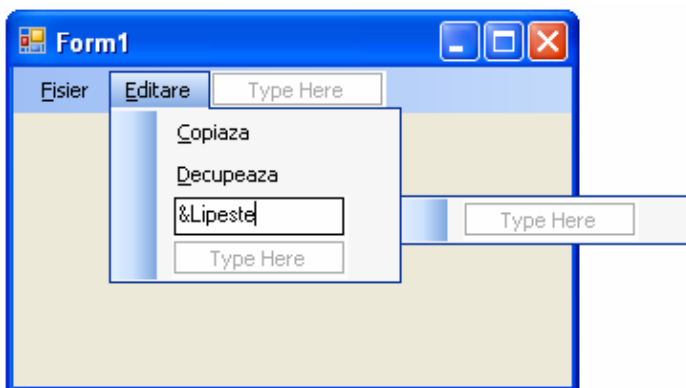
- Un editor de text rudimentar cu ajutorul unui control **TextBox**.
- Operațiile *cut/copy/paste*. Aceste sunt accesibile atât dintr-un meniu cât și dintr-un meniu contextual.

Urmați pașii:

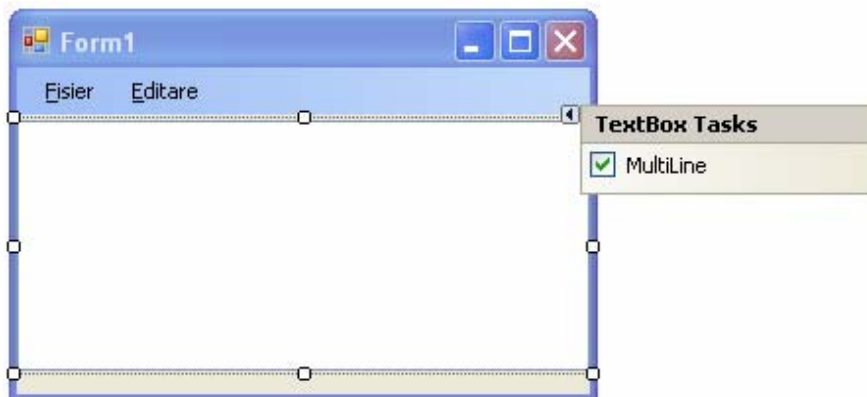
1. Creați un nou proiect de tip **Windows Forms Application**, cu numele *MenuExample*.
2. Din **Toolbox** trageți cu ajutorul mouse-ului un control **MenuStrip** pe suprafața formei.
3. Faceți click pe caseta controlului, acolo unde scrie *Type Here*, și creați un prim meniu numit **Fisier**, cu opțiunile **Nou** și **lesire**:



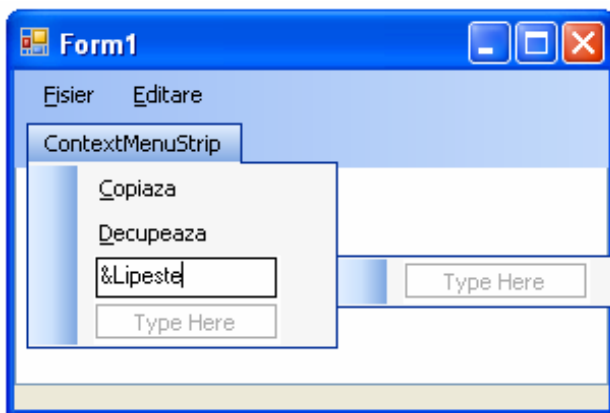
4. Creați un al doilea meniu cu numele **Editare** și opțiunile **Copiază**, **Decupează** și **Lipește**:



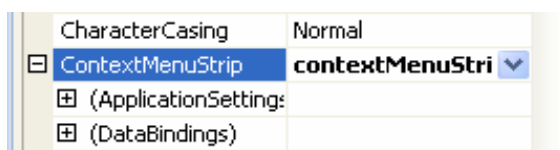
5. Din **Toolbox** alegeți un control **TextBox** și plasați-l pe suprafața formei.
6. Setați pentru **TextBox** proprietatea **Multiline** la **true**. Acest lucru se poate face din **Properties** sau mai simplu cu ajutorul meniului contextual care apare prin click pe săgeata din colțul dreapta sus a controlului:



7. Din **Toolbox** trageți pe suprafața ferestrei aplicației un control de tip **ContextMenuStrip**. Selectați controlul și introduceți opțiunile: **Copiază**, **Decupează**, **Lipește**:

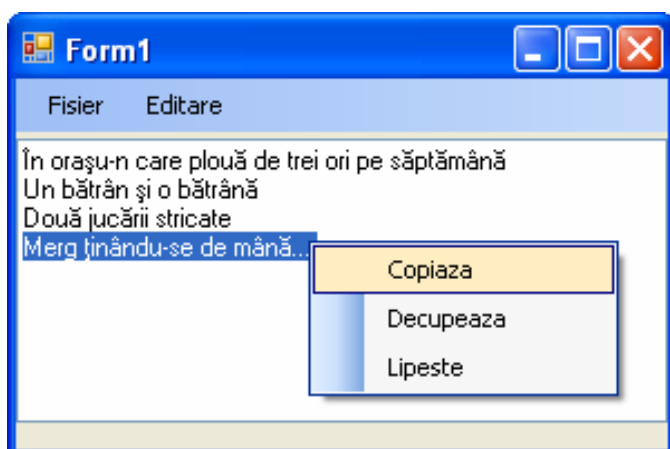


8. Acum veți asigna meniul de context controlului **TextBox**. Selectați controlul **TextBox** și în **Properties** atribuiți proprietății **ContextMenuStrip** valoarea **contextMenuStrip1**, adică referința la obiectul de tip meniu contextual al aplicației:



9. În continuare, veți trata evenimentul **Click** pentru fiecare dintre opțiunile meniului și ale meniului contextual:
- În meniul **Fisier**, dublu click pe opțiunea **Nou**. În corpul metodei *handler*, scrieți: `textBox1.Clear()`; . În felul acesta, ștergeți textul din controlul **TextBox**.
 - În meniul **Fisier**, dublu click pe opțiunea **Iesire**. În corpul metodei *handler*, scrieți: `Application.Exit()`; . Astfel se iese din aplicație.
 - În meniul **Editare**, dublu click pe opțiunea **Copiază**. În corpul metodei *handler*, scrieți: `textBox1.Copy()`; . Metoda copiază în *clipboard* textul selectat.
 - În meniul **Editare**, dublu click pe opțiunea **Decupează**. În corpul metodei *handler*, scrieți: `textBox1.Cut()`; . Metoda șterge din control textul selectat și îl copiază în *clipboard*.
 - În meniul **Editare**, dublu click pe opțiunea **Lipește**. În corpul metodei *handler*, scrieți: `textBox1.Paste()`; . Metoda scrie în control textul memorat în *clipboard*.
 - Selectați meniul contextual din bara gri, din partea de jos a *Form Designer*-ului. Veți face dublu click pe fiecare opțiune a meniului contextual și veți trata evenimentul **Click** în aceeași manieră în care ați procedat cu opțiunile **Copiază**, **Decupează** și **Lipește** ale meniului aplicației.

10. Compilați și rulați cu **F5**.



Forme

O formă este un tip special de control care reprezintă o fereastră. Este folosită ca suport pentru alte controale. Prin setarea proprietăților se obțin diferite tipuri de ferestre, având diverse dimensiuni, aparențe, stiluri și culori. Formele sunt instanțe ale clasei **Form**.

Principalii membri ai clasei Form

Din galeria de membri ai acestei clase, prezentăm:

Proprietăți :

ClientSize	- Returnează și setează suprafața utilă a formei, pe care se pot plasa controale sau se poate desena.
DialogResult	- Setează valoarea pe care o returnează o formă când funcționează ca dialog modal.
Modal	- Stabilește dacă o formă se afișează sau nu ca dialog modal.
Size	- Returnează sau modifică dimensiunile formei.

Metode:

Activate ()	- Activează forma și îi transferă <i>focusul</i>
Close ()	- Închide forma
Invalidate ()	- Invalidează întreaga suprafață a formei, fapt care cauzează redesenarea ei
Show ()	- Afișează forma
ShowDialog ()	- Afișează forma ca dialog modal
Text	- Specifică un text în bara de titlu

Evenimente:

Closing	- Se declanșează când forma se închide
Load	- Se declanșează înainte ca forma să fie afișată pentru prima oară.

Crearea formelor

1. Crearea programatică a formelor

În mod programatic, o formă se crează foarte simplu:

```
Form f = new Form();    // Se instanțiază forma
f.Show();               // Se afișează
```

O metodă obișnuită de creare a unei forme este de a defini o clasă derivată din clasa **Form**:

```
class FormaMea : Form
{
    public FormaMea() // Constructorul clasei
    {
    }
    // Alți membri ai clasei (câmpuri, proprietăți, metode)
}
```

Dintr-o altă metodă a aplicației, instanțiați forma:

```
FormaMea f = new FormaMea();
f.Show();
```

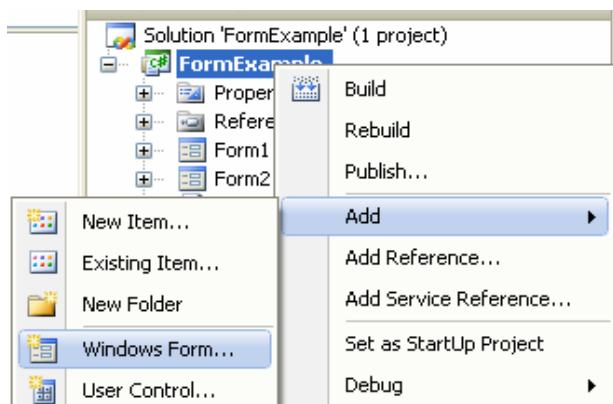
2. Crearea formelor cu ajutorul *Form Designer*-ului

La crearea unui nou proiect de tip *Windows Forms*, mediul integrat generează o clasă cu numele implicit **Form1**, iar această clasă se instanțiază în **Main()**:

```
Application.Run(new Form1());
```

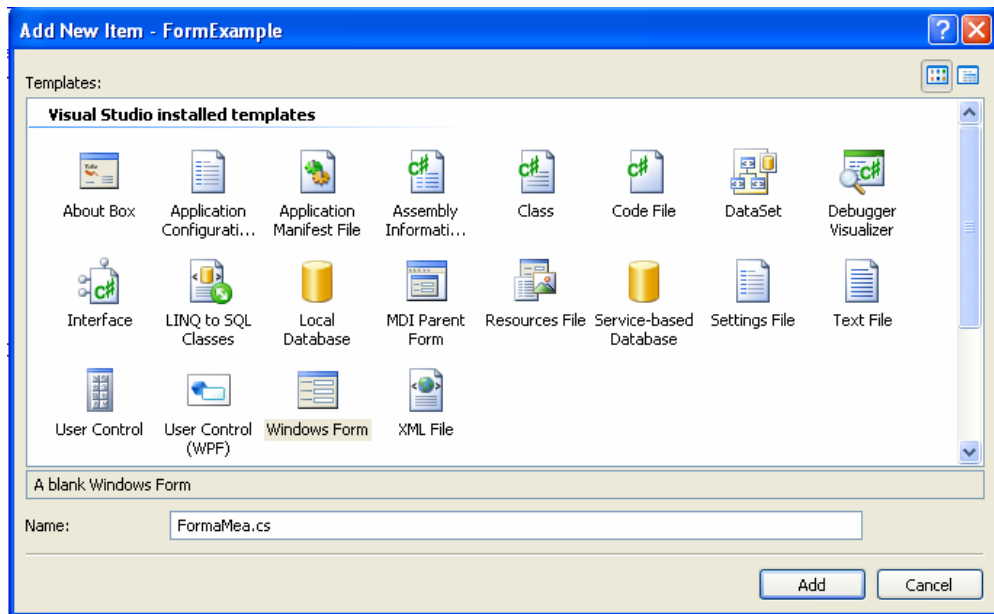
Puteți adăuga proiectului oricâte alte forme, în felul următor:

În **Solution Explorer**, acționați click drept pe numele proiectului, alegeți *Add*, apoi *Windows Form...*:



Ca variantă alternativă, click drept pe numele proiectului, alegeți *Add*, apoi *New Item...*

În fereastra *Add New Item* – [nume proiect] selectați iconul *Windows Form* și introduceți un nume pentru noua formă:



Dialoguri modale și dialoguri nemodale

Una dintre forme este fereastra de bază a aplicației. Celelalte forme reprezintă ferestre care pot fi făcute să apară la un moment dat.

Dialoguri modale

Anumite tipuri de ferestre se prezintă ca dialoguri cu utilizatorul. Acesta introduce date în controale, pe care apoi le validează sau nu (apăsând de exemplu butoanele **OK** sau **Cancel**). Acest tip de fereastră dialog nu permite utilizatorului să acceseze alte ferestre ale aplicației până când dialogul nu este închis. De exemplu, în *Microsoft Word*, când alegeți **Open** în meniul **File**, se deschide un dialog care vă cere să alegeți un fișier. Fereastra blochează aplicația și nu puteți întreprinde nici o altă acțiune în editor, până în momentul în care ați închis dialogul. Un asemenea tip de dialog, se numește **dialog modal**.

Un dialog modal este o formă care se deschide cu metoda `ShowDialog()`:

```
class Forma : Form
{
    // membrii clasei
}

Forma f = new Forma();
f.ShowDialog();
```

Aplicația *DialogModal*

Realizăm un proiect de tip consolă care lansează o formă ca dialog modal. Urmăți pașii:

1. În meniul **File**, alegeți *New Project*.
2. În panoul dialogului *New Project* selectați *Empty Project*, iar în caseta text *Name*, introduceți numele proiectului: *DialogModal*.
3. În Solution Explorer, executați click drept pe numele proiectului și alegeți *Add*, apoi *New Item...*
4. În dialogul *Add New Item*, selectați iconul *Code File*, apoi în caseta *Name*, introduceți numele fișierului: *Modal.cs*.
5. Fișierul *Modal.cs* se deschide în *Editorul de Cod*. Introduceți codul:

```
using System;
using System.Windows.Forms;

class Forma : Form
{
}

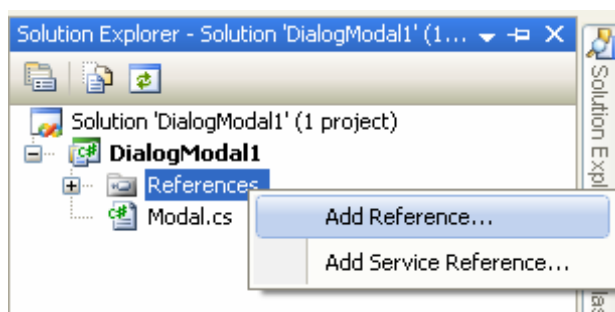
class Program
{
    public static void Main()
    {
        Forma f = new Forma();    // Instanțiere

        // Text în bara de titlu
        f.Text = "Dialog Modal";

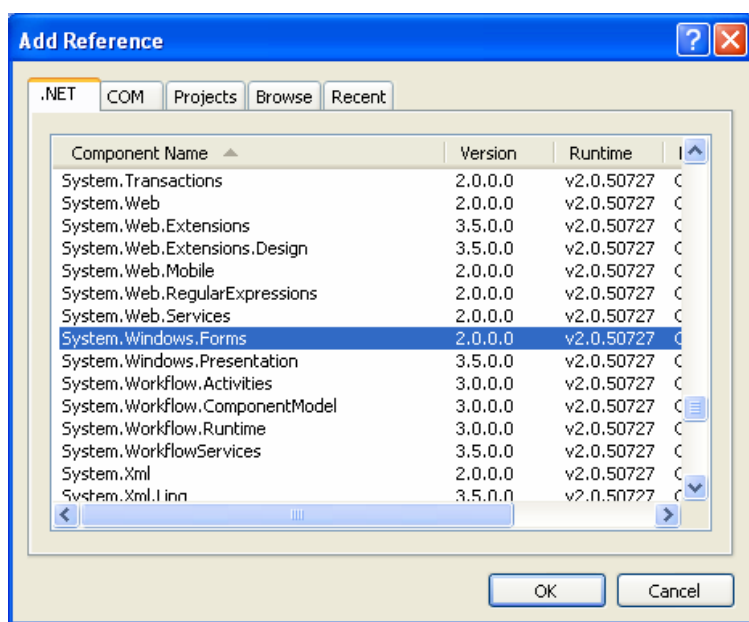
        // Dialogul modal devine vizibil
        f.ShowDialog();

        Application.Run();        // Lansează aplicația
    }
}
```

6. Încă nu putem compila, pentru că proiectul de tip consolă nu introduce în mod automat referințe la bibliotecile dinamice **.NET** care implementează spațiile de nume **System.Windows** și **System.Windows.Forms**. Vom aduce aceste referințe astfel: în **Solution Explorer**, click drept pe folderul *References*:



7. În dialogul **Add Reference**, selectați pe tab-ul **.NET** intrarea **System**, apoi repetați pasul 6 și adăugați o referință la bibliotecile **System.Windows.Forms**:



8. Compilați și rulați aplicația cu **F5**. La rulare obțineți:



Dialoguri nemodale

Dialogurile nemodale sunt ferestre care nu întrerup execuția aplicației. Un dialog nemodal nu împiedică utilizatorul să execute acțiuni în alte ferestre ale aplicației. Se utilizează în special ca bare de instrumente. Gândiți-vă la *toolbox*-urile din *Paint* sau *Word*, sau chiar **Toolbox** din *Visual C# Express 2008*.

Afișarea se face cu metoda `Show()`; La închiderea unei forme cu `Close()`, toți membrii clasei (de exemplu controalele de pe formă) se distrug.

Ca exercițiu de creare programatică a unui dialog nemodal, reluați toți pașii aplicației *DialogModal* (redenumiți ca *DialogNemodal*) și faceți o singură modificare în cod: înlocuiți `f.ShowDialog()` cu `f.Show()`;

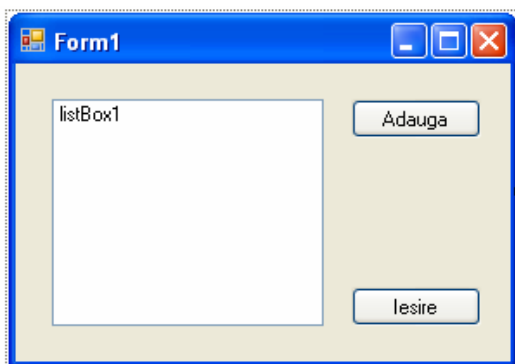
Aplicația *ModalDialogExample*

Descrierea aplicației:

- Construiește un dialog modal care se deschide la apăsarea unui buton al formei părinte a aplicației.
- Datele pe care utilizatorul le introduce într-un control al dialogului modal se transferă unui control al clasei părinte.
- Anticipează modul de utilizare a unui control **ListBox**.

Urmați pașii:

1. Creați un nou proiect de tip **Windows Forms Application**, cu numele *ModalDialogexample*.
2. Din **Toolbox** plasați pe formă cu ajutorul mouse-ului un control de tip **ListBox** și două butoane. Setati proprietatea **Text** pentru celor două butoane astfel:



3. Adăugați o nouă formă proiectului. În **Solution Explorer**, click dreapta pe numele proiectului, alegeți *Add*, apoi *Windows Form...* și validați cu **OK**.

4. În **Solution Explorer**, dublu click pe numele *Form2* (în cazul în care nu ați redenumit noua formă).
5. Aduceți din **Toolbox** un control **TextBox** și un buton, ca în figură:



6. În **Solution Explorer**, acționați click drept pe numele *Form2*, apoi alegeți **View Code**. În clasa **Form2** definiți membrii:

```
// Câmp care reține textul introdus în textBox1
private string item;

// Proprietate care returnează valoarea textului
public string Item
{
    get { return item; }
}
```

7. Tratăm evenimentul **Click** generat la apăsarea butonului **OK**. Acționați dublu click pe buton. Completați codul evidențiat în **Bold**:

```
private void button1_Click(object sender, EventArgs e)
{
    // Salvăm textul introdus în control.
    item = textBox1.Text.Trim();

    // Închidem forma Form2.
    Close();
}
```

8. Dorim ca **Form2** să se închidă și la apăsarea tastei **Enter**. Tratăm evenimentul **KeyDown** pentru controlul **TextBox**. Selectați controlul. În fereastra **Properties** apăsați butonul **Events** (fulgerul) și faceți dublu click pe evenimentul **KeyDown**. Introduceți codul:

```
private void textBox1_KeyDown(object sender,
                               KeyEventArgs e)
{
    // Dacă tasta apăsată are codul tastei Enter
    if (e.KeyCode == Keys.Enter)
    {
        item = textBox1.Text.Trim(); // Memorăm textul
        Close(); // Închidem forma
    }
}
```

9. În scopul de a ni se permite să accesăm proprietatea **Item** a clasei **Form2** din metodele clasei **Form1**, modelăm o *relație de conținere (containment)*. Definim în **Form1** un câmp privat, o referință la **Form2**. În **Solution Explorer**, click drept pe **Form1** și alegeți **View Code**. În fișierul **Form1.cs**, în clasa **Form1**, scrieți:

```
private Form2 f;
```

10. Tratăm evenimentul **Click** care se generează la click pe butonul **Adauga**, aflat pe forma **Form1**. În **Solution Explorer**, faceți dublu click **Form2**, apoi în **Form Designer** acționați dublu click pe butonul **Adauga**. În corpul *handler*-ului introduceți codul:

```
private void button1_Click(object sender, EventArgs e)
{
    f = new Form2();           // Creăm forma

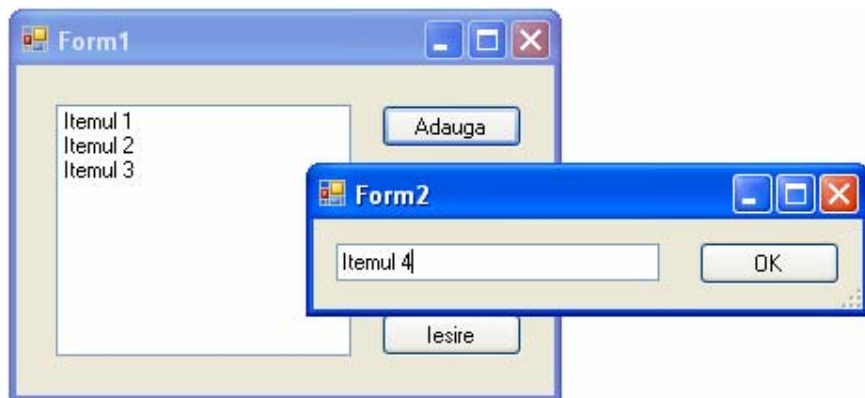
    f.ShowDialog();           // Afișăm dialogul modal
    if ( f.Item != "" )       // Dacă există text
    {
        // Adăugăm itemul în listBox1
        listBox1.Items.Add(f.Item);
    }
}
```

11. Pentru ieșire din aplicație, faceți dublu click pe butonul **Iesire**. În corpul *handler*-ului evenimentului **Click**, scrieți:

```
Application.Exit();          // Ieșire din aplicație
```

12. Compilați și lansați în execuție cu **F5**.

La rulare, prin apăsarea butonului **Adauga**, se deschide dialogul modal **Form2**. Când se apasă **OK** sau **Enter**, se închide dialogul **Form2**, iar textul introdus apare ca nou item în controlul **ListBox** de pe **Form1**.



Notă:

- Metoda **Trim()** a clasei **string** înlătură caracterele albe de la începutul și sfârșitul stringului.
- Metoda **Add()** din apelul **listBox1.Items.Add(f.Item)** ; adaugă un item într-un controlul referit de **listBox1**.
- Metoda **Close()** închide forma.

IMPORTANT !

Când apelezi **Close()** pentru un dialog modal, fereastra nu se distruge ci devine invizibilă. Aceasta, deoarece după închiderea ferestrei, aveți de regulă nevoie de membrii clasei, ca în aplicația de față:

```
f.ShowDialog();           // Afișăm dialogul modal
// În acest punct fereastra este închisă!
// f.Item există și după închiderea dialogului, deoarece
// dialogul (obiectul de tip Form2) nu este distrus,
// ci numai ascuns.
if ( f.Item != "" )
{
    listBox1.Items.Add(f.Item);
}
```

De reținut:

- Dialogurile modale se afișează cu metoda **ShowDialog()**, iar cele nemodale cu metoda **Show()**.
- Afișarea unei forme cu **Show()** se poate face alternativ prin setarea proprietății **visible** la valoare **true**. Ascunderea formei se poate face și prin setarea proprietății **Hide** la **false**.
- La închiderea formei cu **Close()**, dialogurile nemodale se distrug (se distruge instanța clasei).
- Când o formă este afișată ca dialog modal, apelul metodei **Close()** sau un click pe butonul **Close** (butonul cu un **x** în colțul dreapta sus a formei) nu distruge forma ci o ascunde, ea putând fi făcută vizibilă ulterior.
- Codul care urmează apelului **ShowDialog()** nu se execută până când forma se închide. După închiderea dialogului modal, puteți utiliza în continuare referința la clasă și toți membrii clasei.

Butoane de validare a datelor

Când închide un dialog modal, utilizatorul ar trebui să aibă cel puțin două opțiuni: să păstreze datele pe care le-a introdus în controale sau să renunțe la ele. Pentru aceasta, are nevoie de două butoane, să le spunem **OK** și **Cancel**. Evident,

nu este suficient să etichetați cele două butoane astfel ca să lucreze în mod adecvat. Ca să joace rolurile dorite, se setează proprietatea `DialogResult` a clasei `Button`, astfel:

```
Button b1 = new Button();
Button b2 = new Button();
b1.Text = "OK";           // Poate fi și altă etichetă
b2.Text = "Cancel";       // Poate fi și altă etichetă

// Setăm b1 să returneze DialogResult.OK la click
b1.DialogResult = DialogResult.OK;

// Setăm b2 să returneze DialogResult.Cancel la click
b2.DialogResult = DialogResult.Cancel;
```

În felul acesta, `b1` va fi butonul **OK** al formei, iar `b2` cel de închidere cu **Cancel**. `DialogResult` este o enumerare a cărei membrii sunt: `OK`, `Cancel`, `Abort`, `Retry`, `None`, `Ignore`, `Yes`, `No`.

Cum detectăm în cod faptul că o formă a fost închisă cu **OK** sau cu **Cancel**? Vom analiza valoarea de retur a metodei `ShowDialog()`, care este tocmai una dintre valorile membrilor enumerării și anume, valoarea returnată de butonul apăsat. Secvența standard este:

```
DialogResult rez = f.ShowDialog();

switch (rez)
{
    case DialogResult.OK :
        // Ceea ce doriți să se execute dacă s-a ieșit cu OK
        break;
    case DialogResult.Cancel :
        // Cod care se execută în caz că s-a acționat Cancel
        break;
}
```

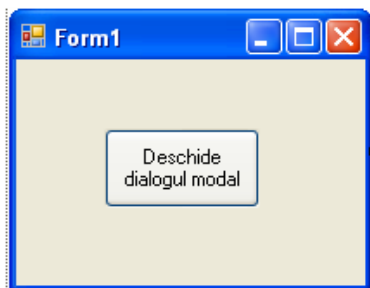
De ce vrem să știm dacă forma a fost închisă cu **OK** sau cu **Cancel**? Foarte simplu: pentru că dacă închideți cu **OK**, forma este doar ascunsă, și o puteți utiliza în continuare, ca și datele introduse, în vreme ce dacă ați închis cu **Cancel**, forma se distruge împreună cu toate controalele și datele reținute în membrii clasei.

Pentru edificare, vom realiza un mic proiect de tip *Windows Forms*.

Aplicația OkCancel

Aplicația are două butoane, cu etichetele **OK** și **Cancel**. Primului buton i se atribuie rolul de validare a datelor, iar celui alt de renunțare. Amândouă închid forma. Se afișează câte un *message box* la închiderea formei pentru fiecare caz.

1. Creați un nou proiect de tip **Windows Forms Application**, cu numele *OkCancel*.
2. Din **Toolbox** plasați pe formă un buton. Atribuiți proprietății **Text** valoarea "Deschide dialogul modal".



3. Adăugați o formă nouă proiectului: în **Solution Explorer**, click dreapta pe numele proiectului, alegeți *Add*, apoi *Windows Form*. Clasa formei se va numi *Form2*.
4. În **Solution Explorer**, faceți dublu click pe *Form2*. Plasați pe suprafața acestei forme două butoane, pentru care setați textele *OK* și *Cancel*.
5. Tratăm evenimentul **Load** pentru *Form2*, pentru a putea seta proprietățile **DialogResult** pentru cele două butoane. Desigur că puteți face acest lucru și din fereastra **Properties**. Executați dublu click pe suprafața formei. Scrieți codul:

```
private void Form2_Load(object sender, EventArgs e)
{
    // Setează button1 să returneze OK la click
    button1.DialogResult = DialogResult.OK;

    // Setează button2 să returneze Cancel la click
    button2.DialogResult = DialogResult.Cancel;
}
```

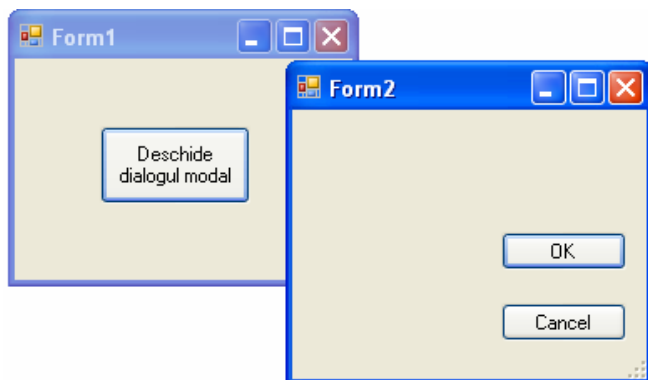
6. Tratăm evenimentul **Click** pentru butonul formei *Form1*. Faceți dublu click pe buton. În metoda *handler*, introduceți codul evidențiat în **Bold**:

```
private void button1_Click(object sender, EventArgs e)
{
    Form2 f = new Form2();
    DialogResult rez = f.ShowDialog();

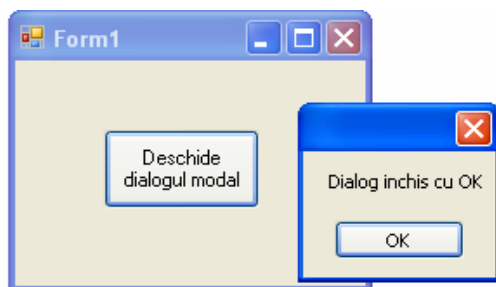
    switch (rez)
    {
        case DialogResult.OK:
            MessageBox.Show("Dialog inchis cu OK");
            break;
    }
}
```

```
        case DialogResult.Cancel:
            MessageBox.Show("Dialog inchis cu Cancel");
            break;
    }
}
```

La execuție, înainte de închiderea *Form2*, avem:



După închiderea formei cu **OK**:



De reținut:

- Puteți desemna butoane care returnează valori egale cu cele ale unor membri ai enumerării **DialogResult**.
- Valoarea returnată la acționarea click asupra unui asemenea buton este aceeași pe care o returnează metoda **ShowDialog()**. Acest lucru permite ca ulterior închiderii formei să se dirijeze fluxul de execuție al programului în funcție de rezultatul întors de formă.

Probleme propuse

1. Testați metoda **MessageBox.Show(text, titlu, butoane)** cu toate butoanele disponibile.

2. Testați metoda `MessageBox.Show()` în versiunea cu **4** parametri, folosind pe rând toate butoanele și toate iconurile disponibile.
3. Realizați o aplicație care deschide un dialog modal la acționarea unui buton de pe forma principală. Dialogul modal conține un formular realizat în principal cu controale *text box*, în care se rețin date personale despre clienții unei firme. La închiderea dialogului modal cu **OK**, datele se afișează într-un format ales de dumneavoastră, într-un control *text box* de pe forma părinte. Dacă dialogul modal s-a închis cu butonul **Cancel**, atunci nu se salvează datele.

Dialoguri predefinite

.NET oferă câteva dialoguri predefinite pe care le puteți invoca în aplicații. Dialogurile sunt forme. Când în *Notepad* sau *Microsoft Word* acționați **File -> Open**, **File -> Save As** sau **File -> Page Setup**, invocați de fapt unul dintre dialogurile standard *Windows*.

Clașele **.NET** care reprezintă dialogurile predefinite sunt: **MessageBox**, **OpenFileDialog**, **SaveFileDialog**, **FontDialog**, **ColorDialog**, **PageSetupDialog**, **PrintDialog** și **PrintPreviewDialog**.

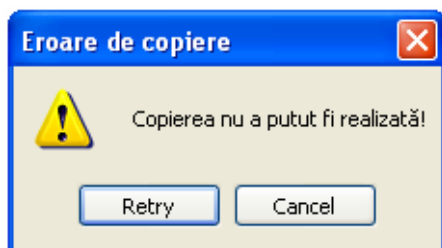
Toate aceste dialoguri sunt **dialoguri modale**.

Descrierea dialogurilor predefinite

MessageBox este cel mai comun dialog predefinit. Clasa definește metoda statică `Show()` în mai multe versiuni. Cea mai simplă versiune, cu un singur parametru, a fost deja folosită în această lucrare. În exemplu se apelează o metodă `Show()` cu patru parametri:

```
MessageBox.Show("Copierea nu a putut fi realizată!",
                "Eroare de copiere",
                MessageBoxButtons.RetryCancel,
                MessageBoxIcon.Exclamation);
```

Efectul pe ecran este :



Icon-urile pe care le poate afișa un *message box* sunt:

Asterisc, Error, Exclamation, Hand, Information, None, Question, Stop și Warning. Aceștia sunt membrii ai enumerării **MessageBoxIcon**, iar semnificația lor este evidentă.

Butoanele pe care le poate afișa un *message box* sunt: **AbortRetryIgnore** (3 butoane), **OK**, **OKCancel** (2 butoane), **YesNo** (2 butoane), **YesNoCancel** (3 butoane). Aceste entități sunt membri ai enumerării **MessageBoxButtons**.

În aplicația pe care o vom realiza în cadrul temei **MDI – Multiple Document Interface** vom arăta cum dați funcționalitate butoanelor de pe formă.

OpenFileDialog moștenește clasa **FileDialog**. Dialogul (forma) este o instanță a clasei. Permite utilizatorului să selecteze un fișier pentru a fi deschis. Atenție, dialogul nu deschide fișierul! Acest lucru cere un mic efort de programare, așa cum vom vedea în subcapitolul următor. Numele fișierului selectat se atribuie în mod automat proprietății **FileName**. Formatul fișierelor care trebuie deschise se stabilesc cu ajutorul proprietății **Filter**.

SaveAsDialog moștenește clasa **FileDialog**. Dialogul este folosit în scopul alegerii unei locații pentru salvarea unui fișier. Dialogul nu salvează fișierul. Numele fișierului selectat se atribuie în mod automat proprietății **FileName**. Clasa poate crea un nou fișier sau poate suprascrie unul existent.

FontDialog derivă din clasa **CommonDialog**. Utilizatorul poate alege un font dintre cele instalate pe calculator prin simpla selectare cu mouse-ul.

PageSetupDialog are ca și clasă de bază directă **CommonDialog**. Utilizatorul poate stabili setări pentru pagină, cum ar fi marginile, formatul, etc.

PrintDialog derivă din clasa **CommonDialog**. Dialogul permite alegerea unei imprimante, setarea formatului de imprimare, alegerea unei porțiuni de document care va fi printată și invocarea imprimantei.

PrintPreviewDialog este derivată din clasa **Form**. Dialogul permite examinarea unui document înainte de imprimare.

Indicație:

Alegeți un utilitar, cum este *Microsoft Word* și revedeți toate aceste dialoguri, accesându-le pe rând din meniul **File**. Merită să o faceți, pentru că acum le vedeți cu ochii programatorului.

Afișarea dialogurilor predefinite

Dialogurile standard Windows sunt forme .NET. În plus, sunt dialoguri modale. Așadar, afișarea lor se face cu ajutorul metodei **ShowDialog()**:

Exemplu:

Iată cum afișați dialogul **OpenFileDialog**:

```
if (openFileDialog.ShowDialog() ==  
    System.Windows.Forms.DialogResult.OK)  
{  
    // Cod care deschide fisierul  
}
```

`openFileDialog` este o referință la un obiect de tip `OpenFileDialog`. Referința se crează în mod automat în momentul în care din **Toolbox** alegeți un control **OpenFileDialog** și îl plasați pe suprafața formei.

Valoarea de retur a metodei este egală cu unul dintre membrii enumerării `DialogResult`. Aceștia sunt: **OK**, **Cancel**, **Abort**, **Retry**, **None**, **Ignore**, **Yes**, **No**. Fiecareia dintre aceste valori îi corespunde un buton pe suprafața dialogului. În mod implicit, dialogurile predefinite afișează butoanele **OK** și **Cancel**, însă prin setarea proprietăților corespunzătoare ale dialogului, le puteți afișa și pe celelalte.

Codul de mai sus se interpretează astfel: `ShowDialog()` afișează dialogul **OpenFileDialog**. Dacă utilizatorul apasă butonul **OK**, atunci se execută codul dintre acolade. Ce anume trebuie să scrieți, vom vedea la secțiunea “**Controlul RichTextBox**”.

Pentru afișarea celorlalte controale predefinite, se procedează similar, cu singura diferență că veți înlocui referința `openFileDialog` cu una dintre referințele: `saveFileDialog`, `printDialog`, `pageSetupDialog`, etc.

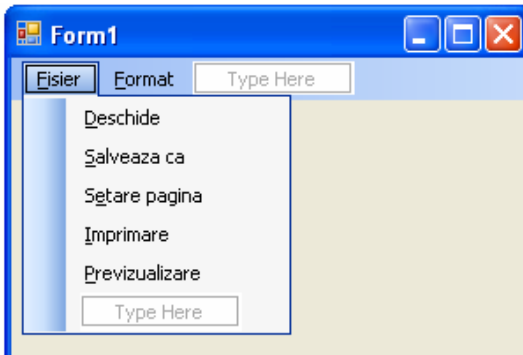
Observație:

Toate cele șapte dialoguri predefinite sunt *dialoguri modale*, deoarece utilizează metoda `ShowDialog()`. Un dialog modal întrerupe aplicația, iar utilizatorul nu poate continua până în momentul în care dialogul a fost închis.

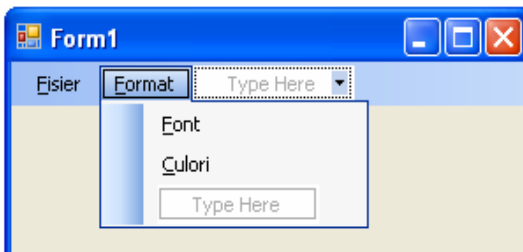
Aplicația *DialoguriPredefinite*

Proiectul pe care îl propunem este o aplicație cu un meniu din care utilizatorul poate selecta opțiuni de deschidere a dialogurilor predefinite. Pentru unele dialoguri, se va scrie și cod suplimentar, mai ales referitor la setarea proprietăților. Vestea bună este că veți scrie întotdeauna aproape la fel atunci când lucrați cu dialoguri predefinite.

1. Creați un nou proiect de tip **Windows Forms Application**, cu numele *DialoguriPredefinite*.
2. Din **Toolbox** plasați cu ajutorul mouse-ului un control **MenuStrip** pe suprafața formei.
3. Creați un prim meniu numit **Fisier**, cu opțiunile **Deschide**, **Salveaza ca**, **Setare pagina**, **Imprimare** și **Previzualizare**:



4. Creați un al doilea submeniu numit **Format**, cu opțiunile **Font** și **Culori**:



5. Din **Toolbox** plasați pe suprafața formei următoarele controale: **OpenFileDialog**, **SaveFileDialog**, **PageSetupDialog**, **PrintDialog**, **PrintPreviewDialog**, **PrintDocument**, **FontDialog**, **ColorDialog**. În "tray"-ul designerului de forme veți remarca imediat referințele la aceste obiecte, care au fost adăugate în cod.
6. În continuare veți trata evenimentul **Click** pentru fiecare dintre opțiunile meniului. Faceți dublu click pe opțiunea **Deschide**. În *handler*-ul evenimentului, scrieți codul:

```
private void deschideToolStripMenuItem_Click
    (object sender, EventArgs e)
{
    // Titlul dialogului
    openFileDialog1.Title = "Deschide Fisier";

    // Setează tipurile de fișiere care apar
    // în combobox-ul "Files of Type"
    openFileDialog1.Filter =
    "Fisiere Rich Text (*.rtf)|*.rtf|Fisiere Text (*.txt)|*.txt";

    // În combobox-ul File Name nu vrem să apară la
    // deschiderea dialogului nici un nume de fișier
    openFileDialog1.FileName = "";
}
```

```
// Directorul care se deschide în mod implicit
openFileDialog1.InitialDirectory = "MyDocuments";

// Afișează o atenționare dacă utilizatorul
// specifică un fișier care nu există
openFileDialog1.CheckFileExists = true;

// Deschide dialogul OpenFileDialog
if (openFileDialog1.ShowDialog() ==
    System.Windows.Forms.DialogResult.OK)
{
    // Cod care deschide fișierul
}
}
```

7. Dublu click pe opțiunea **Salvează ca**. În metoda de tratare, scrieți:

```
private void salveazaCaToolStripMenuItem_Click
    (object sender, EventArgs e)
{
    // Titlul dialogului
    saveFileDialog1.Title = "Salveaza Fisierul";

    // Extensia implicită de salvare a fișierelor
    saveFileDialog1.DefaultExt = ".rtf";

    // Atenționare dacă încercați să suprascrieți
    saveFileDialog1.OverwritePrompt = true;

    // Deschide dialogul SaveFileDialog
    if (saveFileDialog1.ShowDialog() ==
        System.Windows.Forms.DialogResult.OK)
    {
        // Cod care salvează fișierul
    }
}
```

8. Dublu click pe opțiunea **Setare pagina**. În metoda de tratare, scrieți:

```
private void setarePaginaToolStripMenuItem_Click
    (object sender, EventArgs e)
{
    // Se preiau setările din printDocument1
    pageSetupDialog1.Document = printDocument1;

    // Setările de pagină
    pageSetupDialog1.PageSettings =
        printDocument1.DefaultPageSettings;
```

```
// Diverse secțiuni ale dialogului devin active sau
// inactive. De fapt, valoarea implicită a acestor
// proprietăți este true
pageSetupDialog1.AllowMargins = true;
pageSetupDialog1.AllowPaper = true;
pageSetupDialog1.AllowOrientation = true;
pageSetupDialog1.AllowPrinter = true;
pageSetupDialog1.ShowNetwork = true;
pageSetupDialog1.EnableMetric = false;
pageSetupDialog1.ShowHelp = false;

// Se deschide dialogul și se preiau setările
// de pagină în obiectul de tip PrintDocument
if (pageSetupDialog1.ShowDialog() ==
    System.Windows.Forms.DialogResult.OK)
    printDocument1.DefaultPageSettings =
        pageSetupDialog1.PageSettings;
}
```

9. Faceți dublu click pe opțiunea **Imprimare**. În *handler* scrieți codul:

```
private void imprimareToolStripMenuItem_Click
    (object sender, EventArgs e)
{
    // Se preiau setările de printare din
    // obiectul de tip PrintDocument
    printDialog1.Document = printDocument1;

    // Se deschide dialogul PrintDialog
    if (printDialog1.ShowDialog() ==
        System.Windows.Forms.DialogResult.OK)
    {
        // Startează procesul de printare
        printDocument1.Print();

        // Cod care transformă documentul
        // în imagine și îl printează
        // ...
    }
}
```

10. Dublu click pe opțiunea **Previzualizare**. În *handler*, scrieți codul:

```
private void previzualizareToolStripMenuItem_Click
    (object sender, EventArgs e)
{
    if (printPreviewDialog1.ShowDialog() ==
        System.Windows.Forms.DialogResult.OK)
    {
        // Cod care afișează documentul în dialog
    }
}
```

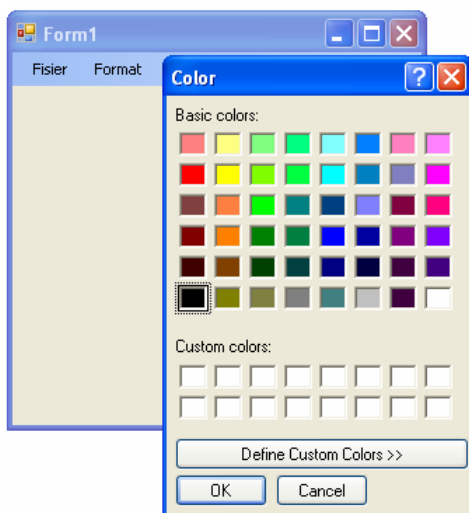
11. Dublu click pe opțiunea **Font**. În *handler*-ul evenimentului, scrieți codul:

```
private void fontToolStripMenuItem1_Click
    (object sender, EventArgs e)
{
    // Se deschide dialogul FontDialog
    if (fontDialog1.ShowDialog() ==
        System.Windows.Forms.DialogResult.OK)
    {
        // Cod care reafișează documentul cu
        // fontul ales de utilizator
    }
}
```

12. Dublu click pe opțiunea **Culori**. În *handler*-ul evenimentului, scrieți codul:

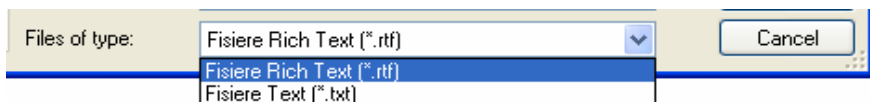
```
private void culoriToolStripMenuItem_Click
    (object sender, EventArgs e)
{
    // Se deschide dialogul ColorDialog
    if (colorDialog1.ShowDialog() ==
        System.Windows.Forms.DialogResult.OK)
    {
        // Cod care actualizează culorile in document
    }
}
```

13. Compilați și rulați cu **F5**.



Comentarii:

- Proprietatea **Filter** este comună dialogurilor **OpenFileDialog** și **SaveAsDialog**, întrucât este moștenită de la clasa de bază comună, **FileDialog**. Caracterul `'|'` (pipe) delimitează tipurile de fișiere pe care le puteți selecta, dar și intrările în *combobox* : `"Fișiere Rich Text (*.rtf)|*.rtf|Fișiere Text (*.txt)|*.txt"`;



- Dialogurile predefinite se afișează cu metoda **ShowDialog()**, iar tipul de retur al acestei metodei se analizează pentru a vedea dacă utilizatorul dorește să păstreze setările pe care le-a făcut sau vrea să renunțe la ele.
- Rețineți că **OpenFileDialog** nu deschide un fișier (document) ci numai vă permite să-l alegeți. Pentru deschidere aveți nevoie de un control suplimentar, capabil să afișeze conținutul fișierului, așa cum este un **TextBox** sau un **RichTextBox**. Aceleași considerații sunt valabile și pentru celelalte dialoguri predefinite.

Notă:

În ce privește printarea efectivă a unui document, aceasta se face prin apelul metodei **Print()** a clasei **PrintDocument**. Clasa **PrintDocument** este definită în spațiul de nume **System.Drawing.Printing**. Obiectul de tip **PrintDocument** preia setările de pagină stabilite de către utilizator în dialogul **PageSetupDialog**, și trimite documentul la imprimantă. Codul necesar poate fi găsit în documentația clasei **PrintDocument** sau în articole din **MSDN (Microsoft Developer Network)**.

Sfat practic

Să aveți în permanență deschis Helpul mediului integrat. Acolo găsiți imediat clasele cu care lucrați, cu descrierea completă a câmpurilor, proprietăților, metodelor și evenimentelor lor, dar și cu exemple de cod care vă pot ajuta. Veți descoperi foarte repede membrii ai claselor care vă pot fi utili.

*Reamintim locația **Bibliotecii de Clase: Help -> Contents -> .NET Framework SDK -> .NET Framework Class Library**.*

MDI – Multiple Document Interface

O aplicație **MDI** se lansează cu o singură fereastră container care reprezintă întreaga aplicație.

În interiorul ferestrei container se pot deschide multiple ferestre *child*. Gândiți-vă la *Microsoft Word*. Ferestrele interioare reprezintă diferite documente pe care utilizatorul le poate edita în același timp sau diferite vederi ale acelorași date.

Aplicațiile **MDI** diferă de cele de tip **SDI** (*Single document interface*) prin aceea că **SDI** au o singură fereastră copil. *Notepad* este un asemenea exemplu. Dacă doriți să editați două documente, trebuie să deschideți două instanțe *Notepad*.

Atât fereastra părinte cât și ferestrele copil sunt de fapt forme **.NET**. O formă devine container în momentul în care proprietatea **IsMdiContainer** este setată **true**.

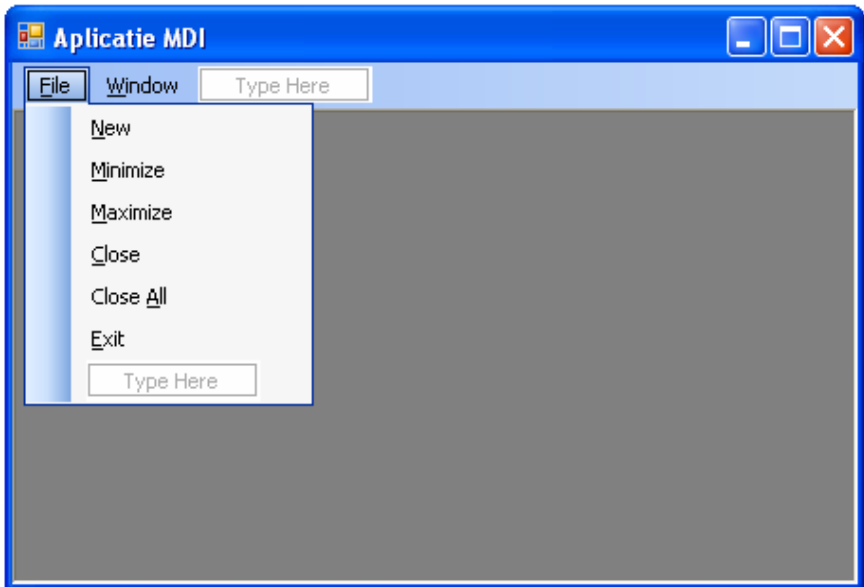
Aplicația *MdiExample*

Aplicația implementează:

- Deschiderea și închiderea ferestrelor *child*.
- Minimizarea și maximizarea ferestrelor.
- Aranjarea ferestrelor.
- Tratarea mesajului de atenționare la ieșirea din aplicație.

Urmați pașii:

1. Creați un nou proiect de tip **Windows Forms Application**, cu numele *MdiExample*.
2. Setati forma ca și container **MDI**. Pentru aceasta, în fereastra **Properties** atribuiți proprietății **IsMdiContainer** valoare **true**.
3. Din **Toolbox** plasați cu ajutorul mouse-ului un control **MenuStrip** pe suprafața formei.
4. Creați un prim meniu numit **File**, cu opțiunile **New**, **Minimize**, **Maximize**, **Close**, **Close All** și **Exit**. Creați un al doilea meniu cu numele **Window**, având opțiunile: **Tile** și **Cascade**:



5. Adăugați în clasa *Form1* un câmp privat care contorizează numărul de ferestre copil și o metodă privată care creează o fereastră copil. Pentru aceasta, acționați dublu click pe numele formei în **Solution Explorer**. În fișierul *Form1.cs*, în clasa **Form1**, scrieți codul:

```
private int nr = 0;    // Numărul de ferestre child

private void CreateChild()
{
    // Creăm o forma child.
    Form f = new Form();
    nr++;

    // Textul din bara de titlu
    f.Text = "Documentul " + nr;

    // Forma f devine document child
    // this este referință la forma container
    f.MdiParent = this;

    // Afișează forma child
    f.Show();
}
```

6. La pornirea aplicației, dorim să se deschidă o fereastră copil. Tratăm evenimentul **Load** generat la încărcarea formei. Dublu click pe suprafața formei. În metoda de tratare a evenimentului, scrieți:

```
private void Form1_Load(object sender, EventArgs e)
```

```
{  
    CreateChild();  
}
```

7. Acum tratăm evenimentele **Click** pentru toate opțiunile din meniuri. Acționați dublu click pe opțiunea **New**. În metoda de tratare, scrieți:

```
private void newToolStripMenuItem_Click(object sender,  
                                         EventArgs e)  
{  
    // Dacă nu există ferestre copil active  
    // setăm contorul la 0  
    if (ActiveMdiChild == null)  
        nr = 0;  
    CreateChild();  
}
```

8. Acționați dublu click pe opțiunea **Minimize**. În metoda de tratare, scrieți:

```
private void minimizeToolStripMenuItem_Click(  
                                             object sender, EventArgs e)  
{  
    // Dacă există o fereastră copil activă,  
    // aceasta se minimizează  
    if (ActiveMdiChild != null)  
        ActiveMdiChild.WindowState =  
            FormWindowState.Minimized;  
}
```

9. Acționați dublu click pe opțiunea **Maximize**. În metoda de tratare, scrieți:

```
private void maximizeToolStripMenuItem_Click(  
                                             object sender, EventArgs e)  
{  
    // Dacă există o fereastră copil activă,  
    // aceasta se maximizează  
    if (ActiveMdiChild != null)  
        ActiveMdiChild.WindowState =  
            FormWindowState.Maximized;  
}
```

10. Acționați dublu click pe opțiunea **Close**. În metoda de tratare, scrieți:

```
private void closeToolStripMenuItem_Click(  
                                           object sender, EventArgs e)  
{  
    // Închide fereastra copil activă  
    if (ActiveMdiChild != null)  
        ActiveMdiChild.Close();  
}
```

11. Acționați dublu click pe opțiunea **Close All** . În metoda de tratare, scrieți:

```
private void closeallToolStripMenuItem_Click(
    object sender, EventArgs e)
{
    // Închide pe rând fiecare formă copil
    foreach (Form f in this.MdiChildren)
        f.Close();
}
```

12. Acționați dublu click pe opțiunea **Exit**. În metoda de tratare, scrieți:

```
private void exitToolStripMenuItem_Click(object sender,
    EventArgs e)
{
    Application.Exit(); // Ieșire din aplicație
}
```

13. Acționați dublu click pe opțiunea **Tile** . În metoda de tratare, scrieți:

```
private void tileToolStripMenuItem_Click(object sender,
    EventArgs e)
{
    // Aranjare Tile pe orizontală
    this.LayoutMdi (MdiLayout.TileHorizontal);
}
```

14. Acționați dublu click pe opțiunea **Cascade** . În metoda de tratare, scrieți:

```
private void tileToolStripMenuItem_Click(object sender,
    EventArgs e)
{
    // Aranjare în cascadă
    this.LayoutMdi (MdiLayout.Cascade);
}
```

15. La închiderea formei, dorim să apară un *message box* cu butoanele **Yes** și **No**, care să ne întrebe dacă suntem siguri că vrem să ieșim. În *Form Designer* selectați forma container. În fereastra **Proprietăți** apăsați butonul **Events** (fulgerul). Acționați dublu click pe evenimentul **FormClosing**. În metoda de tratare a evenimentului, scriți codul evidențiat în **Bold**:

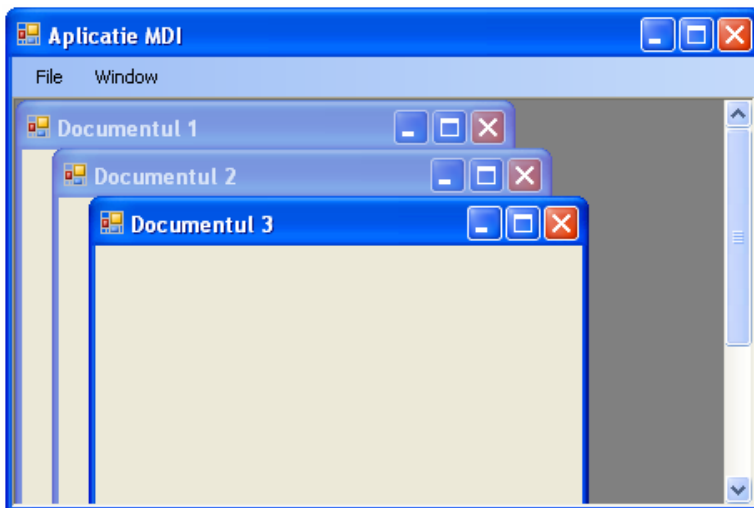
```
private void Form1_FormClosing(object sender,
    FormClosingEventArgs e)
{
    DialogResult result = MessageBox.Show(
        "Esti sigur ca vrei sa parasesti aplicatia?",
        "Avertizare", MessageBoxButtons.YesNo,
        MessageBoxIcon.Question);
}
```

```
// Dacă s-a apăsat No, atunci aplicația
// nu se închide
if (result == DialogResult.No)
    e.Cancel = true;
}
```

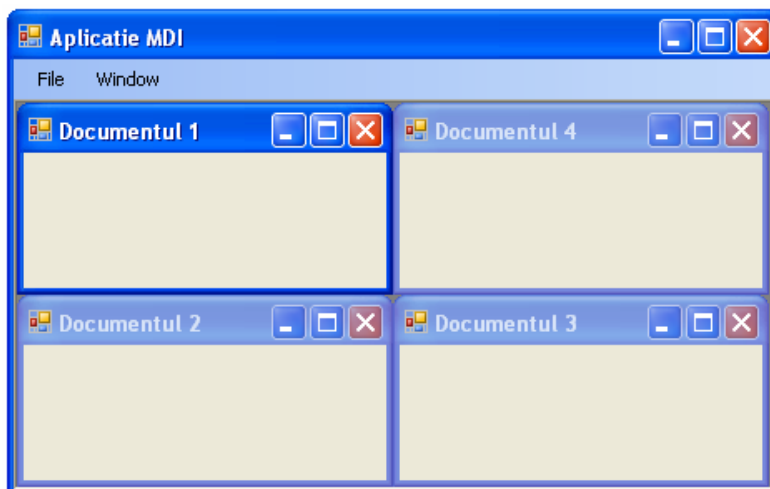
16. Compilați și rulați aplicația cu **F5**.

Câteva capturi de ecran din timpul rulării:

a) După deschiderea câtorva ferestre copil:



b) Aranjare **Tile** :



c) La ieșirea din aplicație:



De reținut:

- O formă devine container **MDI** dacă se setează **true** valoarea proprietății **IsMdiContainer**.
- Ferestrele *child* se construiesc ca oricare altă formă, după care proprietății **MdiParent** i se atribuie o referință la forma părinte:

```
Form f = new Form();
f.MdiParent = this;
f.Show();
```

- Proprietatea **MdiChildren** a formei container reține referințe la toate formele copil.
- Proprietatea **ActiveMdiChild** returnează o referință la fereastra *child* activă.
- Proprietatea **WindowState** a unei forme setează starea curentă a ferestrei. În mod implicit, valoarea ei este **Normal**. Se mai pot seta valorile **Maximized** și **Minimized**. Aceste valori se obțin prin accesarea membrilor enumerării: **FormWindowState**. **Exemplu:**

```
ActiveMdiChild.WindowState = FormWindowState.Maximized;
```

- Metoda **LayoutMdi()** a clasei **Form**, aranjează formele copil **MDI** în forma părinte.
- Metoda **Show()** a clasei **MessageBox** este supraîncărcată. Una dintre versiuni, cea din aplicație, are trei parametri: textul mesajului, textul din bara de titlu și butoanele care trebuie să apară pe formă. Reamintim că un *message box* este un dialog predefinit *Windows*, deci o formă, care

returnează întotdeauna o valoare de tip `DialogResult`. Aplicația arată cum puteți exploata valoarea returnată.

- La tratarea evenimentului `FormClosing`, unul dintre parametri este tipul `FormClosingEventArgs`. Clasa are proprietatea `Cancel`. Dacă valoarea acestei proprietăți este `true`, atunci evenimentul nu se mai declanșează, deci forma nu se mai închide (scrieți `e.Cancel = true`);

Probleme propuse:

- Implementați aplicației *MdiExample* facilitarea de aranjare a ferestrelor în modul *Tile*, dar pe verticală.
- Adăugați aplicației *MdiExample* un control `ToolStrip`, cu butoane cu aceeași funcționalitate cu cea a opțiunilor meniului.
- *Integrați ferestrelor copil în aplicația *MdiExample* un control de tip `TextBox`, pentru ca aplicația să funcționeze ca editor de text.

Controlul `RichTextBox`

`RichTextBox` derivă direct din clasa `TextBoxBase`, la fel ca și `TextBox`. În vreme ce un `TextBox` se utilizează pentru a introduce secvențe scurte de caractere de la tastatură, un `RichTextBox` se utilizează cu precădere ca editor de text.

Controlul are toate facilitățile unui `TextBox`, dar oferă și altele, cum sunt posibilitatea de formatare a fontului (*Italic*, **Bold**, Underline) și a paragrafului. Textul se poate introduce de către utilizator, poate fi încărcat din fișiere de tip **RTF** (*Rich Text Format*) sau din fișiere text. Clasa are metode pentru deschidere și pentru salvare de fișiere în aceste formate. Un control de tip `TextBox` poate fi înlocuit fără modificări majore de cod cu un control `RichTextBox`.

În aplicația care urmează vom utiliza pentru prima oară și o bară de instrumente, cu ajutorul controlului `ToolStrip`. Înainte de a continua, câteva cuvinte despre acest control:

Controlul `ToolStrip`

Este un control care vă permite să creați bare de instrumente. Este de fapt un container pentru butoane standard *Windows*. Iată un exemplu:

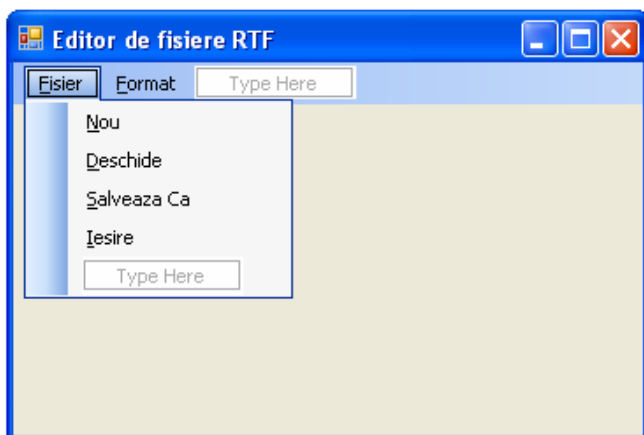


Se pot crea bare de instrumente cu temă sau fără temă, cu aparența și comportamentul barelor *Microsoft Office*, sau *Internet Explorer*, dar și bare cu stil și comportament stabilit de dumneavoastră. Clasa `ToolStrip` abundă de surprize plăcute. Merită să le explorați.

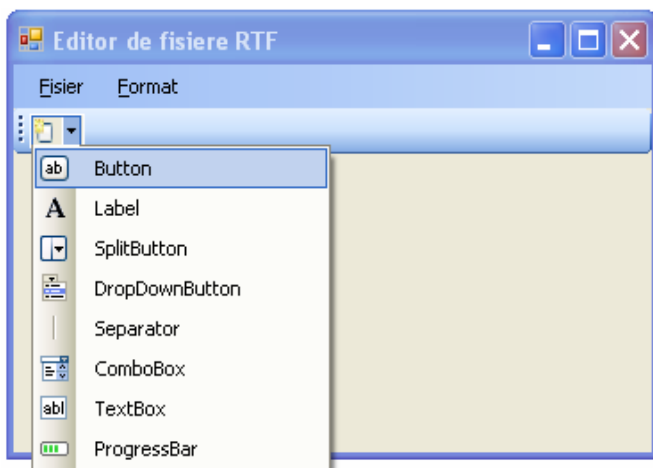
Aplicația *RichTextBoxExample*

Vom realiza un mic editor de text, asemănător celui creat în subcapitolul **TextBox**. Controlul care afișează textul va fi de data aceasta un **RichTextBox**. Aplicația va avea un meniu, facilități de deschidere și salvare a documentelor în format **RTF**, ca și posibilitatea de a seta fontul și culoarea lui. Evident, pentru implementarea acestor facilități, vom utiliza dialoguri predefinite.

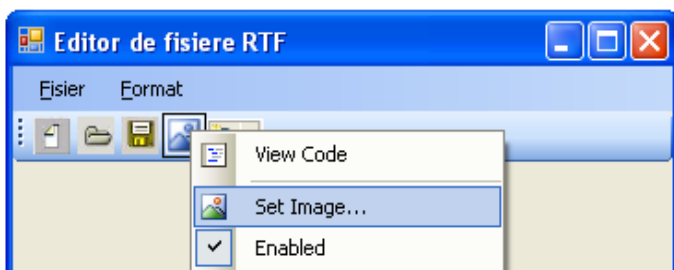
1. Creați un proiect de tip **Windows Forms Application**, cu numele *RichTextBoxExample*. Selectați forma și din fereastra **Properties** atribuiți proprietății **Text** valoarea: "Editor de fisiere RTF".
2. Din **Toolbox** plasați cu ajutorul mouse-ului un control **MenuStrip** pe suprafața formei.
3. Creați un prim meniu numit **Fisier**, cu opțiunile **Nou**, **Deschide**, **Salveaza Ca**, și **Iesire**. Creați un al doilea meniu cu numele **Format**, având opțiunile: **Font** și **Culoare**:



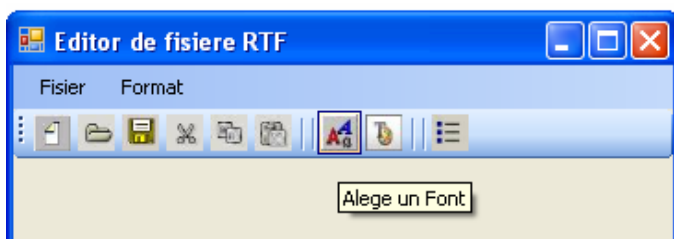
4. Din **Toolbox** plasați pe formă cu ajutorul mouse-ului un control **ToolStrip**.



5. În controlul **ToolStrip** adăugați pe rând **9** butoane din lista controlului, așa ca în figura de mai sus. Rolul acestor butoane poate fi descris prin textele: "Fișier Nou", "Deschide Fișierul", "Salvează Fișierul", "Decupează", "Copiază", "Lipește", "Alege un Font", "Alege o Culoare", "Bullets". Practic au exact funcționalitatea opțiunilor din meniu.
6. Pe fiecare buton adăugat, vom prefera să lipim câte o imagine, decât să scriem text. Căutați pe Internet câteva imagini **.jpg** sau **.gif** adecvate și salvați-le într-un folder oarecare. Selectați controlul **ToolStrip** și din fereastra **Properties** setați proprietatea **ImageScalingSize** la valorile **20** și **20** (implicit e **16** și **16**). Aceasta, pentru a mări puțin dimensiunile imaginilor afișate pe butoane.
7. Click drept pe fiecare buton din **ToolStrip** și alegeți **Set Image**. În dialogul **Select Resource**, apăsați butonul **Import**. Veți lipi astfel pe fiecare buton imaginea dorită:



8. Dorim să atașăm fiecărui buton câte un *Tool Tip Text*, adică un mic text care apare când staționați o clipă cu mouse-ul asupra butonului. Pentru aceasta, selectați pe rând câte un buton și din **Properties** atribuiți proprietății **Text** un text scurt, reprezentând rolul butonului:



9. Luați din **Toolbox** un control **RichTextBox** și plasați-l pe suprafața formei. Acționați click pe sageata din colțul dreapta sus a controlului și alegeți "Dock in parent container".
10. Vom trata evenimentul **Click** pentru fiecare dintre opțiunile meniului, dar și pentru butoanele din **ToolStrip**. Dublu click pe opțiunea **Nou**. În corpul *handler*-ului de eveniment, scrieți codul:

```
private void nouToolStripMenuItem_Click(object sender,
                                         EventArgs e)
{
    // Șterge textul controlului
    richTextBox1.Text = "";
}
```

Dublu click pe butonul cu textul “*Fisier Nou*” și scrieți același cod.

11. Faceți dublu click pe opțiunea **Deschide**. Corpul *handler*-ului trebuie să conțină exact același cod scris în aplicația *Dialoguri Predefinite* din subcapitolul precedent. Precizăm doar codul suplimentar:

```
if (openFileDialog1.ShowDialog() ==
    System.Windows.Forms.DialogResult.OK)
{
    // Metoda LoadFile() deschide fișierul
    richTextBox1.LoadFile(openFileDialog1.FileName);
}
```

Dublu click pe butonul cu textul “*Deschide Fișierul*”. Scrieți același cod.

12. Dublu click pe opțiunea de meniu **Salveaza Ca**. Corpul *handler*-ului trebuie să conțină exact același cod scris în aplicația *Dialoguri Predefinite* din subcapitolul precedent. Precizăm doar codul suplimentar:

```
if (saveFileDialog1.ShowDialog() ==
    System.Windows.Forms.DialogResult.OK)
{
    // Metoda SaveFile() salvează pe disc textul
    // controlului într-un fișier cu formatul RTF
    richTextBox1.SaveFile(saveFileDialog1.FileName,
        RichTextBoxStreamType.RichText);
}
```

Dublu click pe butonul cu textul “*Salvează Fișierul*”. Scrieți același cod.

13. Dublu click pe opțiunea **Ieșire**. În corpul *handler*-ului adăugați codul:

```
Application.Exit(); // Ieșire din aplicație
```

14. Dublu click pe opțiunea de meniu **Font**. *Handler*-ul evenimentului trebuie să arate astfel:

```
private void fontToolStripMenuItem_Click(object sender,
                                         EventArgs e)
{
    if (fontDialog1.ShowDialog() ==
```

```

        System.Windows.Forms.DialogResult.OK)
    {
        // Porțiunea selectată din text preia
        // caracteristicile alese de către utilizator
        richTextBox1.SelectionFont =
            fontDialog1.Font;
    }
}

```

Dublu click pe butonul cu textul “*Alege un Font*”. Scrieți același cod.

15. Dublu click pe opțiunea de meniu **Culoare**. *Handler*-ul evenimentului trebuie este:

```

private void culoareToolStripMenuItem_Click
    (object sender, EventArgs e)
{
    if (colorDialog1.ShowDialog() ==
        System.Windows.Forms.DialogResult.OK)
    {
        richTextBox1.SelectionColor =
            colorDialog1.Color;
    }
}

```

Dublu click pe butonul cu textul “*Alege o Culoare*”. Scrieți același cod.

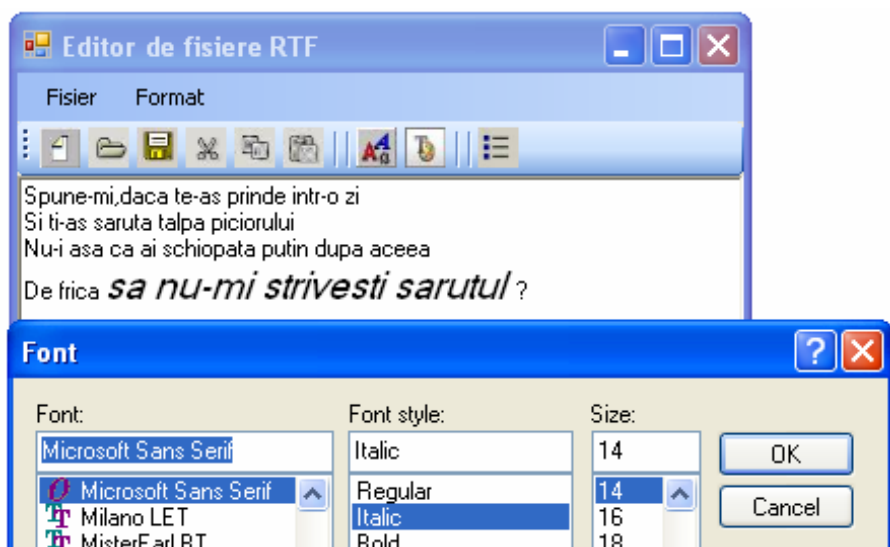
16. Selectați ultimul buton din **ToolStrip**, cu textul “*Bullets*”. În fereastra **Properties**, setați proprietatea **Name** la valoarea “*bulletButton*” și proprietatea **CheckOnClick** la valoarea **true**. Dublu click pe buton pentru a trata evenimentul **Click**. În metoda *handler* scrieți:

```

private void toolStripButton9_Click(object sender,
    EventArgs e)
{
    // Dacă butonul era apăsât atunci textului
    // selectat nu i se aplică stilul bullet
    if ( bulletButton.Checked )
        richTextBox1.SelectionBullet = false;
    else
        richTextBox1.SelectionBullet = true;
}

```

17. Compilați și lansați cu **F5**.

**Observație:**

- Editorul de mai sus lucrează numai cu fișiere **RTF**. La încercarea de deschidere a unui alt tip de fișier, această versiune a metodei **LoadFile()** aruncă o excepție. Din acest motiv, într-o aplicație serioasă, veți apela asemenea metode numai în interiorul blocurilor **try/catch**:

```
try
{
    richTextBox1.LoadFile(openFileDialog1.FileName);
}
catch (System.Exception e)
{
    MessageBox.Show(e.Message);
}
```

Există o a doua versiune a acestei metode care are doi parametri și poate fi adaptată la lucrul cu fișiere text.

- Clasa **RichTextBox** are și alte facilități de editare și formatare a textului, pe care le puteți explora de exemplu, îmbunătățind editorul de mai sus.

Controlul ToolTip

ToolTip se utilizează pentru afișarea unor mesaje de informare sau de avertizare, atunci când utilizatorul plasează cursorul desupra unui control. O

singură instanță a clasei este suficientă pentru a furniza texte de informare pentru toate controalele unei forme.

Exemplu:

```
ToolTip t = new ToolTip();  
Button b = new Button();  
ComboBox c = new ComboBox();  
t.SetToolTip(b, "Valideaza datele");  
t.SetToolTip(c, "Lista elevilor din clasa a XII-a B");
```

Dacă doriți stiluri și afișări diferite pe aceeași formă, veți utiliza mai multe controale **ToolTip**.

Principalii membri ai clasei ToolTip

Proprietăți :

AutoPopDelay	- Returnează sau setează perioada de timp în care ToolTip rămâne vizibil dacă cursorul e staționar deasupra controlului.
IsBallon	- Returnează o valoare de tip bool care indică dacă mesajul este inclus într-o fereastră balon.
ToolTipIcon	- Setează un icon pentru ToolTip .
ToolTipTitle	- Stabilește un titlu pentru ToolTip

Metode:

Hide()	- Ascunde fereastra ToolTip
Show()	- Setează un text pentru ToolTip și îl afișează
RemoveAll()	- Înlătură toate textele asociate cu un control ToolTip

Evenimente:

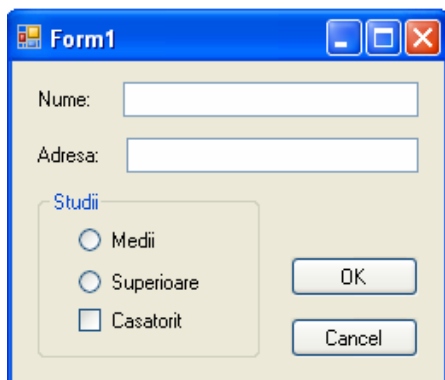
Popup	- Se declanșează înainte ca ToolTip să fie afișat
--------------	--

Aplicația *ToolTipExample*

În proiectul de față se folosește același **ToolTip** pentru mai multe controale de pe aceeași formă. Se utilizează *designer*-ul pentru toate acțiunile necesare.

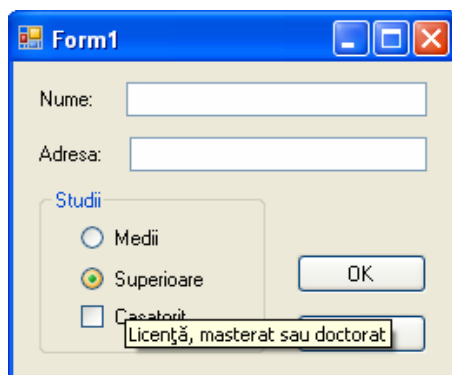
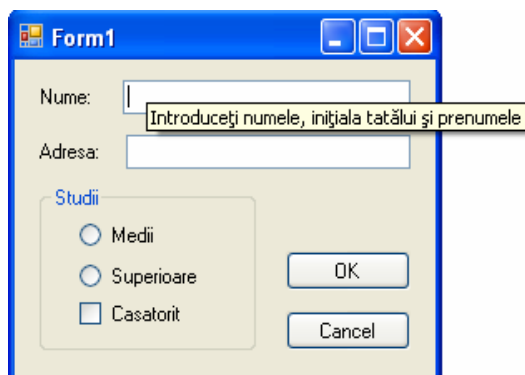
1. Creați un proiect de tip **Windows Forms Application** cu numele *ToolTipExample*.

2. Din **Toolbox** plasați pe formă câteva controale: două controale de tip **TextBox**, un **GroupBox**, două controale de tip **RadioButton**, un **CheckBox** și două butoane:



3. Din **ToolBox** alegeți un **ToolTip** și plasați-l pe suprafața formei. Acesta nu este vizibil pe formă, dar referința `toolTip1` apare în *designer tray*.
4. Selectați primul control **TextBox**. În fereastra **Properties** selectați *ToolTip on toolTip1*. Acționați click pe săgeata din dreapta și în panoul care se deschide introduceți mesajul: "Introduceți numele, inițiala tatălui și prenumele". Apoi acționați **Ctrl + Enter**.
5. Selectați al doilea control **TextBox**. Acționați de aceeași manieră ca la pasul 4, dar introduceți alt mesaj, cum ar fi "Introduceți strada, numărul locuinței și localitatea".
6. Se repetă pasul 4, pentru controalele de tip **RadioButton**, pentru **CheckBox** și pentru cele două controale de tip **Button**.
7. Compilați și rulați cu **F5**.

La rulare, când staționați cu mouse-ul asupra fiecărui control, apar mesajele **ToolTip**:



De reținut:

- Mesajele **ToolTip** se pot seta atât *design time*, cât și *run time*.
- Se poate introduce **ToolTip** fără ajutorul *designer*-ului. Tratați de exemplu evenimentul **Load** pentru forma acestei aplicații (dublu click pe suprafața forme) și introduceți codul:

```
private void Form1_Load(object sender, EventArgs e)
{
    ToolTip t = new ToolTip();
    t.SetToolTip(textBox2,
        "Introduceți numele, inițiala tatălui și prenumele");
    t.SetToolTip(textBox1,
        "Introduceți strada și numărul locuinței");
    t.SetToolTip(radioButton1,
        "Studiile postliceale sunt considerate medii");
    t.SetToolTip(radioButton2,
        "Licență, masterat sau doctorat");
    t.SetToolTip(checkBox1,
        "Nu bifați în cazul în care sunteți divorțat!");
    t.SetToolTip(button1,
        "Se validează datele introduse");
    t.SetToolTip(button2,
        "Se renunță la datele introduse");
}
```

Controlul NotifyIcon

Formele, ca de altfel și alte controale, moștenesc metoda **Hide()** de la clasa **Control**. Dacă utilizatorul ascunde forma principală a aplicației, atunci nu mai are posibilitatea să o reafișeze.

Clasa **NotifyIcon** oferă posibilitatea de a accesa procese care rulează în *background*. Creează un icon în zona de notificare (*notify area*) din *system tray*. Zona de notificare se găsește în dreapta *task bar*-ului, lângă ceas. Iconul răspunde la evenimente, cum sunt **Click** sau **DoubleClick** pe care le puteți trata pentru a reafișa fereastra aplicației. Alternativ, se obișnuiește să se atașeze un meniu contextual care se deschide la click drept pe icon.

Principalii membri ai clasei**Proprietăți :**

ContextMenu	-	Returnează sau setează un meniu de context pentru icon
Icon	-	Returnează sau setează un iconul controlului
Text	-	Returnează sau setează un text <i>ToolTip</i> . când cursorul mouse-ului staționează deasupra iconului
Visible	-	Stabilește dacă iconul e vizibil în zona de notificare

Metode:

ShowBallonTip()	- Afișează mesajul ToolTip într-o formă asemănătoare unui balon
------------------------	--

Evenimente:

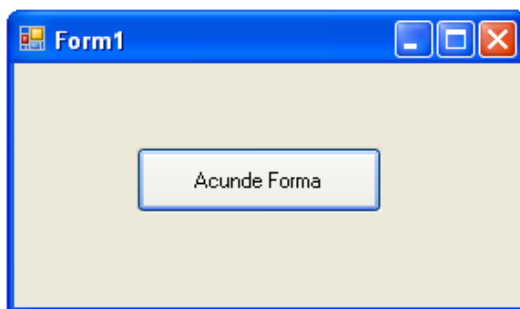
Click	- Se declanșează la click pe icon
DoubleClick	- Se declanșează la dublu click pe icon

Aplicația *NotifyIconExample*

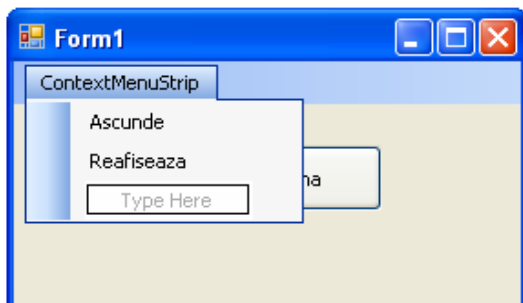
Pe o formă plasăm un buton care la click o ascunde. Folosim un **NotifyIcon** pentru reafișarea formei. Reafișarea se produce la dublu click pe icon sau la click într-un meniu de context.

Urmați pașii:

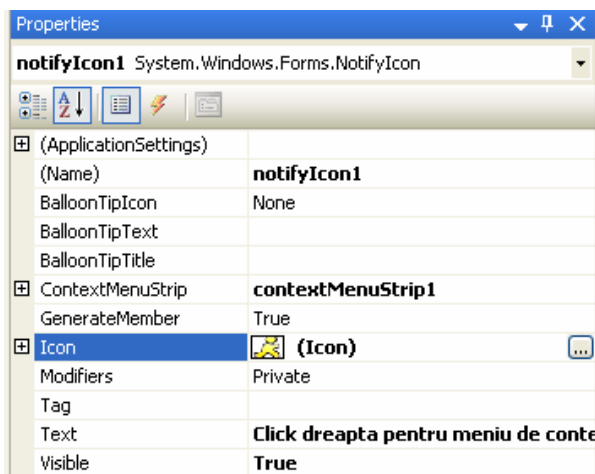
1. Creați un proiect de tip **Windows Forms Application** cu numele *NotifyIconExample*.
2. Din **Toolbox** plasați pe formă un control de tip **Button**. În fereastra **Properties** setați proprietatea **Text** la valoarea "*Ascunde Forma*":



3. Trageți deasupra formei un control de tip **ContextMenuStrip**. Selectați în *designer tray* referința `contextMenuStrip1`. Introduceți itemii *Ascunde* și *Reafiseaza*:



4. Trageți deasupra formei un control de tip **NotifyIcon**. Selectați în *designer tray* referința **notifyIcon1**.
 - a. În fereastra **Properties**, selectați proprietatea **Text** și introduceți textul care doriți să apară ca *tool tip* când staționați cu mouse-ul deasupra iconului.
 - b. Setați proprietatea **Visible** la **true**.
 - c. Selectați proprietatea **Icon** și alegeți iconul care apare în zona de notificare. Evident, trebuie să aveți unul pregătit. Nu uitați să setați un icon. În lipsa lui, nu apare la execuție nimic în *notify area*.
 - d. Selectați proprietatea **ContextMenuStrip** și alegeți valoarea **contextMenuStrip1**.



5. Tratăm evenimentul **Click** pentru butonul *Ascunde Forma*. Executați dublu click pe el și scrieți în corpul *handler*-ului codul evidențiat în **Bold**:

```
private void button1_Click(object sender, EventArgs e)
{
    // Ascunde forma aplicației
    this.Hide();
}
```

6. În *designer tray* selectați referința **notifyIcon1** și faceți dublu click pe opțiunea *Ascunde*, a meniului contextual. În *handler*-ul de tratare a evenimentului **Click**, introduceți codul:

```
private void ascundeToolStripMenuItem_Click(
    object sender, EventArgs e)
{
    // Ascunde forma aplicației
    this.Hide();
}
```

7. Acționați dublu click pe opțiunea *Reafiseaza* a meniului contextual, pentru tratarea evenimentului **Click**. În corpul metodei de tratare, introduceți codul:

```
private void reafiseazaToolStripMenuItem_Click(
                                object sender, EventArgs e)
{
    this.Visible = true;
}
```

8. Dorim ca la dublu click pe icon în *notify area* forma să redevină vizibilă. Pentru aceasta, tratăm evenimentul **DoubleClick** pentru **NotifyIcon**. Selectați în *designer tray* referința **notifyIcon1**, apoi în fereastra **Properties** selectați butonul *Events* (fulgerul). Acționați dublu click pe evenimentul **DoubleClick**. În corpul metodei de tratare scrieți:

```
private void notifyIcon1_DoubleClick(object sender,
                                    EventArgs e)
{
    this.Visible = true;
}
```

9. Tratăm evenimentul **Click** pentru opțiunile din meniul contextual. Selectați în *designer tray* referința **contextMenuStrip1**. Acționați dublu click pe opțiunea *Ascunde*. În editor, scrieți codul:

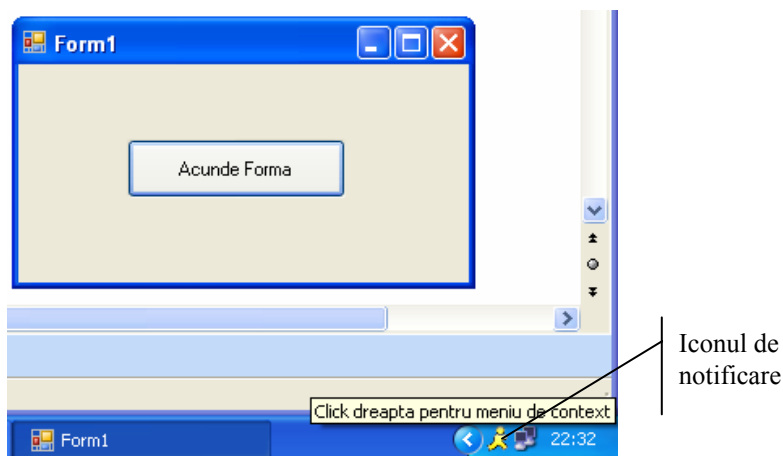
```
private void ascundeToolStripMenuItem_Click(
                                object sender, EventArgs e)
{
    this.Hide();
}
```

10. Acționați dublu click pe opțiunea *Reafiseaza*. În editor scrieți codul:

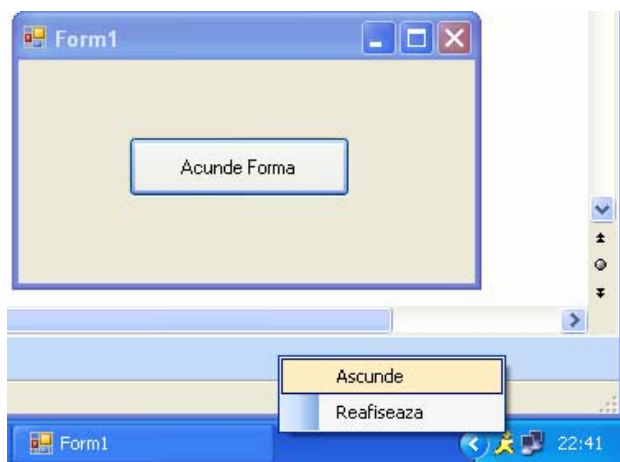
```
private void reafiseazaToolStripMenuItem_Click(
                                object sender, EventArgs e)
{
    this.Visible = true;
}
```

11. Compilați și rulați cu **F5**.

La executare, iconul rămâne în permanență vizibil în zona de notificare.



La click pe butonul *Ascunde Forma*, forma devine invizibilă. La dublu click pe icon, forma redevine vizibilă, iar iconul rămâne vizibil. Din meniul de context puteți de asemenea să controlați vizibilitatea formei:



De reținut:

- Controlul **NotifyIcon** plasează un icon în zona de notificare din *system tray*. Iconul reprezintă o scurtătură pentru procesele care lucrează în *background*.
- Controlul răspunde la evenimente (**Click**, **DoubleClick**) și admite un meniu contextual.

Fonturi

Fonturile nu sunt controale **.NET**. Clasa **Font** este definită în spațiul de nume **System.Drawing**. Un obiect de tip **Font** încapsulează un format particular

de text, incluzând stilul, dimensiunea. Toate controalele de bază au proprietatea **Font**, care vă permite să specificați caracteristicile fontului care va fi afișat în control. Setările fontului se pot face *design time*, (din fereastra **Properties**) sau *run time*.

Exemplu:

Setăm fontul afișat de către o etichetă:

```
Label e = new Label();

// Utilizăm unul dintre constructorii clasei Font
// pentru a crea un nou font
e.Font = new Font("Arial", 10, FontStyle.Italic);
```

Stilurile Fonturilor

Enumerarea **FontStyle** din spațiul de nume **System.Drawing** are ca membri enumeratorii: **Regular**, **Bold**, **Italic**, **Underline** și **Strikeout**. Aceștia determină stilul aplicat textului.

Cînd lucrați cu un editor de text, lucrurile se complică puțin, pentru că mai multe stiluri din cele de mai sus pot fi aplicate aceleași porțiuni selectate de text. Aplicarea unui nou stil nu trebuie să invalideze stilul anterior. De exemplu:

“Această porțiune de text conține stilurile **Bold**, *Italic* și Underline”.

Aplicația *FontStyleExample*

Proiectul introduce stilurile **Bold**, **Italic**, **Underline**, într-un control **RichTextBox**:

1. Creați un nou proiect de tip **Windows Forms Application**, cu numele *FontStyleExample*.
2. Din **Toolbox** alegeți un control **ToolStrip** și plasați-l pe suprafața formei. Adăugați trei butoane în acest control. Selectați pe rând cele trei butoane și din fereastra **Properties** atribuiți proprietății **Text** valorile *Bold*, *Italic*, *Underline*, iar proprietății **DisplayStyle** valoarea *Text*.
3. Aduceți pe suprafața formei un control de tip **RichTextBox**. Setăți din fereastra **Properties**, câmpul **Dock** la valoarea *Fill*.
4. Tratăm evenimentul **Click** pentru fiecare buton. Dublu click pe butonul **Bold**. *Handler*-ul de eveniment este:

```
private void toolStripButton1_Click(object sender,
                                   EventArgs e)
```

```

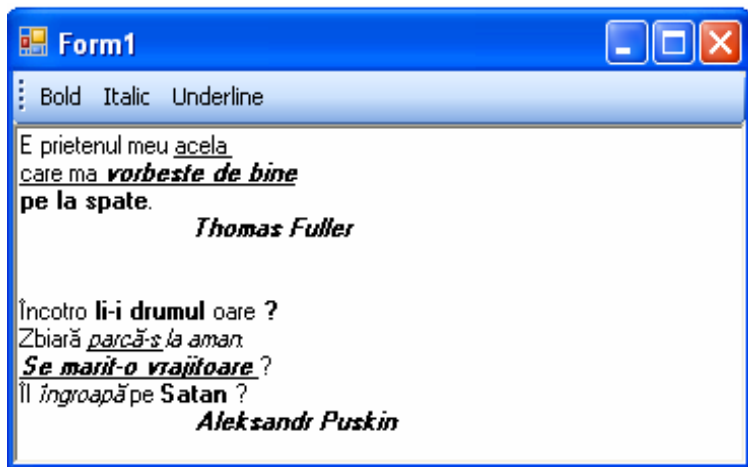
{
    Font vechiulFont, noulFont;

    // Returnează fontul folosit în textul selectat
    vechiulFont = this.richTextBox1.SelectionFont;

    // Dacă vechiul font era stil Bold, înlăturăm
    // formatarea
    if (vechiulFont.Bold)
    {
        noulFont = new Font(vechiulFont,
                            vechiulFont.Style & ~ FontStyle.Bold);
    }
    else
    {
        noulFont = new Font(vechiulFont,
                            vechiulFont.Style | FontStyle.Bold);
    }
    // Inserăm noul font și redăm focusul
    // controlului RichTextBox.
    this.richTextBox1.SelectionFont = noulFont;
    this.richTextBox1.Focus();
}

```

5. Dublu click pe butonul **Italic**. *Handler-ul de eveniment va avea același cod cu cel atașat evenimentului Bold*, cu unica diferență că în toate locurile cuvântul “**Bold**” se înlocuiește cu “**Italic**”.
6. Dublu click pe butonul **Underline**. *Handler-ul de eveniment va avea același cod cu cel atașat evenimentului Bold*, cu unica diferență că în toate locurile cuvântul “**Bold**” se înlocuiește cu “**Underline**”.
7. Compilați și rulați cu **F5**.



Observație:

- Codul prezent în metodele *handler* verifică mai întâi dacă stilul dorit este deja prezent. Dacă este, îl înlătură.
- Pentru adăugarea sau înlăturarea unui stil nou, fără a afecta celelalte stiluri prezente, se efectuează operații pe biți cu enumerarea **FontStyle**. (“*Or logic pe biți*” (!) adaugă un stil nou, iar “*And logic pe biți*”, aplicat negării noului stil, înlătură noul stilul dintre cele prezente. Exemple:

```
// Adaugă stilul Underline:
vechiulFont.Style | FontStyle.Underline

// Înlătură stilul Underline
vechiulFont.Style & ~ FontStyle.Underline
```

Fonturile instalate

Cu ajutorul unui obiect al clasei **InstalledFontCollection**, definită în spațiul de nume **System.Drawing.Text** se obțin fonturile instalate în sistem, astfel:

```
using System.Drawing.Text;
InstalledFontCollection fnt = new InstalledFontCollection();
```

Proprietatea **Families** a clasei **InstalledFontCollection** returnează un tablou de referințe la obiecte de tip **FontFamily**. Fiecare obiect încapsulează o familie de fonturi, iar prin proprietatea **Name**, returnează numele fontului:

```
foreach (FontFamily fam in fnt.Families)
    System.Console.WriteLine(fam.Name);
```

Vom vedea un exemplu după paragraful următor.

Desenarea fonturilor

.NET Framework oferă posibilitatea de a desena pe suprafețele formelor sau ale controalelor. Se pot desena linii, forme geometrice, imagini sau fonturi. În acest paragraf ne vom ocupa numai de desenarea fonturilor. Pentru desenare, aveți nevoie de un obiect de tip **Graphics**. Clasa **Graphics** este definită în spațiul de nume **System.Drawing**.

Fonturile se desenează cu metoda **DrawString()**. Una dintre versiuni este: **DrawString(String, Font, Brush, float, float)**. Desenează textul specificat de obiectele **String** și **Font**, cu pensula indicată de obiectul **Brush**, la locația indicată. Ultimii doi parametrii indică coordonatele **X** și **Y** ale colțului stânga sus al textului. Dacă de exemplu dorim să desenăm pe suprafața

unei forme, atunci cel mai bun moment pentru desenare este acela al desenării formei și vom trata evenimentul `Paint` al formei.

Aplicația *FontDrawingExample*

Proiectul de față determină fonturile instalate în sistem și populează un combobox cu numele acestora. La selectarea unui font din listă, numele fontului se va desenează pe suprafața formei.

1. Creați un nou proiect de tip **Windows Forms Application**, cu numele *FontDrawingExample*.
2. Din Toolbox, plasați pe suprafața formei un control de tip **ComboBox**.
3. Tratăm evenimentul `Load` generat la încărcarea formei. Acționați dublu click pe suprafața formei. Scrieți codul marcat în **Bold**, în *handler*-ul de eveniment:

```
private void Form1_Load(object sender, EventArgs e)
{
    // Determină fonturile instalate
    InstalledFontCollection fnt =
        new InstalledFontCollection();

    // Adaugă numele fiecărui font în listă
    foreach (FontFamily fam in fnt.Families)
        comboFont.Items.Add(fam.Name);
}
```

4. În antetul fișierului `Form1.cs`, adăugați directiva:

```
using System.Drawing.Text;
```

5. Dorim ca desenarea fontului să se producă odată cu redesenarea formei. Forțăm redesenarea formei, la fiecare selectare a unui item în **ComboBox**. Tratăm evenimentul **SelectedIndexChanged**. În acest scop, faceți dublu click pe controlul **ComboBox**. Scrieți codul:

```
private void comboBox1_SelectedIndexChanged
    (object sender, EventArgs e)
{
    // Invalidează suprafața formei, fapt
    // care cauzează redesenarea acesteia
    this.Invalidate();
}
```

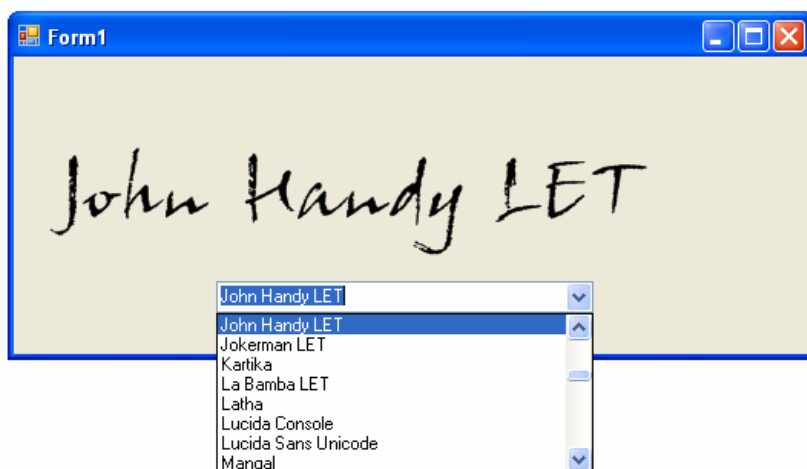
6. Tratăm evenimentul **Paint** pentru formă. Selectați forma în *Form Designer* și din fereastra **Properties** acționați dublu click pe evenimentul `Paint`. Scrieți codul marcat în **Bold**, în *handler*-ul de eveniment:

```

private void Form1_Paint(object sender,
                             PaintEventArgs e)
{
    try
    {
        // Desenează fontul pe suprafața formei
        e.Graphics.DrawString(comboBox1.Text,
                              new Font(comboBox1.Text, 40),
                              Brushes.Black, 10, 50);
    }
    catch (ArgumentException ex)
    {
        MessageBox.Show(ex.Message);
    }
}

```

7. Compilați și lansați în execuție cu **F5**.



TabControl

Un **TabControl** conține pagini suprapuse, numite pagini *tab*. O pagină *tab* este o instanță a clasei **TabPage**. Pagina *tab* curentă se schimbă prin cu click cu mouse-ul pe *tab*-ul dorit sau se poate face programatic. Paginile *tab* suportă alte controale, în mod similar formelor. Astfel, un **TabControl** poate constitui o alternativă unei afișări succesive a mai multor forme.

Principalii membrii ai clasei TabControl

Proprietăți :

SelectedIndex	- Returnează sau setează indexul paginii <i>tab</i> selectate
SelectedTab	- Returnează sau setează pagina <i>tab</i> selectată
TabCount	- Returnează numărul de <i>tab</i> -uri din control
TabPage	- Returnează colecția de <i>tab</i> -uri din TabControl

Metode:

GetControl()	- Returnează obiectul TabPage de la locația specificată
GetItems()	- Returnează colecția de obiecte TabPage aparținând controlului TabControl .

Evenimente:

Selected	- Se declanșează când se selectează un <i>Tab</i>
SelectedIndexChanged	- Se declanșează când proprietatea SelectedIndex s-a modificat

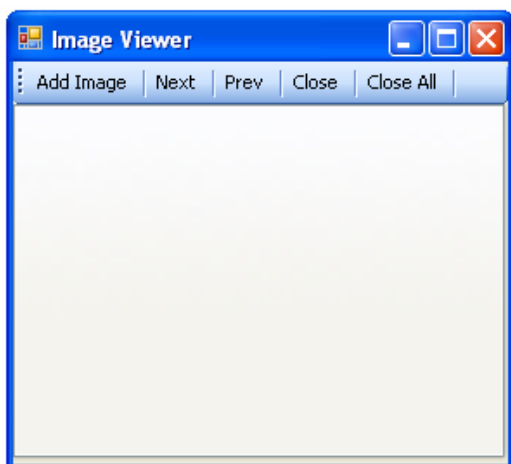
Aplicația **TabControlExample**

Aplicația pe care o propunem utilizează un **TabControl** pentru afișarea imaginilor. *Tab*-urile controlului se adaugă *runtime*, în momentul în care se încarcă o imagine de pe disc. Operațiile se execută la apăsarea butoanelor unui control **ToolStrip**. Acestea sunt:

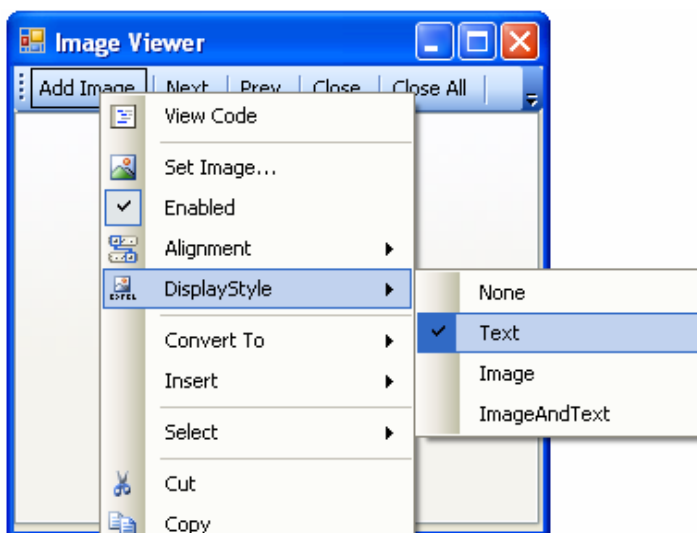
- **Open** – încarcă o imagine de pe disc și o afișează într-un *tab* nou.
- **Next** – selectarea *tab*-ului următor
- **Prev** - selectarea *tab*-ului anterior
- **Close** – închiderea *tab*-ului curent
- **Close All** – închiderea tuturor *tab*-urilor

Urmați pașii:

1. Creați un nou proiect de tip **Windows Forms Application**, cu numele *TabControlExample*.
2. Din Toolbox, plasați pe suprafața formei un control de tip **ToolStrip** și un **TabControl**.
3. Selectați **TabControl**-ul, apoi în fereastra **Properties** setați proprietatea **Dock** la valoarea **Fill**. Ștergeți cele două *tab*-uri implicite din control (acționați click dreapta pe fiecare *tab* și selectați) *Remove Tab*.
4. Adăugați cinci butoane pe **ToolStrip**, despărțite printr-un obiect de tip separator. Forma trebuie să arate astfel:



Butoanele din **ToolStrip** trebuie să afișeze text. Ca să obțineți asta, acționați click dreapta pe fiecare buton, apoi alegeți *DisplayStyle* și selectați *Text*. Mai departe, în fereastra **Properties**, atribuiți proprietății **Text** valorile corespunzătoare:



5. Declarăm un câmp privat în clasa **Form1**, care stabilește caracterul separator pentru căile de director. În **Solution Explorer**, click dreapta pe *Form1*, selectați *View Code*, apoi în fișierul *Form1.cs*, în clasa **Form1**, scrieți:

```
private char[] sep = new char[] { '\\' };
```

6. Din **Toolbox**, trageți pe suprafața formei un control de tip **OpenFileDialog**.

7. Tratăm evenimentul **Click** pentru butonul *Add Image*. Acționați dublu click pe buton. În *handler*-ul de eveniment, scrieți codul evidențiat în **Bold**:

```
private void toolStripButton1_Click(object sender,
                                   EventArgs e)
{
    string[] aux;
    string fullPath; // Calea până la fișierul imagine
    string imgName;   // Numele fișierului imagine

    // Setăm un filtru pentru fișiere de tip imagine
    openFileDialog1.Filter = "Fișiere suportate " +
        " (*.jpg;*.png;*.ico;*.gif;*.bmp;*.tiff)" +
        "|*.jpg;*.png;*.ico;*.gif;*.bmp;*.tiff " +
        "|All files (*.*)|*.*";

    // Dacă dialogul se închide prin apăsarea OK
    if (openFileDialog1.ShowDialog() ==
        DialogResult.OK)
    {
        // Obținem calea până la fișier
        fullPath = openFileDialog1.FileName;

        // Separăm calea în stringuri separate prin '\\'
        aux = fullPath.Split(sep);

        // Numele imaginii e ultimul string
        imgName = aux[aux.Length - 1];

        // Creăm un tab nou și un PictureBox
        TabPage tP = new TabPage(imgName);
        PictureBox pB = new PictureBox();

        // Încărcăm în picture box imaginea
        pB.Image = Image.FromFile(fullPath);

        // Stabilim poziția și dimensiunea imaginii pe
        // tab. (x și y față de colțul stânga sus a
        // tab-ului, lățimea și înălțimea)

        pB.SetBounds(
            (tabControl1.Width/2) - pB.Image.Width / 2),
            (tabControl1.Height/2) - (pB.Image.Height/2),
            pB.Image.Width, pB.Image.Height);

        // Adăugăm pb pe tab
        tP.Controls.Add(pB);

        // Adăugăm tab-ul pe TabControl
        tabControl1.TabPages.Add(tP);
    }
}
```

```
        // Tab-ul adăugat devine cel selectat
        tabControl1.SelectedIndex =
            tabControl1.TabPages.Count - 1;
    }
}
```

8. Tratăm evenimentul **Click** pentru butonul *Next*. Faceți dublu click pe buton. În metoda de tratare, scrieți:

```
private void toolStripButton2_Click(object sender,
                                   EventArgs e)
{
    // Dacă există tab-ul următor
    if (tabControl1.SelectedIndex + 1 <
        tabControl1.TabPages.Count)
    {
        // îl selectăm
        tabControl1.SelectedIndex =
            tabControl1.SelectedIndex + 1;
    }
}
```

9. Tratăm evenimentul **Click** pentru butonul *Prev*. Faceți dublu click pe buton. În metoda de tratare, scrieți:

```
private void toolStripButton3_Click(object sender,
                                   EventArgs e)
{
    // Dacă există tab-ul precedent
    if (tabControl1.SelectedIndex - 1 >= 0)
    {
        // îl selectăm
        tabControl1.SelectedIndex =
            tabControl1.SelectedIndex - 1;
    }
}
```

10. Tratăm evenimentul **Click** pentru butonul *Close*. Faceți dublu click pe buton. În metoda de tratare, scrieți codul evidențiat în **Bold**:

```
private void toolStripButton4_Click(object sender,
                                   EventArgs e)
{
    // Dacă mai există pagini (tab-uri)
    if (tabControl1.TabPages.Count > 0)
    {
        // O ștergem pe cea selectată
        tabControl1.TabPages.RemoveAt (
            tabControl1.SelectedIndex);
    }
}
```

11. Tratăm evenimentul **Click** pentru butonul *Close All*. Faceți dublu click pe buton. În metoda de tratare, scrieți codul evidențiat în **Bold**:

```
private void toolStripButton5_Click(object sender,
                                   EventArgs e)
{
    // Cât timp mai sunt pagini (tab-uri)
    while (tabControl1.TabPages.Count > 0)
    {
        // ștergem pagina selectată
        tabControl1.TabPages.RemoveAt(
            tabControl1.SelectedIndex);
    }
}
```

12. Compilați și rulați cu **F5**.

La rulare, cu *Add Image* se pot încărca mai multe imagini în pagini diferite, cu butoanele *Next* și *Prev* se parcurg tag-urile deschise, cu *Close* se închide *tab*-ul selectat, iar cu *Close All* se închid toate paginile.

Probleme propuse

1. Realizați o aplicație care utilizează un **TabControl** care preia comenzile clienților la o pizzerie. Paginile controlului trebuie să conțină controale de tip **TextBox**, **ComboBox**, **CheckBox**, **RadioButton** și **Button**. La finalul comenzii, aplicația afișează într-un dialog întreaga comanda, împreună cu prețul final.
2. Realizați un proiect care un utilizează un **TabControl** cu șapte pagini. Controlul reprezintă jurnalul unui elev și fiecare pagină corespunde unei zile a săptămânii. Controlul trebuie să conțină cel puțin căsuțe de editare. După completarea zilei a șaptea, se apasă un buton care creează un nou *tab*. În acest *tab* se vor afișa într-un **TextBox** informațiile introduse în timpul săptămânii (în rezumat sau integral - dumneavoastră decideți).

Controalele *ListBox*, *ComboBox* și *CheckedListBox*

Toate cele trei controale moștenesc clasa **ListControl** și servesc la afișarea unei liste de itemuri. Itemurile apar în listă ca șiruri de caractere și pot fi selectate prin click cu mouse-ul.

Controlul **ListBox** poate oferi selecție simplă sau multiplă, în funcție de valoarea proprietății **SelectionMode**.

Controlul **ComboBox** combină un **TextBox** cu un **ListBox**, permițând utilizatorului să selecteze itemi dintr-o listă, sau să introducă noi itemi.

Controlul **CheckedListBox** afișează o listă de itemi care pot selectați cu ajutorul unui checkbox prezent pentru fiecare item.

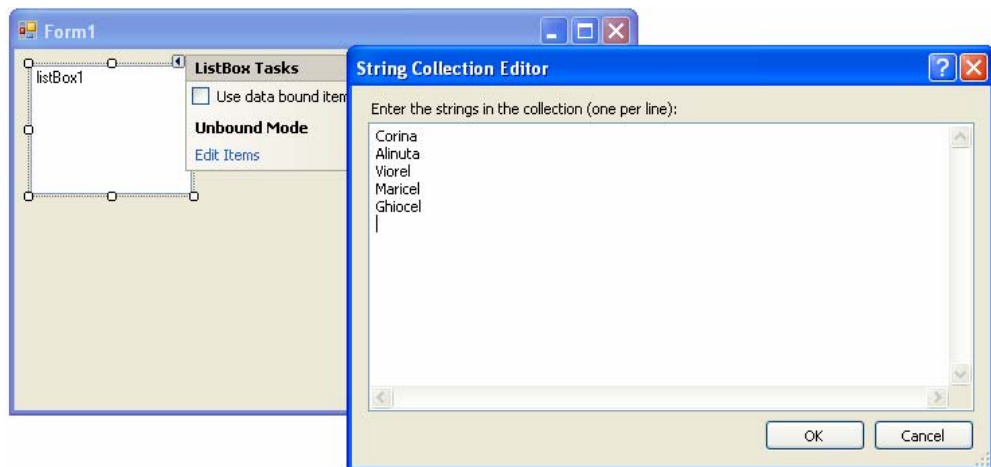
Rețineți că în aceste tipuri de liste se pot însira obiecte, dar că ceea ce se vede este reprezentarea ca șir de caractere a acestor obiecte.

Adăugarea itemilor în liste

Itemii se pot adăuga *design time* sau *run time*.

Adăugarea itemilor *design time*

O variantă simplă este să selectați controlul, faceți click pe săgeata din colțul stânga sus, alegeți *Edit Items* și introduceți itemii manual, câte unul pe linie:



Adăugarea itemilor *run time*

Se folosesc metodele `Insert()`, `Add()` sau `AddRange()`. Cu `Insert()` inserați un item la o poziție specificată. De exemplu:

```
// Înserează în listă la poziția 3 un item cu textul "Item 3"
listBox.Items.Insert(3, "Item 3");
```

Metoda `Add()` adaugă la sfârșitul listei un obiect.

Exemplu:

```
comboBox.Items.Add("Item Nou");
```

Pentru adăugarea eficientă a unui număr mare de itemi în listă, controalele furnizează metodele `BeginUpdate()` și `EndUpdate()`. Prin utilizarea acestora, controlul nu este redesenat de fiecare dată când un item este introdus în listă.

Exemplu:

```
comboBox.BeginUpdate(); // Controlul nu se mai redesenează
for (int i = 0; i < 200; i++)
    comboBox.Items.Add("Itemul " + i);
comboBox.EndUpdate();
```

Adaugarea unui interval de itemuri în liste se face cu metoda `AddRange()`. Aceasta primește un tablou de obiecte, apoi afișează în listă valoarea de tip `string` implicită pentru fiecare obiect.

Exemplu:

```
listBox.Items.AddRange(new object[] { "UNU", "DOI", "TREI" });
```

Listele pot prelua itemi. Sursele de date pot fi tablouri, colecții, *DataSet*-uri.

Exemplu:

```
string[] copii = { "Ionel", "Viorel", "Alinel", "Dorel" };
checkedListBox.DataSource = copii;
```

Itemurile se șterg cu metoda `Remove()`. **Exemplu:**

```
// Șterge obiectul caracterizat de șirul "Alin"
listBox.Items.Remove("Alin");
```

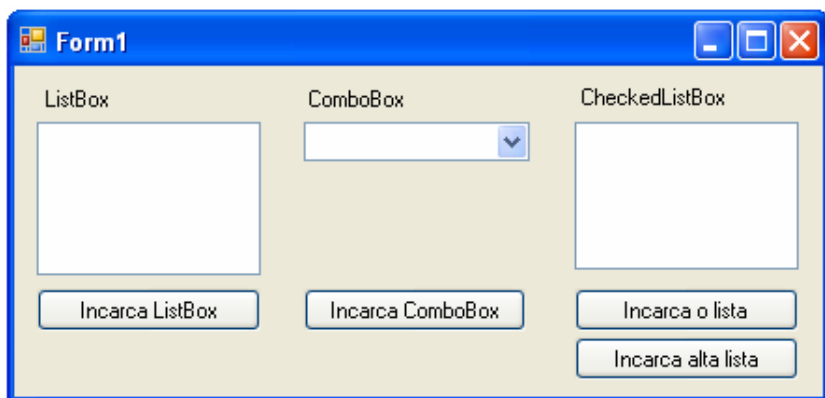
```
// Șterge primul item
listBox.Items.RemoveAt(0);
```

Clasele specifice acestor liste au desigur și alte metode și proprietăți utile. Vom face un mic proiect, pentru introducerea în lucrul cu controalele de tip listă.

Aplicația *TestListControl*

Aplicația pe care o propunem ilustrează diferite modalități de populare *runtime* a controalelor **ListBox**, **ComboBox** și **CheckedListBox**:

1. Creați un nou proiect de tip **Windows Forms Application**, cu numele *TestListControl*.
2. Din **Toolbox** plasați pe formă cu ajutorul mouse-ului un control **ListBox**, un control **ComboBox** și un control **CheckedListBox**. Adăugați de asemenea trei **Label**-uri pe care le veți eticheta corespunzător controalelor de tip listă.
3. Sub primele două controale de tip listă plasați câte un buton. Sub controlul de tip **CheckListBox** plasați două butoane. Stabiliți valorile proprietăților **Name** ale butoanelor astfel: `listButton`, `comboButton`, `checkListBoxButton1` și `checkListBoxButton2`, iar valorile proprietăților **Text**, așa cum se vede în figură:



4. Dorim ca la încărcarea formei să încărcăm și controlul **CheckListBox** cu o primă listă de itemi. Pentru aceasta, tratăm evenimentul **Load** care se declanșează la încărcarea formei. Faceți dublu click pe suprafața formei. Completați astfel corpul *handler*-ului de eveniment:

```
private void Form1_Load(object sender, EventArgs e)
{
    // Un tablou de referințe la obiecte de tip string
    string[] copii = { "Ionel", "Viorel", "Alinel",
                      "Dorel" };
    // Sursa de date este tabloul
    checkedListBox1.DataSource = copii;
}
```

5. Definim clasa, **Elev**, în fișierul **Form1.cs**. Aveți grijă, *Visual C# Express 2008* cere ca definiția oricărei clase să urmeze definiției formei:

```
class Elev
{
    private string nume;
    public Elev(string n) { nume = n; }
    public string Nume
    {
        get { return nume; }
    }
}
```

Vom defini de asemenea un câmp privat **el** în clasa **Form1**, de tip referință la un tablou de obiecte de tip **Elev**:

```
private Elev[] el;
```

6. Tratăm evenimentul **Click** pentru butonul **listButton**. Acționați dublu click pe acesta. Introduceți acest cod în metoda de tratare:


```
private void listButton_Click(object sender,
                               EventArgs e)
{
    // Membrul clasei Elev care va fi afișat în listă
    // ca ultim item
    listBox1.DisplayMember = "Nume";

    // Tabloul de referințe de tip object poate reține
    // referințele ORICĂROR TIPURI de obiecte
    listBox1.Items.AddRange(new object[] { "UNU", "DOI",
                                             "TREI", new Elev("Nelutu") });
}
```

7. Tratăm evenimentul **Click** pentru butonul **comboButton**. Acționați dublu click pe acesta. Completați astfel metoda de tratare:

```
private void comboButton_Click(object sender,
                                EventArgs e)
{
    /* Începând de la apelul BeginUpdate() și până la
    EndUpdate() controlul nu se mai redesenează, ceea
    ce sporește viteza de adăugare în listă */
    comboBox1.BeginUpdate();
    for (int i = 0; i < 20; i++)
        comboBox1.Items.Add("Itemul " + i);
    comboBox1.EndUpdate();
}
```

8. Tratăm evenimentul **Click** pentru butonul **checkListButton1**. Acționați dublu click pe acesta. Completați astfel metoda de tratare:

```
private void checkListButton1_Click(object sender,
                                     EventArgs e)
{
    string[] copii = { "Ionel", "Viorel", "Alinel",
                       "Dorel" };
    checkedListBox1.DataSource = copii;
}
```

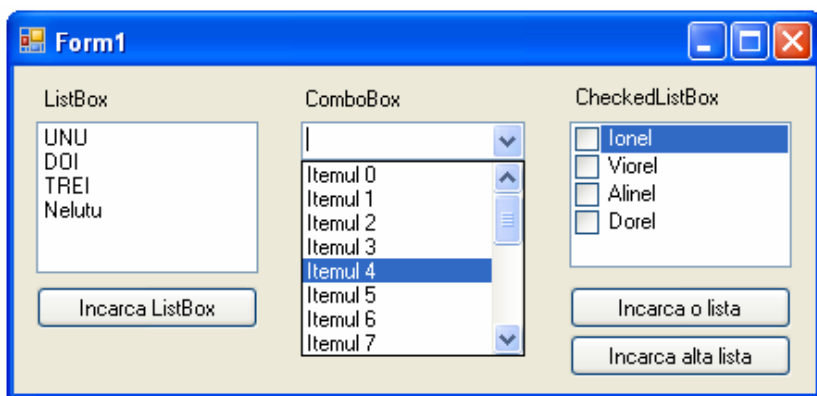
9. Tratăm evenimentul **Click** pentru butonul **checkListButton2**. Acționați dublu click pe acesta. Completați în acest mod metoda de tratare:

```
private void checkListButton2_Click(object sender,
                                     EventArgs e)
{
    // Creăm obiectele care se inserează în listă
    el = new Elev[] { new Elev("marius"),
                      new Elev("lenuta") };
}
```

```
// Sursa de date este e1
checkedListBox1.DataSource = e1;

// Membrul clasei Elev care se va afișa
// în listă este proprietatea Nume
checkedListBox1.DisplayMember = "Nume";
}
```

10. Compilați și rulați cu **F5**.



Cele trei controale se încarcă *runtime*, prin acționarea butoanelor corespunzătoare.

Controalele *TrackBar*, *NumericUpDown* și *DomainUpDown*

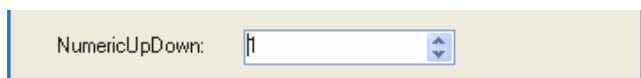
Aceste controale se numesc și controale de domeniu, deoarece utilizatorul poate introduce un număr limitat de valori dintr-un domeniu prestabilit. Introducerea datelor se face în mod vizual, acționând de regulă cu mouse-ul.

TrackBar



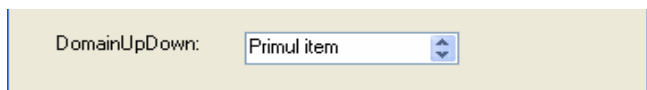
Este un control *scroll*-abil, cu ajutorul căruia utilizatorul poate alege o valoare a proprietății **value**, cuprinsă între o valoare maximă și minimă. Limitele extreme se impun prin atribuirea de valori proprietăților **Minimum** și **Maximum**. Implicit, aceste valori sunt **0** și **10**.

NumericUpDown



Este un control care conține o singură valoare numerică, pe care utilizatorul o poate modifica acționând click pe butoanele *Up* sau *Down* ale controlului. Utilizatorul poate să și introducă valori, dacă proprietatea **ReadOnly** este setată **true**.

DomainUpDown



Este similar cu un **ListBox** în aceea că oferă o listă de opțiuni între care utilizatorul poate alege. Diferența față de **ListBox** este că un singur item este vizibil la un moment dat, iar utilizatorul poate naviga în listă doar acționând săgețile *Up* și *Down* ale controlului. Pentru crearea colecțiilor de obiecte care vor alcătui lista, se utilizează metodele **Add()**, la fel ca pentru celelalte controale de tip listă.

Transparența și opacitatea controalelor

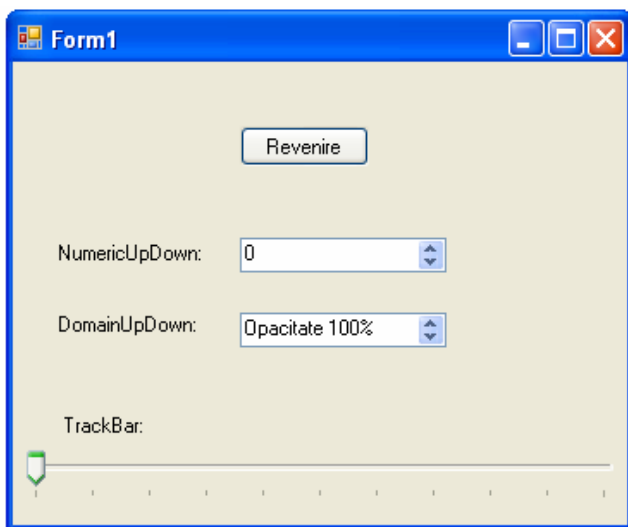
Opacity și **Transparency** sunt două proprietăți ale formelor. Au rolul de a regla gradul de transparență a formei, dar și a controalelor care se găsesc pe suprafața formei. Când proprietatea **Opacity** este setată la valoarea **1,00** (100%), atunci forma este perfect vizibilă. Dacă valoarea este **0**, atunci forma, inclusiv bordurile, marginile și controalele de pe formă sunt invizibile. Valori intermediare, reglează niveluri proporționale de transparență.

În ce privește proprietatea **Transparency**, acesteia îi este atașată o culoare. Întreaga arie a formei care are aceeași culoare (proprietatea **BackColor**) devine transparentă, iar orice acțiune cu mouse-ul pe suprafața devenită transparentă, este transferată ferestrei de sub aria devenită transparentă. Această caracteristică vă permite să definiți controale cu forme neregulate.

Aplicația Opacity

Vom utiliza controalele **TrackBar**, **NumericUpDown** și **DomainUpDown** pentru a regla opacitatea unei forme. Evident, unul singur dintre cele trei controale ar fi fost suficient, însă scopul nostru este de a învăța să le utilizăm pe toate.

1. Creați un nou proiect de tip **Windows Forms Application**, cu numele **Opacity**.
2. Din **Toolbox** plasați pe formă cu ajutorul mouse-ului un control **NumericUpDown**, un control **DomainUpDown** și un control **TrackBar**. Adăugați de asemenea trei **Label**-uri pe care le veți eticheta corespunzător controalelor și un buton căruia setați-i proprietatea **Text**: la *Revenire*:



3. Selectați controlul **DomainUpDown**. Din fereastra **Properties**, atribuiți proprietății **Text** valoarea "*Opacitate 100%*". Selectați proprietatea **Items**, apoi apăsați butonul din dreapta. În dialogul *String Collection Editor*, introduceți **10** itemi, câte unul pe linie, cu textul: "*Opacitate 100%*", "*Opacitate 90%*", ... "*Opacitate 0%*".
4. Tratăm evenimentul **Scroll** pentru **TrackBar**. Dublu click pe control. Scrieți codul marcat în **Bold**, în *handler*-ul de eveniment:

```
private void trackBar1_Scroll(object sender,
                             EventArgs e)
{
    // Odată cu creșterea valorii Value de la 0
    // 10, opacitatea formei trebuie să scadă
    this.Opacity = 1 - (double)trackBar1.Value / 10;
}
```

5. Tratăm evenimentul **SelectedIndexChanged** pentru controlul **DomainUpDown**, în scopul de a modifica opacitatea în raport cu indexul itemului. Dublu click pe control. Scrieți codul:

```
private void domain_SelectedIndexChanged(object sender,
                                       EventArgs e)
{
    // Opacitatea formei scade cu creșterea
    // indexului itemului selectat
    this.Opacity = 1 -
    (double)domainUpDown1.SelectedIndex / 10;
}
```

6. Tratăm evenimentul **ValueChanged** pentru controlul **NumericUpDown**, pentru a modifica opacitatea în funcție de valoarea afișată în control. Acționați dublu click pe control. Scrieți codul:

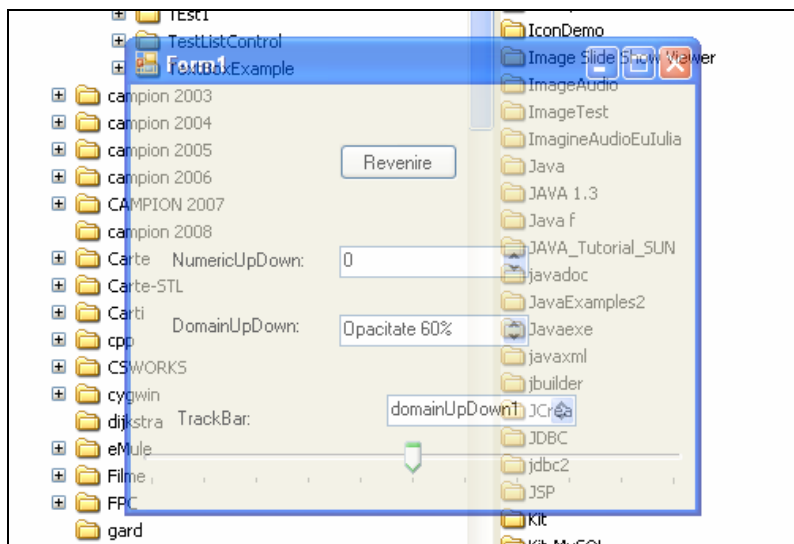
```
private void numericUpDown1_ValueChanged(object sender,
                                         EventArgs e)
{
    // Value este valoarea afișată în control
    this.Opacity = 1 - (double)numericUpDown1.Value/10;
}
```

7. Pentru reinițializarea valorilor controalelor, dar și pentru readucerea formei la opacitate maximă, tratăm evenimentul **Click** pentru butonul “Revenire”. Dublu click pe buton. Scrieți codul marcat în **Bold**, în *handler*-ul de eveniment:

```
private void button1_Click(object sender, EventArgs e)
{
    this.Opacity = 1;           // Forma devine opacă

    // Reinițializarea valorilor în controale
    trackBar1.Value = 0;
    numericUpDown1.Value = 0;
    domain.Text = "Opacitate 100%";
}
```

8. Compilați aplicația și rulați cu **F5**.



Opacitatea formei se poate seta în mod independent cu ajutorul celor trei controale.

Aplicația Transparency

Atunci când doriți ca anumite porțiuni ale formei să fie transparente și acțiunea mouse-ului pe suprafața transparentă să se transfere ferestrelor de sub formă, utilizați proprietatea **Transparency** a formelor.

1. Creați un nou proiect de tip **Windows Forms Application**, cu numele **Opacity**.
2. Din **Toolbox** plasați pe formă cu ajutorul mouse-ului un control de tip **Panel** și două butoane.
3. Selectați panoul și în fereastra **Properties** atribuiți proprietății **BackColor** valoarea **Gray**. Setați pentru cele două butoane proprietatea **Text** la valorile: **“Panou Transparent”**, respectiv **“Forma transparentă”**.
4. Tratăm evenimentul **Click** generat de butonul cu eticheta **“Panou Transparent”**. Faceți dublu click pe buton. Scrieți în metoda de tratare următorul cod :

```
private void button1_Click(object sender, EventArgs e)
{
    // Toate zonele formei de culoare gri,
    // devin transparente (suprafața panoului)
    this.TransparencyKey = System.Drawing.Color.Gray;
}
```

5. Tratăm evenimentul **Click** generat de butonul cu eticheta **“Forma transparentă”**. Dublu click pe buton, iar în metoda de tratare scrieți:

```
private void button2_Click(object sender, EventArgs e)
{
    // Zonele de culoarea implicită a controalelor
    // devin transparente (exteriorul controlului Panel)
    this.TransparencyKey =
        System.Drawing.SystemColors.Control;
}
```

6. Compilați și executați cu **F5**.



Panoul e transparent



Forma e transparentă

Observați că numai suprafața formei devine transparentă, nu și marginile, bara de titlu, sau celelalte controale de pe formă.

Controlul ProgressBar

Controlul nu permite nici o acțiune a utilizatorului. Este destinat să indice în mod vizual progresul unei aplicații care decurge lent, așa cum sunt printarea unor documente, copierea unui număr mare de fișiere, scanarea unui disc.

Stilul de afișare al controlului este determinat de proprietatea **style**. Există trei stiluri:

- O bară continuă care se umple de la stânga la dreapta
- Blocuri segmentate care progresează de la stânga la dreapta.
- Blocuri care oscilează.

Proprietatea **Value** este valoarea care reprezintă progresul curent al aplicației. Poziția curentă a barei de progres este în legătură directă cu **Value**. Proprietățile **Minimum** și **Maximum** sunt limitele de valori pentru **Value**. **Minimum** este de regulă setat la 0, iar **Maximum** este setat la o valoare care indică terminarea operației.

Întreaga operație pentru care vizualizați progresul este formată dintr-un număr de operații pe care le considerați elementare. După fiecare operație elementară, cu ajutorul metodei **PerformStep()** incrementați valoarea barei de progres cu valoarea **Step**.

Exemplu de utilizare:

```
ProgressBar p = new ProgressBar();  
// Maximum poate fi de exemplu numărul de fișiere  
// care trebuie copiate  
p.Maximum = 500;  
p.Minimum = 500;  
p.Value = 0;  
p.Step = 5;  
  
for (int i = p.Minimum; i < p.Maximum; i += p.Step)  
{  
    // Operație elementară (de exemplu, copierea unui fișier)  
    // Incrementează bara de progres  
    progress.PerformStep();  
}
```

Aplicația pe care o vom prezenta în cadrul temei **Controlul Timer** ilustrează și utilizarea unui *progress bar*.

Controlul Timer

Contrulul funcționează ca și cronometru. Declanșează un eveniment la un interval de timp specificat de către programator. **.NET** implementează câteva cronometre dintre care discutăm două:

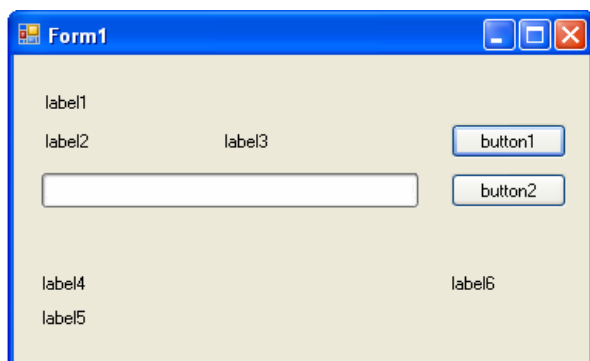
1. Primul, este definit de clasa **Systems.Windows.Forms.Timer**. Se utilizează în aplicații de tip *Windows Forms*. El apare în mod implicit în **Toolbox** și trebuie afișat într-o fereastră. Se utilizează numai în medii cu un singur *thread*. Acuratețea este de aproximativ **55** milisecunde.
2. Al doilea, definit de clasa **System.Timers.Timer**, este mai performant. Poate fi folosit în medii *multithread*. Nu se găsește în mod implicit în **Toolbox**, însă poate fi adus acolo, așa cum vom vedea.

Există asemănări și deosebiri între cele două *timer*-e. Ambele declanșează un eveniment la un interval specificat. Primul declanșează evenimentul **Tick**, iar al doilea, evenimentul **Elapsed**. Ambele au metodele **Start()** și **Stop()**, și proprietatea **Enabled**.

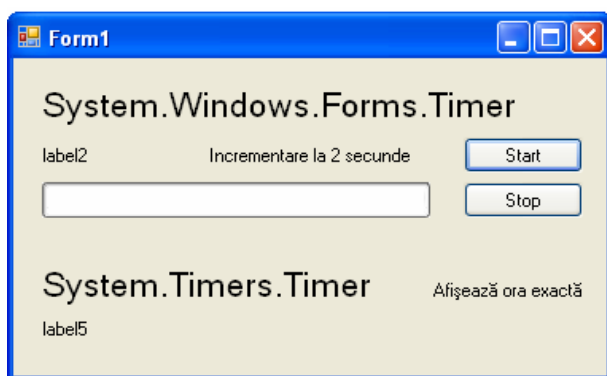
Aplicația Timer.Example

Aplicația execută sarcinile:

- Utilizează *timer*-ul **System.Windows.Timer** care declanșează un eveniment **Tick** la interval de două secunde, timp de 20 de secunde. Numărul de secunde se afișează într-un control **Label**, iar progresul se vizualizează cu o bară de progres. Cronometrul poate fi pornit și oprit cu butoanele *Stop* și *Start*.
 - Utilizează *timer*-ul **System.Timers.Timer** pentru afișarea orei, într-un control **Label**. Evenimentul **Elapsed** se declanșează la interval de o secundă.
1. Creați un proiect de tip **Windows Forms Application**, cu numele *TimerExample*.
 2. Aduceți din **Toolbox** pe suprafața formei un control de tip **Timer**, două controale de tip **Button**, un **ProgressBar** și și șase controale **Label**:



3. Vom seta mai întâi etichetele butoanelor:
 - a. Selectați `label11`. În fereastra **Properties**, setați fontul la dimensiunile (**16, 50**). Atribuiți proprietății **Text** valoarea "*System.Windows.Forms.Timer*". Selectați `label12`. Setați fontul la dimensiunile (**16, 50**). Ștergeți numele etichetei. Selectați `label13`. Atribuiți proprietății **Text** valoarea "*Incrementare la 2 secunde*".
 - b. Selectați `label14`. Setați fontul la dimensiunile (**16, 50**). Atribuiți proprietății **Text** valoarea "*System.Timers.Timer*". Selectați `label15`. Setați fontul la dimensiunile (**16, 50**). Ștergeți numele etichetei. Selectați `label16`. Atribuiți proprietății **Text** valoarea "*Afișează ora exactă*".
 - c. Selectați `button1`. Atribuiți proprietății **Text** valoarea "*Start*". Selectați `button2`. Atribuiți proprietății **Text** valoarea "*Stop*".



4. Selectați referința `timer1` în *designer tray*. În panoul **Properties**, setați proprietatea **Name** la valoarea `winTimer`. Setați la **1000** (o secundă) valoarea proprietății **Interval**. Este intervalul la care se declanșează evenimentul **Tick**.
5. Vom adăuga un *timer*-ul din spațiul de nume **System.Timers**. În **Toolbar**, în partea cea mai de jos, unde nu mai sunt controale, faceți click drept și alegeți *Choose Items...*
6. În dialogul *Choose Toolbox Items*, alegeți din tab-ul *.NET Framework Components*, clasa **Timer** din spațiul de nume **System.Timers**. Un icon tip ceas apare în partea de jos în **Toolbar**.
7. Trageți noul **Timer** pe suprafața formei. În *designer tray*, selectați referința `timer1`. În panoul **Properties**, schimbați numele referinței la `systemTimer`, setați



valoarea proprietății **Enabled** la **true**, iar a proprietății **Interval** la **1000**. Aceasta înseamnă că evenimentul **Elapsed** se declanșează la interval de o secundă.

8. Selectați bara de progres. În panoul **Properties**, setați proprietatea **Step** la valoarea **2** și proprietatea **Maximum** la valoarea **20**.
9. În fișierul *Form1.cs*, în clasa **Form1**, declarați un câmp privat, numit **time**, care contorizează numărul de secunde scurse. În **Solution Explorer**, faceți click drept pe *Form1.cs* și selectați *View Code*. Scrieți:

```
private int time = 0;
```

10. Tratăm evenimentul **Tick** pentru **winTimer**. Selectați referința **winTimer** în *designer tray*, apoi în **Properties**, acționați dublu click pe evenimentul **Tick**. Scrieți codul:

```
private void winTimer_Tick(object sender, EventArgs e)
{
    time += 2;    // Un eveniment Tick la 2 secunde

    // Afișăm timpul în etichetă
    label2.Text = string.Format("{0}", time);

    // Incrementarea barei de progres
    progressBar1.PerformStep();
    if (time == 20)    // Dacă s-au scurs 20 secunde
    {
        // oprim timer-ul și bara de progres
        winTimer.Stop();
        progressBar1.Enabled = false; // oprim
    }
}
```

11. Tratăm evenimentul **Click** pentru butonul *Start*. Faceți dublu click pe buton. Introduceți codul:

```
private void button1_Click(object sender, EventArgs e)
{
    winTimer.Enabled = true;    // Activăm timer-ul
    if (time == 20)    // Dacă au trecut 20 sec
    {
        // se fac reinițializări
        time = 0;
        progressBar1.Value = 0;
    }
}
```

12. Tratăm evenimentul **Click** pentru butonul *Stop*. Faceți dublu click pe buton. Introduceți codul:

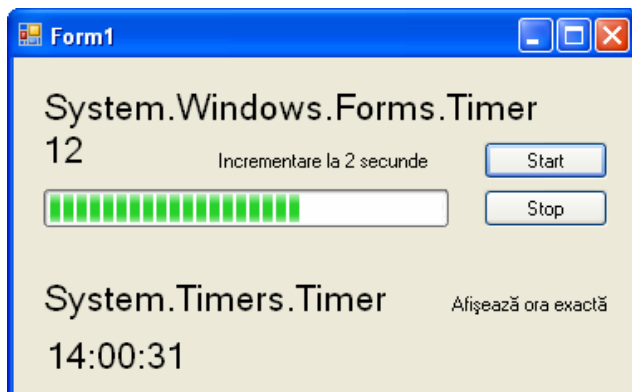
```
private void button2_Click(object sender, EventArgs e)
{
    winTimer.Stop(); // Oprim cronometrul
    progressBar1.Enabled = false; // Oprim progress bar
    if (nrTick == 20)
    {
        nrTick = 0;
        progressBar1.Value = 0;
    }
}
```

13. Tratăm evenimentul **Elapsed** pentru **systemTimer**. Selectați referința **systemTimer** în *designer tray*, apoi în **Properties**, acționați dublu click pe evenimentul **Elapsed**. Scrieți codul:

```
private void systemTimer_Elapsed(object sender,
                                System.Timers.ElapsedEventArgs e)
{
    // Proprietatea Now a clasei DateTime, returnează
    // ora exactă
    label5.Text = System.DateTime.Now.ToString();
}
```

14. Compilați și rulați aplicația cu **F5**.

Captură de ecran la execuție:



Controalele PictureBox și ImageList

Imagini de fundal pentru controale

Multe controale suportă afișarea de imagini, deoarece moștenesc proprietatea **BackgroundImage** de la clasa **Control**. Între aceste controale,

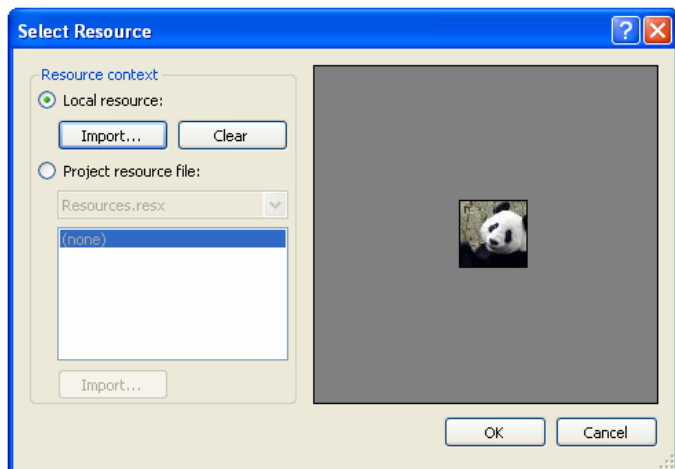
menționăm **Button**, **CheckBox**, **RadioButton**, **PictureBox**. Și controalele de tip container cum sunt **Panel**, **GroupBox** și **Form** suportă imagini de fundal.



Anumite controale suportă și imagini de prim plan. De exemplu, **Button** are în acest scop proprietatea **Image**, moștenită de la clasa **ButtonBase**. În cazul în care utilizați atât o imagine de fundal cât și de prim plan, imaginea de prim plan se afișează deasupra.

Controalele au în plus proprietăți cu ajutorul cărora se pot modifica caracteristicile imaginilor afișate.

Pentru afișarea *design time* a unei imagini, se selectează controlul, se apasă butonul cu eticheta “...” din linia proprietății **BackgroundImageLayout** sau a proprietății **Image**. În dialogul *Select Resource*, bifați “*Local resource*”, apoi apăsați butonul “*Import*”.



Caracteristicilor imaginilor se modifică simplu din panoul **Properties** atribuind valori următoarelor proprietăți:

- **BackgroundImageLayout**
Are valorile posibile: **None**, **Tile**, **Center**, **Stretch** și **Zoom**. De exemplu, mai sus, pentru **CheckBox**, și **RadioButton** am ales **Center**, iar pentru **GroupBox** valoarea **Tile**.
- **ImageAlign**
Label-urile nu suportă **BackgroundImage** deci utilizați **Image** pentru afișare, iar pentru modul de afișare, alegeți dintre valorile disponibile în panoul **Properties**, pentru proprietatea **ImageAlign**.
- **AutoSize**
Pentru *label*-uri, *checkbox*-uri și radiobutoane trebuie să setați această proprietate la valoarea **false**, altminteri nu veți reuși să redimensionați suprafețele controalelor, astfel încât să potrivească imaginile.
- **TextAlign**
Această proprietate vă permite să stabiliți poziția și alinierea textului în raport cu suprafața controlului.

Clasa Image

.NET definește clasa **Image** în spațiul de nume **System.Drawing**, pentru lucrul cu imagini.

Clasa este abstractă, deci nu se pot crea obiecte de acest tip. În schimb, imaginile se citesc din fișiere cu metoda statică **FromFile()**:

Exemplu:

```
Image im = Image.FromFile(
    Path.Combine(Application.StartupPath, "foto.gif"));
Label lbl = new Label(); // Creează o etichetă
lbl.Image = im;          // Afișează imaginea în control
```

Codul de mai sus încarcă imaginea din fișierul "**foto.gif**" aflat în folderul în care se găsește fișierul *assembly* (exe) al aplicației. Metoda nu recunoaște decât formatele **.GIF** și **.BMP**. Eticheta **lbl** afișează imaginea încărcată.

Clasa are proprietăți și metode pentru manipularea imaginilor. Metoda **FromStream()** încarcă imagini de diferite tipuri nu numai în formatele standard **.GIF** sau **.BMP**. Crează un stream din care puteți descărca imaginea. *Stream*-ul poate să fie de conectat la orice dispozitiv de intrare, de exemplu la o adresă de Internet:

```
string s = http://...../foto.jpg; // URL
// Sunt necesare obiectele wrq și wrs pentru crearea
// stream-ului
WebRequest wrq = WebRequest.Create(s);
WebResponse wrs = wrq.GetResponse();
Stream str = wrs.GetResponseStream(); // Crează stream-ul
```

```
Label lbl = new Label();

// Afișează imaginea într-un control de tip Label
lbl.Image = Image.FromStream(str);
str.close();
```

Vom reveni asupra tipului **Image**.

Controlul PictureBox

PictureBox este un control aflat la îndemână în **Toolbox**. E folosit pentru afișarea imaginilor grafice, obținute din fișiere **JPEG**, **GIF**, **BMP**, **PNG**, iconuri sau metafișiere.

Prezentăm câțiva membri reprezentativi ai clasei **PictureBox**.

Proprietăți

Image	Se setează pentru afișarea <i>design time</i> sau <i>run time</i> a imaginilor
SizeMode	Specifică modul de afișare a imaginii. Are valorile: StretchImage , CenterImage , Normal , Zoom .
ClientSize ImageLocation	Permite modificarea dimensiunii imaginii în timpul rulării. Reprezintă calea sau URL-ul spre imaginea care se afișează în PictureBox .

Metode

Load()	Afișează imaginea specificată de proprietatea ImageLocation . Are două versiuni.
SetBounds()	Setează limitele controlului, la o anumită locație, și la dimensiunile specificate.

Evenimente

Click	Se declanșează la click pe control
Paint	Se declanșează când controlul trebuie redesenat
ClientSizeChanged	Se declanșează când proprietatea ClientSize se modifică

Resursele unui proiect Visual C#

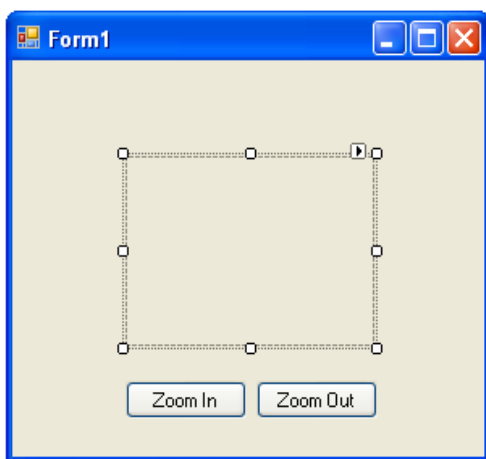
Ați văzut că un program C# poate accesa fișiere externe de tip imagine. problema apare când asemenea resurse sunt șterse sau mutate din greșeală. În aceste condiții aplicația nu le mai poate utiliza.

În Visual C# există posibilitatea să incorporați resurse de care aplicația are nevoie: imagini, iconuri, sunete, stringuri în același fișier *assembly*(.exe) care conține codul compilat al aplicației. Pentru a vedea cum se realizează acest lucru, realizăm un mic proiect.

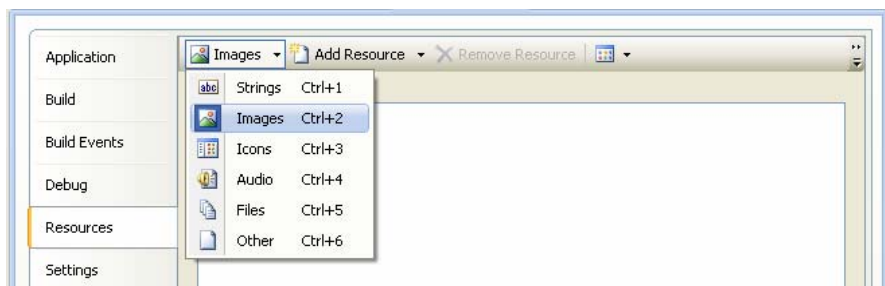
Aplicația *ImageResourceExample*

Proiectul va fi unul simplu: pe o formă, avem un control **PictureBox**. Imaginea pe care o afișăm în control, va fi una luată din resursele proiectului. Pentru divertisment, implementăm și facilitatea de mărire și de micșorare a imaginii (*Zoom*).

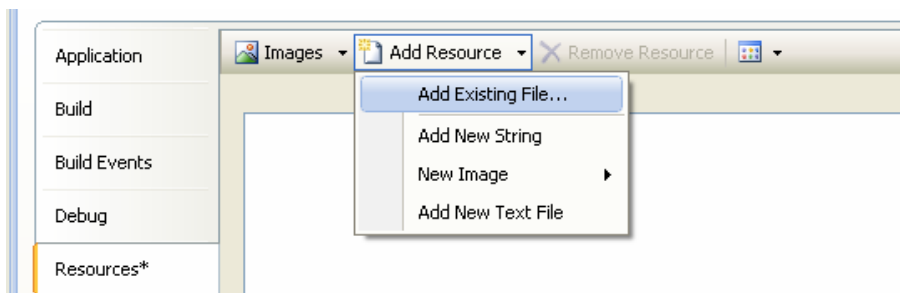
15. Creați un nou proiect de tip **Windows Forms Application**, cu numele *ImageResourceExample*.
16. Aduceți din **Toolbox** pe suprafața formei un control de tip **PictureBox** și două butoane.
17. Selectați controlul **PictureBox**. În panoul **Properties**, setați pentru proprietatea **SizeMode** valoarea **zoom**. Aceasta permite controlului să se redimensioneze odată cu imaginea.
18. Selectați pe rând cele două butoane și stabiliți pentru proprietatea **Text**, valorile *ZoomIn*, respectiv *ZoomOut*:



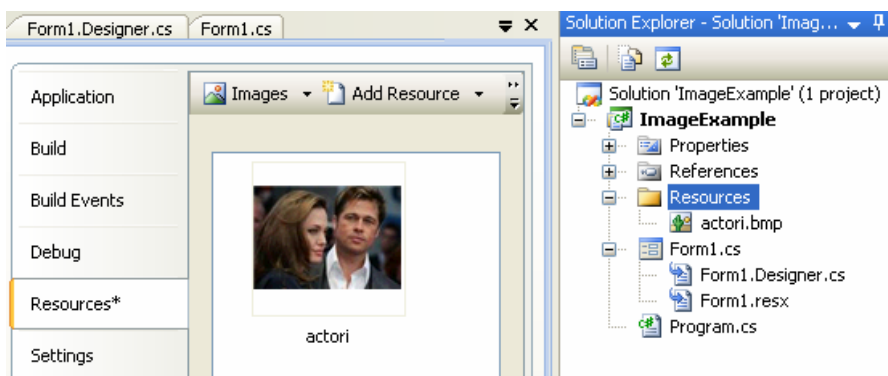
19. Acum adăugăm o imagine ca resursă a proiectului. În **Solution Explorer**, dublu click pe **Properties**. Se deschide fereastra **Editorului de Resurse**. Pe coloana din stânga ferestrei, selectați *Resources*. Din lista primului drop down button din stânga sus, selectați *Images*:



20. Din lista butonului **Add Ressource**, selectați **Add Existing File ...**:



21. Selectați o imagine pe care ați pregătit-o în acest scop:



22. Selectați controlul **PictureBox**. În panoul **Properties**, acționați butonul cu eticheta (...) din dreapta proprietății **Image**. În dialogul **Select Resource** veți selecta **Project Resource File** și încărcați imaginea pe care ați adus-o la resursele proiectului.

23. Declarați un câmp privat **zoom**, în clasa **Form1**. El reprezintă cantitatea cu care se incrementează dimensiunile controlului la fiecare apăsare a butoanelor:

```
private int zoom;
```

24. Tratăm evenimentul **Click** pentru butonul cu textul **ZoomIn**. Acționați dublu click asupra lui. Se deschide **Editorul de Cod**. Scrieți codul evidențiat cu **Bold**, în handler-ul de eveniment:

```
private void button1_Click(object sender, EventArgs e)
{
    zoom = 2;
    // Lățimea și înălțimea după o apăsare de buton
    int W = pictureBox1.Size.Width + zoom;
    int H = pictureBox1.Size.Height + zoom;
}
```



```
// Are loc actualizarea dimensiunilor prin  
// modificarea proprietății ClientSize  
pictureBox1.ClientSize = new Size(W, H);  
}
```

25. Tratăm evenimentul **Click** pentru butonul cu textul *ZoomOut*. Acționați dublu click asupra lui. În *handler*-ul evenimentului veți scrie același cod, cu singura diferență că prima linie este: **zoom = -2;** (dimensiunile scad).
26. Compilați și rulați cu **F5**.



Prin apăsarea celor două butoane, se obține efectul de mărire, respectiv de micșorare a imaginii.

De reținut:

- Resursele adăugate proiectului se integrează în mod automat în fișierul *assembly* compilat (fișierul *.exe*).
- Resursele sunt imagini, iconuri, sunete, sau stringuri.
- Editarea resurselor se face *design time* cu ajutorul *Editorului de Resurse*, sau *runtime* în mod programatic.
- Între folderele proiectului apare unul nou, numit **Resources**, care poate fi vizualizat din **Solution Explorer**.
- Se pot elimina elemente folderul *Resources*, fără ca aceasta să afecteze fișierul *assembly*. La fiecare **build**, compilatorul consultă acest folder și numai dacă a apărut o versiune nouă a unui fișier, acesta este integrat în *assembly*.

Manipularea *runtime* a resurselor

Resursele se accesează ușor din cod, deoarece mediul integrat crează o clasă **Resources**, aflată în spațiul de nume **Properties**, iar acesta din urmă se găsește în interiorul spațiului de nume al aplicației. În clasa **Resources**, toate resursele se accesează simplu, ca oricare membru public.

Iată de exemplu cum puteți atribui **runtime** unui control o altă imagine din resursele proiectului:

```
pictureBox1.Image = Image.Properties.Resources.pisica;
```

Resursa pisica este un tip nou creat, pe baza unei imagini obținute dintr-un fișier care se numea de exemplu *pisica.bmp*.

Setarea iconului aplicației și a iconului unei forme

Ca să atașați un icon fișierului **assembly** (.exe) al aplicației, în **Solution Explorer** acționați dublu click pe itemul **Properties**. În fereastra care se deschide, selectați **Application**, apoi selectați **check box**-ul **Icon and manifest** și alegeți iconul dorit.

Ca să atașați un icon unei forme, selectați forma în **Form Designer**, apoi în fereastra **Properties** selectați proprietatea **Icon** și alegeți un icon-ul dorit.

Clasa ImageList

ImageList este un control care încapsulează o colecție de imagini sau icon-uri. Imaginile se folosesc de către alte controale, cum este **ListView**, **TreeView**, sau **ToolStrip**. Principalii membrii ai clasei sunt:

Proprietăți

Images	Colecția de imagini utilizate de către alte controale
ImageSize	Dimensiunea imaginilor în colecție. Indiferent de dimensiunile inițiale ale imaginilor, acestea se convertesc la un format specificat în momentul în care se adaugă la colecție
ColorDepth	Valoare care indică adâncimea de culoare. Valorile obișnuite sunt 5 biți (256 culori), 16 biți (<i>high color</i>), 24 biți (<i>true color</i>).

Metode

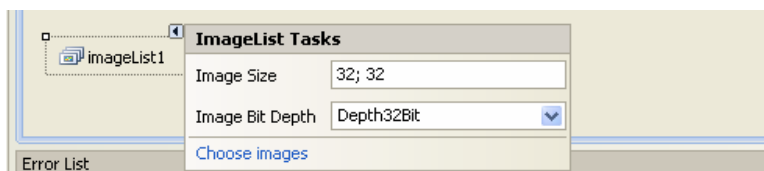
Draw()	Metodă supraîncărcată care desenează imaginile pe suprafața unui control.
---------------	---

Aplicația *ImageListExample*

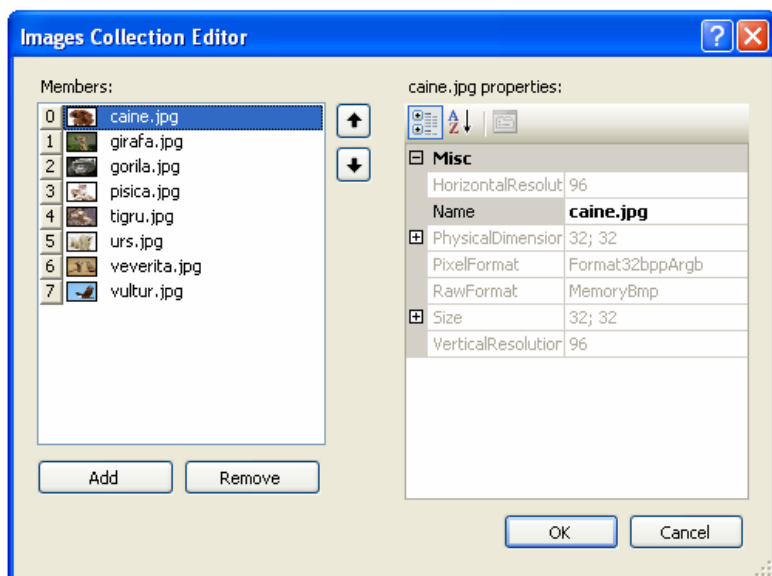
Vom utiliza un control de tip **ImageList** ca sursă de imagini pentru un control **PictureBox**. Rețineți că un **PictureBox** nu se poate conecta direct la întreaga listă de imagini, așa cum o fac alte controale mai evoluate, ca **ListView** sau **TreeView**.

În această aplicație, la fiecare acționare unui buton, se accesează câte o imagine din **ImageList** și imaginea se afișează în **PictureBox**.

1. Creați un nou proiect de tip **Windows Forms Application**, cu numele *ImageListExample*.
2. Aduceți din **Toolbox** pe suprafața formei un control de tip **PictureBox** și un buton.
3. Căutați opt imagini de mărime medie și salvați-le într-un folder oarecare.
4. Selectați controlul **PictureBox**. În panoul **Properties**, setați pentru proprietatea **SizeMode** valoarea **StretchImage**. Aceasta "întinde" imaginea pe toată suprafața controlului. Setați ca imagine inițială pentru proprietatea **Image**, una dintre cele 8 imagini pe care le-ați pregătit pentru acest proiect:
5. Selectați butonul și stabiliți pentru proprietatea **Text**, valoarea "Urmatoarea imagine".
6. Din **Toolbox**, trageți pe suprafața formei un control de tip **ImageList**. În mod automat, în **tray**-ul *Form Designer*-ului apare numele referinței: **imageList1**.
7. Apăsați click pe săgeata mică din colțul dreapta sus al numelui **imageList1** în *designer tray*. Stabiliți **ImageSize** la valorile **32** și **32** și adâncimea de culoare la valoarea **Depth32Bit**.



8. Alegeți *Choose images*. În *Editorul Colecțiilor de Imagini*, adăugați pe rând câte o imagine la **ImageList**. Cifrele din stânga reprezintă pozițiile imaginilor în colecție. Pozițiile se pot schimba, acționând săgețile:



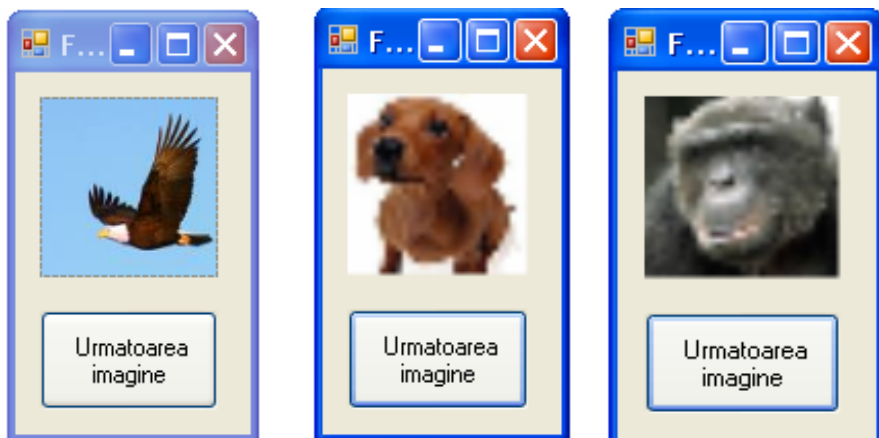
9. Declarați în clasa *Form1*, în fișierul *Form1.cs*, câmpul privat **count**. Acesta reprezintă indexul imaginii din **ImageList**, care se afișează la un moment dat în **PictureBox**:

```
private int count = 0;
```

10. Vom trata evenimentul **Click** pentru buton. Acționați dublu click pe suprafața lui. Scrieți codul evidențiat cu **Bold**, în *handler*-ul de eveniment:

```
private void button1_Click(object sender, EventArgs e)
{
    // Atribuim proprietății Image imaginea din
    // ImageList corespunzătoare indexului count % 8
    pictureBox1.Image = imageList1.Images[count % 8];
    count++;
}
```

11. Compilați și lansați în execuție cu **F5**.



Imaginea se schimbă la fiecare click.

De reținut:

- Obiectele de tip **ImageList** rețin colecții de imagini, care se utilizează de către alte controale.
- Toate imaginile se formează în mod automat la aceleași dimensiuni și adâncimi de culoare precizate în momentul adăugării în **ImageList**, indiferent de formatul inițial.
- Imaginile se accesează cu ajutorul operatorului de indexare. De exemplu, `imageList1.Images[3]` ; reprezintă a patra imagine din colecția referită de `imageList1`. Atenție la acest exemplu! În colecția aceasta trebuie să existe cel puțin 4 imagini, altfel `Images[3]` duce la aruncarea unei excepții.

Controlul *ListView*

ListView este un control complex, destinat afișării unei liste de itemi.

Panoul din dreapta al aplicației *Windows Explorer* implementează un **ListView**. *Control Panel* este un alt exemplu cunoscut de către toată lumea.

Pentru fiecare item din listă controlul afișează un text și un icon.

Itemii se pot afișa în cinci moduri diferite. Cele cinci moduri sunt determinate de valorile pe care le poate avea proprietatea **view**, a clasei **ListView**.

Modurile de afișare într-un *ListView*

Valorile posibile ale proprietății **view** determină cele cinci moduri de afișare. Aceste valori sunt:

LargeIcon	- Afișează iconuri mari (32x32 pixeli), cu text dedesupt, aliniate de la stânga la dreapta și de sus în jos
------------------	---

SmallIcon	- Afișează iconuri mici (16x16 pixeli), cu eticheta în dreapta
List	- Afișează iconuri mici (16x16 pixeli), cu eticheta în dreapta. Itemii sunt aranjați în coloane, fără <i>header</i> de coloană.
Details	- Fiecare item apare pe o singură linie. Informațiile suplimentare despre item sunt date de subitemi. Subitemii sunt aranjați pe coloane diferite, pe aceeași linie. Coloana cea mai din stânga conține un icon mic și un text.
Tile	- Afișează iconuri mari, cu text în dreapta acestuia, reprezentând informațiile din subitemi.

IMPORTANT !

*Fiecare item într-un control **ListView**, este o instanță a clasei **ListViewItem**.*

Principalii membri ai clasei ListView

Clasa conține numeroase metode, proprietăți și evenimente, grație cărora are multiple calități. Iată câteva :

- ✓ Suportă selecția simplă și multiplă a itemilor.
- ✓ Tratatând corespunzător a evenimentele de la tastatură și mouse, itemii pot deveni activi, declanșând acțiuni ca deschidere de fișiere, lansare de aplicații, etc.
- ✓ Se poate folosi pentru afișarea informațiilor dintr-o bază de date, a informațiilor din fișiere, de pe disc, sau a informațiilor din aplicații.

Principalii membri ai clasei sunt :

Proprietăți :

Items	- Depozitează colecția de obiecte de tip ListViewItem care reprezintă subitemii.
Columns	- Returnează colecția de obiecte de tip ColumnHeader care sunt afișate în ListViewControl .
View	- Permite specificarea modului de afișare a itemilor în listă
LargeImageList	- Indică obiectul ImageList care conține iconurile folosite la afișarea itemilor, atunci când proprietatea View are valoarea LargeIcon
SmallImageList	- Indică obiectul ImageList care conține iconurile folosite atunci când proprietatea View are oricare altă valoare în afară de LargeIcon
MultiSelect	- Când este setat true , se pot selecta mai mulți itemi odată

Metode:

Clear ()	- Înlătură toți itemii din control
BeginUpdate () EndUpdate ()	- Prima metodă suspendă redesenarea controlului ListView la fiecare adăugare a unui item în situația în care se adaugă un mare număr de itemi. - Se apelează a doua metodă după terminarea operației de update, pentru a permite controlului să se redeseneze.

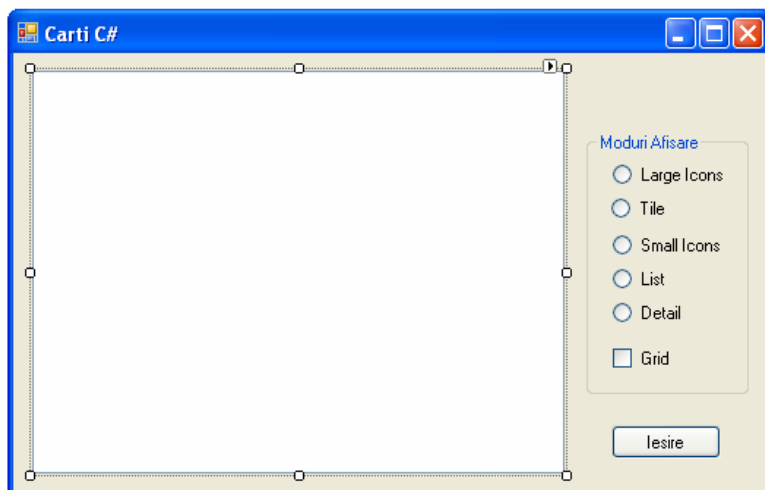
Evenimente:

ColumnClick	- Se declanșează la click pe header-ul unei coloane
SelectedIndexChanged	- Se declanșează când utilizatorul selectează sau deselectează un item

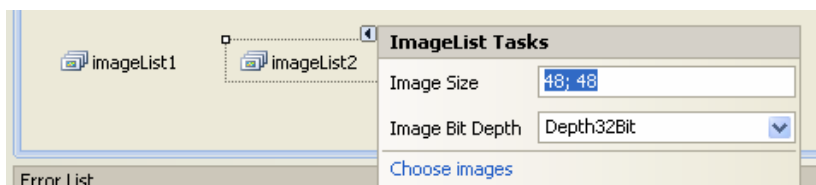
Aplicația *ListViewExample*

Proiectul afișează o listă titluri de cărți, în toate modurile de afișare: *LargeIcon*, *SmallIcon*, *List*, *Tile*, *Detail*. În modul *Detail*, fiecare item (carte) are câte doi itemi suplimentari, afișați în coloana a doua și a treia: numele autorului și editura care a publicat cartea. De asemenea, tot în modul *Detail* este disponibilă opțiunea de *Grid*.

1. Creați un nou proiect de tip **Windows Forms Application**, cu numele *ListViewExample*.
2. Trageți din **Toolbox** pe suprafața formei un control de tip **ListView** un control **GroupBox**, cinci controale **RadioButton**, un **CheckBox** și un buton de apăsare și aranjați-le astfel:



3. Căutați câteva fișiere de tip imagine cu coperti de cărți și salvați-le într-un folder oarecare. Notați-vă pentru fiecare carte, titlul, autorul și editura.
4. Aduceți din **Toolbox** două controlale de tip **ImageList**. În *tray*-ul *Form Designer*-ului apar *imageList1* și *imageList2*. Ele reprezintă referințele spre colecția de icon-uri mici necesare în modurile *SmallIcon*, *List* și *Detail*, respectiv spre colecția de icon-uri mari, necesare în modurile *LargeIcon* și *Tile*.
5. Setati pentru *imageList1*, dimensiunile (16, 16) și adâncimea de culoare 8 biți. Setati pentru *imageList2* dimensiunile (48, 48) (puteți decide și alte dimensiuni) și adâncimea de culoare 32 de biți:



6. Adăugați aceleași imagini în ambele obiecte de tip **ImageList**, acționând *Choose images*.
7. Dorim să populăm controlul **ListView** la încărcarea formei. Faceți dublu click pe suprafața formei, pentru a trata evenimentul **Load**. În corpul *handler*-ului evenimentului, scrieți codul evidențiat în **Bold**:

```
private void Form1_Load(object sender, EventArgs e)
{
    // Controlul listView1 preia colecțiile de iconuri
    // mici și de iconuri mari
    listView1.SmallImageList = imageList1;
    listView1.LargeImageList = imageList2;

    // Suspendăm redesenarea listei cât timp adăugăm
    // itemi
    listView1.BeginUpdate();

    // Pentru modul Detail, adăugăm trei coloane având
    // în header-e textele "Carti", Autor, "Editura".
    // Lățimile coloanelor sunt 150, 100 și 110 și
    // alinierea orizontală
    listView1.Columns.Add("Carti C#", 150,
        HorizontalAlignment.Left);
    listView1.Columns.Add("Autor", 100,
        HorizontalAlignment.Left);
    listView1.Columns.Add("Editura", 110,
        HorizontalAlignment.Left);

    // Adăugăm primul item în listă. Este un obiect de
    // tip ListViewItem
    ListViewItem it1 = new ListViewItem();

    // Indexul icon-ului din listImage1 și listImage2.
    // Puteți avea mai mulți itemi cu același index,
    // deci cu același icon
    it1.ImageIndex = 0;

    // Textul itemului
    it1.Text = "C# 2008 Code Book";

    // Creăm cei doi subitemi. Puteți avea oricâți
    // subitemi pentru un item, dar aveți grijă să
    // creați tot atâtea coloane.
```



```

it1.SubItems.Add("Jürgen Bayer");
it1.SubItems.Add("Addison-Wesley 2007");

// Adaugăm itemul în controlul ListView
listView1.Items.Add(it1);

// Se crează al doilea item, în același mod
ListViewItem it2 = new ListViewItem();
it2.ImageIndex = 1;
it2.Text = "Beginning C# 2008";
it2.SubItems.Add("Karl Watson");
it2.SubItems.Add("WROX 2008");
listView1.Items.Add(it2);

// Creați ceilalți itemi ca mai sus
// ...

// Am terminat adăugarea de itemi.
// Acum permitem controlului să se redeseneze
listView1.EndUpdate();
}

```

8. Acționați dublu click pe radio butonul cu eticheta *Large Icons*. Tratăm evenimentul **CheckedChanged**. În *Editorul de Cod*, scrieți:

```

private void radioButton1_CheckedChanged(object sender,
                                         EventArgs e)
{
    // Proprietatea View primește valoarea LargeIcon.
    // LargeIcon este un membru al enumerării View
    listView1.View = View.LargeIcon;

    // Dezactivăm căsuța de validare pentru Grid.
    // Efectul Grid e posibil numai în modul Detail
    checkBox1.Enabled = false;
}

```

9. Acționați dublu click pe radio butonul cu eticheta *Tile*. În corpul *handler*-ului de eveniment, scrieți codul:

```

listView1.View = View.Tile;
checkBox1.Enabled = false;

```

10. Acționați dublu click pe radio butonul cu eticheta *Small Icons*. În corpul *handler*-ului de eveniment, scrieți codul:

```

listView1.View = View.SmallIcon;
checkBox1.Enabled = false;

```

11. Acționați dublu click pe radio butonul cu eticheta *List*. În corpul *handler*-ului de eveniment, scrieți:

```
listView1.View = View.Tile;
checkBox1.Enabled = false;
```

12. Acționați dublu click pe checkbox-ul cu eticheta *Grid*. Tratăm evenimentul **CheckedChanged**. În *handler-ul de eveniment*, scrieți codul marcat cu **Bold**:

```
private void checkBox1_CheckedChanged(object sender,
                                     EventArgs e)
{
    // Dacă tocmai s-a selectat checkbox-ul
    if (checkBox1.Checked)
    { // atunci punem griduri.
        listView1.GridLines = true;
    }
    else
    { // Dacă tocmai s-a deselectat controlul
      // scoatem gridurile
        listView1.GridLines = false;
    }
}
```

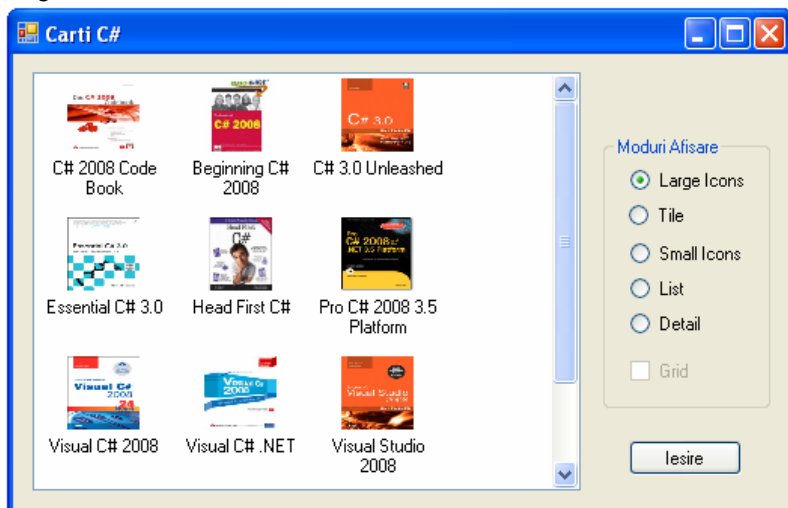
13. Acționați dublu click pe butonul *Iesire*. Tratăm evenimentul **Click**. În corpul metodei de tratare, scrieți:

```
Application.Exit(); // Ieșire din aplicație
```

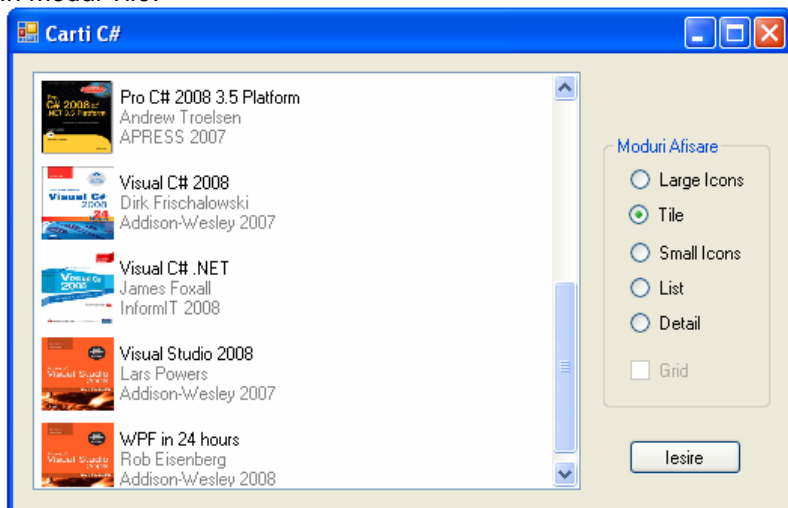
14. Compilați proiectul și rulați cu **F5**.

La rulare, se obține:

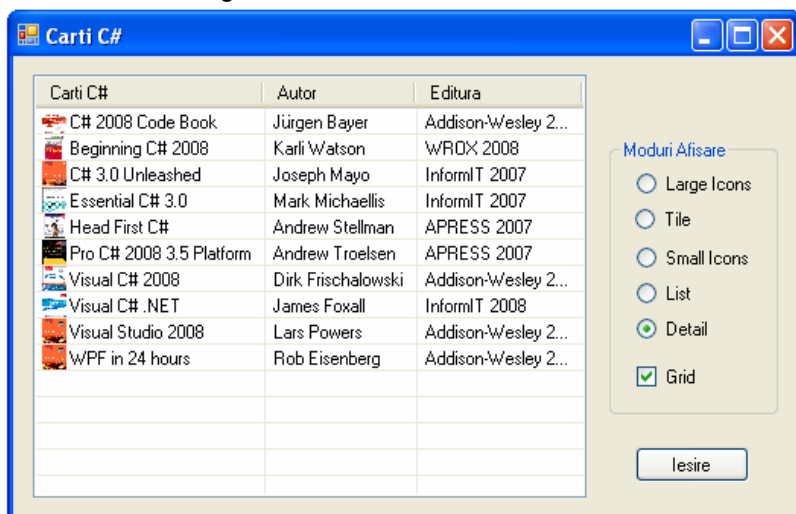
Vedere *Large Icons*:



Vedere în modul *Tile*:



Vedere în modul *Detail* cu gridurile activate:



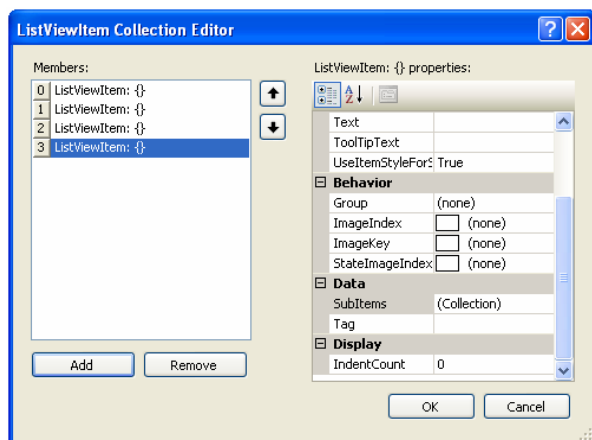
Celelalte moduri de afișare funcționează de asemenea în mod corespunzător.

Observații:

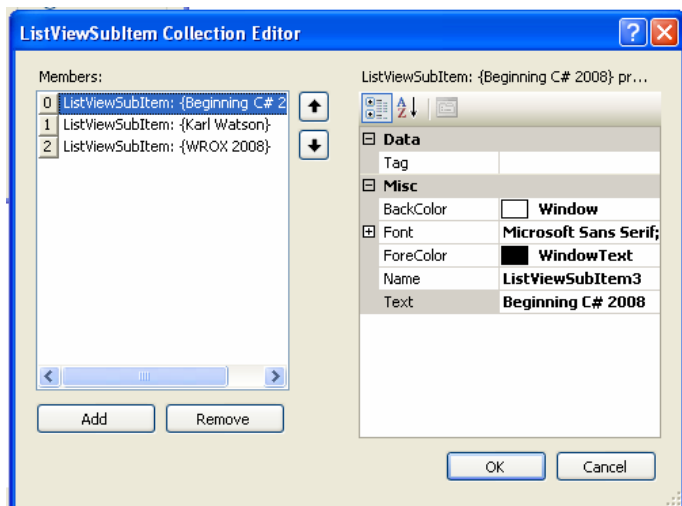
- În multe situații itemii nu se introduc în mod manual. Aceștia pot fi încărcăți din fișiere **XML**, sau din baze de date, așa cum vom vedea în capitolele următoare.
- În proiectul acesta, itemii au fost adăugați programatic. Este una dintre cele mai simple variante, dar nu unica. Mai există și alte variante de adăugare programatică a itemilor, pe care le puteți găsi în literatura de specialitate.

Adăugarea *design time* a itemilor într-un ListView

Selecționați controlul **ListView**. În fereastra **Properties** apăsați butonul cu eticheta (...) din dreapta proprietății **Items**. Se deschide dialogul *ListViewItem Collection Editor*. Apăsați butonul **Add** pentru a adăuga itemi.



În panoul din dreapta, selecționați *Subitems* și apăsați butonul din dreapta. În dialogul *ListViewSubItem Collection Editor*, adăugați pe rând subitemii necesari pentru fiecare item:



De reținut:

- Dacă implementați afișarea *Detail*, atunci este nevoie să adăugați coloane (obiecte de tip **ColumnHeader**).
- Itemii sunt obiecte diferite de tip **ListViewItem**.
- Numărul subitemilor trebuie să coincidă cu numărul coloanelor.

Probleme propuse

1. Aduceți următoarea facilitate aplicației *ListViewExample*: la apăsarea unui buton, se activează opțiunea de selectare a întregului rând, atunci când utilizatorul face click pe un item în modul *Detail*.
2. În aplicația *ListViewExample*, introduceți câte un mic icon care precedă textul în *header*-ul fiecărei coloane (modul *Detail*).
Indicație: Utilizați constructorul adecvat al clasei **ColumnHeader**, astfel:

```
ColumnHeader c = new ColumnHeader(3);  
// 3 este indexul iconului  
c.Text = "C# 2008"  
listView1.Columns.Add(c);  
// etc
```

3. * Creați o aplicație care preia într-un **ListView** informațiile despre fișierele și folderele care se găsesc într-un folder dat. Folderul se alege cu ajutorul controlului predefinit **FolderBrowserDialog**.
4. Creați o aplicație care permite editarea etichetelor itemilor unui **ListView**.

Controlul *TreeView*

Controlul afișează o colecție ierarhică de elemente care se numesc noduri. Fiecare nod este o instanță a clasei **TreeNode**. Proprietatea **Nodes** a controlului memorează această colecție de noduri.

Nodurile se creează programatic relativ simplu. Creați un proiect de tip *Windows Forms* și tratați evenimentul **Load** generat de formă (dublu click pe suprafața ei), apoi scrieți codul:

```
private void Form1_Load(object sender, EventArgs e)  
{  
    // Instanțiem un TreeView  
    TreeView tw = new TreeView();  
  
    // Creăm un prim nod cu eticheta "Baieti"  
    TreeNode tn1 = new TreeNode();  
    tn1.Text = "Baieti";  
  
    // Adăugăm nodul în control  
    tw.Nodes.Add(tn1);  
  
    // Creăm al doilea nod cu eticheta "Fete"  
    TreeNode tn2 = new TreeNode();  
    tn2.Text = "Fete";  
  
    // Adăugăm nodul în control  
    tw.Nodes.Add(tn2);  
}
```

```
// Adăugăm controlul pe formă  
this.Controls.Add(tw);  
}
```

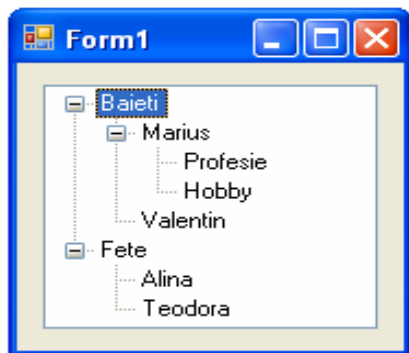


Crearea ierarhiilor

Fiecare nod într-un **TreeView** poate fi *părinte* pentru alte noduri. Se pot crea ierarhii arborescente cu oricâte niveluri. Modificăm exemplul anterior, prin adăugarea de noduri *child* nodurilor “*Baieti*” și “*Fete*”:

```
// Instanțiem un TreeView  
TreeView tw = new TreeView();  
// Stabilim poziția pe formă și dimensiunea controlului:  
tw.Location = new System.Drawing.Point(12, 12);  
tw.Size = new System.Drawing.Size(153, 133);  
  
// Creăm un prim nod cu eticheta Baieti  
TreeNode tn1 = new TreeNode();  
tn1.Text = "Baieti";  
  
// Creăm un nod child pentru Baieti:  
TreeNode tn11;  
// Referința tnn1 indică nodul nou creat (cu eticheta Marius)  
tn11 = tn1.Nodes.Add("Marius");  
  
// Nodul Marius va fi părinte pentru alte două noduri:  
tn11.Nodes.Add("Profesie");  
tn11.Nodes.Add("Hobby");  
  
// Al doilea nod child pentru Baieti:  
tn1.Nodes.Add("Valentin");  
  
// Adăugăm nodul Baieti în control:  
tw.Nodes.Add(tn1);
```

```
// Creăm al doilea nod rădăcină cu eticheta Fete:  
TreeNode tn2 = new TreeNode();  
tn2.Text = "Fete";  
tn2.Nodes.Add("Alina");  
tn2.Nodes.Add("Teodora");  
  
// Adăugăm nodul Fete în control  
tw.Nodes.Add(tn2);  
// Adăugăm controlul pe formă  
this.Controls.Add(tw);
```



Imaginea anterioară reprezintă ceea ce se obține la rulare.

Important:

Metoda `Add()` returnează întotdeauna o referință la nodul nou creat. Aceasta înlesnește adăugarea de noduri child.

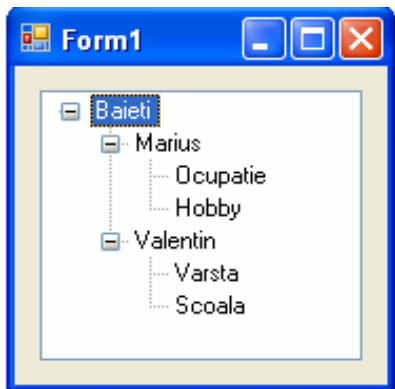
Accesarea indexată a nodurilor

Subnodurile care fac parte din colecția **Nodes** a nodului curent se pot accesa cu ajutorul operatorului de indexare. Reluăm exemplul anterior. Prezentăm o alternativă de construire programatică a controlui, bazată pe accesarea indexată:

```
// Instanțiem un TreeView  
TreeView tw = new TreeView();  
  
// Stabilim poziția pe formă și dimensiunea controlului:  
tw.Location = new System.Drawing.Point(12, 12);  
tw.Size = new System.Drawing.Size(153, 133);  
  
// Adăugăm în control un prim nod:  
tw.Nodes.Add("Baieti");  
  
// Adăugăm subnodurile Marius și Valentin pentru Baieti  
tw.Nodes[0].Nodes.Add("Marius");  
tw.Nodes[0].Nodes.Add("Valentin");  
  
// Adăugăm subnodurile Ocupatie și Hobby pentru Marius:  
tw.Nodes[0].Nodes[0].Nodes.Add("Ocupatie");  
tw.Nodes[0].Nodes[0].Nodes.Add("Hobby");
```

```
// Adăugăm subnodurile Varsta și Scoala pentru Valentin:  
tw.Nodes[0].Nodes[1].Nodes.Add("Varsta");  
tw.Nodes[0].Nodes[1].Nodes.Add("Scoala");  
  
// Adăugăm controlul pe formă:  
this.Controls.Add(tw);
```

La execuție se obține :



Principalii membri ai clasei TreeView

Controlul **TreeView** are multiple capabilități. Descriem câteva dintre acestea:

- Poate afișa imagini asociate fiecărui nod.
- Poate afișa opțional *checkbox*-uri asociate nodurilor.
- Își poate schimba aparența setând corespunzător proprietăți de stil.
- Poate răspunde la diverse evenimente, cum ar fi click sau dublu click pe eticheta unui nod.
- Se poate folosi pentru afișarea informațiilor dintr-o bază de date, a informațiilor din fișiere, de pe disc, sau a informațiilor din aplicații.

Aceste calități se datorează multimei de metode, proprietăți și evenimente ale clasei. Dintre acestea menționăm:

Proprietăți :

Nodes	- Depozitează colecția de obiecte de tip TreeNode a controlului. Fiecare nod are la rândului lui proprietatea Nodes , care găzduiește propriile noduri child .
LabelEdit	- Stabilește dacă eticheta text a unui nod poate fi editată
CheckBoxes	- Stabilește dacă se afișează un checkbox lângă etichetele nodurilor
SelectedNode	- Returnează sau modifică nodul selectat.
PathSeparator	- Returnează sau setează stringul separator folosit în calea spre noduri. Calea spre un nod este un set de etichete de noduri separate prin delimitatorul PathSeparator .

Metode:

CollapseAll()	- Colapsează ramurile arborelui (Inversul expandării)
BeginUpdate() EndUpdate()	- Suspendă redesenarea controlului TreeView la fiecare adăugare a unui item în situația în care se adaugă un mare număr de itemi. - Se apelează a doua metodă după terminarea operației de update, pentru a permite controlului să se redeseneze.
GetNodeAt()	- Returnează nodul care se găsește la locația specificată

Evenimente:

BeforeCollapse AfterCollapse	- Se declanșează înainte, respectiv după colapsul unui nod
BeforeExpand AfterExpand	- Se declanșează înainte, respectiv după expandarea unui nod

Parcurgerea și prelucrarea nodurilor

Uneori trebuie să vizitați toate nodurile unui **TreeView** pentru o prelucrare oarecare. Amintiți-vă că un **TreeView** este o structură de date arborescentă. Veți înțelege din ce cauză parcurgerea unei asemenea structuri se face cu o metodă algoritmică. Propunem o parcurgere în adâncime:

```
private void ParcurgTreeView(TreeNodeCollection noduri)
{
    foreach (TreeNode nod in noduri)
    {
        PrelucrezNod(nod);
        ParcurgTreeView(nod.Nodes);
    }
}

private void PrelucrezNod(TreeNode nod)
{
    // Prelucrarea dorită pentru nod
    // Aici, pentru verificare afișăm doar eticheta
    MessageBox.Show(nod.Text + "\n");
}

private void button1_Click(object sender, EventArgs e)
{
    //
    ParcurgTreeView(treeView1.Nodes);
}
```

De reținut:

- `treeview1.Nodes` reprezintă colecția de noduri *rădăcină* în arbore. Fiecare nod din această colecție are la rândul lui proprietatea `Nodes`, care referă nodurile de pe al doilea nivel, etc.
- **TreeNodeCollection** este un tip de date care reprezintă o colecție de obiecte **TreeNode**.

Aplicația *TreeViewExample*

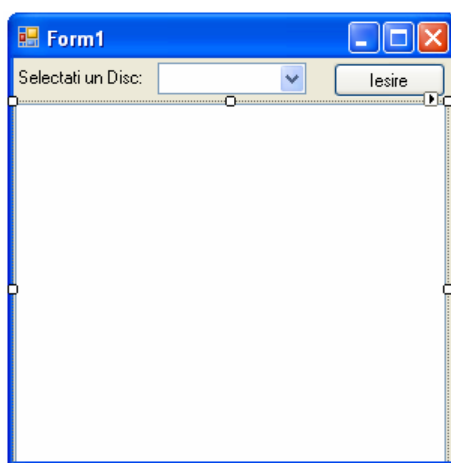
Proiectul pe care îl propunem utilizează un *combobox* pentru selectarea *drive*-urilor sistemului. În funcție de discul selectat, într-un control **TreeView** se afișează structura arborescentă de directoare discului.

Elemente de noutate:

- Obținerea informațiilor despre discuri cu metoda:
`DriveInfo.GetDrives()`
- Obținerea numelor subdirectorilor unui director specificat, cu metoda:
`Directory.GetDirectories()`
- Parcurgerea recursivă a structurii de directoare a unui disc, în scopul populării controlului **TreeView**.

Urmați pașii:

1. Creați un nou proiect de tip **Windows Forms Application**, cu numele *TreeViewExample*.
2. Aduceți din **Toolbox** pe suprafața formei controalele: un **Label**, un **CheckBox**, un **Button** și un **ListView** și aranjați-le astfel:



3. Modificați culorile controlului **TreeView** după preferință: culoarea fundalului (proprietatea `BackColor`), culoarea etichetelor (`ForeColor`) și culoarea liniilor punctate (`LineColor`).

4. Adăugați directiva `using System.IO`;
5. În fișierul *Form1.cs*, în corpul clasei **Form1**, scrieți secvențele de cod următoare:

```
public Form1() // Constructoul clasei
{
    InitializeComponent();
    // Populează comboBox1 cu toate drive-urile
    DriveInfo[] drives = DriveInfo.GetDrives();
    comboBox1.Items.AddRange(drives);
}

// Câmpuri private. Reprezintă caracterul de separare
// a căii de director, respectiv litera discului
private char[] sep = new char[] { '\\' };
private string drvLetter = null;
// Deep First în structura de directoare
private void PopulateTreeView(string path,
                             TreeNode node)
{
    string[] dirs = Directory.GetDirectories(path);
    string[] auxs;
    foreach (string dir in dirs)
    {
        auxs = dir.Split(sep);
        TreeNode tn =
            new TreeNode(auxs[auxs.Length - 1]);
        node.Nodes.Add(tn); // Adaug subdirectorul
        PopulateTreeView(dir, tn);
    }
}
```

6. Aplicația trebuie să reacționeze la alegerea unui item în *combobox*. Tratăm evenimentul **SelectedIndexChanged** pentru **comboBox1**. Pentru aceasta, acționați dublu click pe acest control. În corpul *handler*-ului de eveniment, scrieți codul evidențiat în bold:

```
private void comboBox1_SelectedIndexChanged(
    object sender, EventArgs e)
{
    // Obținem textul itemului selectat în combobox
    drvLetter =
    comboBox1.Items[comboBox1.SelectedIndex].ToString();

    // Suspendăm redesenarea controlului
    treeView1.BeginUpdate();

    // Ștergem toată colecția de noduri
    treeView1.Nodes.Clear();
}
```

```

// Adăugăm eticheta drive-ului ca nod radacină
treeView1.Nodes.Add(drvLetter);
drvLetter += "\\";    // Concatenez caracterul '\\'

// Explorarea începe de la rădăcina drive-ului
PopulateTreeView(drvLetter, treeView1.Nodes[0]);
treeView1.EndUpdate();
}

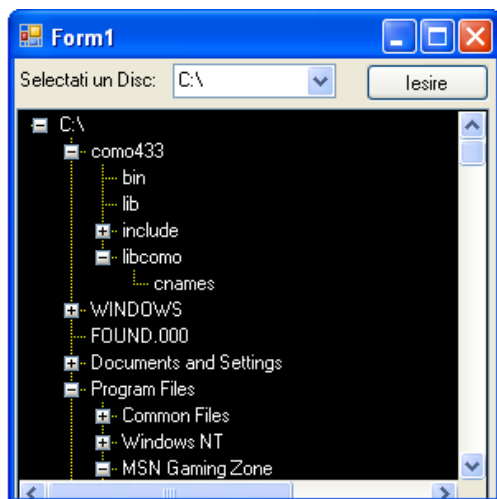
```

7. Acționați dublu click pe butonul *Iesire*. În *handler*-ul evenimentului **Click**, scrieți:

```
Application.Exit();
```

8. Compilați și rulați cu **F5**.

La rulare obțineți:



Observație:

Cu metoda recursivă se parcurge întreaga structură de director a *drive*-ului curent. Din această cauză, actualizarea controlului **TreeView** este lentă pentru un disc încărcat.

De reținut:

- Clasa **Directory** din spațiul de nume **System.IO** furnizează metode statice pentru creare, copiere, mutarea, redenumire de directori.
- Clasa **DriveInfo** din spațiul de nume **System.IO** conține metode și proprietăți pentru determinarea informațiilor referitoare la *drive*-uri.
- Metoda `treeView1.Nodes.Clear();` șterge toate nodurile din colecție.

Probleme propuse

1. Creați o aplicație de tip *WindowsForms*. Forma conține un control **TreeView**. Populați *design time* controlul cu noduri.
Indicație: În fereastra **Properties**, acționați butonul cu eticheta (...) aflat în dreapta proprietății **Nodes**. În dialogul *TreeNode Editor*, adăugați noduri apăsând butoanele *Add Root* și *Add Child*.
2. Implementați aplicației *TreeViewExample* următoarea facilitare: fiecare nod are lângă etichetă un *checkbox*.
3. * Modificați aplicația *TreeViewExample* de așa manieră încât nici un nod să nu se adauge în *TreeView* decât în momentul când se expandează părintele său în control. Un nod se expandează prin click pe cruciulița din dreptul său. În felul acesta aplicația va lucra mai rapid.
4. * Implementați un *file browser* cu funcționalitate asemănătoare *Windows Explorer*. Aplicația trebuie să afișeze în panoul din dreapta fișierele aflate în directorul care este selectat în panoul din stânga. De asemenea, la dublu click pe un fișier executabil în panoul din dreapta, acesta să se lanseze în execuție.
Indicație: Utilizați un **SplitContainer**. În panoul din stânga aduceți un control **TreeView**, iar în cel din dreapta un **ListView**. Pentru obținerea fișierelor dintr-un director oarecare, apelați metoda **Directory.GetFiles()**. Pentru lansare în execuție, utilizați metoda **System.Diagnostics.Process.Start()**.

Controalele Web Browser și StatusStrip

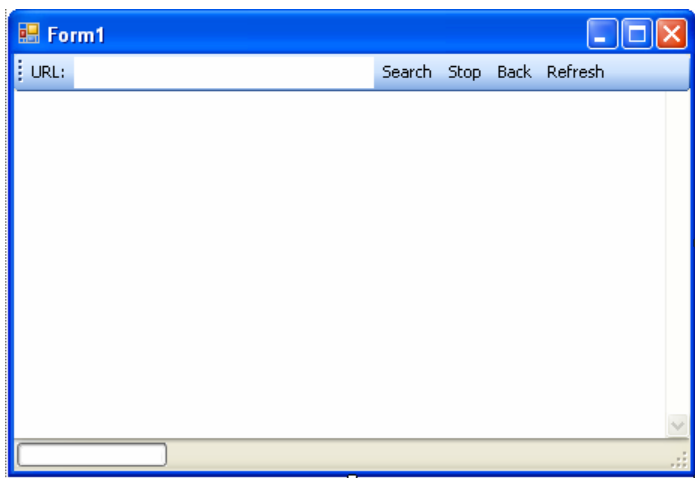
Dacă v-ați gândit să integrați un *browser Web* în aplicația dumneavoastră, atunci cu *Visual C# Express Edition 2008* aceasta este o sarcină simplă, deoarece mediul include un control de tip **WebBrowser**.

Aplicația WebBrowserExample

În acest proiect utilizăm un control de tip **WebBrowser**, un control **StatusStrip** pentru vizualizarea încărcării paginii curente și un **ToolStrip** pentru butoane și bara de navigare. Urmăți pașii:

1. Creați un nou proiect de tip *Windows Forms* cu numele *WebBrowserExample*.
2. Pe forma aplicației aduceți din **Toolbox** un control de tip **ToolStrip**, un control de tip **StatusStrip** și un control de tip **WebBrowser**.

3. Selectați controlul **ToolStrip**. Adăugați pe el un **Label**, un **TextBox** și patru butoane. Pentru fiecare buton, acționați click drept și alegeți *DisplayStyle*, apoi *Text*, pentru a afișa text în loc de imagine.
4. Selectați controlul **StatusStrip**. Acționați click drept și alegeți un **ProgressBar**.
5. Selectați controlul **WebBrowser**. În fereastra **Properties** atribuiți proprietății **Dock** valoarea *Fill* și proprietății **URL** adresa: <http://www.google.com>. Setăți textele pentru butoane și aranjați totul astfel:



6. Tratăm evenimentul **Click** pentru butonul *Search*. Acționați dublu click pe buton. În corpul metodei *handler*, scrieți:

```
private void toolStripButton1_Click(object sender,
                                   EventArgs e)
{
    // Încarcă pagina de la adresa specificată
    webBrowser1.Navigate(toolStripTextBox1.Text);
}
```

7. Tratăm evenimentul **Click** pentru butonul *Stop*. Acționați dublu click pe buton. În corpul metodei *handler*, scrieți:

```
private void toolStripButton2_Click(object sender,
                                   EventArgs e)
{
    // Oprește încărcarea paginii
    webBrowser1.Stop();
}
```

8. Tratăm evenimentul **Click** pentru butonul *Back*. Dublu click pe buton. În corpul metodei de tratare, scrieți:

```
private void toolStripButton3_Click(object sender,
                                   EventArgs e)
{
    // Revenire la pagin aanterioară
    webBrowser1.GoBack() ;
}
```

9. Tratăm evenimentul **Click** pentru butonul *Refresh*. Dublu click pe buton. În corpul metodei de tratare, scrieți:

```
private void toolStripButton4_Click(object sender,
                                   EventArgs e)
{
    // Reîncărcarea paginii de la URL-ul curent
    webBrowser1.Refresh() ;
}
```

10. În *progress bar*-ul integrat în **StatusStrip** dorim să vizualizăm progresul încărcării paginii. În acest scop, tratăm evenimentul **ProgressChanged** pentru controlul **WebBrowser**. Selectați controlul și acționați dublu click pe acest eveniment în fereastra **Properties**. Introduceți codul:

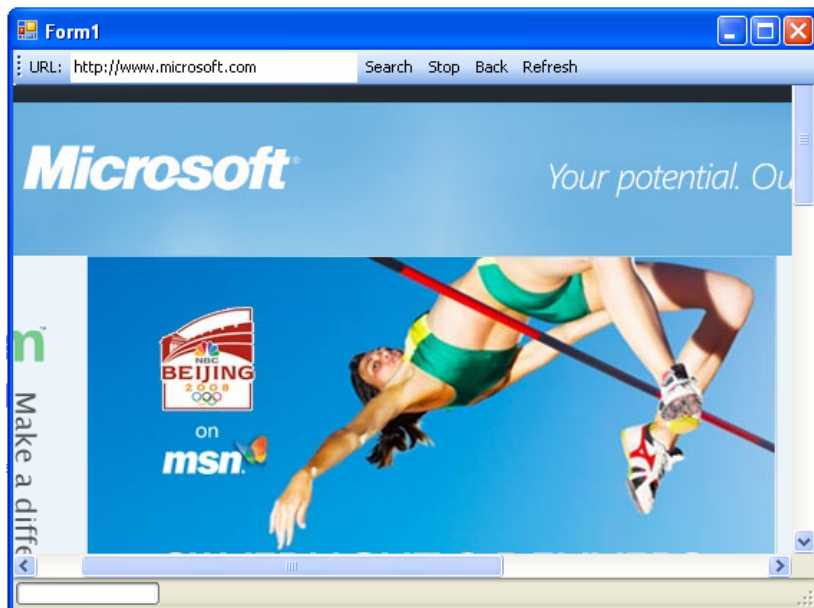
```
private void webBrowser1_ProgressChanged(object sender,
                                         WebBrowserProgressChangedEventArgs e)
{
    // Setează Maximum la numărul total de bytes pe
    // care îl ocupă documentul care se descarcă
    toolStripProgressBar1.Maximum =
                                   (int)e.MaximumProgress;
    // Setează Value la numărul de bytes care au fost
    // downloadați până în prezent
    toolStripProgressBar1.Value =
                                   (int)e.CurrentProgress;
}
```

11. În momentul în care documentul este descărcat, dorim să ne asigurăm că *progress bar*-ul este la valoarea maximă. Tratăm evenimentul **DocumentCompleted** declanșat de *WebBrowser* când documentul este complet descărcat. Dublu click pe acest eveniment în **Properties**. În metoda de tratare, introduceți codul:

```
private void webBrowser1_DocumentCompleted(
    object sender, WebBrowserDocumentCompletedEventArgs e)
{
    toolStripProgressBar1.Value =
                                   toolStripProgressBar1.Maximum;
}
```

12. Compilați și rulați cu **F5**.

La rulare, aplicația încarcă în mod automat pagina stabilită *design time* prin proprietatea **URL**. În fereastra de navigare puteți introduce oricarea altă adresă:



Observații:

- Dacă lucrați în **Visual C# Express Edition 2005**, atunci controlul **WebBrowser** nu se găsește în mod implicit în **Toolbar**. Aceasta nu este o problemă, pentru că el există ca și componentă **.NET** și îl puteți aduce în **Toolbar** cu un click dreapta pe suprafața liberă de jos, apoi selectați *Chose Items...* iar în tab-ul *.NET Framework Components*, alegeți **Web Browser**.
- Clasa **WebBrowser** are metode prin care implementați acțiunile specifice ale unui browser: navigare, reîncărcare, întoarcere la *URL*-ul anterior și așa mai departe.

Integrarea WindowsMediaPlayer în aplicații

Suntem siguri că v-ați întrebat dacă este greu să implementați *player*-e video sau audio. Nu este ușor. Însă există o cale mai simplă de a avea toate acestea, prin integrarea componentei COM, **WindowsMediaPlayer** în aplicația dumneavoastră.

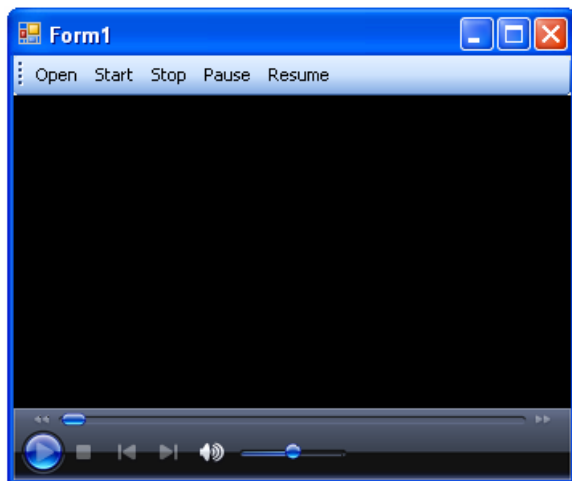
Ca să vedeți cum se procedează, vom face următorul proiect:

Aplicația WindowsMediaPlayerExample

Aplicația utilizează componenta **Windows Media Player** și un control de tip **ToolStrip** cu 5 butoane. Rolul butoanelor este acela de a efectua aceleași operații ca și butoanele implicite ale controlului. Ne vom servi și de dialogul predefinit **OpenFileDialog** pentru alegerea unui fișier audio sau video de pe disc.

Urmați pașii:

1. Creați un nou proiect de tip *Windows Forms*.
2. Pe suprafața designerului plasați un control de tip **ToolStrip**, pe care adăugați apoi cinci butoane care afișează text. Etichetați cele cinci butoane astfel: *Open*, *Start*, *Stop*, *Pause*, *Resume*.
3. Plasați pe suprafața formei un control de tip **OpenFileDialog**.
4. **Windows Media Player** nu este disponibil în **Toolbox** în mod implicit, dar poate fi adus acolo manual. Îl aduceți cu un click dreapta pe suprafața liberă de jos, apoi selectați *Chose Items...* iar în *tab*-ul *COM Components*, alegeți **Windows Media Player**. În continuare, plasați componenta pe suprafața *designer*-ului și setați-i proprietatea *Dock* la valoarea *Fill* și proprietatea *Name* la valoarea *wmp*.



5. Tratăm evenimentul **Click** pentru butonul *Open*. Acționați dublu click pe buton și scrieți în metoda *handler*:

```
private void toolStripButton1_Click(object sender,
                                   EventArgs e)
{
    // Dacă s-a ales un fișier media și s-a apăsât OK
    if (openFileDialog1.ShowDialog() ==
        DialogResult.OK)
    {
        // Playerul nu va executa play în mod automat
        // la încărcarea fișierului
        wmp.settings.autoStart = false;

        // Se încarcă fișierul ales de utilizator
        wmp.URL = openFileDialog1.FileName;
    }
}
```

6. Tratăm evenimentul **Click** pentru butonul *Start*. Acționați dublu click pe buton și introduceți codul:

```
private void toolStripButton2_Click(object sender,
                                   EventArgs e)
{
    // Player-ul rulează fișierul încărcat
    wmp.Ctlcontrols.play();
}
```

7. Tratăm evenimentul **Click** pentru butonul *Stop*. Dublu click pe buton și introduceți codul:

```
private void toolStripButton3_Click(object sender,
                                   EventArgs e)
{
    // Oprește rularea
    wmp.Ctlcontrols.stop();
}
```

8. Tratăm evenimentul **Click** pentru butonul *Pause*. Dublu click pe buton. În metoda de tratare introduceți codul:

```
private void toolStripButton4_Click(object sender,
                                   EventArgs e)
{
    // Pauză. La un nou apel play(), redarea continuă
    // din această poziție în fișier
    wmp.Ctlcontrols.pause();
}
```

9. Tratăm evenimentul **Click** pentru butonul *Resume*. Dublu click pe buton. În metoda de tratare introduceți codul:

```
private void toolStripButton5_Click(object sender,
                                     EventArgs e)
{
    // Redarea se reia din poziția în care s-a ajuns
    // la ultimul apel pause()
    wmp.Ctlcontrols.play() ;
}
```

10. Compilați și rulați cu **F5**.

Puteți rula atât fișiere audio cât și video.



Observație:

Controalele adăugate manual au aceleași funcții cu cele ale butoanelor implicite. A fost un exercițiu pentru cazul în care vreți să mascați bara standard și să utilizați propriile controale pentru manevrarea player-ului.

Capitolul 8

Desenare în .NET cu Visual C#

GDI+ (*Graphics Device Interface*) este o interfață de programare a aplicațiilor integrată sistemelor de operare *Microsoft Windows XP* și *Windows Server 2003*.

Aplicațiile de tip Windows desenează elemente grafice cu ajutorul bibliotecii de clase **GDI+**. O interfață pentru dispozitive grafice, așa cum este **GDI+**, permite programatorilor să deseneze pe dispozitive grafice: ecran, scanner, imprimantă, cu ajutorul acelorași funcții, fără să trebuiască să țină seama de detaliile unui dispozitiv de afișare particular.

Se pot desena linii, curbe, figuri geometrice, text, imagini. Clasele **GDI+** se găsesc în marea lor majoritate în spațiul de nume **System.Drawing**. Dintre acestea, clasa **System.Drawing.Graphics** este cea mai utilizată.

Clasa Graphics

Clasa definește o suprafață de desenare **GDI+**. Metodele sale desenează pe această suprafață, iar biblioteca **GDI+** știe să trimită imaginea pe dispozitivul grafic (monitor, imprimantă, sau altceva).

Principalele metode ale clasei Graphics

Clear ()	- Curăță suprafața de desenare și o umple cu o culoare de fundal specificată
DrawArc ()	- Desenează un arc de elipsă
DrawImage ()	- Desenează o imagine la o locație dată
DrawLine ()	- Desenează un segment specificat prin capetele sale
DrawEllipse ()	- Desenează o elipsă specificată printr-un dreptunghi care o mărginește
DrawRectangle ()	- Desenează un dreptunghi
DrawString ()	- Desenează un text la o locație specificată
FillEllipse ()	- Umple interiorul unei elipse cu ajutorul unei pensule (obiect de tip Brush)
FillRectangle ()	- Umple interiorul unui dreptunghi cu ajutorul unei pensule

Clasa nu răspunde la evenimente fiindcă nu are membri de tip **event**. Dar folosește evenimentul **Paint** al controalelor pentru a desena pe suprafața lor.

Penițe pentru desenarea formelor

Desenăm cu ajutorul unui obiect de tip **Graphics**. Obiectul se obține în două moduri:

1. Prin intermediul parametrului de tip **PaintEventArgs** al *handler*-ului evenimentului **Paint**.
2. Cu ajutorul metodei **CreateGraphics()**, moștenită de către toate controalele de la clasa de bază **Control**.

Detaliem cele două cazuri:

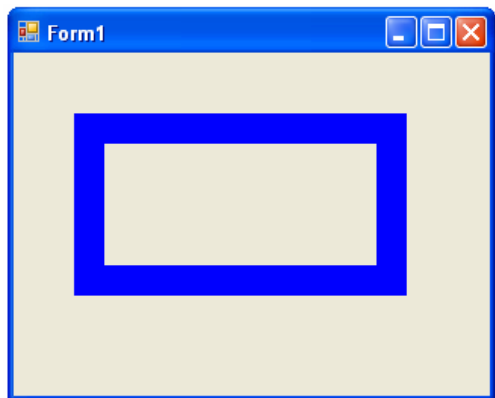
Cazul 1.

Creați un proiect nou de tip *Windows Forms*. Tratăm evenimentul **Paint** al formei. Selectați forma și în fereastra **Properties** acționați dublu click pe evenimentul **Paint**. Scrieți codul:

```
private void Form1_Paint(object sender, PaintEventArgs e)
{
    // Peniță de culoare roșie și grosime 20
    Pen p = new Pen(Color.Blue, 20);

    Rectangle r = new Rectangle(new Point(50, 50),
                                new Size(200, 100));
    e.Graphics.DrawRectangle(p, r);
    p.Dispose();
}
```

Compilați și rulați cu **F5**. Pe formă s-a desenat un dreptunghi.



Handler-ului evenimentului **Paint** are un parametru **e** de tip **PaintEventArgs**. Acesta returnează un obiect de tip **Graphics** prin proprietatea **Graphics**: **e.Graphics**.

IMPORTANT !

Pentru desenarea liniilor și a curbelor, aveți nevoie de un obiect de tip **Graphics** și un obiect de tip **Pen** (peniță). Obiectul **Graphics** furnizează metodele care desenează, iar obiectele de tip **Pen** depozitează atribute ale liniei, cum ar fi culoarea, grosimea și stilul.

Rețineți că este o practică bună aceea de a elibera resursele deținute de obiectul **Pen**, cu metoda **Dispose()**, în momentul în care nu mai aveți nevoie de el.

Cazul 2.

Metoda **CreateGraphics()**, se regăsește în toate clasele derivate din **Control**. Returnează un obiect de tip **Graphics**. Este nevoie de această metodă când nu doriți să folosiți un eveniment **Paint** pentru a desena.

Realizați un nou proiect de tip *Windows Forms*. Pe formă plasați un buton. Acționați dublu click pe buton pentru tratarea evenimentului **Click**. În *handler*-ul evenimentului introduceți secvența de cod evidențiată cu **Bold**:

```
private void button1_Click(object sender, EventArgs e)
{
    // Obținem un obiect Graphics
    Graphics g = this.CreateGraphics();

    // Fabricăm o peniță
    Pen p = new Pen(Color.Red, 3);

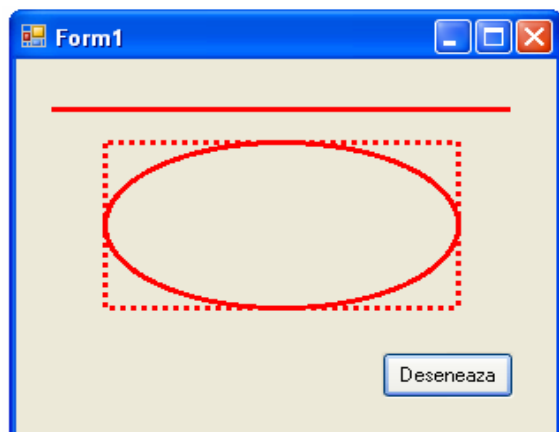
    // Desenăm o linie
    g.DrawLine(p, new Point(20, 30), new Point(280, 30));

    // Desenăm un arc definit de dreptunghiul (50, 50, 200,
    // 100). Unghiul de început e 0, iar cel de sfârșit 360.
    // Aceasta înseamnă elipsă
    g.DrawArc(p, 50, 50, 200, 100, 0, 360);

    // Schimbăm stilul peniței (linie punctată) și desenăm
    // un dreptunghi
    p.DashStyle = System.Drawing.Drawing2D.DashStyle.Dot;
    Rectangle r = new Rectangle(new Point(50, 50),
                                new Size(200, 100));
    g.DrawRectangle(p, r);

    p.Dispose();
}
```

La click pe butonul *Desenează*, pe suprafața formei se desenează figurile:



Observații

- Un arc este o porțiune a unei elipse. Ca să desenați o elipsă, apăsați metoda `DrawEllipse()`. Parametrii metodei `DrawArc()` sunt aceiași cu cei ai metodei `DrawEllipse()`, cu diferența că `DrawArc()` necesită un unghi de start și unul de baleiere. Iată cum desenați o elipsă:

```
Graphics g = this.CreateGraphics();
Pen p = new Pen(Color.Red, 3);
Rectangle r = new Rectangle(new Point(50, 50),
                             new Size(200, 100));

g.DrawEllipse(p, r);
p.Dispose();
```

- Metodele de desenare ale arcelor și elipselor primesc ca parametri coordonatele dreptunghiului care le încadrează. Metodele sunt supraîncărcate. De exemplu, puteți înlocui apelul:
`g.DrawArc(p, 50, 50, 200, 100, 0, 360);` cu apelul
`g.DrawArc(p, r, 0, 360);` unde `r` este dreptunghiul definit anterior.

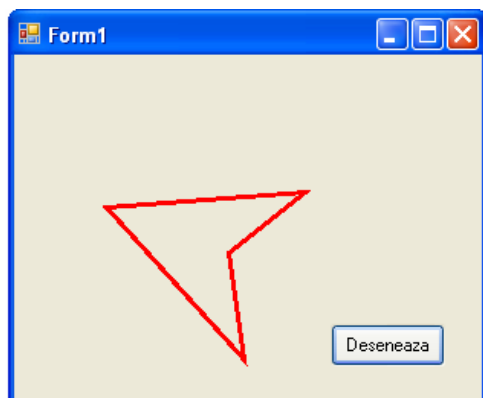
Poligoane

Desenarea unui poligon se face foarte simplu, prin apelul metodei `DrawPolygon()`. Metoda are ca parametri un obiect de tip `Pen` și un tablou de puncte:

```
Graphics g = this.CreateGraphics();
Pen p = new Pen(Color.Red, 3);
Point[] pt = { new Point(190, 90), new Point(140, 130),
               new Point(150, 200), new Point(60, 100) };

// Desenează un poligon cu patru vârfuri
g.DrawPolygon(p, pt);
p.Dispose();
```

Dacă ne folosim de evenimentul Click al butonului în aplicația anterioară, atunci la click pe *Desenează*, avem:



Pensule pentru umplerea formelor

Figurile închise se desenează cu penițe. Interiorul lor se umple cu pensule. Pensulele sunt obiecte de tip **Brush**.

Biblioteca **GDI+** oferă câteva clase care definesc pensule: **SolidBrush**, **HatchBrush**, **TextureBrush**, **LinearGradientBrush**, și **PathGradientBrush**.

Clasa **SolidBrush** este definită în spațiul de nume **System.Drawing**. Celelalte pensule sunt în spațiul de nume **System.Drawing.Drawing2D**.

Cel mai simplu mod de a învăța, este de a lucra cu aceste pensule. Vom realiza o aplicație ca model.

Aplicația BrushesExample

Aplicația umple trei elipse cu pensule diferite: o pensulă solidă, una de hașurare și una gradient liniar. În dreapta formei se va umple un dreptunghi cu o pensulă de tip **TextureBrush**, care utilizează ca element de umplere o imagine.

Urmați pașii:

1. Creați un nou proiect de tip **Windows Forms Application**, cu numele *BrushesExample*.
2. În fișierul *Form1.cs*, în clasa **Form1**, introduceți la secțiunea directivelor **using**, secvența:

```
using System.Drawing.Drawing2D;
```


3. Tratăm evenimentul **Paint** pentru formă. Selectați forma. În fereastra **Properties**, acționați dublu click pe evenimentul **Paint**. În corpul *handler*-ului introduceți codul:

```
private void Form1_Paint(object sender,
                                PaintEventArgs e)
{
    Pen p = new Pen(Color.Red, 3);

    // Creăm un dreptunghi pentru elipse
    int x = 10, y = 10, width = 200, height = 100;
    Rectangle r = new Rectangle(x, y, width, height);

    // Creăm o pensulă solidă
    SolidBrush s = new SolidBrush(Color.Aquamarine);

    // Umplem elipsa cu pensula solidă
    e.Graphics.FillEllipse(s, r);

    y = 120;
    r = new Rectangle(x, y, width, height);
    // Creăm o pensulă de hașurare
    HatchBrush h =
        new HatchBrush(HatchStyle.DiagonalCross,
                       Color.Azure, Color.Black);

    // Umplem elipsa cu pensula de hașurare
    e.Graphics.FillEllipse(h, r);

    y = 230;
    r = new Rectangle(x, y, width, height);

    // Creăm o pensulă gradient liniar
    LinearGradientBrush lg = new LinearGradientBrush(r,
                                                       Color.Aqua, Color.BlueViolet,
                                                       LinearGradientMode.Horizontal);

    // Umplem elipsa cu pensula gradient liniar
    e.Graphics.FillEllipse(lg, r);

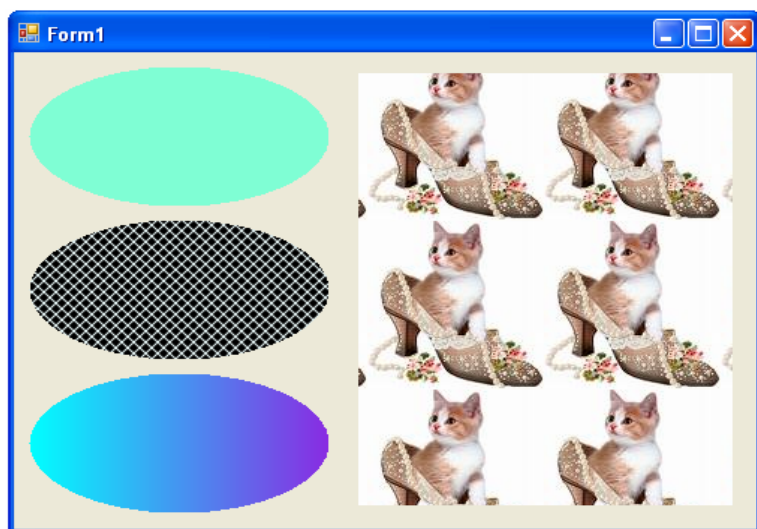
    x = 230; y = 15; width = 250; height = 310;
    r = new Rectangle(x, y, width, height);

    // Creăm o pensulă de textură
    TextureBrush t =
        new TextureBrush(Image.FromFile("pisica.jpg"));

    // Umplem dreptunghiul cu pensula de textură
    e.Graphics.FillRectangle(t, r);
    p.Dispose();
}
```

4. Compilați, rulați cu **F5**.

La rulare, avem:



Observații:

- Constructorii claselor de tip *brush* primesc ca argumente culori, stiluri, etc, prin intermediul membrilor anumitor enumerări. Ați remarcat enumerările **Color**, **LinearGradientMode**, **HatchStyle**.
- Constructorul `TextureBrush((Image.FromFile("pisica.jpg")))`; încarcă o imagine. Dacă se specifică doar numele fișierului imagine, atunci acesta este căutat în folderul în care se găsește fișierul *assembly*, adică `/bin/Debug` sau `/bin/Release`.

Desenarea textului

GDI+ furnizează câteva clase care se ocupă cu desenarea textului. Clasa **Graphics** are în acest scop mai multe metode `DrawString()`. Vă veți acomoda cu utilizarea acestora, urmărind exemplul de mai jos:

Aplicația *DrawStringExample*

Ca să desenați cu metodele `DrawString()` trebuie să pregătiți un font și o pensulă. Aplicația testează două dintre metodele `DrawString()`. Prima scrie un text la o locație specificată de un punct, iar a doua scrie un text încadrat într-un dreptunghi.

Urmați pașii:

1. Creați un nou proiect de tip **Windows Forms Application**, cu numele *DrawStringExample*.
2. Tratăm evenimentul **Paint** pentru formă. Selectați forma. În fereastra **Properties**, acționați dublu click pe evenimentul **Paint**. În corpul *handler*-ului introduceți codul:

```
private void Form1_Paint(object sender,
                                PaintEventArgs e)
{
    String s = "Dau un regat pentru un cal";

    // Creăm un font precizând familia, dimensiunea
    // și stilul, apoi o pensulă solidă
    Font f = new Font("Arial", 22, FontStyle.Bold);
    SolidBrush b = new SolidBrush(Color.Red);

    // Punctul colțului stânga sus al stringului
    PointF pt = new PointF(20.0F, 30.0F);

    // Desenează stringul
    e.Graphics.DrawString(s, f, b, pt);

    float x = 100.0F, y = 100.0F, width = 200.0F,
           height = 80.0F;
    RectangleF r = new RectangleF(x, y, width, height);

    // Desenează dreptunghiul pe ecran
    Pen p = new Pen(Color.Black);
    e.Graphics.DrawRectangle(p, x, y, width, height);

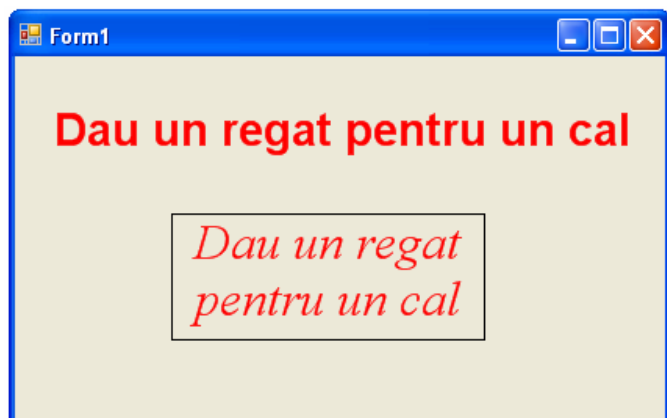
    // Formatăm stringul. Aliniere centru
    // (în dreptunghi)
    StringFormat stF = new StringFormat();
    stF.Alignment = StringAlignment.Center;
    f = new Font("Times New roman", 24,
                FontStyle.Italic);

    // Desenez textul în dreptunghiul r
    e.Graphics.DrawString(s, f, b, r, stF);

    p.Dispose();
}
```

3. Compilați și rulați cu **F5**.

La rulare, obțineți:



Observații:

- Clasa **StringFormat** are proprietatea **Alignment**, care permite să setați alinierea textului în dreptunghiul specificat.
- **PointF** și **RectangleF** sunt structuri similare **Point** și **Rectangle**, cu diferența că admit coordonate reale.
- A doua metodă **DrawString()** constrânge stringul să se încadreze în dreptunghiul specificat și îl trunchiază în așa fel încât cuvintele individuale nu se frâng.

De reținut:

- ✓ Cu ajutorul bibliotecii **GDI+** se scriu aplicații care desenează pe dispozitive grafice (ecran, imprimante).
- ✓ Clasa **System.Drawing.Graphics** este nucleul metodelor de desenare.
- ✓ Pentru a desena o curbă, aveți nevoie de o peniță (obiect de tip **Pen**), iar pentru a umple sau hașura o curbă închisă, aveți nevoie de o pensulă (obiect de tip **Brush**).
- ✓ Obiectele de tip **Graphics**, cu care apelați metodele **Draw**, se obțin în două moduri:
 1. Din parametrul de tip **PaintEventArgs** al metodei de tratare a evenimentului **Paint**.
 2. Prin apelul metodei **CreateGraphics()**.
- ✓ Textul se desenează cu metodele **DrawString()** ale clasei **Graphics**.

Probleme propuse

1. Să presupunem că în *handler*-ul evenimentului **Paint** desenați pe formă un text. Stringul care trebuie desenat este itemul pe care s-a făcut click într-un **Listbox**. Cum forțați redesenarea formei (declașarea evenimentului **Paint**) la click pe acel item ? Realizați o aplicație care implementează această cerință.
Indicație: În *handler*-ul evenimentului **SelectedIndexChanged** al controlului **ListBox**, preluați textul necesar și forțați evenimentul **Paint** cu metoda **this.Invalidate()**. Aceasta invalidează toată suprafața formei și obligă redesenarea.
2. Există patru versiuni ale metodei **DrawEllipse()**. Testați-le pe toate într-un mic proiect.
3. Realizați un proiect în care veți desena icon-uri cu ajutorul celor două metode **DrawIcon()** și veți afișa imagini, utilizând câteva versiuni diferite ale metodei **DrawImage()**.
4. Utilizați metoda **Graphics.RotateTransform()**, pentru a roti un string-uri și figuri geometrice care au fost desenate cu metodele **Draw()**.
5. Realizați un mic proiect care utilizează metodele **DrawPie()**, **DrawPolygon()**, **FillPie()** și **FillPolygon()**.

Capitolul 9

XML cu C#

XML (eXtensible Markup Language) este un limbaj folosit pentru descrierea datelor. Menirea XML este de a oferi un format standard, cu ajutorul căruia aplicații diferite, rulând pe calculatoare diferite, pot să citească datele, să le procese și să le scrie. Sistemele de calculatoare și bazele de date conțin date în formate incompatibile. Datele XML se depozitează în format text. Aceasta a oferit o cale de a depozita și de a transmite datele în mod independent de software-ul și hardware-ul folosit.

XML specifică date, dar și forma în care datele sunt organizate. Formatul XML este folosit pentru depozitarea informațiilor din documente care conțin cuvinte, pentru menținerea listelor de prețuri pe site-urile Web, detaliile *post*-urilor de pe bloguri. XML este vehiculul prin care se trimit cantități mari de informație prin Internet. Datele schimbate între serviciile *Web* și aplicațiile clienților sunt XML.

Sintaxa XML

Spre deosebire de HTML, XML este destinat depozitării și transportului datelor și nu afișării datelor. Tag-urile XML nu sunt predefinite. Trebuie să definiți dumneavoastră aceste tag-uri. Un fișier XML este un fișier cu text.

```
<?xml version="1.0" encoding="utf-8" ?>
<biblioteca>
  <carte nrvol ="12">
    <titlu>Oameni si soareci</titlu>
    <autor>John Steinbeck</autor>
  </carte>
  <carte nrvol ="7">
    <titlu>Darul lui Humboldt</titlu>
    <autor>Saul Bellow</autor>
  </carte>
</biblioteca>
```

Elementele sunt ceea ce scrieți între parantezele unghiulare. Exemplu: *biblioteca*, *carte*, *titlu*, *autor*, sunt elemente. Primul element, `<?xml version="1.0" encoding="utf-8" ?>` indică versiunea XML cu care este conformă acest document și că e conformă cu standardul de codificare *Unicode UTF-8*.

Atributele descriu elementele.

Sintaxa pentru un element este:

```
<nume_element nume_atribut ="valoare_atribut">
  Conținutul elementului
</nume_element>
```

De exemplu, **12** și **7** sunt atributele elementelor **carte**. Atributele pot să lipsească.

Comentariile în document se scriu astfel: `<!--Comentariu-->`

Criteriile pe care trebuie să le îndeplinească un document valid XML sunt:

- Documentul are exact un singur element **root**. În cazul de față, elementul root este **biblioteca**. Acesta îndeplinește rolul de container pentru restul datelor.
- Pentru fiecare element “start” trebuie să existe un element corespunzător “final”. Exemplu: `<carte>` și `</carte>`.
- Elementele nu se pot suprapune. Documentul următor este invalid:

```
<biblioteca>
  <carte>
    <titlu>Iarna vrajbei noastre</carte>
  </titlu>
</biblioteca>
```

Clase .NET pentru Xml

Clasele .NET care lucrează cu XML se găsesc în spațiul de nume **System.Xml**. Documentele XML au o structură arborescentă. Trebuie să conțină întotdeauna un unic element **root**. Acesta la rândul lui, conține atribute, valori, alte elemente, și așa mai departe. Clasa **XmlDocument** se bazează pe aceeași idee. Un document se obține prin instanțierea clasei **XmlDocument**. Obiectul are proprietatea **ChildNodes**, care este o colecție referințe la nodurile *child*. Fiecare element al colecției este un obiect de tip **XmlNode**. Fiecare obiect de tip **XmlNode**, are la rândul lui proprietatea **ChildNode**. Astfel, deoarece în fiecare nod se mențin referințe spre nodurile child, se poate explora întregul document.

Citirea informațiilor dintr-un document XML

Un document XML poate îndeplini rolul unei baze de date pentru aplicația dumneavoastră. Aplicațiile citesc deseori informații din asemenea fișiere XML. Este important, pentru că dacă doriți să modificați acele informații, schimbați conținutul fișierului XML și nu e nevoie de refacerea aplicației.

Aplicațiile .NET pot să și scrie în fișiere .xml. De exemplu, dacă setările pe care le face utilizatorul în aplicație trebuie fie persistente, atunci e o idee bună să le salvați într-un document XML.

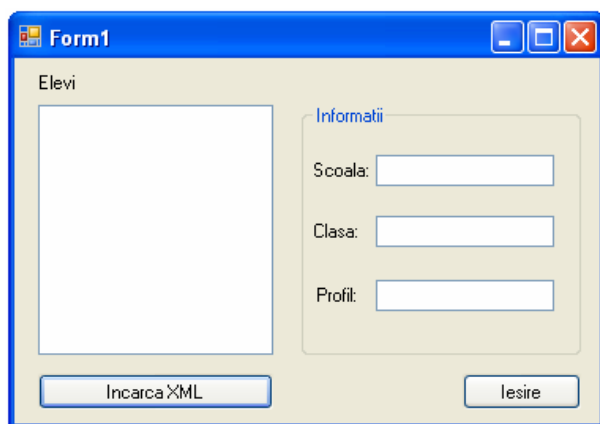
Vom lucra în continuare cu clasa **XmlDocument**, pentru citirea datelor dintr-un document (fișier) XML.

Aplicația *XmlExample*

Aplicația citește date din fișierul **Elevi.xml** și le afișează în mai multe controale. Într-un control de tip **ListBox** se afișează numele elevilor, iar în alte trei controale de tip **TextBox**, se afișează școala, clasa și profilul clasei pentru elevul care a fost selectat în *list box*.

Urmați pașii:

1. Creați un nou proiect de tip **Windows Forms Application**, cu numele *XmlExample*.
2. Aduceți pe forma aplicației un control de tip **ListBox**, trei controale de tip **TextBox**, două butoane, un **GroupBox**, patru controale **Label** și aranjați-le astfel:



3. Vom edita fișierul XML. Mediul integrat include un editor de fișiere XML. Acesta se invocă acționând click drept în **Solution Explorer** pe titlul proiectului, apoi *Add*, și *New Item...* În dialogul care apare, alegeți din panoul *Template*, iconul *XML File*. Completați câmpul *Name* cu *Elevi.xml*. În editor scrieți următorul document:

```
<?xml version="1.0" encoding="utf-8" ?>
<absolventi>
  <elev>
    <nume>Ardelean Eugen</nume>
    <scoala>C.N. Liviu Rebreanu</scoala>
    <clasa>a XII-a A</clasa>
    <profil>matematica-informatica</profil>
  </elev>

  <elev>
    <nume>Costea Andrei</nume>
    <scoala>C.N. Andrei Muresanu</scoala>
    <clasa>a IX-a C</clasa>
    <profil>stintele naturii</profil>
  </elev>
</absolventi>
```



```

    <elev>
        <nume>Nicoara Alina</nume>
        <scoala>Gr. Sc. Industrial nr 1</scoala>
        <clasa>a XI-a B</clasa>
        <profil>telecomunicatii</profil>
    </elev>
</absolventi>

```

După ce ați editat și ați salvat, copiați fișierul *Elevi.xml* din folderul proiectului, în folderul */bin/Debug*.

4. În secțiunea se directive a fișierului fișierul *Form1.cs*, introduceți :

```
using System.Xml;
```

5. Declarați două câmpuri private în clasa *Form1*:

```

// d reține întregul document XML
private XmlDocument d = null;

// elevi reține colecția de noduri child a
// nodului curent.
private XmlNodeList elevi = null;

```

6. Tratăm evenimentul **Click** pentru butonul "*Incarca Xml*". În fereastra **Properties** acționați dublu click pe buton. În corpul *handler*-ului introduceți codul:

```

private void button1_Click(object sender, EventArgs e)
{
    d = new XmlDocument(); // Creează un document XML
    d.Load("Elevi.xml");   // Încarcă fișierul

    // Returnează colecția de elemente (noduri) "elev"
    elevi = d.SelectNodes("absolventi/elev");

    // Parcurge elementele "elev"
    for (int i = 0; i < elevi.Count; i++)
    {
        // În colecția elevi, nodurile se accesează
        // indexat. Pentru fiecare elev, se selectează
        // nodul "nume"
        XmlNode elev =
            elevi.Item(i).SelectSingleNode("nume");

        // Numele elevului se adaugă în listă
        listBox1.Items.Add(elev.InnerText);
    }
}

```

7. La selectarea unui nume în listă, dorim să afișăm în controalele **TextBox** informațiile referitoare la acel nume.

Tratăm evenimentul **SelectedIndexChanged** pentru **ListBox**. Selectați controlul și în fereastra **Properties**, faceți dublu click pe eveniment. În metoda de tratare, introduceți codul evidențiat:

```
private void listBox1_SelectedIndexChanged(
    object sender, EventArgs e)
{
    // Se obține indexul numelui selectat
    int index = listBox1.SelectedIndex;

    // Se selectează lista de elemente "elevi"
    elevi = d.SelectNodes("absolventi/elev");

    // În colecția elevi se accesează cel cu indexul
    // index. Apoi se selectează subelementul "scoala"
    XmlNode nume =
        elevi.Item(index).SelectSingleNode("scoala");

    // Textul aferent elementului "scoala"
    textBox1.Text = nume.InnerText;

    // Operațiile se repetă pentru "clasa" și "profil"
    XmlNode clasa =
        elevi.Item(index).SelectSingleNode("clasa");
    textBox2.Text = clasa.InnerText;
    XmlNode profil =
        elevi.Item(index).SelectSingleNode("profil");
    textBox3.Text = profil.InnerText;
}
```

8. Compilați și rulați cu **F5**.

La rulare, după acționarea butonului *Incarca XML*:

Important!

- Clasele `XmlDocument` și `XmlNodeList` moștenesc clasa `XmlNode`. Astfel ne explicăm faptul că proprietățile `ChildNodes` și `InnerText` se regăsesc printre membrii ambelor clase.
- `InnerText` este valoarea nodului concatenată cu valorile tuturor copiilor săi.
- `XmlNodeList` reprezintă o colecție ordonată de noduri.
- Metoda `XmlNode Item(int i)` a clasei `XmlNodeList` returnează nodul aflat la indexul `i`.

Descărcarea fișierelor XML de pe Internet

În ultimii ani, multe site-uri și bloguri expun clienților un format XML cunoscut sub numele **RSS** (*Really Simple Syndication*), în scopul distribuirii de conținut care se schimbă frecvent (știri, sumarul conținutului unui site sau întregul conținut, posturi pe blog, etc).

Un furnizor de conținut (site, blog) publică de regulă un link (*feed link*) la care utilizatorii subscriu cu ajutorul unor programe specializate (*feed reader*). Subscrierea se face simplu, copiind link-ul din *Web browser* în *feed reader*. Mai departe, conținuturile noi se descarcă în mod automat, iar utilizatorul este înștiințat când acest lucru se petrece. Practic, se descarcă fișiere XML.

Cu .NET puteți crea propriul *feed reader* (agregator). Nu vom face acest lucru aici, ci vom arăta cât de simplu puteți descărca un fișier XML de pe Internet sau un *feed RSS*.

Aplicația `DownloadXml`

Aplicația utilizează clasa `WebClient` din spațiul de nume **System.Net** pentru a descărca un fișier de tip XML.

Urmați pașii:

1. Creați o aplicație de tip *Windows Forms* în Visual C# Express 2008.
2. Pe forma aplicației, aduceți din **Toolbox** un buton.
3. La secțiunea directivelor, în fișierul `Form1.cs`, adăugați directivele:

```
using System.Xml;  
using System.Net;
```

4. Tratăm evenimentul **Click** generat la click pe buton. Acționați dublu click pe buton. În metoda de tratare scrieți:

```
private void button1_Click(object sender, EventArgs e)
{
    // Descărcăm în stringul xml conținutul XML (RSS
    // feed) de la un anumit URL
    string xml = new
    WebClient().DownloadString("http://news.google.com/"
                                + "?output=rss");

    // Creăm o instanță de tip XmlDocument
    XmlDocument doc = new XmlDocument();

    // Încărcăm stringul (cu conținut XML) în document
    doc.LoadXml(xml);

    // Salvăm pe disc conținutul XML descărcat
    doc.Save("continut.xml");
}
```

5. Compilați și rulați cu **F5**.

După executare, veți găsi în folderul **/Bin/Debug** fișierul **continut.xml**.

Observație:

Pe numeroase site-uri și blog-uri veți găsi *feed link*-uri pentru subscriere. Iată câteva link-uri alese aleatoriu:

```
http://www.preferredjobs.com/rss/rss2.asp
http://sourceforge.net/export/rss2_sfnews.php?feed
http://newsrss.bbc.co.uk/rss/newsonline_uk_edition/world/rss.
xml
```

Citirea și analiza unui document XML cu XmlTextReader

Transportul datelor la distanță se face în mod frecvent cu ajutorul *stream*-urilor.

Clasa **XmlTextReader** este un instrument foarte rapid de accesare *stream*-urilor de date XML. Clasa reprezintă un *cititor* (*reader*), care se deplasează în *stream* doar înainte (*forward only*), citește documentul nod după nod și nu modifică *stream*-ul. Reader-ul vă permite să vă deplasați progresiv în documentul XML și să examinați fiecare nod, cu elementul, atributele și valoarea sa.

Propunem o aplicație care ilustrează o parte din capabilitățile acestei clase.

Aplicația *XmlTextReaderExample*

Programul C# de mai jos folosește clasa **XmlTextReader** pentru a citi un document XML aflat pe disc. Documentul este citit nod cu nod, apoi este reconstituit și se afișează pe ecran.

Urmați pașii:

1. În *Visual C#* creați un proiect de tip consolă cu numele *XmlTextReaderExample*.
2. Editați fișierul *carti.xml* cu următorul conținut:

```
<!--Citire fisier XML-->
<biblioteca>
  <carte ISBN="186-383-537">
    <Titlu>Amintiri din copilărie</Titlu>
    <Pret>13.20</Pret>
    <Autor>Ion Creanga</Autor>
  </carte>
</biblioteca>
```

3. În fișierul *Program.cs* introduceți codul:

```
using System;
using System.Xml;

class Program
{
    static void Main(string[] args)
    {
        // Creăm un reader (instanță a clasei) și
        // încărcăm fișierul
        XmlTextReader reader =
            new XmlTextReader ("carti.xml");

        // Citim pe rând toate nodurile din document
        while (reader.Read())
        {
            // În funcție de tipul nodului curent
            switch (reader.NodeType)
            {
                // Dacă nodul este un element
                case XmlNodeType.Element:
                    Console.WriteLine("<" + reader.Name);
                    Console.WriteLine(">");
                    break;

                // Afișăm textul din fiecare element
                case XmlNodeType.Text:
```

```

        Console.WriteLine ("    " +
                           reader.Value);

        break;

    // Afișăm "sfârșitul" elementului
    case XmlNodeType.EndElement:
        Console.Write("</" + reader.Name);
        Console.WriteLine(">");
        break;

    }

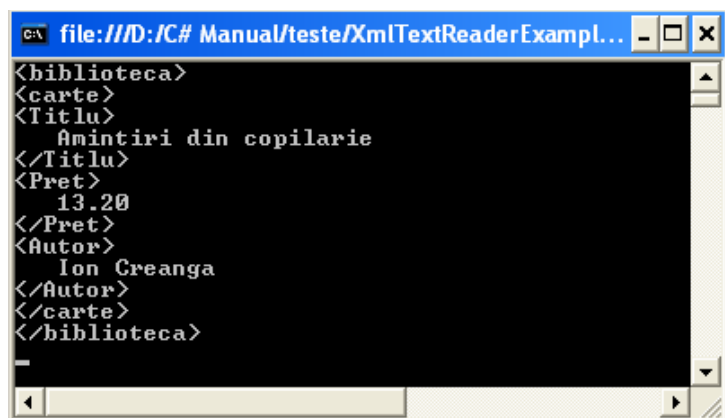
}

} // Main()
}

```

4. Compilați și rulați cu **F5**.

La rulare, pe ecran se va afișa:



```

C:\ file:///D:/C# Manual/teste/XMLTextReaderExamp...
<biblioteca>
<carte>
<Titlu>
  Amintiri din copilarie
</Titlu>
<Pret>
  13.20
</Pret>
<Autor>
  Ion Creanga
</Autor>
</carte>
</biblioteca>
-

```

De reținut:

- Metoda `Read()` a clasei `XmlTextReader` citește și returnează următorul nod din *stream*.
- Proprietatea `NodeType` a clasei `XmlNode` este o enumerare ai cărei membri reprezintă tipurile de noduri din document. Studiați această enumerare pentru a vedea că există mai multe tipuri de noduri decât în acest exemplu.
- Cu ajutorul clasei `XmlTextReader` obțineți conținutul XML dintr-un fișier sau dintr-un *stream*. În plus, are calitatea că vă permite să analizați și să prelucrați acest conținut.

Crearea conținutului XML cu *XmlTextWriter*

Clasa **XmlTextWriter** oferă o cale rapidă cu care puteți genera fișiere sau *stream*-uri cu conținut XML. Clasa conține un număr de metode și proprietăți cu care se generează conținut XML. Pentru a le folosi creați un obiect de tip **XmlTextWriter**, apoi adăugați pe rând entități XML obiectului. Există metode cu care puteți adăuga orice tip de conținut XML. Aplicația pe care urmează să o utilizăm folosește câteva dintre aceste metode.

Aplicația *XmlTextWriterExample*

Vom crea un fișier cu conținut XML. Urmăți pașii:

1. Creați o aplicație de tip consolă în *Visual C# Express Edition*.
2. În fișierul *Program.cs*, introduceți codul:

```
using System;
using System.IO;
using System.Xml;

public class Program
{
    // Fișierul în care se scrie conținutul XML
    private string filename = "continut.xml";

    public static void Main()
    {
        // Creăm obiectul de tip XmlTextWriter
        XmlTextWriter writer =
            new XmlTextWriter(filename, null);

        // Folosim indentarea tag-urilor
        writer.Formatting = Formatting.Indented;

        // Scriem un comentariu XML
        writer.WriteComment("Creeare fisier XML");

        // Scriem primul element (root).
        writer.WriteStartElement("bibliotecarie");

        // Scriem elementul carte(child pentru
        // bibliotecarie)
        writer.WriteStartElement("carte");
        // Scriem un atribut pentru carte
        writer.WriteAttributeString("ISBN",
            "186-383-523");
    }
}
```

```
// Titlul cărții
writer.WriteStartElement("Titlu");
writer.WriteString("Amintiri din copilărie");
writer.WriteEndElement();

// Prețul și autorul cărții
writer.WriteElementString("Pret", "13.20");
writer.WriteStartElement("Autor");
writer.WriteString("Ion Creanga");

// Scrie tag-ul de sfârșit pentru elementul
// carte
writer.WriteEndElement();

// Scrie tag-ul de sfârșit pentru elementul
// librărie
writer.WriteEndElement();

// Scrie XML în fișier și închide writer
writer.Flush();
writer.Close();

// Afișăm conținutul XML și pe ecran
XmlDocument doc = new XmlDocument();

// Păstrăm spațiile albe pentru lizibilitate
doc.PreserveWhitespace = true;
// Încărcăm fișierul
doc.Load(filename);

// Scriem conținutul XML pe ecran
Console.Write(doc.InnerXml);

Console.ReadLine();
} // Main()
}
```

3. Compilați și rulați cu F5.

În folderul **/bin/Debug** veți găsi fișierul *continut.xml*. Același conținut se afișează și pe ecran:

De reținut:

- Clasa `XmlTextWriter` oferă o cale programatică de a crea conținut XML
- Metodele clasei au nume intuitive.
De exemplu, `writer.WriteStartElement("titlu");` scrie în obiectul `writer` `<titlu>`.
- Cu `WriteString()` scrieți textul asociat elementului curent.
- Există și alte metode și proprietăți corespunzătoare altor entități XML. Acestea se utilizează în mod similar. Merită efortul să le explorați.

Probleme propuse

1. Realizați o aplicație care utilizează controale *text box* pentru a introduce datele personale ale clienților unei bănci. Datele clienților se vor salva într-un fișier în format XML.
2. Realizați o aplicație care deschide un fișier XML și afișează într-un *textbox* concatenarea tuturor textelor asociate elementelor *child* ale elementului *root*.
3. Realizați o aplicație de tip consolă care descarcă un **feed RSS** de pe Internet și listează numele tuturor elementelor din conținutul XML.
4. Realizați o aplicație care citește conținutul XML dintr-un fișier și cu ajutorul clasei `XmlTextReader` și afișează într-un **ListBox** toate elementele XML.
5. Realizați o aplicație care salvează într-un fișier cu conținut XML informații despre produsele unui magazin. La repornirea aplicației, aceste informații se pot încărca de către aplicație pentru a fi afișate în controale.

Partea a III – a

Baze de date

Scopul acestei părți a lucrării este acela de a vă ajuta să accesați bazele de date relaționale cu C#.

Capitolul 10

Baze de date și ADO.NET. Noțiuni introductive

Capitolul are următoarele obiective:

- O scurtă prezentare a sistemului de gestiune pentru baze de date relaționale **MS SQL Server**.
- Generalități despre sistemele de gestiune a bazelor de date relaționale.
- Primele noțiuni despre tehnologia **ADO.NET**.
- Crearea unei baze de date în *Visual C# 2008 Express Edition*.
- Interogarea bazei de date cu ajutorul *Query Designer*.

Instrumente de lucru

Instrumentele software pe care le vom utiliza în lucrul cu bazele de date sunt:

- **Visual C# Express Edition 2008 (VCSE)**.
- **Microsoft SQL Server Express Edition 2005 (SSE)**.

Ambele instrumente funcționează pe platforma **Microsoft .NET** versiunea **3.5**. Sunt versiuni *free*, puternice, destinate să lucreze împreună. În momentul când instalați **VCSE**, aveți opțiune de instalare și pentru **SSE**.

SSE, este un subset al serverului de baze de date *Microsoft SQL Server 2005*. Acesta din urmă este unul dintre cele mai avansate sisteme de gestiune a bazelor de date (SGDB) existente. Deși SSE nu include toate capabilitățile **SQL Server 2005**, este perfect funcțional și utilizabil în aplicații industriale. Suportă procesarea online a tranzacțiilor (OLAP), poate să gestioneze baze de date mai mari de **4 Gb** și admite sute de conexiuni concurente.

Când instalați **SSE**, veți avea de fapt două versiuni ale serverului: **SQL Server Express Edition** și **SQL Server Express Compact Edition 2005**. Acesta din urmă, este o versiune cu mai puține facilități, fiind destinat să gestioneze baze de date locale. În momentul în care construiți o bază de date în VCSE, aveți opțiunea de a alege unul dintre cele două servere.

SGDB-uri – noțiuni generale

Sistemele de gestiune ale bazelor de date, așa cum numele indică, se ocupă cu crearea și întreținerea bazelor de date. O bază de date este o colecție de informații structurate. Bazele de date poate înmagazina mari cantități de informație care poate apoi să fie accesată și modificată în mod eficient de către utilizatori prin intermediul SGDB-urilor.

Un SGDB admite multiple baze de date. Utilizatorii obțin informații din bazele de date scriind interogări (*query*) care se adresează serverului de baze de date. Acesta le execută, efectuând operații specifice asupra bazei de date și întoarce utilizatorului un set de rezultate. Operațiile obișnuite sunt: de modificare a datelor, de introducere de noi date, de ștergere.

SQL (*Structured Query Language*) este limbajul cu care trebuie să ne adresăm unui SGDB care operează cu baze de date relaționale. În această lucrare vom presupune că cititorii au cunoștințe elementare referitoare la acest limbaj.

Un **Sistem de Gestiune a Bazelor de Date Relaționale** (SGDBR) este un SGDB bazat pe **modelul relațional**. Datele sunt depozitate în tabele. Între tabele există relații, iar relațiile se memorează de asemenea în baza de date.

Calitățile SGDBR-urilor

Sistemele de gestiune a bazelor de date asigură depozitarea și regăsirea datelor. În raport cu alte forme de păstrare a datelor, au *atu*-uri care le fac inegalabile:

- **Viteza mare a operațiilor.**
Operațiile asupra datelor (căutare, sortare, modificare) sunt foarte rapide.
- **Compactează informația.**
Bazele de date lucrează cu cantități uriașe de date. Acestea se memorează pe spații relativ reduse.
- **Asigură securitatea datelor.**
Datele sunt foarte bine protejate împotriva oricărui acces neautorizat.
- **Întreține datele.**
Un SGDB ține evidența fiecărui fragment de informație. Utilizatorii nu trebuie să se preocupe acest aspect.
- **Controlează redundanța datelor.**
Previn crearea de duplicate multiple ale acelorași date, care ar duce la ocuparea unui volum mare pe disc.
- **Previne inconsistența datelor.**

Nu permite utilizatorului operații care distruge logica bazei de date. În același timp, când un SGDB elimină date redundante, această operație se face cu păstrarea consistenței datelor.

- **Asigură atomicitatea operațiilor.**

Operațiile elementare, dar și grupurile de operații (tranzacțiile) se garantează fie că vor fi complet executate, fie că nu vor avea nici un efect. În oricare dintre situații, baza de date rămâne într-o stare consistentă.

Furnizori de baze de date relaționale

Pe piață există multe SGDB-uri. O să amintim doar câteva firme producătoare importante:

- **Oracle**, cu SGDB-ul **Oracle**. Deține recordul de vânzări în acest moment.
- **IBM** cu produsele **DB2** și **Informix**.
- **Microsoft** cu **SQL Server 2005**.
- **Sybase**, cu SGDB-ul **Sybase**.

Există și alte firme ale căror sisteme de gestiune a bazelor de date au un segment de piață bine delimitat, dar mult mai restrâns decât a celor menționate.

În acest punct trebuie să adăugăm rolul tot mai important pe care îl are în acest moment SGDB-ul **MySQL**. Se utilizează ca *free software*, sub *GNU General Public Licence* (GPL). Este foarte popular îndeosebi pentru aplicații Web, fiind disponibil pentru mai multe sisteme de operare.

Tehnologia ADO.NET – introducere

Aproape toate aplicațiile software interacționează cu baze de date. Este nevoie de un mecanism prin care aplicațiile se conectează și utilizează bazele de date. Pentru aplicațiile **.NET**, acest mecanism se numește **ADO.NET**.

Caracteristicile tehnologiei ADO.NET

ADO.NET este format dintr-un subset al **Bibliotecii de Clase .NET**, folosite în programare pentru accesarea surselor de date, în special a bazelor de date relaționale.

Înainte ca platforma **.NET** să existe, în programarea aplicațiilor pentru sistemele *Windows* se utilizau următoarele tehnologii de accesare a bazelor de date: **ODBC** (*Open Database Connectivity*), **OLE DB** (*Object Linking and Embedding, Database*) și **ADO** (*ActiveX Data Objects*). **ADO** este o colecție de obiecte pentru accesarea surselor de date.

Pentru compatibilitate cu SGDB-uri mai vechi, **ADO.NET** include în continuare **ODBC** și **OLE DB**, însă **ADO.NET nu este ADO**.

ADO.NET este o tehnologie complet nouă de acces la date. Este parte integrantă a platformei .NET și nu este formată din obiecte *ActiveX*. Rațiunile pentru care Microsoft a păstrat cuvântul ADO în numele ADO.NET țin de ideea că interfața de utilizare a celor două tehnologii este oarecum asemănătoare.

ADO.NET facilitează dezvoltarea de aplicații mai performante cu baze din următoarele considerente :

1. Accesul deconectat la bazele de date

ADO.NET este conceput să permită atât **accesul conectat** cât și **deconectat** la bazele de date. Imaginați-vă că v-ați conectat prin intermediul unei aplicații la un server de baze de date, aflat undeva la distanță în rețea. Deschideți o conexiune, citiți datele, modificați datele și la sfârșit vă deconectați. Acesta este **modelul conectat**. Dezavantajul este că în tot acest timp comunicați prin rețea direct cu serverul care trebuie să mențină conexiunea și să execute interogările. Dacă ne gândim că poate fi vorba de sute sau mii de utilizatori care transferă cantități mari de date, atunci înțelegem că în **modelul conectat**, utilizat cu vechile tehnologii, pot să survină supraîncărcări ale serverului și ale traficului în rețea, un consum mare de resurse și încetinirea sau blocarea aplicațiilor.

În **modelul deconectat** datele sunt trimise de la server și sunt stocate local la client într-o structură de date numită **dataset**. Clientul operează doar asupra datelor din acest dataset, iar când a terminat, datele schimbate se trimit la server, acolo unde se găsește adevărata bază de date. Astfel, serverul este eliberat de sarcina de a menține conexiunile în mod permanent și poate servi mai mulți cu clienți. Pe partea clientului de asemenea, toate operațiile se desfășoară mai rapid.

2. Integrarea XML.

ADO.NET este strâns legat de **XML**. XML este folosit intern de ADO.NET ca să mențină datele în **dataset**-uri și să rețină relațiile și constrângerile între tabele. Mai mult, suportul pentru XML este integrat în ADO.NET. Se pot produce *scheme XML*, se pot transmite date cu ajutorul *documentelor XML*.

Arhitectura ADO.NET

ADO.NET are două componente centrale:

1. **Furnizorii de date (data provider)**
2. **Seturile de date (datasets)**

1. Furnizorii de date

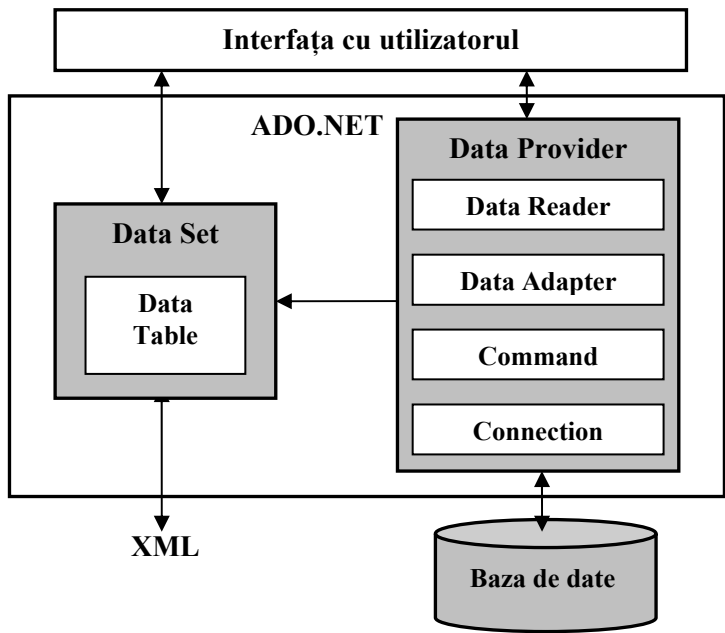
Un furnizor de date (*data provider*) asigură conectarea la o *sursă de date (data source)*. Sursa de date poate fi un fișier text, un document XML, o bază de date, etc. Totodată un *furnizor de date* suportă accesul la date și manipularea

datelor. Este legătura între aplicația dumneavoastră și sursa de date. De regulă sursa de date este o bază de date și atunci un *furnizor de date* asigură legătura între aplicație și sistemul de gestiune a bazei de date respective.

Un *furnizor de date* se compune dintr-un număr de obiecte ale unor clase specializate. Aceste clase sunt definite în interiorul unor spații de nume speciale. Tabelul prezintă câteva dintre spațiile de nume în care sunt grupate componentele .NET:

Spațiul de nume	Descriere
System.Data	Clase, interfețe, delegări care definesc arhitectura .NET. Aici sunt clasele care definesc <i>dataset</i> -urile
System.Data.Odbc	<i>Data provider</i> .NET pentru ODBC
System.Data.OleDb	<i>Data provider</i> .NET pentru OLE DB
System.Data.Sql	Clase care suportă funcționalitate specifică SQL Server
System.Data.OracleClient	<i>Data provider</i> .NET pentru Oracle
System.Data.SqlClient	<i>Data provider</i> .NET pentru SQL Server

Figura 10.1. Schema părților componente ale arhitecturii .NET.



Sarcinile pe care le îndeplinește un *data provider*:

- Furnizează accesul la date printr-o conexiune activă cu sursa de date.
- Asigură transmitia datelor la și dinspre **dataset**-uri (în modelul deconectat).
- Asigură transmitia datelor la și dinspre aplicație (în modelul conectat)

Clasele unui *data provider* sunt:

Connection	Creează conexiunea cu sursa de date.
Command	Este utilizată pentru operații asupra sursei de date: citire, modificare, ștergere de date.
Parameter	Describe un singur parametru al unei comenzi. De pildă, parametrul unei proceduri stocate.
DataAdapter	Este un adaptor pentru transferul datelor între sursa de date și <i>dataset</i> .
DataReader	Este folosit pentru accesarea și citirea rapidă a datelor într-o sursă de date.

Pentru a înțelege cum sunt definite în .NET aceste clase, vom lista numele acestor clase pentru doi furnizori de date: **Sql Server** și **OLE DB .NET**. Reamintim că în spațiul de nume **System.Data.SqlClient** sunt clasele furnizorului de date **Sql Server**, iar în spațiul de nume **System.Data.OleDb** sunt cele ale furnizorului de date, **OLE DB**.

Clasele specifice celor doi *data provider*-i sunt:

Furnizorul OLE DB .NET	Furnizorul SQL Server	Corespunde clasei
OleDbConnection	SqlConnection	Connection
OleDbCommand	SqlCommand	Command
OleDbDataReader	SqlDataReader	DataReader
OleDbDataAdapter	SqlDataAdapter	DataAdapter

Pentru ca lucrurile să se lămurească pe deplin, enumerăm clasele **DataReader** specifice tuturor furnizorilor .NET:

- **System.Data.SqlClient.SqlDataReader**
- **System.Data.OleDb.OleDbDataReader**
- **System.Data.Odbc.OdbcDataReader**
- **Oracle.OracleClient.OracleDataReader**

2. Seturile de date

Dataset-ul este o componentă majoră a arhitecturii .NET. **DataSet** este clasa care definește un *dataset*. Un *dataset* este o colecție de obiecte **DataTable** care pot fi legate între ele cu obiecte de tip **DataRelation**. *Dataset*-urile preiau de la *data provider*-i informațiile necesare din sursa de date sub forma unei copii reduse a bazei de date. Prin urmare, un *dataset* poate include o bază de date relațională cu tabele, relații, vederi. Utilizatorul operează asupra tabelelor din *dataset*, care sunt practic o copie a tabelelor reale din baza de date. Deoarece un *dataset* este un obiect, *cache*-ul pentru datele sale este în memorie.

Dataset-urile implementează *modelul de lucru deconectat* de baza de date. Când este necesar, *dataset*-urile transmit bazei de date modificările operate asupra datelor.

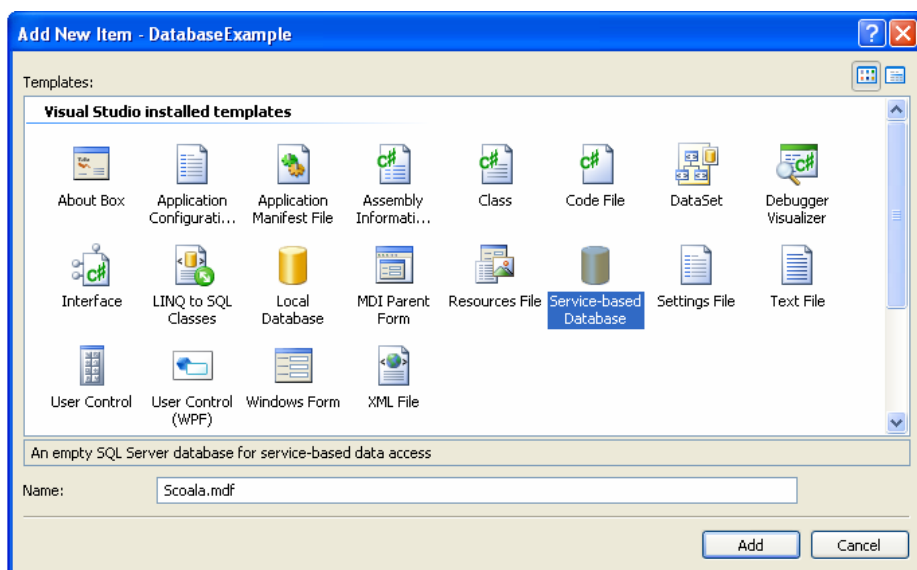
Clasa **DataSet**, ca și celelalte clase care lucrează cu un *dataset* se găsesc în spațiul de nume **System.Data**. Tabelul următor descrie aceste clase:

DataSet	Obiectele sale descriu schema întregii baze de date sau a unei submulțimi a sa. Conține tabele și relațiile între ele.
DataTable	Obiectele reprezintă o singură tabelă din baza de date. Conține rânduri și coloane.
DataRow	Reprezintă un singur rând într-o tabelă.
DataColumn	Reprezintă o coloană într-o tabelă.
DataRowView	Obiectele sortează datele. Nu sunt incluse într-un DataTable
DataRowView	Obiectele reprezintă un singur rând într-un DataRowView .
DataRelation	Reprezintă o relație între tabele. De exemplu, o relație de <i>cheie primară-cheie străină</i>
Constraint	Describe o constrângere în tabelă, cum ar fi unicitatea valorilor într-o coloană cheie primară.

Crearea unei baze de date în VCSE

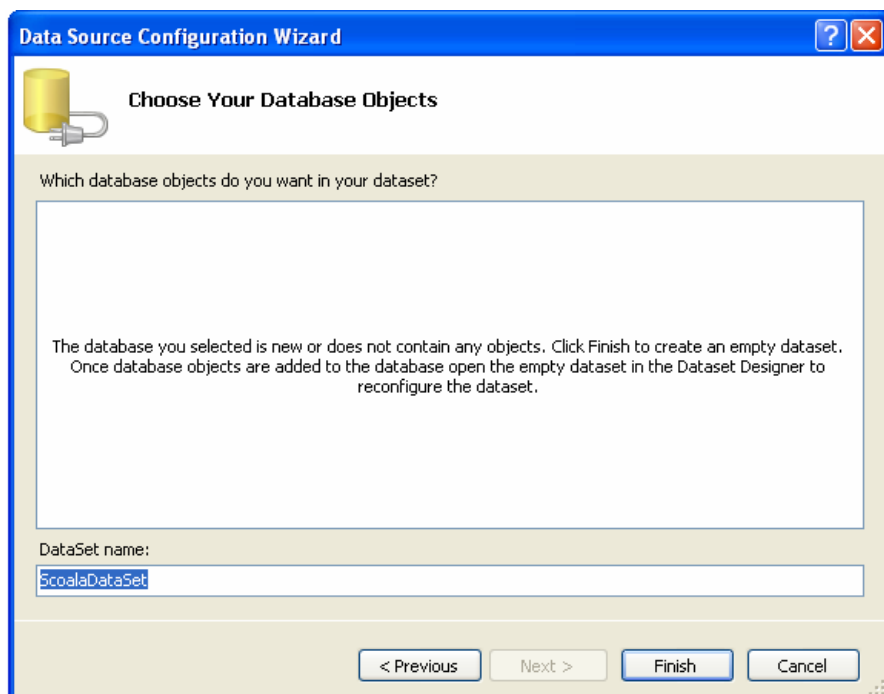
Construim o bază de date în cadrul unui proiect de tip consolă. Pentru aceasta, deschideți *Visual C# Express 2008* și urmați următoarele indicații:

1. Creați un nou proiect de tip consolă cu numele *DatabaseExample*.
2. În meniul *View*, selectați *Other Windows*, apoi *Database Explorer*. Aceasta este o fereastră importantă, în care vizualizați și manipulați elementele bazei de date pe care o veți crea.
3. În *Solution Explorer*, acționați click drept pe numele proiectului, alegeți *Add*, apoi *New Item...* În dialogul *Add New Item*, selectați iconul *Service-based Database*, apoi numiți baza de date *Scoala*.



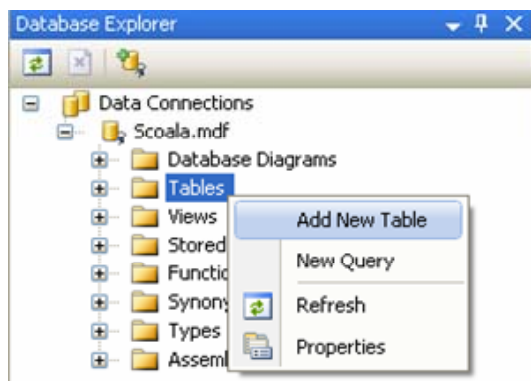
Remarcați că există și alternativa de a alege *Local Database*. În această situație, baza de date este creată de către **SQL Server Compact Edition**. Are extensia **.sdf**, și este destinată să fie utilizată local. Testați și această variantă.

4. În panoul *Add New Item*, acționați butonul *Add*. Dialogul *Data Source Configuration Wizard* vă anunță că va construi un *dataset* vid, pe care îl puteți configura ulterior. Acceptați numele `ScoalaDataSet` pentru acesta:

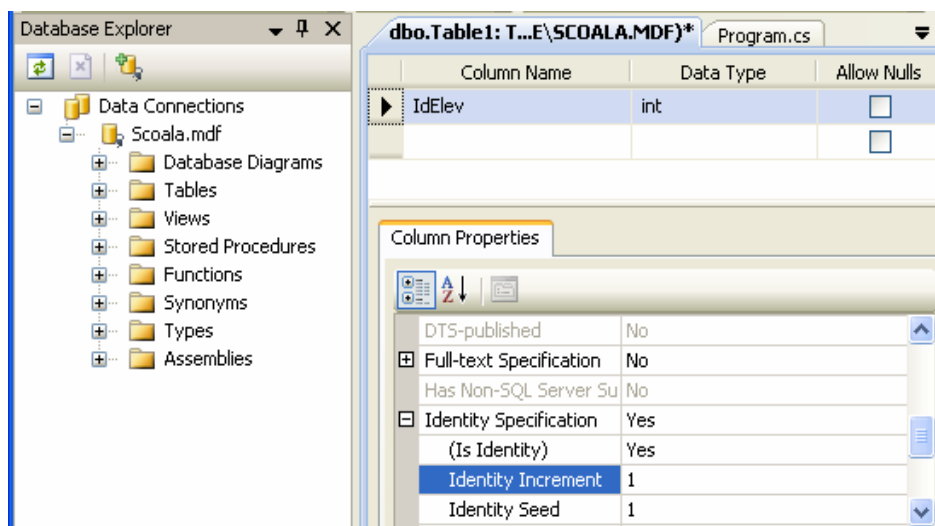


Apăsați butonul *Finish*.

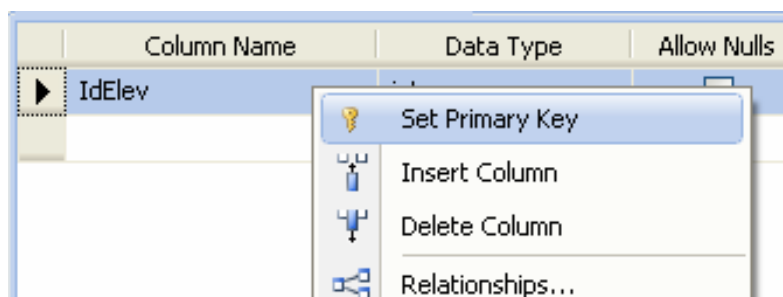
5. În panoul **Database Explorer**, expandați nodul *Scoala.mdf*, care reprezintă baza de date. Click dreapta pe **Tables**, și selectați *Add New Table*:



6. Panoul care se deschide este **Table Designer**. Acesta vă permite să definiți schema tabelului, adică numele, tipul și constrângerile pentru fiecare coloană. Completați prima linie ca în figură. În fereastra *Column Properties*, setați pentru câmpul **IdElev**, proprietatea **IsIdentity** la valoarea **Yes** și **Identity Increment** la valoarea **1**. Aceasta înseamnă că tipul coloanei trebuie să fie **int**, iar valorile pe coloană se incrementează în mod automat începând de la valoarea **1**, pentru fiecare rând nou introdus. În acest fel, **IdElev** poate identifica în mod unic un rând în tabelă:



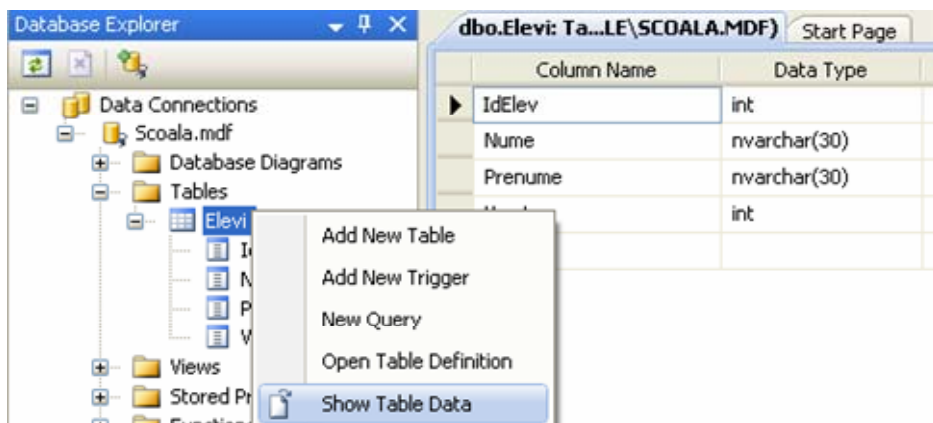
7. Vom stabili pentru coloana **IdElev**, o constrângere de **cheie primară** (*primary key*). Facem acest lucru pentru a vedea cum se procedează și nu pentru că ar fi necesar, deoarece **IdElev** este deja o coloană **Identity**. Deselectați *check box*-ul **Allow Nulls**, deoarece o coloană **Identity** sau o cheie primară nu admite valori nule. Selectați în **Table Designer** primul rând, faceți click dreapta și alegeți **Set Primary Key**:



8. Definiți celelalte coloane astfel:

Column Name	Data Type	Allow Nulls
IdElev	int	☐
Nume	nvarchar(30)	☐
Prenume	nvarchar(30)	☒
Varsta	int	☒
	int	☐
	money	
	nchar(10)	

9. Salvați proiectul și tabela apăsând butonul *Save All*. În dialogul *Choose Name* introduceți numele tablei: **Elevi**.
10. Pentru a însera (*design time*) câteva rânduri cu date în tabelă, expandați nodul *Tables* în **Database Explorer**. Click dreapta pe tabela **Elevi** și selectați *Show Table Data*.



11. S-a deschis *Query Designer*. Aici puteți introduce manual date în tabelă și puteți efectua interogări. Completați câteva rânduri, ca mai jos. Observați că pe coloana **IdElev** serverul nu vă permite să introduceți valori, deoarece este coloană **Identity** și aceasta înseamnă că **SQL Server** îi atribuie valori în mod automat:

IdElev	Nume	Prenume	Varsta
1	Pascu	Ioan	18
2	Nedelcu	Valentin	17
3	Pop	Andrei	19
4	Ilie	Ioan	18
*	NULL	NULL	NULL

12. Dacă doriți să adăugați și alte tabele bazei de date **Scoala**, de pildă tabelele **Clase**, **Profesori**, **Materii**, e foarte simplu: faceți click dreapta pe **Tables** în **Database Explorer** și alegeți **New Table.**, apoi repetați pașii **6...11**.

Nu veți face **build** acestui proiect, pentru că aplicația nu execută nimic. Vom vedea în secțiunile următoare cum ne conectăm la baza de date și cum o interogăm programatic.

De reținut:

- O **cheie primară (primary key)** identifică în mod unic fiecare rând într-o tabelă. Cheia primară include o coloană sau o combinație de coloane. Nu pot exista două rânduri distincte într-o tabelă care să aibă aceeași valoare (sau combinație de valori) în aceste coloane.
- O **coloană identitate (Identity column)** identifică în mod unic fiecare rând într-o tabelă, iar valorile sale sunt generate de către baza de date. Deseori o coloană identitate e desemnată și cheie primară.
- **varchar(n)** este un tip de dată specifică SGDB-urilor, care reține un set de caractere de lungime variabilă, dar care nu depășește **n**. Pentru **SQL Server 2005**, **n** poate fi maxim 8000 *bytes*.

Interogări cu Query Designer

În această secțiune vom face câteva exerciții de manipulare a datelor, în scopul împrăștiării cunoștințelor referitoare la comenzile limbajului **SQL**. Utilizăm baza de date **Scoala**, construită anterior.

Deschideți proiectul **DatabaseExample**. În **Database Explorer**, acționați click dreapta pe **Tables** și alegeți **New Query**. În dialogul **Add Table** selectați tabela **Elevi** și acționați butonul **Add**. Se deschide **Query Designer**-ul mediului integrat, care vizual consistă din patru panouri: **Panoul Diagramelor (Diagram Pane)**, **Panoul Criteriilor (Criteria Pane)**, **Panoul SQL (SQL Pane)**, **Panoul de Vizualizare a Rezultatelor (Show Results Pane)**.

În **Panoul SQL** vom introduce interogări, iar rezultatele interogării, adică setul de rezultate, este vizualizat în **Panoul Rezultatelor**. Panoul diagramelor permite selectarea coloanelor iar cel al criteriilor servește la stabilirea criteriilor de sortare. Toate selecțiile făcute în cele două panouri generează cod în **Panoul SQL**.

Reamintim faptul că limbajul **SQL (Structured Query Language)** include două secțiuni importante:

1. **Data Manipulation Language (DML).**
DML constă din interogări și comenzi pentru modificarea datelor. Acestea sunt: **SELECT**, **UPDATE**, **DELETE** și **INSERT INTO**.
2. **Data Definition Language (DDL).**
 Comenzile **DDL** permit crearea și ștergerea tabelor într-o bază de date. Cele mai importante comenzi de acest tip sunt: **CREATE TABLE**, **ALTER TABLE**, **DROP TABLE**.

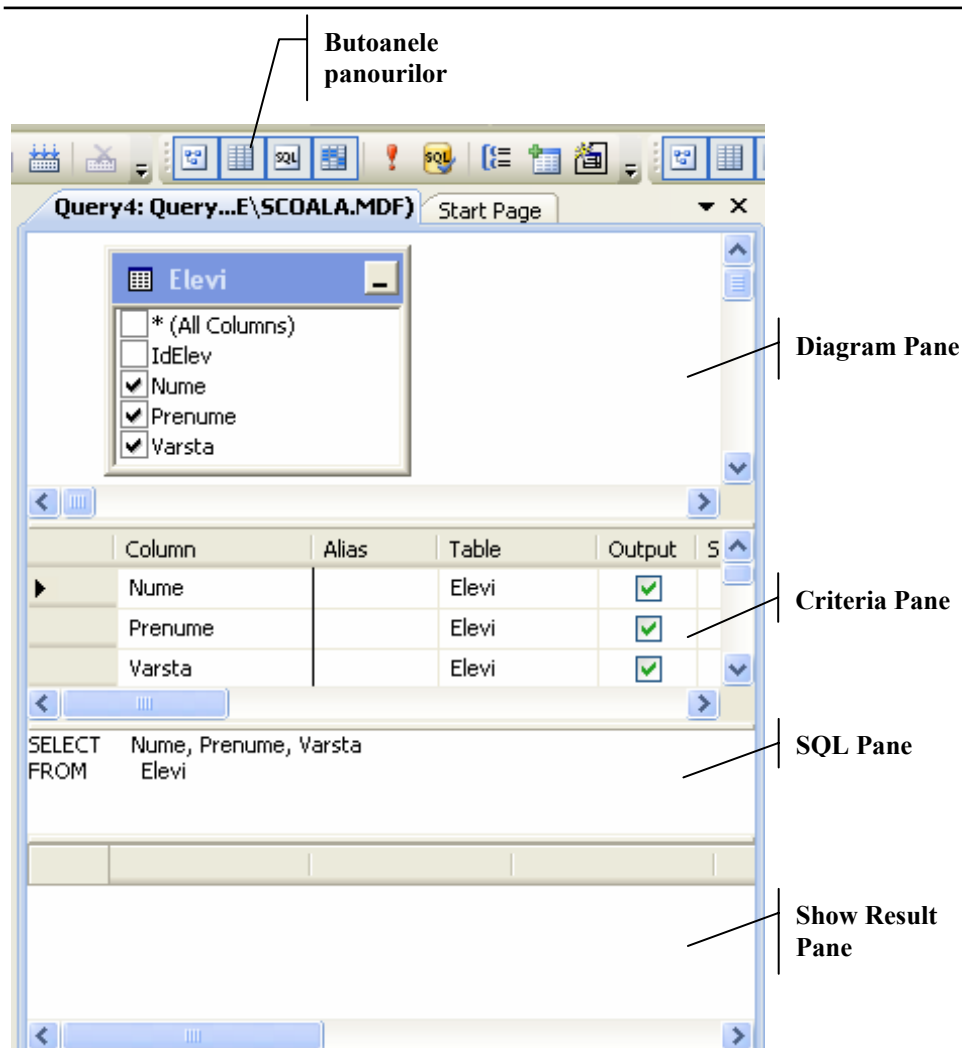


Figura 10.2 Query Designer

Executarea interogării

Pentru executare acționați click pe butonul *Execute SQL* (butonul cu semnul exclamării) sau de la tastatură **Ctrl + R**.

În panoul de rezultate, se afișează:

	Nume	Prenume	Varsta
►	Pascu	Ioan	18
	Nedelcu	Valentin	17
	Pop	Andrei	19
	Ilie	Ioan	18

Comanda SELECT

SELECT este utilizată pentru selectarea datelor dintr-o tabelă. Rezultatul este returnat într-o formă tabelară, numită *result set*.

Sintaxă:

```
SELECT nume_coloane FROM tabela
```

Exemplu:

Introduceți în *SQL Pane* comanda următoare, după care o executați cu un click pe butonul *Execute SQL*:

```
SELECT Nume, Prenume, Varsta FROM Elevi
```

Puteți face acest lucru manual, sau bifați în *Diagram Pane* coloanele corespunzătoare.

Sortare

Ca să introduceți criterii de sortare în comanda **SELECT**, utilizați *Panoul Criteriilor* (selectați o opțiune din coloana *Sort Type*) sau le introduceți manual.

Exemplu:

Introduceți în *SQL Pane* comanda:

```
SELECT Nume, Prenume FROM Elevi  
ORDER BY Nume
```

Rezultat:

	Nume	Prenume
►	Ilie	Ioan
	Nedelcu	Valentin
	Pascu	Ioan
	Pop	Andrei

Clauza WHERE

WHERE permite selecția condiționată. Cu ***** selectați toate coloanele tablei.

Exemplu:

Introduceți în *SQL Pane* comanda:

```
SELECT * FROM Elevi  
WHERE Varsta >= 18
```

Rezultat:

	IdElev	Nume	Prenume	Varsta
►	1	Pascu	Ioan	18
	3	Pop	Andrei	19
	4	Ilie	Ioan	18

Comanda INSERT INTO

Comanda inserează un rând nou în tabelă.

Sintaxă:

```
INSERT INTO nume_tabelă
VALUES valoare1, valoare2, ...
```

Puteți de asemenea să specificați coloanele în care se înserează date:

```
INSERT INTO nume_tabelă (coloana1, coloana2, ...)
VALUES valoare1, valoare2, ...
```

Exemplu:

Introduceți în *SQL Pane*:

```
INSERT INTO Elevi (Nume, Prenume, Varsta)
VALUES ('Florea', 'Constantin', 99)
```

Rezultat:

Pentru a vedea rezultatul, avem nevoie de un *result set* întors de o instrucțiune **SELECT**. Introduceți interogarea:

```
SELECT * FROM Elevi
```

	IdElev	Nume	Prenume	Varsta
►	1	Pascu	Ioan	18
	2	Nedelcu	Valentin	17
	3	Pop	Andrei	19
	4	Ilie	Ioan	18
	7	Florea	Constantin	99

Comanda UPDATE

Comanda este utilizată pentru modificarea datelor într-o tabelă.

Sintaxă:

```
UPDATE nume_tabelă
SET coloană1 = valoare1, coloană2 = valoare2, ...
WHERE coloană = valoare
```

Cu **UPDATE** puteți modifica valorile pe una sau mai multe coloane, în rândurile selectate cu clauza **WHERE**.

Exemplu:

Introduceți în *SQL Pane*:

```
UPDATE Elevi
SET Varsta = 104
WHERE Prenume = 'Ioan'
```

Rezultat:

Pentru a vizualiza rezultatul introduceți în *SQL Pane* interogarea:

```
SELECT * FROM Elevi
```

	IdElev	Nume	Prenume	Varsta
▶	1	Pascu	Ioan	104
	2	Nedelcu	Valentin	17
	3	Pop	Andrei	19
	4	Ilie	Ioan	104
	7	Florea	Constantin	99

Comanda DELETE

Comanda este utilizată pentru ștergerea rândurilor dintr-o tabelă.

Sintaxă:

```
DELETE FROM nume_tabelă
WHERE coloană = valoare
```

Exemplu:

Introduceți în *SQL Pane* comanda următoare, după care executați-o cu click pe butonul *Execute SQL*:

```
DELETE FROM Elevi
WHERE Varsta > 19
```

Rezultat:

Pentru a vizualiza rezultatul introduceți în *SQL Pane* interogarea:


```
SELECT * FROM Elevi
```

	IdElev	Nume	Prenume	Varsta
▶	2	Nedelcu	Valentin	17
	3	Pop	Andrei	19

Comanda CREATE TABLE

Comanda creează o tabelă într-o bază de date.

Sintaxă:

```
CREATE TABLE nume_tabelă
(
coloana1 tip_dată,
coloana2 tip_dată,
...
)
```

Exemplu:

Lucrăm cu aceeași bază de date, *scoala*. Introduceți în *SQL Pane* comanda următoare, după care executați cu un click pe butonul *Execute SQL*:

```
CREATE TABLE Clase (IDClasa varchar(5), Profil varchar(20),
NrElevi int)
```

Rezultat:

În *Database Explorer* a apărut tabela *Clase*. Acționați click drept asupra numelui clasei și selectați *Open Table Definition*:

	Column Name	Data Type	Allow Nulls
▶	IDClasa	varchar(5)	☒
	Profil	varchar(20)	☒
	NrElevi	int	☒

Comanda ALTER TABLE

Comanda este utilizată pentru a adăuga sau a șterge coloane într-o tabelă.

Sintaxă:

```
ALTER TABLE tabelă
ADD coloană tip
```

```
ALTER TABLE tabelă
DROP COLUMN coloană
```

Exemplu:

Introduceți în *SQL Pane* comanda următoare, după care o executați cu un click pe butonul *Execute SQL*:

```
ALTER TABLE Clase
ADD Diriginte varchar(20)
```

Rezultat:

În *Database Explorer* acționați click drept asupra numelui clasei și selectați *Open Table Definition*:

	Column Name	Data Type	Allow Nulls
	IDClasa	varchar(5)	☒
	Profil	varchar(20)	☒
	NrElevi	int	☒
▶	Diriginte	varchar(20)	☒

Exemplu:

Introduceți în *SQL Pane* comanda pentru ștergerea coloanei *Diriginte*:

```
ALTER TABLE Clase
DROP COLUMN Diriginte
```

Rezultat:

	Column Name	Data Type	Allow Nulls
	IDClasa	varchar(5)	☒
	Profil	varchar(20)	☒
	NrElevi	int	☒

Comanda DROP TABLE

Comanda este utilizată pentru a șterge o tabelă într-o bază de date.

Sintaxă:

```
DROP TABLE tabelă
```

Exemplu:

Vom șterge tabela *clase*. Introduceți în *SQL Pane* comanda următoare, după care o executați cu un click pe butonul *Execute SQL*:

```
DROP TABLE Clase
```

Rezultat:

În *Database Explorer*, click dreapta pe **Tables** și selectați *Refresh*. Se observă că tabela **Clase** a fost ștersă.

Controlul DataGridView

Acest control este vedeta controalelor pentru afișarea datelor. Clasa **DataGridView** permite construirea unei tabele adaptabile. Celulele, rândurile, coloanele se setează prin intermediul proprietăților **Rows** și **Columns**.

Controlul se poate popula manual sau programatic. Ca alternativă, controlul se poate “lega” la o sursă de date prin intermediul proprietăților **DataSource** și **DataMember**. Astfel popularea se face în mod automat.

Prezentăm un exemplu de manipulare simplă a acestui control.

Aplicația DataGridViewExample

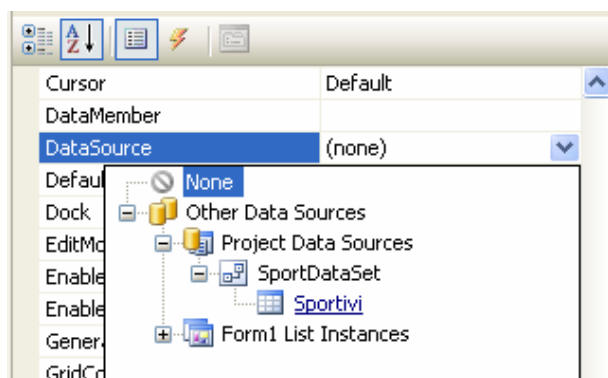
Realizăm un proiect de tip consolă, care conține o bază de date cu o singură tabelă. Informațiile din tabelă se vor afișa într-un control **DataGridView** în momentul pornirii aplicației. Urmăți pașii:

1. Creați un proiect de tip *Windows Forms* cu numele *DataGridViewExample*.
2. Construiți o bază de date cu numele **Sport**. Adăugați în baza de date tabela **Sportivi**. Tabela are următoarea structură:

	Column Name	Data Type	Allow Nulls
►	IdSportiv	int	☐
	Nume	nvarchar(15)	☐
	Prenume	nvarchar(15)	☒
	DataNasterii	datetime	☒
	SportulPracticat	nvarchar(30)	☐

Pentru construirea bazei de date și a tabelii, urmați indicațiile temei: “**Crearea unei baze de date în VCSE**”.

3. Introduceți câteva înregistrări în tabelă. Pentru aceasta, selectați tabela **Sportivi** în *Database Explorer*, acționați click dreapta și alegeți *Show Table Data*.
4. Din **Toolbox**, trageți pe suprafața formei un control de tip **DataGridView** și setați proprietatea **Dock** la valoarea **Fill**.
5. Selectați controlul. În fereastra **Properties**, atribuiți proprietății **DataSource** valoarea **Sportivi**. Se face acest lucru, expandând în panoul din dreapta în mod succesiv nodurile: *Other Data Sources*, *Project Data Sources* și *SportDataSet*.



Astfel, în mod vizual, ați legat controlul la sursa de date.

6. Faceți **Build** și rulați cu **F5**.

La rulare obțineți:



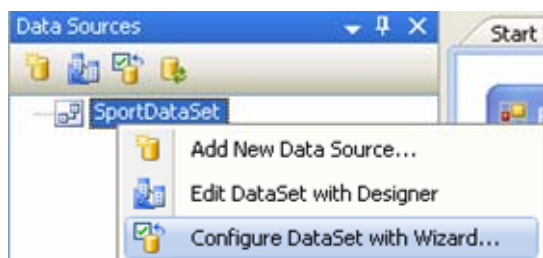
Există și alte variante vizuale pentru popularea controlului.

Varianta a 2-a

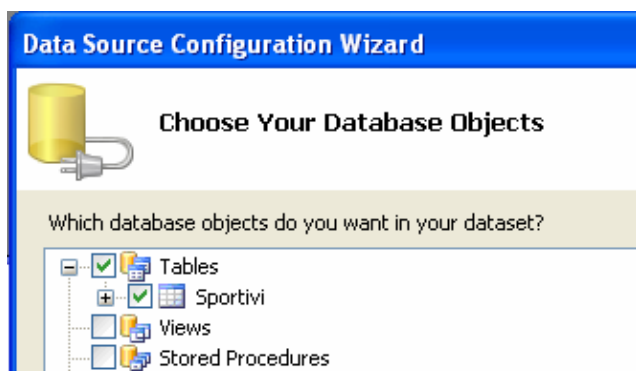
Procedați după cum urmează:

Pașii 1, 2, 3, 4 sunt identici cu cei specificați anterior. Continuați astfel:

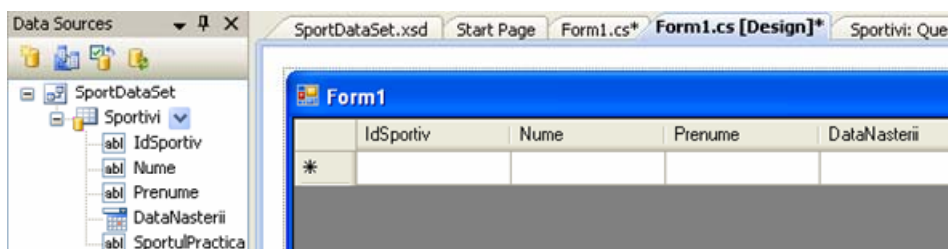
5. În meniul **Data**, selectați **Show Data Sources**. În fereastra **Data Sources**, acționați click dreapta pe elementul **SportDataSet**, apoi selectați **Configure DataSet with Wizard...**



6. În dialogul *Data Source Configuration Wizard*, selectați tabela **Sportivi**, apoi apăsați butonul *Finish*:



7. În fereastra **Data Source**, selectați tabela **Sportivi** și trageți-o pur și simplu cu mouse-ul deasupra controlului **DataGridView**, în *Form Designer*:



8. Faceți **Build** și rulați cu **F5**. Veți obține popularea controlului cu datele din tabelă.

Varianta a 3-a

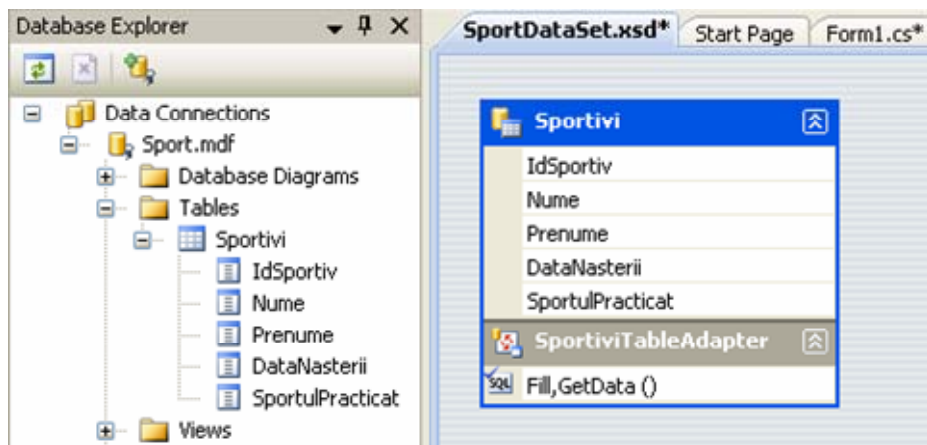
Procedați astfel:

Pașii 1, 2, 3, 4 sunt identici cu cei ai aplicației anterioare. Continuați astfel:

5. Deschideți **DataSet Designer**. Pentru aceasta, aveți două posibilități: în **Solution Explorer** faceți dublu click pe itemul **SportDataSet.xsd** sau în panoul **Data Source**, click dreapta pe itemul **SportDataSet** și selectați *Edit DataSet with Designer*:



6. În **Database Explorer**, selectați tabela **Sportivi** și o trageți cu mouse-ul pe suprafața **DataSet Designer**-ului:



7. Deschideți **Form Designer**-ul cu dublu click pe **Form1.cs** în **Solution Explorer**. În panoul **Data Source**, selectați cu click tabela **Sportivi** și plasați-o pe suprafața controlului **DataGridView**.

8. Faceți **build** și rulați cu **F5**.

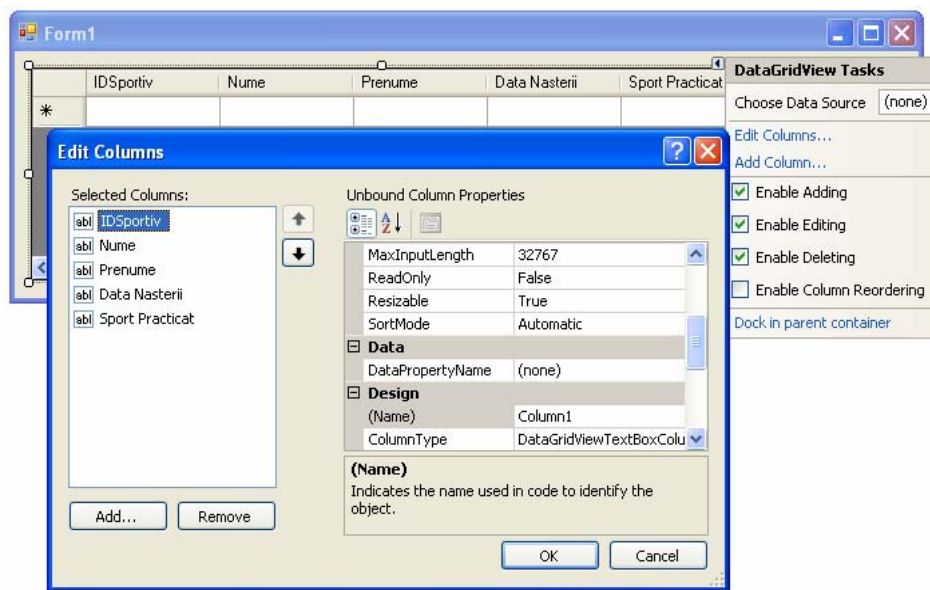
Observații:

- Toate cele trei variante de populare a controlului **DataGridView** realizează o **legare** a controlului la sursa de date printr-un obiect de tip **BindingSource**.
- Despre clasele **DataSet**, **BindingSource**, **Table Adapter**, dar și despre modul în care populați programatic un control **DataGridView**, vom discuta în secțiunile următoare.

Configurarea *design time* a coloanelor unui DataGridView

Coloanele se adaugă și se configurează astfel:

1. În **Form Designer** acționați click pe săgeata din colțul dreapta sus al controlului **DataGridView**, apoi alegeți **Edit Columns**.
2. În dialogul **Edit Columns** adăugați coloane. Pentru fiecare coloană aveți în panoul **Unbound Column Properties**, diverse proprietăți care pot fi setate, între care și textele *header*-elor.



Capitolul 11

Aplicații cu baze de date în modelul conectat

ADO.NET permite aplicațiilor să lucreze atât în *modelul conectat* la baza de date, cât și în *modelul deconectat*.

În modelul conectat se utilizează provider-ii specifici diferitor surse de date. De exemplu, pentru comunicarea cu **SQL Server 2005**, folosim clasele specifice acestui provider: **SqlConnection** și **SqlReader**. Aplicația se conectează, execută interogările, apoi se deconectează.

În modelul deconectat se crează un **dataset** care preia din baza de date tablele necesare. Aplicația se deconectează de la baza de date, execută interogări asupra acestui **dataset**, apoi face *update*-ul necesar în baza de date.

În continuare propunem câteva modele de lucru cu bazele de date, atât în modelul conectat cât și deconectat. Vor fi aplicații de tip consolă, dar și de tip *Windows Forms*.

Vom realiza câteva aplicații care folosesc *provider*-ul pentru **SQL Server 2005**, pentru **OLE DB** și pentru **ODBC**. Reamintiți-vă că **ADO.NET** include *data provideri* pentru diferite surse de date.

Utilizarea provider-ului pentru SQL Server 2005

Realizăm o aplicație este de tip consolă. Reluăm aplicația *DatabaseExample* din capitolul anterior. Aceasta conține baza de date **Scoala**, cu o singură tabelă: **Elevi**. Să presupunem că tabela are doar două rânduri:

	IdElev	Nume	Prenume	Varsta
	1	Nedelcu	Valentin	17
	2	Pop	Andrei	19

Executăm două operații asupra tablei **Elevi**: inserarea unui rând nou și selectarea rândurilor.

Urmați pașii:

1. Deschideți aplicația de tip consolă, *DatabaseExample*. În **Solution Explorer**, redenumiți **Program.cs** astfel: **SQLDataProvider.cs** (click dreapta și *Rename*).
2. Deschideți fișierul **SQLDataProvider.cs** (click dreapta și *View Code*) apoi înlocuiți codul existent cu următorul:

```
using System;  
using System.Data.SqlClient;
```



```
namespace ConsoleApplication1
{
    class SQLDataProvider
    {
        static void Main(string[] args)
        {
            // Construim stringul de conectare
            // (connection string)
            string connString =
                @"server = .\sqlexpress;
                Database=Scoala.mdf;
                trusted_connection=True;
                AttachDbFileName =
C:\DatabaseExample\DatabaseExample\bin\Debug\Scoala.mdf";

            // Pregătim interogările
            string sql1 = @"SELECT * FROM Elevi";
            string sql2 = @"INSERT INTO Elevi
                            (Nume, Prenume, Varsta)
                            VALUES
                            ('Axente', 'Mihai', 20)";

            // Declarăm variabilele conexiune
            // și cititorul de date
            SqlConnection conn = null;
            SqlDataReader reader = null;
            try
            {
                // Deschidem conexiunea
                conn = new SqlConnection(connString);
                conn.Open();

                // Executăm prima interogare
                SqlCommand cmd = new SqlCommand(sql1, conn);
                reader = cmd.ExecuteReader();

                // Închidem reader-ul
                // pentru a putea fi refolosit
                reader.Close();

                // Executăm a doua interogare - inserarea
                // unui rand
                cmd = new SqlCommand(sql2, conn);
                reader = cmd.ExecuteReader();
                reader.Close();

                // Din nou un SELECT pentru a avea
                // un result set
                cmd = new SqlCommand(sql1, conn);
```

```

        reader = cmd.ExecuteReader();

        Console.WriteLine("Baza de date " +
            conn.Database + " se interogheaza cu "
            + cmd.CommandText);

        // Afișăm informațiile din result set
        while (reader.Read())
            Console.WriteLine("{0} {1} {2}",
                reader[1], reader["Prenume"],
                reader[3]);
        Console.WriteLine();
    }
    catch (Exception e)
    {
        Console.WriteLine("Eroare: " + e);
    }
    finally
    {
        // Conexiunea se închide indiferent dacă se
        // aruncă sau nu excepții
        reader.Close();
        conn.Close();
        conn.Dispose();
    }
} //Main()
}

```

3. Faceți **Build** și rulați cu **Ctrl + F5** (Modul depanare)

```

C:\WINDOWS\system32\cmd.exe
Baza de date Scoala.mdf se interogheaza cu SELECT * FROM Elevi
Nedelcu Valentin 17
Pop Andrei 19
Axente Mihai 20
Press any key to continue . . . _

```

Rețineți etapele comunicării cu baza de date:

Pașii următori sunt aceeași indiferent de providerul cu care lucrați:

- **Se include spațiul de nume `System.Data.SqlClient`.** Clasele acestuia definesc providerul pentru **MS SQL Server**. Pentru alți provideri, se înlocuiește prefixul `sql` cu cel corespunzător.
- **Se deschide o conexiune cu baza de date**

- Clasa `SqlConnection` permite obținerea unei conexiuni la server și deschiderea conexiunii cu metoda `Open()`.
- Constructorul acestei clase, `SqlConnection(connectionString)`, primește stringul de conectare.
- Proprietățile `Database` și `CommandText` ale clasei `SqlConnection` returnează baza de date, respectiv interogarea curentă.

▪ Executarea interogării

- Ca să executați o interogare, trebuie mai întâi să construiți un obiect de tip `SqlCommand`.
- Constructorul `SqlCommand(sql, conn)` primește ca argumente interogarea care trebuie executată și conexiunea prin care va lucra.
- Executarea propriuzisă a interogării, se face cu metoda `ExecuteReader()`. Aceasta face două lucruri: construiește un obiect (`reader`) de tip `SqlDataReader` și apoi execută interogarea.
- Metoda `ExecuteReader()` returnează un *result set* care poate fi vid pentru comenzi *non-query*, cum este `INSERT INTO`, dar nevid pentru `SELECT`. Acest *result set* se poate imagina ca o tabelă în care se regăsesc rezultatele interogării.

▪ Accesarea rezultatelor interogării

- Ca să analizați setul de rezultate, parcurgeți setul în buclă cu metoda `Read()`, care la fiecare apel, avansează `reader`-ul la următoarea linie.
- Datele de pe linie se accesează indexat. `reader[0]` este valoarea corespunzătoare primei coloane, `reader[1]` corespunde celei de a doua coloane, etc. Un alt mod de accesare este prin numele coloanei respective, așa cum am arătat în aplicație: `reader["Prenume"]` este valoarea aflată pe linia curentă și coloana `Prenume`. Se preferă accesarea prin indice pentru că este mai rapidă.

▪ Protecția împotriva excepțiilor

- Deschiderea conexiunii (`new SqlConnection(connectionString)`) trebuie protejată într-un bloc `try ...catch`. pentru tratarea excepțiilor. Astfel ne asigurăm că blocul `finally`, în care eliberăm resursele va fi executat. Se face acest lucru, deoarece **ADO.NET** aruncă excepții la erorile de baze de date. Este foarte simplu să aveți erori. De exemplu, greșiți ceva în stringul de conectare, sau baza de date este blocată de către o altă aplicație.

Blocul `using` – o tehnică sigură

Clasele `Connection`, așa cum este `SqlConnection` moștenesc interfața `IDisposable`. Aceasta înseamnă că un obiect de acest tip poate referi resurse negestionate în mod automat de **.NET** (*unmanaged resources*). Poate fi vorba de surse de date, fișiere pe disc și altele. Toate clasele care moștenesc această interfață au metoda `Dispose()`, care trebuie invocată de către programator pentru eliberarea acestor resurse. În mod obișnuit, se apelează în blocul `finally`.

O metodă mai elegantă pentru tratarea excepțiilor este utilizarea *blocului* `using`. Nu confundați cu *directiva* `using`. Obiectele `IDisposable` se declară și se instanțiază în declarația `using`. În acest fel, vă asigurați că metoda `Dispose()` este apelată chiar în cazul în care se aruncă excepții.

Iată cum se poate scrie codul cuprins în blocul `try ... catch` în programul de mai sus:

```
using (conn = new SqlConnection(connString) )
{
    conn.Open();

    SqlCommand cmd = new SqlCommand(sql1, conn);
    reader = cmd.ExecuteReader();

    // cod ...

    while (reader.Read())
        Console.WriteLine("{0} {1} {2}",
            reader[1], reader["Prenume"],
            reader[3]);
    reader.Close();
}
```

Observați că nu mai este nevoie nici măcar să închideți conexiunea, deoarece de acest lucru se ocupă acum C#.

Ce reprezintă *connection string* ?

Stringul de conectare este un string care specifică informații despre o sursă de date, precum și mijloacele de conectare la aceasta. Se transmite *data provider*-ului pentru inițierea conexiunii. Conține atribute cum sunt: numele driverului, numele serverului de baze de date, numele bazei de date, informații de securitate cum ar fi user și parolă și altele.

În aplicația anterioară, `server = .\sqlexpress`; specifică faptul că se utilizează o instanță a *SQL Server 2005 Express Edition*, aflat pe mașina locală (prin `.\` se precizează că e vorba de *localhost*).

Se indică deasemenea numele bazei de date prin `Database=Scoala.mdf`; Mai departe, `trusted_connection=True`; permite userului curent autentificat să facă o conexiune.

Atributul **AttachDbFileName** specifică calea de director pe discul local până la baza de date. Procesul prin care aduceți la cunoștință serverului **SQL Server** de existența unei baze de date se numește **atașarea** bazei de date.

Ajunși în acest punct, precizăm că baza de date se compune din două fișiere strâns legate: fișierul cu extensia (.mdf) și cel cu extensia (.ldf). Primul depozitează datele, iar al doilea este fișier de **log**, care conține informații despre tranzacții.

Există mai multe atribute care pot fi incluse în stringul de conectare. De multe ori, alegerea lor corectă este o problemă. Din fericire, există destulă documentație și chiar site-uri dedicate. Recomandăm unul dintre acestea: **www.connectionstrings.com**.

Utilizarea provider-ului pentru OLE DB

Cu ajutorul furnizorului de date **OLE DB**, se pot accesa date depozitate în diverse tipuri de baze de date, de la **MS Access** până la **Oracle**. Vă puteți conecta inclusiv la **SQL Server 2005**, însă acesta are propriul lui provider, așa cum ați văzut, care este mai rapid. Acest data provider este definit în spațiul de nume **System.Data.OleDb**. Cele mai importante clase ale sale sunt:

OleDbCommand	Execută interogări sau proceduri stocate
OleDbConnection	Reprezintă o conexiune la o sursă de date
OleDbDataReader	Permite citirea numai înainte, a unui stream de rânduri dintr-o sursă de date
OleDbDataAdapter	Reprezintă o legătură între sursa de date și <i>dataset</i> , folosită pentru obținerea și salvarea datelor.

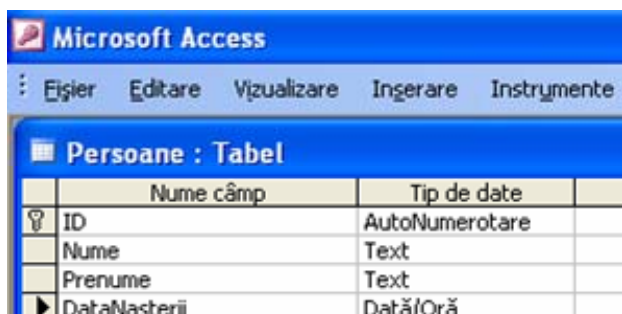
Este ușor de remarcat similitudinea între clasele furnizorului **OLE DB** și cele ale **SQL Server 2005**. Metodele acestor clase sunt de asemenea similare.

Aplicația OleDbProviderWinApp

Vom accesa o bază de date *Microsoft Access 2003* cu ajutorul providerului **Microsoft.Jet.OLEDB.4.0**. Acest furnizor este specific pentru bazele de date **Access**.

Urmați procedura:

1. Deschideți *Microsoft Access* și creați baza de date Access **Persoane.mdb**, cu următoarea schemă:



	Nume câmp	Tip de date
ID		AutoNumerotare
Nume		Text
Prenume		Text
DataNasterii		Data/Oră

Completați câteva rânduri cu date în tabelă. Salvați baza de date **Persoane.mdb** în folderul **C:\teste**.

2. Creați în **VCSE** un nou proiect de tip *Windows Forms*, cu numele *OleDbProviderWinApp*.
3. Pe suprafața formei plasați un control de tip **DataGridView**.
4. În fișierul *Form1.cs* introduceți directiva: **using System.Data.OleDb;**
5. Intenționăm să populăm controlul la încărcarea formei. Prin urmare, tratăm evenimentul **Load** pentru formă. Acționați dublu click pe suprafața formei. În corpul *handler*-ului, introduceți codul marcat cu **Bold**:

```
private void Form1_Load(object sender, EventArgs e)
{
    // Setăm numărul coloanelor și textul header-elor
    dataGridView1.ColumnCount = 4;
    dataGridView1.Columns[0].HeaderText = "ID-ul";
    dataGridView1.Columns[1].HeaderText = "Numele";
    dataGridView1.Columns[2].HeaderText = "Prenumele";
    dataGridView1.Columns[3].HeaderText = "Data Nasterii";

    // Stringul de conectare
    string connString =
        @ "Provider=Microsoft.Jet.OLEDB.4.0;Data Source=
        C:\teste\Persoane.mdb";

    // Stringul de interogare
    string sql = @ "select * from Persoane";

    // Declarăm variabilele conexiune și data reader
    OleDbConnection conn = null;
    OleDbDataReader reader = null;

    // Creăm conexiunea
    using (conn = new OleDbConnection(connString))
    {

```

```

conn.Open(); // Deschidem conexiunea

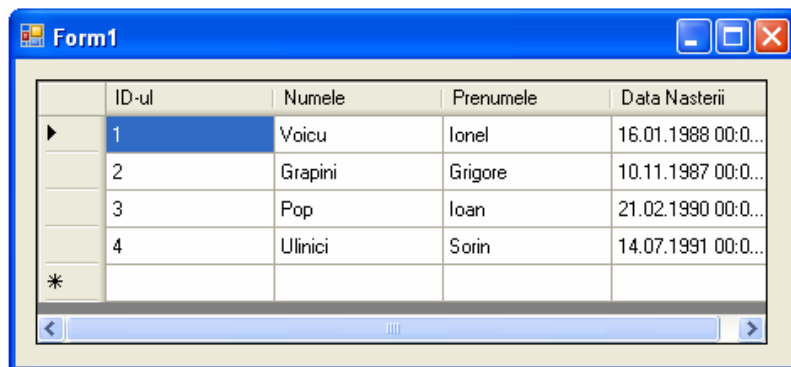
// Se execută interogarea
OleDbCommand cmd = new OleDbCommand(sql, conn);
reader = cmd.ExecuteReader();

// Tablou de stringuri. Reține valorile unui rând
// din tabelă
string[] r = null;
// Metoda Read() avansează reader la un rând nou
while ( reader.Read() )
{
    r = new string[4]{ reader[0].ToString(),
                      reader[1].ToString(),
                      reader[2].ToString(),
                      reader[3].ToString() };
    // Adăugăm rândul în control
    dataGridView1.Rows.Add(r);
}
reader.Close();
} // using
}

```

6. Compilați și executați cu **F5**.

Veți avea ceva asemănător:



De reținut:

- Coloanele, împreună cu textul *header*-elor se pot adăuga *design time*, însă am dorit să arătăm cum le setați programatic.
- Stringul de conectare are două atribute: **Provider** și **Data Source**. Primul indică providerul **OLE DB** pentru bazele de date **Access**, iar al doilea indică sursa de date și locația sa.

- Rândurile se adaugă în control cu metoda `Add()`: `dataGridView1.Rows.Add(r);`, unde `r` este un șir de stringuri care reține valorile *reader*-ului.
- Declarația `using` asigură eliberarea resurselor alocate atât pentru o ieșire normală din bloc, cât și la aruncarea unei excepții. Observați că nu este nevoie să mai închideți conexiunea, ci doar *reader*-ul.
- Remarcăm similitudinea codului și a tipurilor utilizate pentru *provider*-ii **SQL Server** și **OLE DB**.

Utilizarea provider-ului pentru ODBC

ADO.NET include furnizorul de date **ODBC** în spațiul de nume **System.Data.Odbc**. Este prima tehnologie Microsoft de accesare a surselor de date, fiind încă folosită pentru surse de date care nu au provider **OLE DB**. Poate fi folosită cu aproape oricare tip de sursă de date. **ODBC** comunică cu sursa de date prin intermediul *driver*-ului sursei de date, deci apare un nivel intermediar și aceasta face ca viteza de lucru să fie mai mică în raport cu alți provideri.

Cele mai importante clase asociate provider-ului **ODBC** sunt ușor de recunoscut, după prefixul **Odbc**: **OdbcCommand**, **OdbcConnection**, **OdbcDataAdaptor**, **OdbcDataReader**. Semnificația acestora este aceeași cu a claselor similare ale celorlalți provideri.

Vom realiza două aplicații. Prima aplicație se conectează la un fișier **Excel** iar a doua la un server **MySQL**.

Folosim în mod intenționat metode diferite de conectare **ODBC** pentru cele două aplicații: în caul primei aplicații includem în stringul de conectare informații despre *driver*-ul **Excel**, iar pentru a doua aplicație realizăm o sursă de date cu un utilitar integrat sistemului de operare. Precizăm că ambele metode pot fi folosite și lucrează la fel de bine pentru aplicațiile cu provider **ODBC**.

Aplicația *OdbcExcelExample*

Vom realiza o aplicație de tip *Windows Forms* care se conectează prin providerul **ODBC** la un fișier **Excel 2003**, în care pe prima foaie este un tabel. Datele se citesc din sursa de date și se afișează într-un control **DataGridView**. Urmăți indicațiile:

1. Deschideți *Microsoft Excel* și scrieți tabelul:

	A	B	C	D	E	F
1	IdProdus	Denumire	Pret	Cantitate		
2	1	Paine	2	200		
3	2	Lapte	3	23		
4	3	Iaurt	4	45		
5	4	Cola	5	21		
6						

Salvați fișierul cu numele **Produse.xls** în folder-ul **C:\teste**.

2. Creați în **VCSE** un nou proiect de tip *Windows Forms*, cu numele *OdbcExcelExample*.
3. Pe suprafața formei plasați un control de tip **DataGridView**.
4. În fișierul *Form1.cs* introduceți directiva: `using System.Data.Odbc;`
5. Dorim să populăm controlul la încărcarea formei. Deci tratăm evenimentul **Load** pentru formă. Acționați dublu click pe suprafața formei. În corpul *handler*-ului, introduceți codul marcat cu **Bold**:

```
private void Form1_Load(object sender, EventArgs e)
{
    // Setăm numărul coloanelor și textul header-elor
    dataGridView1.ColumnCount = 4;
    dataGridView1.Columns[0].HeaderText = "ID-ul";
    dataGridView1.Columns[1].HeaderText = "Nume";
    dataGridView1.Columns[2].HeaderText = "Pret";
    dataGridView1.Columns[3].HeaderText = "Nr Produse";

    // Stringul de conectare
    string connString =
        @"Driver={Microsoft Excel Driver (*.xls)};
        DriverId=790;
        Dbq=C:\teste\Produse.xls;
DefaultDir=C:\OdbcExcelExample\OdbcExcelExample\bin\Debug";

    // Stringul de interogare
    // [Foaie1$] poate fi [sheet1$] după caz
    string sql = @"select * from [Foaie1$]";

    // Declarăm variabilele conexiune și data reader
    OdbcConnection conn = null;
    OdbcDataReader reader = null;

    // Creăm conexiunea
    using (conn = new OdbcConnection(connString))
    {
        conn.Open(); // Deschidem conexiunea
        // Se execută interogarea
        OdbcCommand cmd = new OdbcCommand(sql, conn);
        reader = cmd.ExecuteReader();

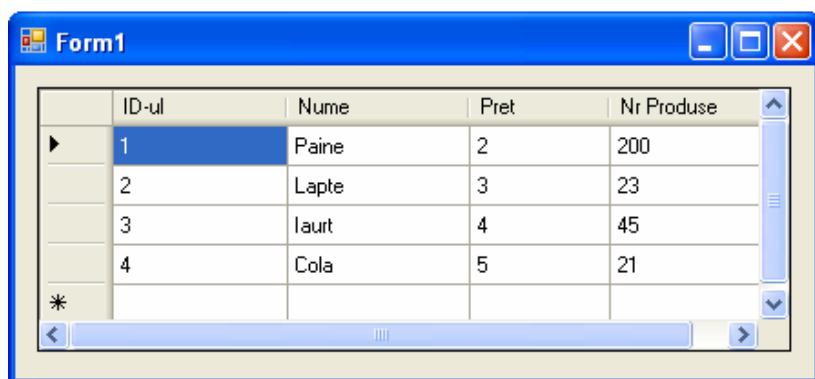
        // Tabloul de stringuri. Reține valorile unui rând
        // din tabelă
        string[] r = null;
    }
}
```

```
// Metoda Read() avansează reader la un rând nou
while (reader.Read())
{
    r = new string[4]{ reader[0].ToString(),
                      reader[1].ToString(),
                      reader[2].ToString(),
                      reader[3].ToString() };

    dataGridView1.Rows.Add(r);
}
reader.Close();
} // using
}
```

6. Compilați și rulați cu **F5**.

La rulare obțineți:



Observații:

- Stringul de conectare precizează driver-ul *Excel* și ID-ul său. **Dbq** indică sursa de date și calea de director, iar **DefaultDir** este directorul implicit.
- Codul C# este similar cu cel pentru provider-ii **OLE DB** sau **SQL Server**.

Aplicația *OdbcMySQL*

Scopul acestei aplicații este acela de a arăta modul în care vă puteți conecta la o bază de date **MySQL** cu C#, utilizând provider-ul **ODBC**.

Diferența esențială față de modul de conectare al aplicației anterioare constă în faptul că vom construi un **Data Source Name (DSN)** cu ajutorul unui utilitar windows. Acest **DSN** se utilizează apoi în stringul de conectare.

Separăm tema în trei secțiuni:

A) Instalarea serverului **MySQL 5.0** și crearea unei baze de date **MySQL**

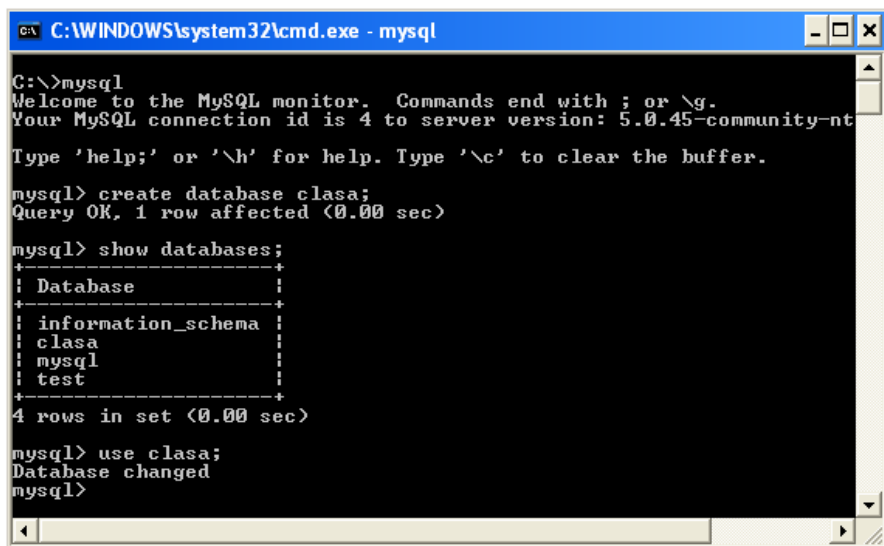
Presupunând că nu aveți **MySQL** instalat și că nu ați mai lucrat cu această bază de date, procedați după cum urmează:

- Intrați pe site-ul **www.mysql.com**. La secțiunea *download*, descărcați fișierul **mysql-essential-5.0.45-win32.msi**.
- Wizard-ul de instalare vă cere anumite informații, pe care le completați selectând următoarele opțiuni (bifați *check box*-urile):
 - ✓ “Configure the MySQL Server now”.
 - ✓ “Install as Windows Service”.
 - ✓ “Launch the MySQL Server Automatically”.
 - ✓ “Include Bin Directory in Windows PATH”.
 - ✓ “Modify Security Settings”. Introduceți parola de **root** în cele două *text box*-uri.
 - ✓ “Enable root access from remote machines”.
 - ✓ “Create An Anonymous Account”.

În felul acesta, ați instalat **MySQL Server 5.0** ca **serviciu Windows** care startează în mod automat la pornirea calculatorului. Aveți un cont de administrator (**root**) și un cont **Anonymous**. Acesta din urmă vă permite să vă conectați fără să introduceți user sau parolă. Este bine să aveți un asemenea user numai în perioada de învățare, deoarece în practică creează probleme de securitate.

Acum serverul e lansat. Îl putem folosi. Ne conectăm cu clientul **mysql.exe** la serverul **MySQL** pentru a crea o bază de date cu o tabelă. Uurmați indicațiile :

1. În *taskbar*-ul sistemului de operare, click *Start->Run*. Introduceți **cmd** și apăsați **OK**.
2. Tastați la promptul sistemului: **mysql**. În acest fel v-ați conectat ca user *Anonymous*, deoarece nu ați specificat user și parolă.
3. Tastați **create database clasa**;
Ați construit astfel o bază de date cu numele **clasa**.
4. Tastați **show databases**;
Vizualizați astfel bazele de date preinstalate și noua bază de date creată.
5. Introduceți comanda **use clasa**; pentru a utiliza această bază de date.



```
C:\WINDOWS\system32\cmd.exe - mysql

C:\>mysql
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 4 to server version: 5.0.45-community-nt

Type 'help;' or '\h' for help. Type '\c' to clear the buffer.

mysql> create database clasa;
Query OK, 1 row affected (0.00 sec)

mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| clasa      |
| mysql     |
| test      |
+-----+
4 rows in set (0.00 sec)

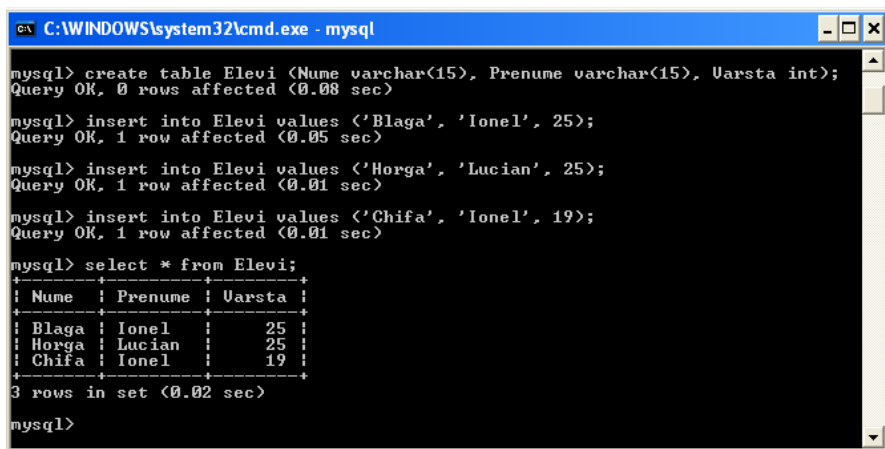
mysql> use clasa;
Database changed
mysql>
```

6. Creăm o tabelă. Introduceți comanda:

```
create table Elevi  
(Nume varchar(15), Prenume varchar(15), Varsta int);
```

7. Inserăm câteva rânduri în tabelă:

```
insert into Elevi values ('Blaga', 'Ionel', 25);  
insert into Elevi values ('Horga', 'Lucian', 25);  
insert into Elevi values ('Chifa', 'Ionel', 19);
```



```
mysql> create table Elevi (Nume varchar(15), Prenume varchar(15), Varsta int);  
Query OK, 0 rows affected (0.08 sec)  
  
mysql> insert into Elevi values ('Blaga', 'Ionel', 25);  
Query OK, 1 row affected (0.05 sec)  
  
mysql> insert into Elevi values ('Horga', 'Lucian', 25);  
Query OK, 1 row affected (0.01 sec)  
  
mysql> insert into Elevi values ('Chifa', 'Ionel', 19);  
Query OK, 1 row affected (0.01 sec)  
  
mysql> select * from Elevi;  
+-----+-----+-----+  
| Nume | Prenume | Varsta |  
+-----+-----+-----+  
| Blaga | Ionel | 25 |  
| Horga | Lucian | 25 |  
| Chifa | Ionel | 19 |  
+-----+-----+-----+  
3 rows in set (0.02 sec)  
  
mysql>
```

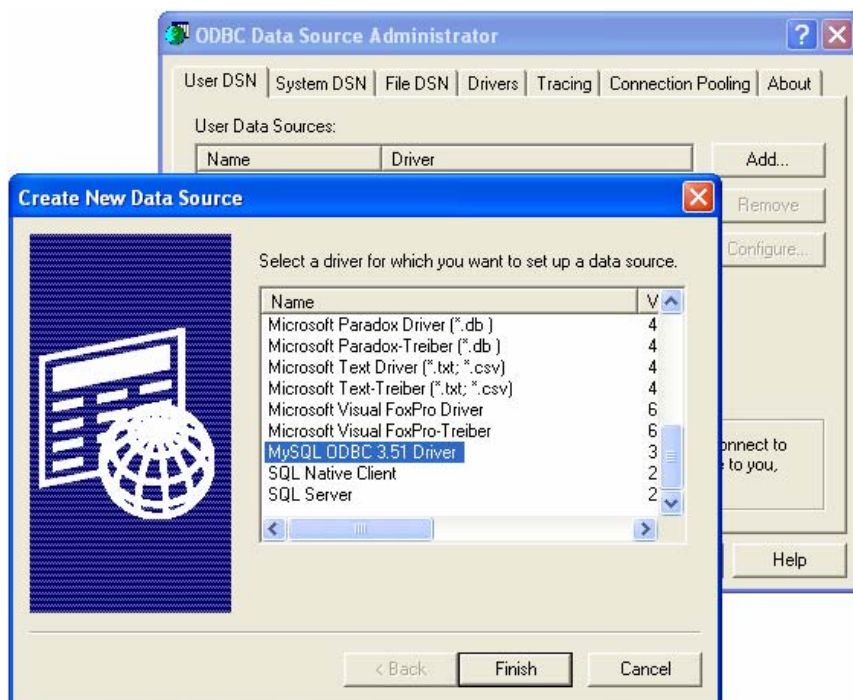
8. Acum avem baza de date **Clase** cu o tabelă numită **Elevi**. Tabela conține trei rânduri. Vă deconectați de la server cu comanda **quit**.

B) Crearea unui *Data Source Name* (DSN)

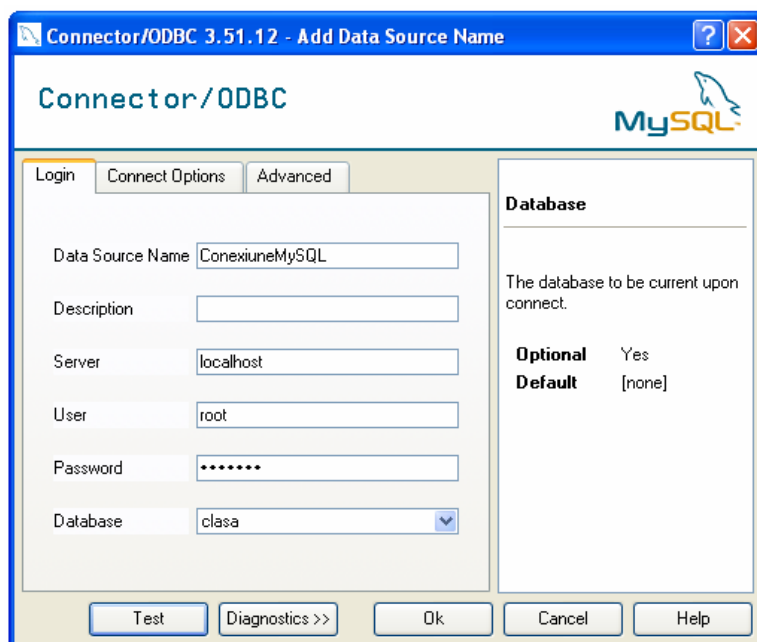
Informațiile necesare conectării la o sursă de date cu provider **ODBC** se pot centraliza cu utilitarul *ODBC Data Source Administrator*. Ceea ce se obține se numește *Data Source Name* (DSN).

Procedați astfel:

1. În bara de task-uri a sistemului de operare, accesați *Start->All Programs->Control Panel*. Dublu click pe iconul *Administrative Tools*, apoi lansați *Data Sources(ODBC)*. Se deschide dialogul *ODBC Data Source Administrator*.
2. În pagina *User DSN*, apăsați butonul *Add*. Se deschide dialogul *Create New Data Source*. Selectați din listă *driver-ul MySQL*, apoi click pe butonul *Finish*:



3. Completați formularul de mai jos. În câmpul *Data Source Name*, introduceți *ConexiuneMySQL*. Dacă doriți să vă conectați ca user **Anonymous**, nu este necesar să completați câmpurile **User** și **Password**.



Dacă serverul **MySQL** nu este pe mașina locală, atunci în câmpul *Server* treceți **IP-ul** *host*-ului respectiv. De exemplu: **80.97.66.192**

4. Apăsați butonul **OK** pe dialogul de mai sus. Ați creat deja un **DSN** cu numele *ConexiuneMySQL*, pe care îl pasăm stringului de conectare în aplicația C# care urmează.

C) Crearea aplicației C#

Aplicația de tip *Windows Forms* se conectează la serverul **MySQL** de pe mașina locală și accesează baza de date *clasa*, creată la pașii anteriori. Apoi afișează într-un control **DataGridView** datele din tabela *elevi*. Procedați astfel:

1. Creați în **VCSE** un nou proiect de tip *Windows Forms*, cu numele *OdbcMySQL*.
2. Pe suprafața formei plasați un control de tip **DataGridView**.
3. În fișierul *Form1.cs* introduceți directiva: **using System.Data.Odbc;**
4. Controlul se va popula la încărcarea formei. Prin urmare, tratăm evenimentul **Load** pentru formă. Acționați dublu click pe suprafața formei. În corpul *handler*-ului, introduceți codul marcat cu **Bold**:

```
private void Form1_Load(object sender, EventArgs e)
{
    dataGridView1.ColumnCount = 3;
    dataGridView1.Columns[0].HeaderText = "Numele";
    dataGridView1.Columns[1].HeaderText = "Prenumele";
    dataGridView1.Columns[2].HeaderText = "Varsta";

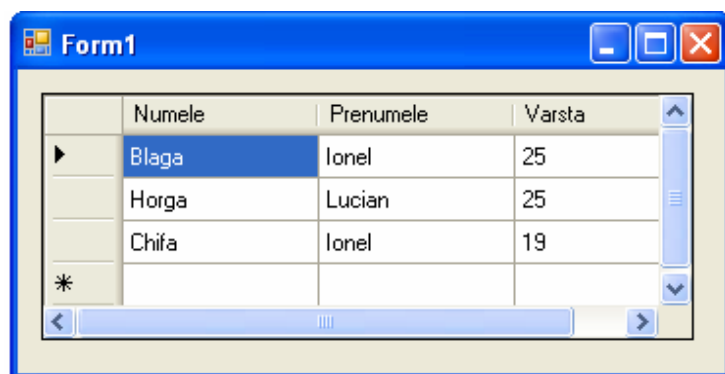
    string connString =
        "dsn=ConexiuneMySQL;User=root;Password=iuliuta";
    string sql = @"select * from elevi";
    OdbcConnection conn = null;
    OdbcDataReader reader = null;

    using (conn = new OdbcConnection(connString))
    {
        conn.Open();
        // Se execută interogarea
        OdbcCommand cmd = new OdbcCommand(sql, conn);
        using (reader = cmd.ExecuteReader())
        {
            string[] r = null;
            while (reader.Read())
            {
```

```
        r = new string[3]{ reader[0].ToString(),
                           reader[1].ToString(),
                           reader[2].ToString() };
        dataGridView1.Rows.Add(r);
    }
}
}
```

5. Compilați și executați cu **F5**.

La rulare, controlul **DataGridView** afișează datele din tabela **Elevi**, obținute în urma interogării **SELECT**:



Precizări:

- **MySQL** oferă propriul provider **ODBC**, numit **MyODBC**. Este driver-ul pe care l-am utilizat la crearea **DSN**.
- **ADO.NET** are un provider dedicat pentru **MySQL**, definit în spațiul de nume **MySql.Data.MySqlClient**.
- Nu am comentat codul, deoarece este similar cu cel al aplicațiilor anterioare cu provider ODBC.

Capitolul 12

Aplicații cu baze de date în modelul deconectat

Dacă ceea ce doriți este doar să citiți și să afișați mari cantități de date, atunci un obiect de tip **DataReader** este foarte potrivit. În cazul în care trebuie să efectuați mai multe operații asupra acestor date și la final să actualizați baza de date, atunci un *reader* nu este cea mai potrivită alegere, deoarece el parcurge datele o singură dată, doar înainte și pentru fiecare operație este nevoie să construiți un alt obiect de tip **DataReader**. Ceea ce trebuie să faceți în aceste situații, este să folosiți un *dataset*.

Un *dataset* este un obiect de tip **System.Data.DataSet**. *Dataset*-urile sunt capabile să depoziteze mari cantități de date în memoria internă a calculatorului. Un *dataset* vă permite să preluați date din sursa de date, să închideți conexiunea cu baza de date, apoi să prelucrați datele *offline*, adică în modul deconectat de la sursa de date. La final, când prelucrarea datelor s-a încheiat, baza de date se actualizează.

Un *dataset* memorează datele într-o colecție de tabele (obiecte de tip **DataTable**) și de relații între tabele.

În momentul în care construiți un obiect de tip **DataSet**, acesta nu conține date. Ca să primească date este nevoie de un obiect de tip **DataAdapter**. Un **DataAdapter** este parte a furnizorului de date, în vreme ce un *dataset* este exterior provider-ului. Fiecare provider are propriul său adaptor de date. Spre exemplu, adaptorul provider-ului pentru **SQL Server** este **SqlDataAdapter**.

Construirea și utilizarea dataset-urilor

Ca să creați o instanță a clasei **DataSet**, utilizați unul dintre constructorii clasei:

```
DataSet ds = new DataSet();  
DataSet ds = new DataSet("numeDataSet");
```

Al doilea constructor precizează numele dataset-ului. Primul constructor, atribuie *dataset*-ului numele **NewDataSet**.

În *dataset*, datele sunt organizate într-un set de tabele. Fiecare tabelă se memorează într-un obiect de tip **DataTable**. Fiecare obiect **DataTable** consistă din obiecte de tip **DataRow** și **DataColumn**, care păstrează datele.

Să presupunem că un *dataset* primește datele de la un **SqlDataAdapter**. Secvența standard de cod care construiește și umple *dataset*-ul este :

```
// Creează adaptorul de date  
SqlDataAdapter da = new SqlDataAdapter(sql, conn);
```



```
// Creează dataset-ul
DataSet ds = new DataSet();

// Încarcă dataset-ul cu tabela elevi. Dacă există și alte
// tabele în sursa de date, acelea nu se încarcă
da.Fill(ds, "elevi");
```

Așadar, constructorul clasei `SqlDataAdapter` primește stringul de conectare și conexiunea (referința la un obiect de tip `SqlConnection`), iar metoda `Fill` a adaptorului umple *dataset*-ul cu tabela "elevi".

Există încă două versiuni ale constructorului clasei `SqlDataAdapter`. Primul nu are parametri, iar al doilea primește un obiect de tip `SqlCommand`:

```
SqlDataAdapter da = new SqlDataAdapter();
SqlDataAdapter da = new SqlDataAdapter(cmd);
```

Pentru a încărca în *dataset* toate tabelele din baza de date, utilizați altă versiune a metodei `Fill`:

```
da.Fill(ds);
```

Accesarea tabelelor într-un dataset

Un *dataset* poate să conțină mai multe tabele. Acestea sunt obiecte de tip `DataTable` care se pot accesa indexat, începând cu indexul `0`. de exemplu :

```
// Obține tabela a treia din dataset.
DataTable dt = ds.Tables[2];
```

Altă variantă este accesarea prin numele tabelii:

```
// Obține tabela produse
DataTable dt = ds.Tables["produse"];
```

Accesarea rândurilor și coloanelor într-o tabelă

Rândurile unui obiect `DataTable` se obțin din proprietatea `Rows`, care suportă indexarea :

```
// Obține al patrulea rând al tablelei
DataRow r = dt.Rows[3];
```

Colecția de coloane a unui obiect `DataTable` se poate obține din proprietatea `Columns`. Proprietatea suportă operatorul de indexare, cu pornire de la `0`:

```
// Obține coloana a 4-a
DataColumn col = dt.Columns[3];

// Obține coloana "Nume"
DataColumn col = dt.Columns["Nume"];
```

Accesarea valorilor dintr-o tabelă a unui dataset

Se utilizează proprietățile **Rows** și **Columns** ale clasei **DataTable**:

```
// Obținem tabela dorită
DataTable dt = ds.Tables["produse"] ;

// Pentru fiecare rând din colecție (din tabela produse)
foreach (DataRow row in dt.Rows)
{
    // Pentru fiecare coloană
    foreach (DataColumn col in dt.Columns)
        Console.Write(row[col]); // Accesare indexată a
                                   // valorii unei celule
    Console.WriteLine();
}
```

Propagarea schimbărilor din dataset spre baza de date

Am văzut că metoda **Fill()** a clasei **TableAdaptor** umple un *dataset* cu datele preluate din baza de date. Dacă metoda este apelată după ce s-a închis conexiunea, atunci **Fill()** deschide conexiunea, transferă datele în *dataset* și apoi închide conexiunea. Dacă conexiunea era deschisă înainte de apelul **Fill()**, atunci metoda lasă conexiunea deschisă.

Modificările operate asupra datelor unui *dataset*, se propagă înapoi în baza de date la apelul metodei **Update()** a clasei **TableAdaptor**. Prezentăm două versiuni ale acestei metode. Ambele returnează numărul de rânduri updatate cu succes.

Prima versiune actualizează baza de date operând toate modificările care s-au făcut în *dataset*:

```
int Update(DataSet ds);
```

A doua versiune actualizează baza de date operând toate modificările care s-au făcut în tabela transmisă ca argument:

```
int Update(DataTable dt)
```

Actualizarea bazei de date se produce conform scenariului de mai jos:

```
// Creează adaptorul de date
SqlDataAdapter da = new SqlDataAdapter(sql, conn);

// Creează dataset-ul
DataSet ds = new DataSet();

// Umple dataset-ul cu date din baza de date
da.Fill(ds);

// Se modifică datele în dataset
// ...

// Se propagă modificările în baza de date
ta.Update(ds);
```

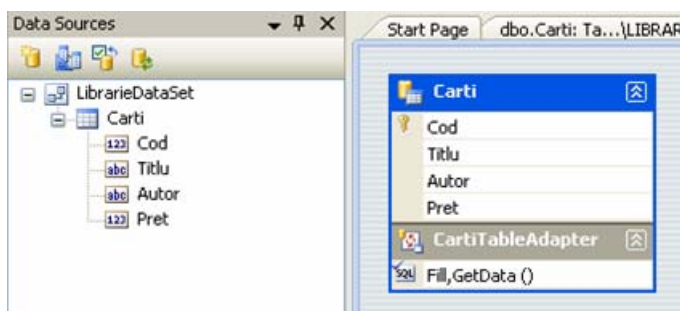
Aplicația *UpdateDatabase*

Aplicația de tip *Windows Forms* accesează o bază de date **SQL Server 2005** și afișează într-un control **DataGridView** datele dintr-o tabelă. La apăsarea unui buton, modificările operate de utilizator în celulele gridului se propagă în baza de date.

1. Creați în **VCSE** un nou proiect de tip *Windows Forms*, cu numele *UpdateDatabase*.
2. Pe suprafața formei plasați un control de tip **DataGridView** și un buton cu textul *Salveaza* și o etichetă cu textul *0 randuri afectate*.
3. Adăugați proiectului o bază de date cu numele **Librarie**. În acest scop, urmați indicațiile subcapitolului **“Crearea unei baze de date în VCSE”**. *Wizardul* propune numele **LibrarieDataset** pentru clasa pe care o generează. Această clasă moștenește **DataSet**, așadar **Este Un dataset**.
4. Adăugați bazei de date **Librarie** tabela **Carti**, cu următoarea structură:

	Column Name	Data Type	Allow Nulls
	Cod	int	☐
	Titlu	nvarchar(50)	☐
	Autor	nvarchar(20)	☒
	Pret	int	☒

5. În fereastra **Data Source**, acționați click dreapta pe tabela **Carti** și selectați *Edit DataSet with Designer*. După ce *designer*-ul se deschide, trageți tabela **Carti** cu mouse-ul, din fereastra **Data Source** pe suprafața designerului:



Prin această acțiune am cerut ajutorul mediului integrat, care a generat pentru noi o clasă **CartiTableAdapter**. Această clasă conține un obiect de tip **SqlDataAdapter**, un obiect de tip **SqlConnection** și un obiect de tip **SqlCommand**, astfel încât nu mai trebuie să ne preocupăm de deschiderea manuală a unei conexiuni la server.

6. Deschideți în *Editorul de Cod* fișierul *Form1.cs*. În clasa **Form1** declarați câmpurile:

```
// Declarăm o referință la dataset
private LibrarieDataSet libDataSet = null;

// Referință la un obiect de tip CartiTableAdapter
private LibrarieDataSetTableAdapters.CartiTableAdapter
    ta = null;
```

7. Populăm controlul **DataGridView** la încărcarea formei. Deci tratăm evenimentul **Load** pentru formă. Acționați dublu click pe suprafața formei. În corpul *handler*-ului, introduceți codul marcat cu **Bold**:

```
private void Form1_Load(object sender, EventArgs e)
{
    // Instanțiem adaptorul de date. Observați că este
    // necesar să specificăm spațiul de nume în care este
    // definită clasa CartiTableAdaptor
    ta = new
        LibrarieDataSetTableAdapters.CartiTableAdapter();

    // Creăm un dataset
    libDataSet = new LibrarieDataSet();

    // Umplem dataset-ul cu datele din tabela Carti
    ta.Fill(libDataSet.Carti);

    // bs permite legarea controlului dataGridView1
    // la sursa de date din dataset
    BindingSource bs = new BindingSource(libDataSet,
        "Carti");
```

```
// Legarea se face prin proprietatea DataSource
dataGridView1.DataSource = bs;
}
```

8. La acționarea butonului *Salveaza*, se transmit modificările din *dataset* în baza de date. Tratăm evenimentul **Click**. Acționați dublu click pe buton. Metoda de tratare se editează astfel:

```
private void button1_Click(object sender, EventArgs e)
{
    // Propagăm modificările spre baza de date.
    int afectate = ta.Update(libDataSet.Carti);
    label1.Text = afectate + " randuri afectate";
}
```

9. Compilați și rulați cu **F5**.

Aplicația afișează forma:

Form1

0 randuri afectate

Salveaza

	Cod	Titlu	Autor	Pret
▶	231	Ion	Liviu Rebreanu	32
	1437	Rosu si Negru	Stendhal	33
	3512	Quo Vadis	Sienkiewicz	35
	6423	Ana Karenina	Tolstoi	48
*				

Dacă spre exemplu operăm modificări în celulele din randurile 3 și 4, iar apoi apăsăm *Salveaza*, obținem:

Form1

2 randuri afectate

Salveaza

	Cod	Titlu	Autor	Pret
	231	Ion	Liviu Rebreanu	32
	1437	Rosu si Negru	Stendhal	33
	3512	Quo Vadis	Henrik Sienkiewicz	35
▶	6423	Ana Karenina	Lev Tolstoi	48
*				

La o nouă rulare cu **F5** veți constata că modificările aduse bazei de date sunt persistente.

Observații:

- Mediul VCSE creează cele două fișiere ale bazei de date (**.mdf** și **.LDF**) în folderul de bază al aplicației, la un loc cu fișierul **.csproj**. În momentul în care faceți **Build** cu **F5** (modul depanare), se face o copie a celor două fișiere în folderul **\Bin\Debug**, iar aplicația se lansează și lucrează cu aceste copii. La noi rulări cu **F5**, nu se mai face **Build** dacă nu ați adus modificări în cod, ci numai lansări în execuție. Constatați că baza de date din **\Bin\Debug** se actualizează. Dacă ați operat modificări în cod, atunci la un **F5** se face și **Build** și baza de date originală se copiază peste cea din **\Bin\Debug**.
- Stringul de conectare se generează în mod automat la crearea bazei de date cu ajutorul mediului integrat. Se poate vizualiza astfel: în **Solution Explorer**, click dreapta pe fișierul **app.config** și selectați **Open**. Veți constata informațiile de conectare se păstrează în format **XML**.
- Atribuirea **dataGridView1.DataSource = bs; leagă** controlul de **dataset**, astfel încât orice modificare operată în celulele controlului, se transmite în mod automat tabelii din **dataset**.

Dataset-urile și XML

Când lucrați cu **dataset**-uri, aveți posibilitatea să salvați datele local, într-un fișier de tip **XML**, astfel încât informațiile să persiste și în alt loc decât în baza de date după încheierea sesiunii.

Salvați datele cu ajutorul metodei **WriteXml()** a clasei **DataSet** astfel:

```
ds.WriteXml(@"C:\date.xml");
```

Când deschideți o nouă sesiune a aplicației, aveți alternativa de a reîncărca în **dataset** datele preluate din fișierul XML. Citirea se face cu metoda **ReadXml()**:

```
ds.ReadXML("C:\date.xml");
```

Exemplu:

Ne întorcem la aplicația **UpdateDatabase**, realizată anterior. Introduceți ca ultimă linie în corpul metodei de tratare a evenimentului **Click** pentru butonul **Salveaza**, apelul:

```
libDataSet.WriteXml(@"C:\carti_bune.xml");
```

Rulați cu **F5**, apoi căutați fișierul **carti_bune.xml**. Veți constata că toate informațiile din **dataset** s-au salvat în format **XML**.

Controalele și legarea datelor

Anumite proprietăți ale controalelor se pot lega la o sursă de date. Sursele de date sunt diverse: o proprietate a altui control, o celulă, un rând sau o tabelă într-un dataset, sau o simplă variabilă. Conceptul se numește *Data Binding*. După ce au fost legate, valorile din sursa de date schimbă valorile din proprietatea legată și invers.

Controalele *Windows Forms* suportă două tipuri de legare a datelor (*data binding*):

- **Legarea simplă.**
- **Legarea complexă.**

Legarea simplă a datelor

Acest tip de legare vă permite să atașați o proprietate a unui control la o singură valoare din sursa de date. Este utilă pentru controale ca **TextBox** sau **Label**, care afișează o singură valoare. Aplicația următoare arată cum puteți lega proprietățile a două controale.

Aplicația *SimpleDataBinding*

1. Creați un nou proiect *WindowsForms* cu numele *SimpleDataBinding*.
2. Pe suprafața formei plasați două controale: un **TextBox** și un **Label**.
3. În constructorul clasei *Form1*, introduceți codul evidențiat în **Bold**:

```
public Form1()
{
    InitializeComponent();
    // Legăm proprietatea Text a butonului (primul
    // parametru) de proprietatea Text a text box-ului
    button1.DataBindings.Add("Text", textBox1, "Text");
}
```

4. Tratăm evenimentul **Click** al butonului. Acționați dublu click pe buton și introduceți codul marcat în **Bold**, în metoda de tratare:

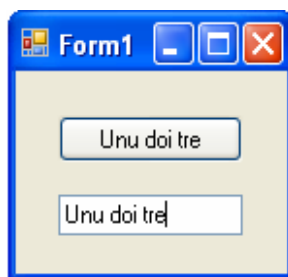
```
private void button1_Click(object sender, EventArgs e)
{
    button1.Text = "Salutare!";
}
```

5. Rulați cu **F5**.

Faceți click pe buton, apoi un click în *text box* ca să primească focusul. În *text box* se afișează „**Salutare!**”. Dacă introduceți text în *textbox*, eticheta butonului se modifică corespunzător.



Click pe buton



Editare în text box

Legarea complexă a datelor

Aplicați acest tip de legare atunci când vreți să afișați o listă de valori din sursa de date. Controalele care suportă legarea complexă sunt **DataGridView**, **ListBox**, **ComboBox**, **CheckedListBox**. Aceste controale au de proprietățile **DataSource** și **DataMember**, cu ajutorul cărora le puteți lega la tabela unei baze de date, astfel:

```
dataGridView1.DataSource = librarieDataSet;  
dataGridView1.DataMember = "carti";
```

În exemplu, proprietății **DataSource** a gridului i se atribuie *dataset*-ul aplicației, iar proprietății **DataMember** i se atribuie numele unei tabele din *dataset*. Aceste setări se pot face ușor în fereastra **Properties**, așa cum vom vedea în continuare.

Aplicația *ComplexDataBinding1*

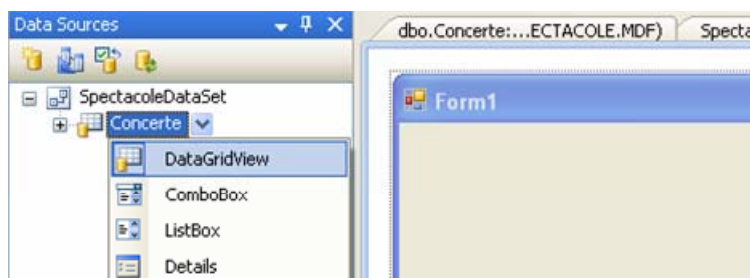
Aplicația utilizează utilizează facilitățile mediului integrat pentru a lega proprietățile unui control **DataGridView** la o tabelă a unei baze de date *SQL Server 2005 Express*. Urmăți pașii:

1. Creați în **VCSE** un nou proiect de tip *Windows Forms*, cu numele *ComplexDataBinding1*.
2. Adăugați proiectului o bază de date cu numele **Spectacole**. În acest scop, urmați indicațiile subcapitolului "Crearea unei baze de date în VCSE". *Wizardul* propune numele *SpectacoleDataSet* pentru clasa pe care o generează. Aceptați acest nume.
3. Adăugați bazei de date **Spectacole** tabela **Concerte**, cu următoarea structură:

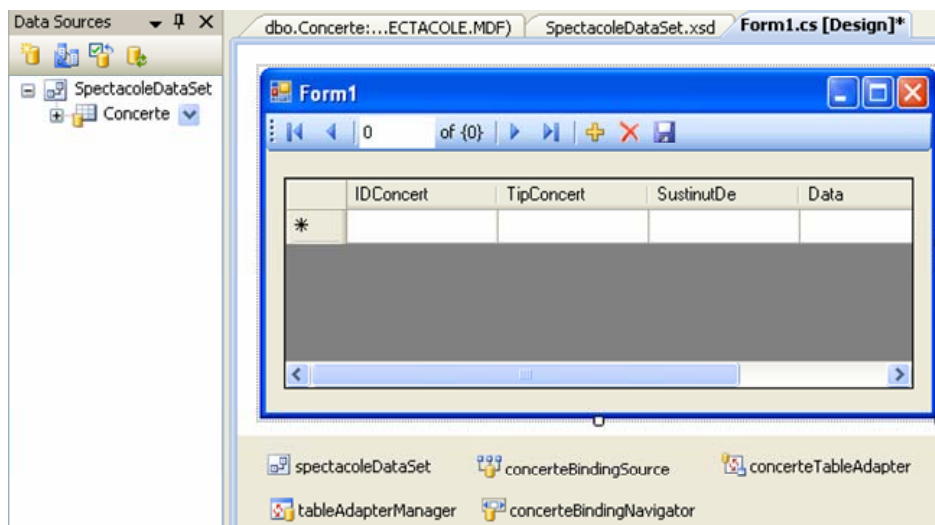
Column Name	Data Type	Allow Nulls
IDConcert	int	☐
TipConcert	nvarchar(20)	☒
SustinutDe	nvarchar(50)	☐
Data	datetime	☐
Locatie	nvarchar(20)	☒

Completați câteva rânduri cu date în tabelă.

- În fereastra **Data Source** acționați click dreapta pe tabela **Concerte** și selectați **Edit DataSet with Designer**. După ce **designer**-ul se deschide, trageți tabela **Concerte** cu mouse-ul din fereastra **Data Source** pe suprafața designerului. Prin această acțiune am cerut ajutorul mediului integrat, care a generat pentru noi o clasă **ConcerteTableAdapter**.
- În **View Designer** selectați forma, iar în fereastra **Data Sources** deschideți lista atașată tablei **Concerte** și selectați **DataGridView**.



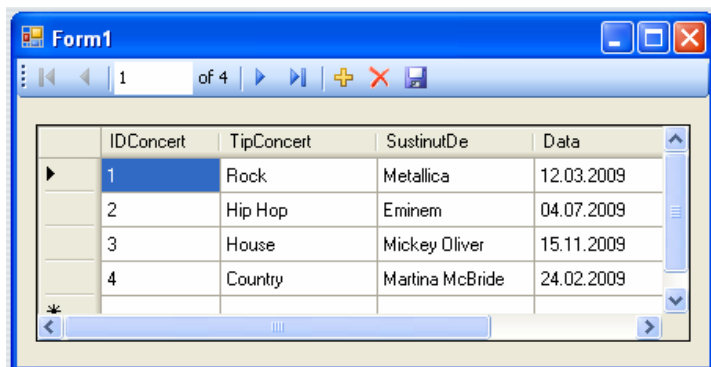
- Urmează o acțiune simplă: în fereastra **Data Sources** selectați cu mouse-ul tabela **Concerte**, apoi o trageți peste formă. Pe suprafața formei apare un control **DataGridView**, cu celulele legate la tabela **Concerte**:



Observați apariția în *designer tray* a câtorva referințe la obiecte de tip **TableAdaptor** sau **BindingNavigator**.

7. Rulați cu **F5**.

Veți obține:



	IDConcert	TipConcert	SustinutDe	Data
▶	1	Rock	Metallica	12.03.2009
	2	Hip Hop	Eminem	04.07.2009
	3	House	Mickey Oliver	15.11.2009
	4	Country	Martina McBride	24.02.2009

Încercați singuri:

În proiectul anterior, ștergeți din *designer tray* apăsând tasta *delete* toate referințele. Ștergeți și gridul de pe formă. În fereastra **Data Sources**, selectați *Details*, apoi trageți cu mouse-ul tabela pe suprafața formei. Pe suprafața formei vor apărea patru controale de tip **TextBox** și un **DateTimePicker**. Fiecare control este legat la una dintre coloanele tabelului.



IDConcert: 2

Tip Concert: Hip Hop

Sustinut De: Eminem

Data: 4 iulie 2009

Locatie: Bucuresti

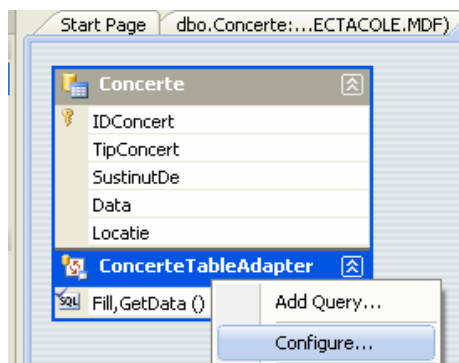
Toate controalele indică același rând din tabelă. Navigarea de la un rând la altul se face cu ajutorul butoanelor săgeată din controlul **ToolStrip** din partea de sus a formei.

Aplicația **ComplexDataBinding2**

Modificăm proiectul anterior astfel încât să legăm proprietăți ale controalelor **ListBox** și **TextBox** la baza de date, ca să fie posibilă editarea datelor și transmiterea modificărilor bazei de date.

Pașii 1, 2, 3, 4 sunt identici cu cei ai aplicației precedente. Schimbați doar numele proiectului : *ComplexDataBinding2*.

5. În *Dataset Designer*, acționați click dreapta pe itemul **ConcerteTableAdapter** și selectați *Configure...*

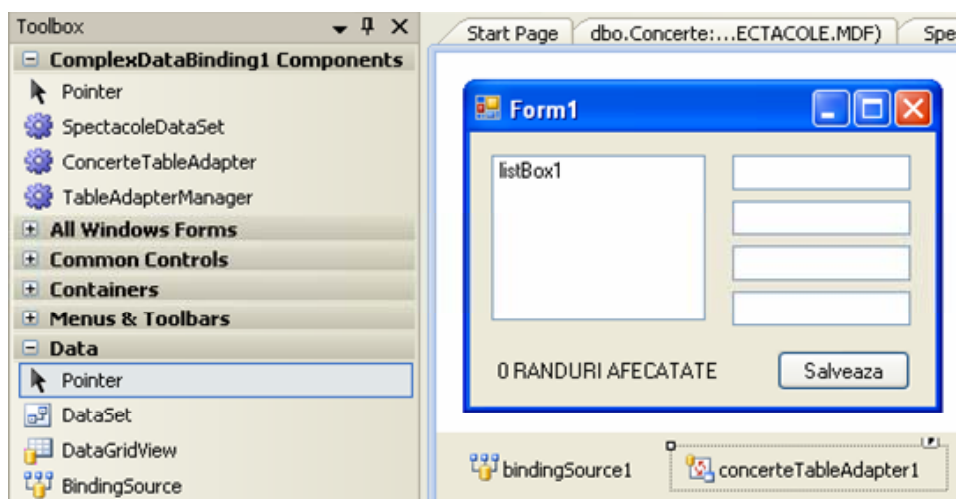


Configurăm adaptorul, deoarece vom apela metoda **Update()** pentru actualizarea bazei de date și această metodă utilizează interogări **SQL** în acest scop. Vom genera interogările în mod automat. Apăsăm butonul *Next* al primului dialog. Al doilea dialog ne spune că va genera pentru noi metodele **Fill**, **GetData**, **Insert**, **Delete** și **Update**. Apăsăm din nou *Next*, apoi pe ultimul dialog, *Finish*.

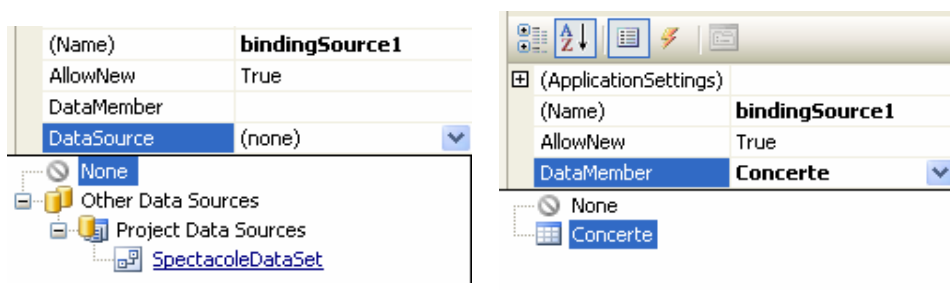
6. Plasați în *Form Designer* pe suprafața formei un control **ListBox**, patru controale **TextBox**, un **Label** și un buton :



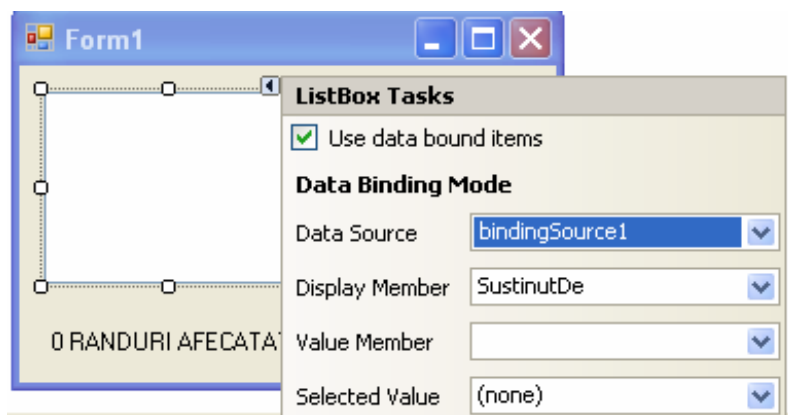
7. Din **Toolbox** alegeți și plasați pe suprafața formei, componentele: **ConcerteTableAdapter** și **Binding Source**. Referințele se creează automat și apar în *designer tray*.



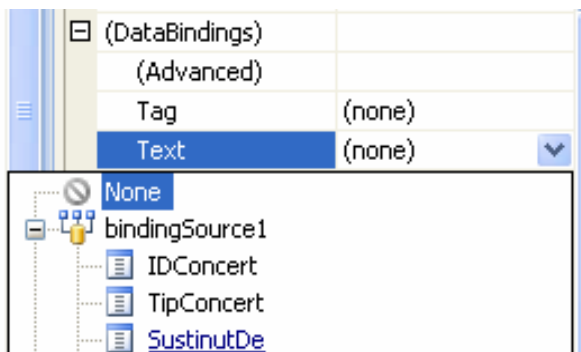
8. Selectați în *designer tray* **bindingSource1**. În fereastra **Properties** atribuiți proprietății **DataSource** valoarea **SpectacoleDataSet**, iar proprietății **DataMember**, tabela **Concerte**:



9. Selectați controlul **ListBox**. Acționați click pe săgeata dreapta sus. În fereastra care se deschide, faceți selecțiile ca în figură:



10. Selectați controlul **TextBox** cel mai de sus. În fereastra **Properties**, expandați itemul **Data Bindings** și setați proprietatea **Text** la valoarea **SustinutDe** :



11. Executați aceleași operații de la pasul **10**, pentru următoarele trei **textbox**-uri, cu diferența că veți lega proprietatea **Text** la celelalte coloane ale tabelului.
12. La încărcarea formei, vom umple *dataset*-ul. Dublu click pe suprafața formei. În *Editorul de Cod* scrieți:

```
private void Form1_Load(object sender, EventArgs e)
{
    // Fill() încarcă datele din baza de date în dataset
    concerteTableAdapter1.Fill(spectacoleDataSet.Concerte);
}
```

13. La click pe butonul *Salvează*, transmitem bazei de date modificările făcute de utilizator. Dublu click pe buton. Scrieți în metoda *handler*:

```
private void button1_Click(object sender, EventArgs e)
{
    // Aplicăm sursei de date modificările făcute
    bindingSource1.EndEdit();

    // Updatăm baza de date
    int afectate =
    concerteTableAdapter1.Update(spectacoleDataSet);

    label1.Text = afectate + " randuri afectate";
}
```

14. Rulați cu **F5**.

Se obține:

Form1

Metallica
Eminem
Mickey Oliver
Martina McBride

Eminem
Hip Hop
Bucuresti
04.07.2009

1 randuri afectate

Salveaza

La click în *list box*, textele se modifică în controalele text box în mod automat, astfel încât toate controalele de pe formă indică același rând din tabelă. Valorile celulelor tabelii se pot edita din *text box*-uri, iar modificările în baza de date sunt persistente.

Capitolul 13

Relații între tabele

Între tabelele unei baze de date (și a unui *dataset*) pot să existe relații de tip **părinte-copil**. Să presupunem că în baza de date **Scoala.mdf** există două tabele: **Clase** și **Elevi**. O clasă are mai mulți elevi. Construim deci o relație între cele două tabele, de tip **una la mai multe** (*one to many*).

Constrângerea Cheie Străină-Cheie Primară

În tabela **Clase** avem un câmp **Clasa**, care reprezintă numele clasei (ex. XII-B). Îl setăm cheie primară. În tabela **Elevi** introducem un câmp cu același nume. Nu dorim să existe elevi pentru care câmpul **Clasa** are o valoare care nu există în tabela **Clase**. Impunem o constrângere de tip **foreign key-primary key**. Este o **constrângere referențială**. **Clase** este tabela părinte, iar **Elevi**, tabela copil. Un rând al tabelului părinte poate să refere multiple rânduri în tabela copil.

Aplicația Scoala

Aplicația creează baza de date și tabelele precizate mai sus, impunând o constrângere de **cheie străină-cheie primară**. Dacă un rând în tabela **Clase** se șterge, atunci dorim să se șteargă toate rândurile corespunzătoare din tabela **Elevi** pentru a păstra **integritatea datelor**. Vom seta regula de ștergere pentru această relație: **ÎN CASCADĂ**.

Atenție !

Vă sugerăm să nu ocoliți acest proiect, deoarece pe structura lui vom lucra în temele următoare, unde va fi dezvoltat și completat.

1. Realizați în **VCSE** un proiect de tip *Windows Forms*, cu numele *Scoala*.
2. Adăugați proiectului o bază de date cu numele **Scoala**. În acest scop, urmați indicațiile subcapitolului “**Crearea unei baze de date în VCSE**”. Aceptați numele **ScoalaDataSet** pentru clasa de tip **DataSet** care se generează.
3. Adăugați bazei de date **Scoala** tabelele **Clase** și **Elevi** cu următoarele structuri:

Tabela **Clase**

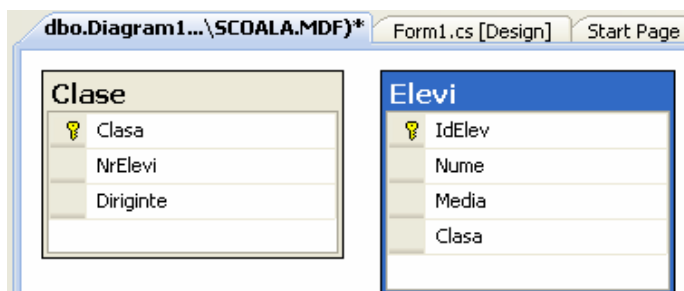
	Column Name	Data Type	Allow Nulls
	Clasa	nvarchar(50)	☐
	NrElevi	int	☐
	Diriginte	nvarchar(50)	☒

Tabela **Elevi**

	Column Name	Data Type	Allow Nulls
	IdElev	int	☐
	Nume	nvarchar(50)	☒
	Media	real	☒

După completarea schemei, introduceți câteva rânduri în fiecare tabelă.

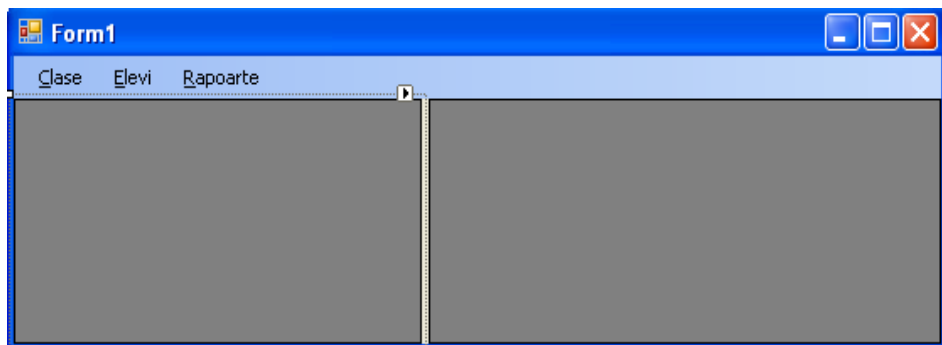
4. În **Database Explorer**, click dreapta pe **Database Diagrams**. În dialogul **Add Table** selectați pe rând tabelele **Clase** și **Elevi** și apăsați butonul **Add**. Diagramele celor două tabele apar în **designer**:



5. Introducem constrângerea de cheie străină. Punctați cu mouse-ul în câmpul **Clasa** din tabela **Clase** și trageți ținând butonul apăsat până pe câmpul **Clasa** al tabelii **Elevi**. Se deschide un dialog care identifică cheia primară, cheia străină și numele relației care se crează:

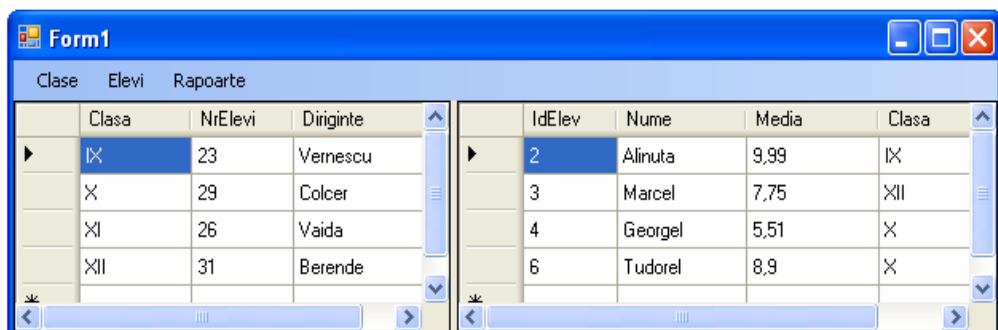
6. Apăsați **OK** să închideți dialogul anterior. Următorul dialog vă permite să stabiliți regulile relației de cheie străină. Setati pentru **Delete** și **Update** regula **Cascade**, apoi apăsați **OK**:

7. În *designer*, relația se vizualizează ca o linie cu o cheie spre tabela părinte și simbolul infinit spre tabela copil (relație *una la mai multe*). Salvați diagrama sub numele *Diagram1*.
8. Plasați pe suprafața formei un control **MenuStrip**, un **SplitContainer**, iar în cele două panouri ale acestuia, câte un control **DataGridView**:



9. Adăugați meniului **Clase**, opțiunile: *Adauga clasa*, *Sterge clasa*, *Modifica clasa*. Adăugați meniului **Elevi**, opțiunile: *Adauga elev*, *Sterge elev*, *Modifica elev*. Adăugați meniului **Rapoarte**, opțiunile: *Diriginti-clase* și *Media pe clase*.
10. Configurăm *dataset*-ul aplicației cu ajutorul *Wizard*-ului. În fereastra **Data Sources**, acționați click dreapta pe itemul *ScoalaDataSet* și selectați *Configure DataSet with Wizard*. În dialogul care se deschide, selectați toate categoriile: *tabele*, *vederi*, *proceduri stocate*, *funcții*, apoi apăsați *Finish*.
11. Din fereastra **Data Sources**, trageți cu mouse-ul tabela **Clase** peste gridul din stânga al formei, apoi trageți tabela **Elevi** peste gridul din dreapta.
12. Rulați cu **F5**.

La rulare, se obține:



Interogări. Proceduri stocate.

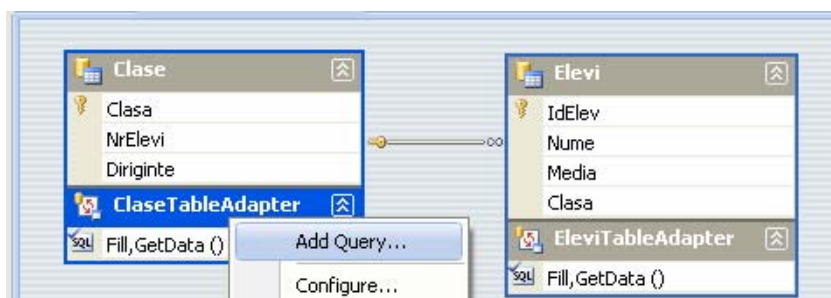
Continuăm să dezvoltăm proiectul **Scoala**, început anterior. Opțiunile din meniuri nu sunt încă funcționale. Ca să fie, trebuie să interogăm baza de date. Dacă vrem să introducem noi clase sau elevi, avem nevoie de comanda **SQL INSERT**. Când vom dori să ștergem din baza de date o clasă sau un elev, aplicăm **DELETE**, iar când modificăm datele ne trebuie **UPDATE**.

O întrebare firească ar fi: cum trimitem aceste interogări serverului de baze de date, din codul nostru C# ? O primă variantă este să manevrăm clasele **SqlConnection**, **SqlCommand**, **SqlDataAdapter**. Este laborios dacă scriem tot codul necesar.

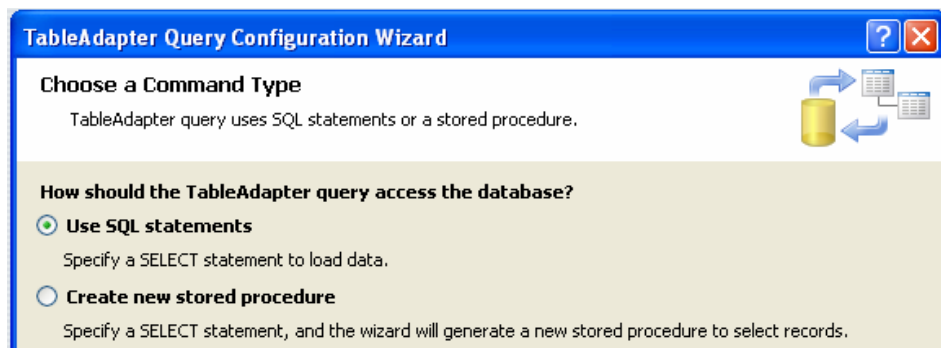
Din fericire, mediul integrat ne ajută mult în această privință. Vom genera în mod automat metode C# pentru clasele **ClaseTableAdapter** și **EleviTableAdapter**. Aceste metode încapsulează interogările SQL sau *proceduri stocate*. În continuare adăugăm interogări aplicației **Scoala**.

Aplicația **Scoala** – adăugarea interogărilor

1. În **Solution Explorer**, acționați dublu click pe itemul **ScoalaDataSet.xsd**. În **Dataset Designer**, pe diagrama tabeli **Clasa**, click dreapta pe **ClaseTableAdapter**. Selectați **Add Query...**

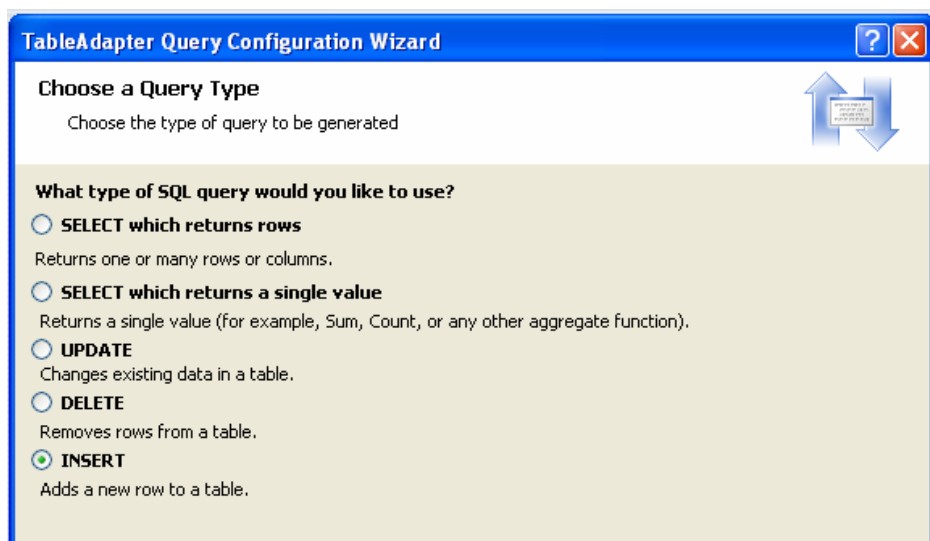


2. Dialogul următor vă permite să generați o metodă care conține o interogare SQL, o procedură stocată, sau o metodă care apelează o procedură stocată existentă:

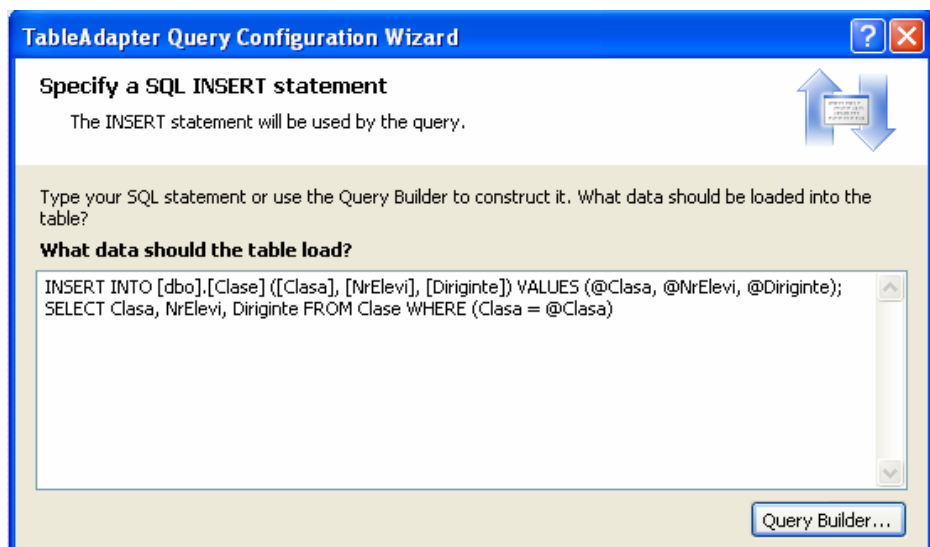


Selectați *Use SQL statements* și apăsați **OK**.

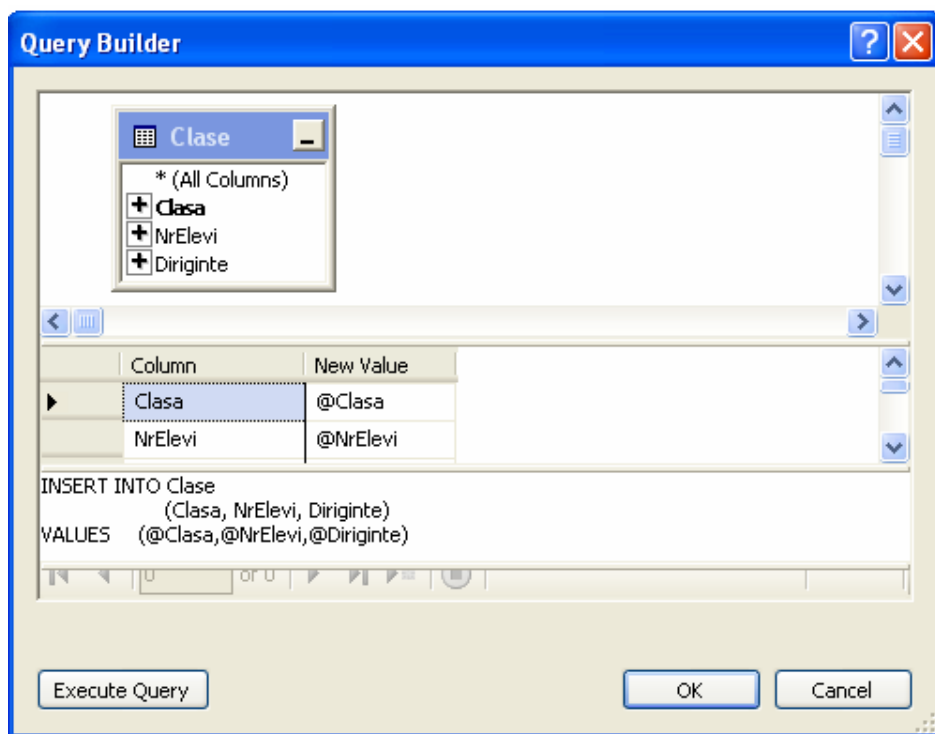
3. În dialogul următor, selectați **INSERT** și apăsați *Next*:



4. Dialogul care se deschide propune o interogare **INSERT TO** pentru adăugarea unei clase. Apăsați butonul *Query Builder...*

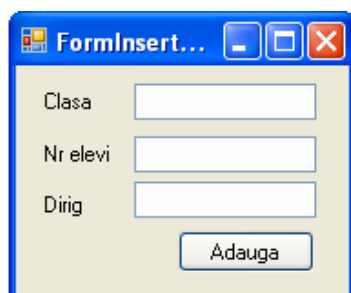


5. Se deschide dialogul *Query Builder*. Acesta este un instrument util, pentru că vă permite să selectați pe diagrama *Clase* câmpurile pe care le înserați, iar codul se generează automat. Evident, puteți edita și manual:



Ați remarcat modul în care se specifică valorile rândului care se inserează? De fapt, @Clasa, @NrElevi, @Diriginte reprezintă numele **parametrilor metodei Insert()** care se va genera, ca membră a clasei **ClaseTableAdapter**. În dialogul următor, setați numele metodei: **InsertClasa**.

6. Ca să introduceți datele, creați o nouă formă. În **Solution Explorer**, click dreapta pe numele proiectului, selectați **Add**, apoi **Windows Form...** Numeți forma **FormInsertClasa**:



7. Din **Toolbox**, selectați componenta **ClaseTableAdapter** și plasați-o pe suprafața formei. În **designer tray**, apare referința **claseTableAdapter1**. Facem acest lucru deoarece avem nevoie de adaptor pentru apelul **Insert()**,

dar în clasa **FormInsertClasa** nu este vizibilă adaptorul **claseTableAdapter**, definit în clasa **Form1**.

8. Acționați dublu click pe butonul *Adaugă*. În metoda de tratare, scrieți:

```
private void button1_Click(object sender, EventArgs e)
{
    if (textBox1.Text == "" || textBox2.Text == "" ||
        textBox3.Text == "")
    {
        MessageBox.Show(
            "Nu ati completat toate campurile!");
        return;
    }

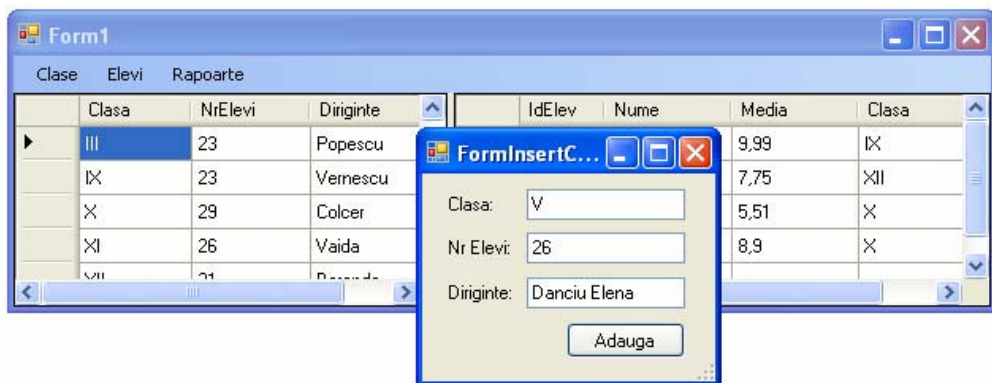
    claseTableAdapter1.Insert(textBox1.Text,
        int.Parse(textBox2.Text), textBox3.Text);
    Close();
}
```

9. Pe forma *Form1*, acționați dublu click pe opțiunea *Adauga clasa*. În metoda de tratare scrieți:

```
private void adaugaClasaToolStripMenuItem_Click(
    object sender, EventArgs e)
{
    // Instanțiem forma FormInsertClasa
    FormInsertClasa fInsCl = new FormInsertClasa();
    fInsCl.ShowDialog();
    // Acum în baza de date avem un rând nou
    // Încărcăm datele din baza de date în dataset
    claseTableAdapter.Fill(scoalaDataSet.Clase);
}
```

10. Compilați și rulați cu **F5**.

La rulare, introduceți clase cu nume diferite, cu ajutorul opțiunii *Adauga clasa* :



Generarea procedurilor stocate

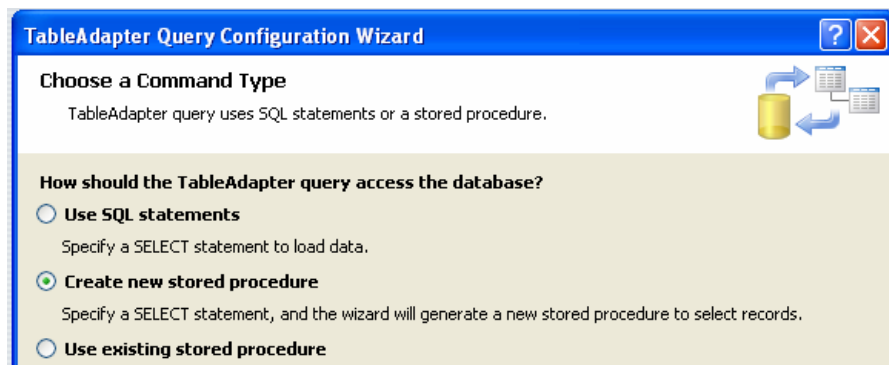
O **procedură stocată** (*stored procedure*) consistă dintr-un grup de comenzi **SQL** adunate sub un nume. Comenzile au fost anterior create și stocate pe serverul de baze de date. Procedurile stocate sunt compilate o singură dată și apoi aplicația client le poate apela de câte ori este necesar. Sunt foarte rapide. Acceptă date prin parametri de intrare. Datele sunt specificate în timpul execuției.

În aplicația anterioară, înainte de apelul `Insert()`, am verificat dacă toate câmpurile sunt completate. Nu am verificat alte două aspecte importante: dacă valoarea introdusă pentru *Nr Elevi* este sau nu un număr întreg și dacă clasa introdusă nu există deja în baza de date. În ambele situații, se aruncă excepții.

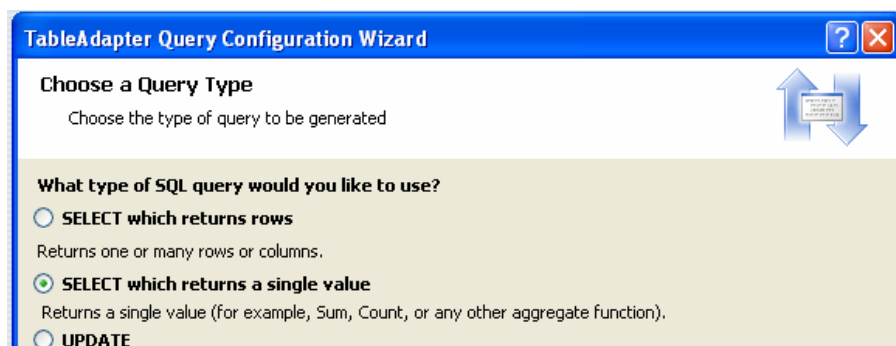
Prima problemă se rezolvă relativ ușor, cu ajutorul metodei `int.TryParse(string, int)`, care încearcă să convertească primul argument în număr, iar dacă nu reușește returnează *false*.

A doua problemă necesită interogarea bazei de date (un **SELECT**) pentru a vedea dacă clasa nu există deja în baza de date. Pentru aceasta putem genera o nouă metodă în clasa adaptorului, însă vom prefera să creăm o procedură stocată, pentru a vedea cum se procedează. Urmați indicațiile :

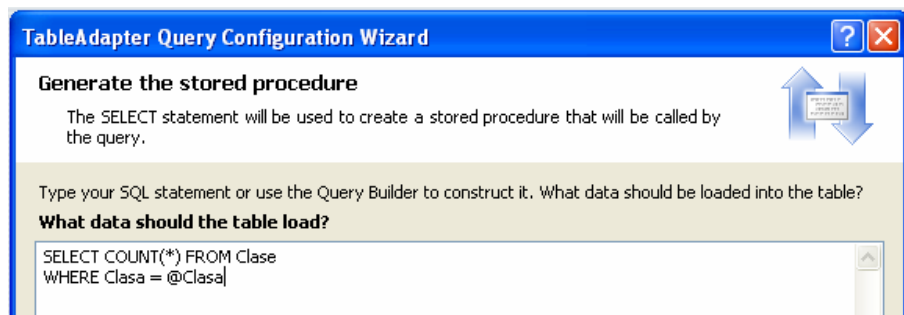
- În *Dataset Designer*, selectați diagrama **Clase** și faceți click dreapta pe adaptorul **ClassTableAdapter**. Selectați *Add Query*, apoi în dialogul care apare, selectați *Create new stored procedure*:



- În dialogul următor, alegeți **“SELECT wich returns a single value”**:

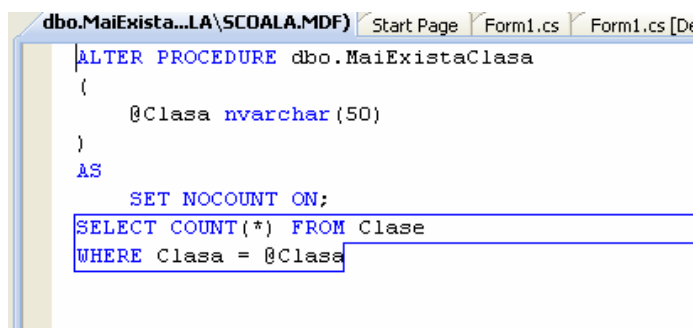


- În dialogul următor, editați interogarea astfel:



Am adăugat clauza **WHERE**. Parametrul @Clasa devine în mod automat parametru de intrare al procedurii stocate. COUNT() returnează numărul de rânduri care conțin clasa @Clasa.

- Apăsați butonul *Next* și stabiliți numele procedurii stocate **MaiExistaClasa**. Puteți să păstrați în dialogul următor același nume și pentru funcția care apelează procedura stocată. Apăsați butonul *Finish*.
- Dacă vreți să vedeți sau să editați codul procedurii stocate, mergeți în **Database Explorer**, expandați itemul *Stored Procedures* și dublu click pe procedura **MaiExistăClasa**:



- Acum suntem în măsură să completăm codul metodei `button1_Click`:

```
private void button1_Click(object sender, EventArgs e)
{
    if (textBox1.Text == "" || textBox2.Text == "" ||
        textBox3.Text == "")
    {
        MessageBox.Show(
            "Nu ati completat toate campurile!");
        return;
    }
}
```

```
int intreg = 0;
if (!int.TryParse(textBox2.Text, out intreg))
{
    MessageBox.Show(
        "Nr elevi trebuie să fie numar intreg!");
    return;
}

if( (int)claseTableAdapter1.MaiExistaClasa(
    textBox1.Text) == 1)
{
    MessageBox.Show("Clasa exista deja!");
    return;
}
claseTableAdapter1.Insert(textBox1.Text,
    int.Parse(textBox2.Text), textBox3.Text);
Close();
}
```

IMPORTANT !

*Metoda **Insert**, ca de altfel toate metodele care conțin interogări, pot să arunce excepții. De aceea, trebuie incluse în blocuri **try-catch**. Noi nu am făcut-o aici, pentru claritate.*

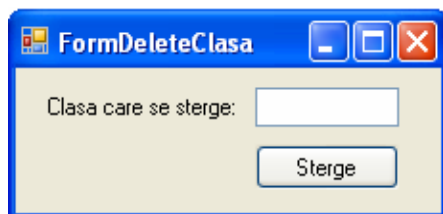
Temă de lucru:

Implementați facilitatea de adăugare a unui nou elev în baza de date la selectarea opțiunii **Adauga elev** din meniul **Elevi**. Trebuie să creați o funcție suplimentară care interoghează baza de date determinând dacă valoarea pe care o introduce utilizatorul pentru câmpul **Clasa** există sau nu în tabela **Clase**.

Ștergerea unei înregistrări din baza de date

Pentru a șterge o clasă din baza de date, este nevoie de o funcție care încapsulează comanda SQL **DELETE**. Procedăm astfel:

1. Adăugați o nouă formă cu numele **FormDeleteClasa**:



2. Din **Toolbox**, selectați componenta **ClaseTableAdapter** și plasați-o pe suprafața formei **FormDeleteClasa**. În *designer tray*, apare referința **claseTableAdapter1**.
11. În **Solution Explorer**, acționați dublu click pe itemul **ScoalaDataSet.xsd**. În *Dataset Designer*, pe diagrama tabelii **Clasa**, click dreapta pe **ClaseTableAdapter**. Selectați *Add Query...*
12. În primul dialog, selectați *Use SQL statements*. Apăsați *Next*. În al doilea dialog, selectați **DELETE**. În *Query Builder*, editați interogarea astfel:

```
DELETE FROM Clase
WHERE      (Clasa = @Original_Clasa)
```

@Original_Clasa este numele parametrului de intrare al funcției care se generează.

13. Stabiliți în următorul dialog, numele funcției: **DeleteClasa**.
14. Pe **Form1**, în *View Designer*, efectuați dublu click pe opțiunea **StergeClasa**. În corpul *handler*-ului introduceți:

```
private void stergeClasaToolStripMenuItem_Click(
    object sender, EventArgs e)
{
    FormDeleteClasa fDelCl = new FormDeleteClasa();
    fDelCl.ShowDialog();

    // Reîncărcăm cele două tabele din dataset
    // cu datele modificate din baza de date
    claseTableAdapter.Fill(scoalaDataSet.Clase);
    eleviTableAdapter.Fill(scoalaDataSet.Elevi);
}
```

15. Acționați dublu click pe butonul *Sterge*, al formei **FormDeleteElev**. În metoda de tratare introduceți:

```
private void button1_Click(object sender, EventArgs e)
{
    claseTableAdapter1.DeleteClasa(textBox1.Text);
    Close();
}
```

16. Rulați cu **F5**.

La execuție veți constata că atunci când ștergeți o clasă, se șterg și toate rândurile orfane din tabela **Elevi**, conform regulii de ștergere în cascadă pe care am impus-o pentru menținerea integrității datelor la crearea constrângerii de cheie străină.

Temă de lucru:

Implementați funcționalitatea de ștergere a unui elev, la selectarea opțiunii *Sterge elev* din meniul *Elevi*. Elevul va fi căutat după câmpul `IdElev`.

Vederile unei baze de date (Views)

Un **View** este o tabelă virtuală, compusă din rezultatul unei interogări. Vederile se creează dinamic. Includ date selectate din una sau mai multe tabele ale bazei de date, și eventual date rezultate în urma unui calcul.

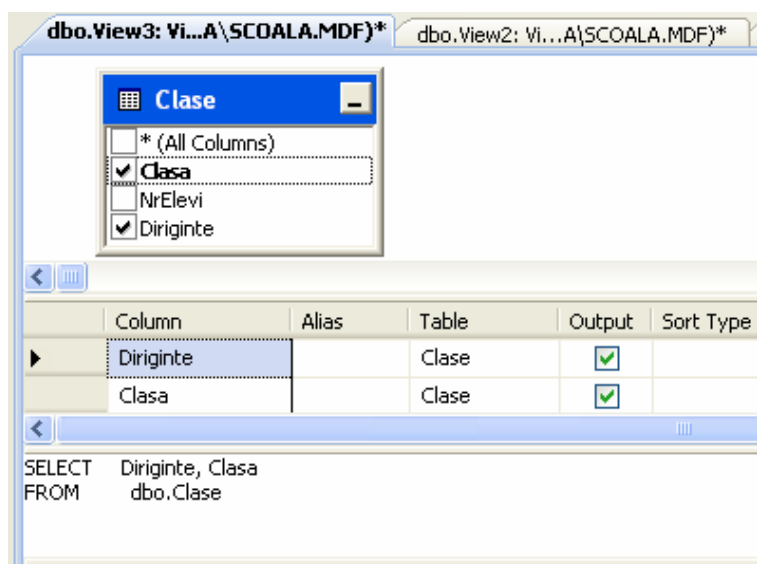
Modificarea datelor în tabele duce automat la modificarea datelor afișate în vederi. Au avantajul că nu ocupă spațiu fizic pe disc.

Vederile se folosesc frecvent atunci când se dorește afișarea rezultatului *join*-ului tabelelor și când nu se dorește modificarea datelor ci doar vizualizarea lor. Ne vom lămuri în cele ce urmează.

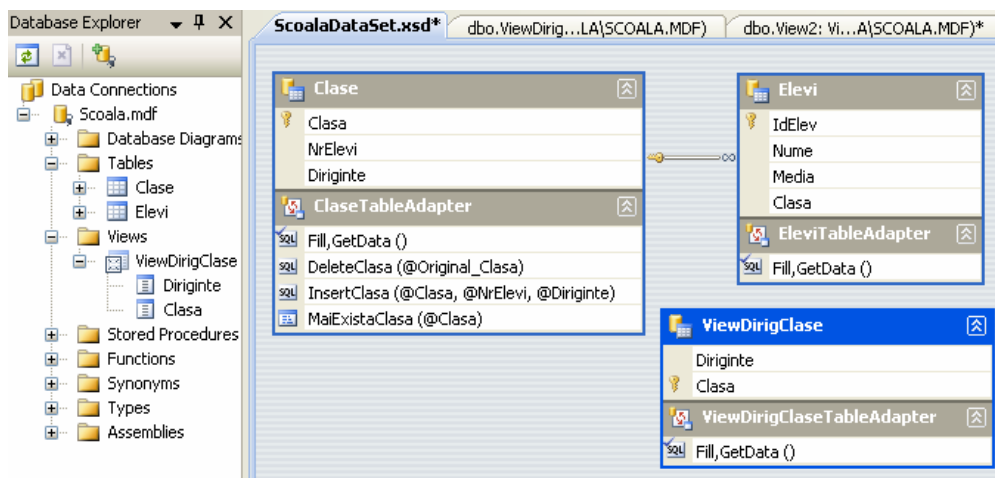
Aplicația *Scoala* – adăugarea vederilor**Vedere cu date selectate dintr-o singură tabelă**

Să presupunem că vrem să vedem o tabelă care conține numai coloanele **Diriginte** și **Clasa** din tabela **Clase**. Nu este cazul să definim o tabelă nouă. Construim o vedere care afișează doar datele din aceste coloane. Procedați astfel:

1. Vederea va fi afișată într-un grid. Avem nevoie de o formă nouă. Adăugați o formă nouă proiectului, acționând click dreapta în **Solution Explorer**, alegeți *Add* și selectați *Windows Form...* Salvați clasa formei cu numele **FormViewDirigClasa**.
2. Din **Toolbox** alegeți un control **DataGridView** și plasați-l pe suprafața formei.
3. În fereastra **Database Explorer**, click dreapta pe itemul **Views**, apoi selectați *Add New View*.
4. În dialogul *Add Table*, selectați tabela **Clase**, apăsați *Add*, apoi click pe butonul *Close*.
5. În *View Designer*, selectați câmpul **Diriginte** pe diagrama **Clase**, apoi câmpul **Clasa**. Interogarea **SELECT** se generează în mod automat:



6. Apăsați butonul **Save**, apoi introduceți numele vederii: **ViewDirigClase**.
7. Avem o vedere, dar aceasta nu este încă integrată în *dataset*. Pentru aceasta, deschideți **DataSet Designer** cu dublu click pe itemul **ScoalaDataSet.xsd** în **Solution Explorer**. Din panoul **Database Explorer**, trageți cu ajutorul mouse-ului vederea pe suprafața designerului. Obțineți:



8. În **Solution Explorer**, dublu click pe forma vederii, pentru a deschide **Form Designer**. Din fereastra **Data Sources**, trageți vederea pe suprafața formei.
9. Acționați dublu click pe opțiunea *Diriginti-clase* din meniul **Rapoarte**, aflat pe **Form1**. În corpul metodei de tratare scrieți:

```
private void dirigintiClaseToolStripMenuItem_Click(
    object sender, EventArgs e)
{
    FormClaseDirig fClDir = new FormClaseDirig();
    fClDir.ShowDialog();
}
```

10. Faceți Build și rulați cu **F5**.

La rulare, în momentul în care selectați opțiunea *Diriginti-Clase*, obțineți:

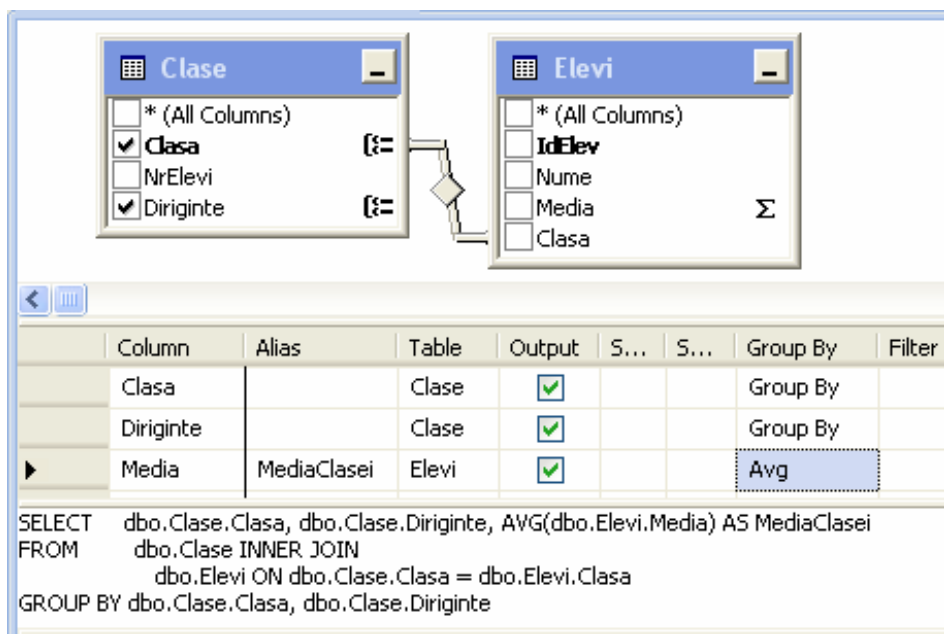


Vedere cu date selectate din două tabele

Așa cum am spus, tabela temporară rezultată în urma unui **JOIN** poate fi afișată într-un **View**.

Ne întoarcem la aplicația **Școala**. Să presupunem ni se cere media pe clase. Construim o vedere care afișează clasa, dirigintele și media clasei. Avem nevoie de informații din ambele tabele, deoarece **Diriginte** este câmp în tabela **Clase**, iar **Media** (unui elev) este câmp în tabela **Elev**. Reamintim că între cele două tabele există o constrângere de cheie străină. **Clasa** este cheie străină în tabela **Elev**. Procedați ca mai jos:

1. Adăugați o formă nouă proiectului, cu numele **FormViewMediaClase**.
2. Din **Toolbox** alegeți un control **DataGridView** și plasați-l pe suprafața formei.
3. În fereastra **Database Explorer**, click dreapta pe itemul **Views**, apoi selectați **Add New View**.
4. În dialogul **Add Table**, **Clase**, adăugați pe rând ambele tabele cu ajutorul butonului **Add**, apoi închideți cu butonul **Close**.
5. În **View Designer**, selectând opțiunile din figura de mai jos, veți obține interogarea necesară:



The screenshot shows a SQL query window with the following SQL code:

```
SELECT  dbo.Clase.Clasa, dbo.Clase.Diriginte, AVG(dbo.Elevi.Media) AS MediaClasei
FROM    dbo.Clase INNER JOIN
        dbo.Elevi ON dbo.Clase.Clasa = dbo.Elevi.Clasa
GROUP BY dbo.Clase.Clasa, dbo.Clase.Diriginte
```

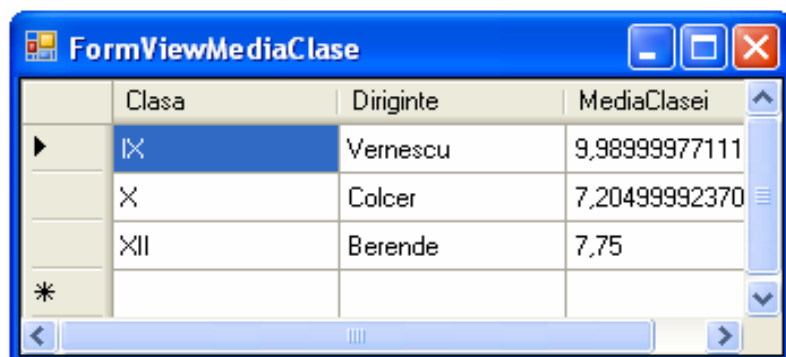
Evident, puteți să completați și manual codul **SQL**. Observați că se face un **JOIN INTERIOR** care returnează câte un rând pentru fiecare clasă. Media clasei se calculează cu funcția **AVG()**.

6. Apăsați butonul **Save**, apoi introduceți numele vederii: **ViewMediaClase**.
7. Vederea trebuie inserată în *dataset*. Deschideți **DataSet Designer** cu dublu click pe itemul **ScoalaDataSet.xsd** în **Solution Explorer**. Din panoul **Database Explorer**, trageți cu ajutorul mouse-ului vederea pe suprafața designerului.
11. În **Solution Explorer**, dublu click pe forma vederii, pentru a deschide **Form Designer**. Din fereastra **Data Sources**, trageți noua vedere pe suprafața formei **FormViewMediaClase**.
12. Acționați dublu click pe opțiunea **Media pe clase** din meniul **Rapoarte**, aflat pe **Form1**. În corpul metodei de tratare scrieți:

```
private void mediaPeClasaToolStripMenuItem_Click(
    object sender, EventArgs e)
{
    // Instanțiem forma și o afișăm
    FormViewMediaClase fv = new FormViewMediaClase();
    fv.ShowDialog();
}
```

8. Rulați cu **F5**.

La execuție, dacă selectați opțiunea de meniu *Media pe clase*, obțineți:



	Clasa	Diriginte	MediaClasei
▶	IX	Vernescu	9,98999977111
	X	Colcer	7,20499992370
	XII	Berende	7,75
*			

Probleme propuse

1. Adăugați aplicației **Scoala** un **View** care afișează pe trei coloane: numele diriginților care au elevi cu medii peste o valoare introdusă de la tastatură, numărul de elevi din fiecare clasă care îndeplinesc această restricție și clasa.
2. Creați o aplicație care întreține o bază de date cu numele **Firma**. Baza de date conține două tabele: *Cienti* și *Comenzi*. Tabelele vor defini cel puțin următoarele coloane: *Cienti*: (*IDClient*, *Nume*, *Prenume*, *Telefon*) și *Comenzi*: (*IDClient*, *Data*, *ValoareComanda*). Aplicația va implementa operații de adăugare de *Cienti* și de *Comenzi*. Va permite de asemenea afișarea tuturor clienților, iar pentru fiecare client, se va afișa valoarea totală a comenzilor sale.
3. Implementați o aplicație care să permită eliminarea dintr-o bază de date a tuturor produselor care au prețul cuprins între două limite introduse de la tastatură. Dacă în baza de date nu există nici o înregistrare cu proprietatea de mai sus, să se afișeze un mesaj. Pentru un produs se memorează: codul, denumirea, prețul, data recepției.
4. Să se realizeze o aplicație care să permită manipularea unei baze de date care conține un tabel cu structura: *nume medicament*, *compensat sau nu*, *procent compensare* și *preț întreg*. Pentru această tabelă să se poată actualiza prețul unui medicament localizat prin nume. Să se realizeze o listă cu medicamentele care nu beneficiază de compensare.

Bibliografie

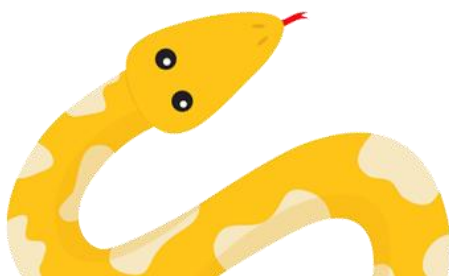
- [1] **Ecma Technical Committee 39 Task Group**
C# Language Specification 4-th Edition, june 2006
- [2] **Jesse Liberty**. *Programming C#*. 2-nd Edition. O'Reilly 2002.
- [3] **Andrew Troelsen** . *Pro C# 2008 and the .Net 3.5 Platform*. 4-th Edition. Apress 2008.
- [4] **Microsoft MSDN Express Library 2008**
- [5] **Trey Nash**. *Accelerated C# 2008*. Apress 2008
- [6] **Stanley Lippman**. *C# Primer: A Practical Approach*. Pearson Education Inc. 2003
- [7] **Herbert Schildt**. *C# 2.0 The Complete Reference*. 2-nd Edition
- [8] **Donis Marshall**. *Programming Microsoft Visual C# 2005: The Language*. Microsoft Press 2006.
- [9] **Rob Harrop**. *Effective Data Access in C#*. Wrox Press 200
- [10] **James Huddleston, Ranga Raghuram**. *Beginning C# 2005 Databases from Novice to Professional*. Apress 2006
- [11] **Paul Kimmel**. *Advanced C# Programming*. McGraw Hill/Osborne 2002
- [12] **John Sharp, Jon Jagger**. *Microsoft Visual C# .NET Step by Step*. Microsoft Press 2003
- [13] **F. Scott Barker**. *Visual C# 2005 Express Edition Starter Kit*. Wiley Publishing Inc. 2005
- [14] **Matthew MacDonald**. *Pro .NET 2.0 Windows Forms and Custom Controls*. Apress, 2006.
- [15] **John Paul Mueller**. *Visual C# .NET Developer's Handbook*. SYBEX Inc. 2002
- [16] **Daniel Solis**. *Illustrated C# 2008*. Apress 2008.
- [17] **Matthew MacDonald**. *Pro WPF in C# 2008: Windows Presentation Foundation with .NET 3.5*, Second Edition. Apress 2008
- [18] **Peter Wright**. *Beginning Visual C# 2005 Express Edition From Novice to Professional*. Apress 2006.
- [19] **MAHESH CHAND**. *A Programmer's Guide to ADO.NET in C#*. Apress 2003.
- [20] **Mickey Williams**. *Microsoft Visual C# .NET*. Microsoft Corporation 2002

infobits Academy

Compania noastră oferă de peste 28 de ani **manuale școlare aprobate M.E.N.** și **auxiliare ce respectă programa școlară, aplicații online**, precum și **cursuri de Informatică și T.I.C.**, utile oricărei persoane care dorește să se pregătească în aceste domenii. *Autorii și colaboratorii noștri* sunt oameni **dedicați procesului educațional**, cu o *experiență pedagogică vastă* și **autori de manuale școlare și auxiliare avizate M.E.N.**

www.infobits.ro

Vezi și alte proiecte, precum



www.pythonisti.ro

Cursul online interactiv respectă programa școlară **aprobată M.E.N.** și poate fi folosit în cadrul oricărui ciclu de învățământ ca o introducere în limbajul de programare [[Python](#)]. Informațiile prezentate sunt optimizate ca dimensiune și se pune accentul pe interacțiunea frecventă cu utilizatorul.

